

Lucid

An Apple Silicon Native Machine Learning Framework
A Comprehensive Technical Report

Chan Lee

May 24, 2026

© 2026 Chan Lee. All rights reserved.

This document describes **Lucid**, an Apple Silicon native machine learning framework. The author makes no warranties, express or implied, about the accuracy or completeness of the information herein. **Lucid** is an evolving open-source project; some material may be superseded by later releases.

Contents

List of Figures	xxxvi
List of Tables	xxxvii
Preface	xxxix
Acknowledgements	xl
Notation	xli
Abbreviations	xlii
I Hardware and Runtime Foundations	1
1 Apple Silicon Architecture	2
1.1 Overview	3
1.2 SoC Floorplan and Coherent Fabric	3
1.3 The CPU Complex	5
1.3.1 Performance cores	5
1.3.2 Efficiency cores	6
1.3.3 Big.LITTLE scheduling and QoS	6
1.4 Apple Matrix Coprocessor (AMX) and SME	6
1.4.1 Instruction format and execution model	7
1.4.2 Throughput characteristics	7
1.4.3 Relationship to ARM SME in the M4 generation	7
1.5 Apple Neural Engine (ANE)	7
1.6 The GPU	8
1.6.1 Compute hierarchy	8
1.6.2 Memory hierarchy on the GPU side	8
1.6.3 Dynamic Caching (M3 and later)	9
1.6.4 Hardware ray tracing and mesh shaders	9
1.6.5 Metal Performance Shaders and MPSGraph	9
1.7 Memory System	9
1.7.1 LPDDR generations and channels	10
1.7.2 System Level Cache	10
1.7.3 Bandwidth versus arithmetic intensity	11
1.8 The Apple Silicon Family: M1 through M4	11
1.8.1 M1 (2020)	11
1.8.2 M2 (2022)	11

1.8.3	M3 (2023)	11
1.8.4	M4 (2024)	12
1.9	Implications for ML Workloads	12
1.9.1	The dispatch trade-off	12
1.9.2	The memory-bandwidth ceiling	12
1.9.3	The power envelope	12
1.9.4	The compile-vs-eager trade-off	13
1.10	Summary and Forward References	13
2	Unified Memory Model and Memory Pressure	14
2.1	The Unified Memory Contract	15
2.2	Address Spaces and the macOS Virtual Memory Subsystem	15
2.2.1	Page residency states	16
2.2.2	Compressed memory	16
2.2.3	Swap and the NVMe controller	16
2.3	Metal Resource Storage Modes	17
2.3.1	Shared (MTLStorageModeShared)	17
2.3.2	Private (MTLStorageModePrivate)	17
2.3.3	Managed (MTLStorageModeManaged)	17
2.3.4	Memoryless (MTLStorageModeMemoryless)	18
2.3.5	What Lucid uses	18
2.4	Memory Pressure on macOS	18
2.4.1	Pressure-induced allocator behaviour	18
2.4.2	User-visible pressure symptoms	18
2.5	Lucid's Memory Accounting	19
2.5.1	Pool versus provider	19
2.5.2	Why the pool exists	19
2.6	The Zero-Copy CPU–GPU Bridge	20
2.6.1	The bridge interface	20
2.6.2	The stride trap	21
2.6.3	Why this is free	21
2.7	Memory Pressure and ML Workloads	21
2.7.1	The working-set decomposition	21
2.7.2	Activation memory and gradient checkpointing	22
2.7.3	The non-preallocation policy	22
2.8	Pathologies and Failure Modes	22
2.8.1	The lazy-graph leak (3.2.1)	23
2.8.2	The InstanceNorm OOM (3.2.2)	23
2.8.3	Cross-process sharing is not supported	23
2.9	Summary and Forward References	23
3	MLX Runtime: Graph-Eager Hybrid	25
3.1	Overview	25
3.1.1	What MLX is, what MLX is not	26
3.1.2	Why MLX, not Metal directly	26
3.2	The Graph-Eager Hybrid Execution Model	27
3.2.1	The lazy graph	27
3.2.2	Evaluation triggers	27
3.2.3	Why lazy by default	28
3.2.4	Asynchronous evaluation	28
3.3	The Array Primitive	28
3.3.1	Structure	28
3.3.2	Dtype and conversion	29

3.3.3	The data<T> accessor	29
3.4	Streams and Devices	30
3.4.1	The default device	30
3.4.2	Stream-aware primitives	30
3.4.3	The linalg carve-out	30
3.5	The Primitive Set	30
3.5.1	Elementwise and reduction	30
3.5.2	Shape manipulation	31
3.5.3	Indexing and gather/scatter	31
3.5.4	Linear algebra and FFT	31
3.5.5	Convolution and neural-network primitives	31
3.6	Function Transformations	31
3.7	Lucid's Use of MLX	32
3.7.1	The MlxBackend interface	32
3.7.2	Dispatch into MLX from the engine	32
3.7.3	Bridge from MLX to Lucid Tensor	32
3.8	Quirks and Pitfalls	33
3.8.1	data<T> ignores strides	33
3.8.2	Two scatter variants	33
3.8.3	Axis vs. axes nomenclature	33
3.9	Version Compatibility and the Wheel Constraint	33
3.9.1	API stability per minor version	33
3.9.2	Platform constraint: macOS $\geq$ 26.0	33
3.9.3	Per-version conditional code	34
3.10	Summary and Forward References	34
4	Apple Accelerate: BLAS, LAPACK, vDSP, vForce, BNNS	35
4.1	Overview	35
4.1.1	Linkage and headers	36
4.1.2	Two SDK transitions	36
4.2	BLAS and LAPACK	37
4.2.1	The CBLAS interface	37
4.2.2	LAPACK routines used by Lucid	37
4.2.3	The linalg carve-out, revisited	38
4.3	vDSP	38
4.3.1	Primitives used by Lucid	38
4.3.2	Stride support	39
4.3.3	Why not OpenBLAS or MKL	39
4.4	vForce	39
4.5	BNNS	39
4.5.1	The convolution fast path	40
4.5.2	The deprecation warning	40
4.5.3	Why Lucid still uses BNNS	40
4.6	AccelerateBackend in Lucid	41
4.6.1	File layout	41
4.6.2	Per-op routing inside the backend	41
4.7	Threading and Concurrency	41
4.7.1	The implicit thread pool	42
4.7.2	Lock-free reentrancy	42
4.8	Quirks and Pitfalls	42
4.8.1	LAPACK error returns	42
4.8.2	Row-major vs. column-major	42
4.8.3	SDK version drift	42

4.9	Summary and Forward References	43
5	Metal and MPSGraph Integration Surface	44
5.1	Overview	44
5.2	Metal Fundamentals	45
5.2.1	The device and its command queue	45
5.2.2	Command buffers and encoders	45
5.2.3	Buffers and textures	46
5.2.4	Compute pipeline state	46
5.2.5	Synchronisation primitives	46
5.3	Metal Shading Language	47
5.3.1	Kernel functions	47
5.3.2	Threadgroups, SIMD groups, and lanes	47
5.3.3	Memory address spaces	47
5.3.4	Why Lucid does not write MSL	48
5.4	MPSGraph	48
5.4.1	Graph construction	48
5.4.2	Compilation and the executable cache	49
5.4.3	Backward pass	49
5.4.4	Operation primitives	49
5.5	MPSGraph versus MLX	50
5.5.1	Abstraction level	50
5.5.2	Compilation cost	50
5.5.3	Optimisation aggressiveness	51
5.5.4	Coexistence in Lucid	51
5.6	Lucid's Use of Metal and MPSGraph	51
5.7	The Compilation Cost Model	51
5.7.1	First-call latency	52
5.7.2	Shape specialisation	52
5.7.3	The executable cache	52
5.8	Limitations and Quirks	52
5.8.1	No backward for some forward operations	52
5.8.2	SDK version drift	53
5.8.3	Compile failure modes	53
5.8.4	Debugging compiled graphs	53
5.9	Summary and Forward References	53
II	System Architecture	54
6	The Layered Dependency DAG	55
6.1	Why a Layered Architecture	55
6.2	The Seven Layers	56
6.2.1	Layer 0: core	56
6.2.2	Layer 1: tensor	57
6.2.3	Layer 2: backend	58
6.2.4	Layer 3: kernel	58
6.2.5	Layer 4: autograd	58
6.2.6	Layer 5: ops, nn, optim, linalg, fft, einops, complex, compile	59
6.2.7	Layer 6: bindings	59
6.3	Dependency Rules	59
6.3.1	Acyclicity	60
6.3.2	Edge directness	60

6.3.3	Sibling independence within layer 5	60
6.4	Enforcement	60
6.4.1	CMake target boundaries	60
6.4.2	Code review policy	60
6.4.3	The <code>tools/check_dag.py</code> script	61
6.5	The Python Mirror	61
6.6	Cross-cutting Concerns	61
6.6.1	The allocator	62
6.6.2	Error handling	62
6.6.3	Dispatch	62
6.7	Adding a Layer or a File	62
6.8	Summary and Forward References	62
7	Tensor Storage Model	64
7.1	The <code>TensorImpl</code> Class	64
7.1.1	Shape	65
7.1.2	Strides	66
7.1.3	Dtype and device	66
7.1.4	Storage offset	66
7.1.5	The Storage handle	66
7.1.6	The autograd metadata pointer	66
7.1.7	The version counter	66
7.2	The Storage Class	67
7.2.1	Provider tag	67
7.2.2	Reference counting	67
7.3	Views	67
7.3.1	What operations produce views	67
7.3.2	When a view is not contiguous	68
7.3.3	The <code>is_contiguous</code> predicate	68
7.4	The <code>AutogradMeta</code> Class	68
7.4.1	Why <code>AutogradMeta</code> is allocated lazily	69
7.5	Reference Counting and Lifetime	69
7.5.1	The user-visible Tensor count	69
7.5.2	The autograd graph count	69
7.5.3	The view chain	70
7.6	Bridge to MLX Arrays	70
7.6.1	How a GPU tensor's storage is constructed	70
7.6.2	How a CPU tensor's storage is constructed	70
7.6.3	Round-tripping	70
7.7	Invariants	71
7.8	Summary and Forward References	71
8	Allocator and Memory Accounting	72
8.1	Why a Pool	72
8.2	Size Classes	73
8.2.1	Internal fragmentation	73
8.2.2	External fragmentation	74
8.3	Freelist and Depth Limits	74
8.3.1	The freelist as a stack, not a queue	74
8.4	The Allocation Fast Path	74
8.4.1	Thread safety	75
8.5	The Deallocation Path	75
8.6	Memory Accounting	75

8.6.1	Pool vs. provider stats	76
8.7	The <code>empty_cache</code> API	76
8.7.1	When to call	76
8.7.2	What <code>empty_cache</code> does not do	77
8.8	Out-of-Memory Handling	77
8.8.1	Memory pressure events versus OOM	77
8.9	Coordination with the Provider's Pool	77
8.9.1	When to disable Lucid's pool	78
8.10	Summary and Forward References	78
9	Backend Dispatch	79
9.1	The IBackend Interface	79
9.1.1	Why a virtual interface	80
9.2	The Dispatcher Class	80
9.2.1	Cross-device errors	81
9.2.2	Per-op routing inside the backend	82
9.3	The Two Backends	82
9.3.1	AccelerateBackend	82
9.3.2	MlxBackend	82
9.4	The Two Carve-Outs	82
9.4.1	The linalg carve-out	83
9.4.2	The data-dependent output carve-out	83
9.5	The Per-Op MPSGraph Carve-out	83
9.5.1	When to add a new MPSGraph carve-out	84
9.6	Type Promotion	84
9.6.1	Promotion in AMP mode	84
9.7	The Python Side	84
9.7.1	The wrap/unwrap pair	85
9.7.2	Per-op wrappers	85
9.8	Performance Characteristics	85
9.9	Summary and Forward References	86
10	Bridge Boundaries to the External World	87
10.1	Hard Rule H4 Restated	88
10.2	Boundary 1: External Array Ingest	89
10.2.1	Why DLPack is routed through numpy	89
10.3	Boundary 2: Tensor-Side Egress	89
10.3.1	Why repr is a separate boundary	90
10.4	Boundary 3: Display Only	90
10.4.1	The display carve-out is read-only	90
10.5	Boundary 4: Typing Protocol	90
10.5.1	Why <code>TYPE_CHECKING</code> and not a string annotation	91
10.6	Boundary 5: Checkpoint Serialisation	91
10.6.1	The <code>state_dict</code> format	91
10.6.2	The safetensors path	91
10.6.3	Why lbfgs is part of this boundary	91
10.7	Boundary 6: External Data Ingest	92
10.7.1	Why the data path is its own boundary	92
10.8	Enforcement	92
10.8.1	The bridge whitelist	92
10.9	Adding a Seventh Boundary	93
10.10	Summary and Forward References	93

11 Python–C++ Binding via pybind11	94
11.1 Overview	94
11.2 Module Initialisation	95
11.2.1 The module-name convention	95
11.3 Type Bindings: The Tensor Class	96
11.3.1 Property versus method	96
11.3.2 Method-like dunders	96
11.4 Function Bindings	96
11.4.1 Code generation for repetitive bindings	97
11.5 Exception Translation	97
11.5.1 Non-Lucid C++ exceptions	97
11.5.2 The <code>LucidError</code> message format	97
11.6 The <code>_C_{name}</code> Alias Convention (H2)	98
11.7 GIL Handling	98
11.7.1 When not to release	99
11.8 Memory and Reference Counting	99
11.8.1 <code>shared_ptr</code> held by Python	99
11.8.2 <code>shared_ptr</code> passed between C++ functions	99
11.8.3 The risk: cycles	99
11.9 Build System Integration	99
11.9.1 The wheel and its constraints	100
11.10 Summary and Forward References	100
12 Lazy Import Surface	101
12.1 Why Lazy Imports	101
12.1.1 The constraint	102
12.2 The <code>__init__.py</code> Structure	102
12.2.1 The eager preamble	102
12.2.2 The loader registry	103
12.2.3 The deferred-resolution fallback	103
12.2.4 The <code>__dir__</code> extension	104
12.3 Subsystem Loaders	104
12.3.1 Subsystem loaders as module attributes	104
12.4 Type Stubs (.pyi)	105
12.4.1 The H9 rule	105
12.5 Startup Cost Measurement	105
12.6 The Tier 1 / Tier 2 / Tier 3 Namespace Layout	106
12.6.1 The namespace-leak rule	106
12.7 Adding a New Subsystem	106
12.8 Summary and Forward References	107
III Operations and Kernels	108
13 Op Registry and Schema	109
13.1 Overview	109
13.2 The OpSchema Structure	110
13.2.1 Name and version	110
13.2.2 ArgSpec for inputs and outputs	110
13.2.3 BackwardSpec	111
13.2.4 AmpPolicy	111
13.2.5 Complexity, determinism, and flags	111
13.3 The OpRegistry	112

13.3.1 Self-registration	112
13.3.2 Initialisation order	112
13.4 Binding-Code Generation	112
13.5 Adding a New Operation	113
13.6 Schema Versioning and Checkpoint Portability	113
13.7 Introspection	113
13.8 The Op Lifecycle	113
13.9 Implementation: From Schema to Bound Function	114
13.10 Schema Statistics	114
13.11 Summary and Forward References	115
14 Kernel Abstraction Layer	116
14.1 Why a Kernel Layer	116
14.2 The IKernel Interface	117
14.3 UnaryKernel	118
14.3.1 Forward dispatch	118
14.3.2 Backward dispatch	118
14.4 BinaryKernel	118
14.4.1 Broadcast unbroadcast	119
14.5 ReduceKernel	119
14.5.1 Axis selection	119
14.5.2 Numerical stability	120
14.6 NaryKernel and VariadicKernel	120
14.7 Contiguity Inference	120
14.7.1 The is_contig_along predicate	120
14.7.2 When to materialise	120
14.8 Primitive Kernels (Data-Dependent Operations)	121
14.8.1 The CPU stream forcing	121
14.9 Summary and Forward References	121
15 Unary Functions	122
15.1 Arithmetic Family	122
15.1.1 Convention at non-differentiable points	123
15.2 Exponential Family	123
15.2.1 Numerical stability for small inputs	123
15.3 Trigonometric Family	123
15.4 Hyperbolic Family	124
15.5 Rounding Family	124
15.5.1 IEEE 754 conformance	124
15.6 Special Functions	124
15.6.1 Numerical precision of erfinv	124
15.7 Activations	124
15.7.1 GELU exact vs approximate	125
15.7.2 The MPSGraph dispatch carve-out	126
15.8 Predicates	126
15.9 Numerical Analysis of Activations	126
15.9.1 Vanishing gradient at saturation	126
15.9.2 Dying ReLU	126
15.9.3 Saturating ranges and gradient flow	126
15.9.4 The exp / log family precision	127
15.10 Activation Selection in Practice	127
15.11 Benchmarks	128
15.12 Summary and Forward References	128

16 Binary Functions	129
16.1 Broadcasting	129
16.1.1 The rule	129
16.1.2 Implementation: stride-0 expansion	130
16.1.3 Backward through broadcasting	130
16.2 Type Promotion	130
16.2.1 Scalar-tensor promotion	130
16.3 Cross-Device Check	130
16.4 Arithmetic	131
16.4.1 The pow special case	131
16.5 Comparison	131
16.5.1 Equality on floating-point	131
16.6 Logical	131
16.7 Bitwise	132
16.8 Operator Overloading	132
16.8.1 Scalar interaction	132
16.9 The Broadcasting Algorithm in Detail	132
16.9.1 Shape compatibility check	132
16.9.2 Stride translation	132
16.9.3 Broadcast unbroadcast on backward	133
16.10 Worked Examples	133
16.10.1 Example: bias addition	133
16.10.2 Example: outer product via broadcast	133
16.10.3 Example: incompatible shapes	134
16.11 Complexity and Performance	134
16.11.1 Memory bandwidth bound	134
16.11.2 The stride-0 efficiency	134
16.12 Summary and Forward References	134
17 Reductions	136
17.1 Axis Semantics	136
17.1.1 The keepdims flag	137
17.1.2 The <code>dim</code> vs <code>axis</code> naming	137
17.2 Simple Reductions	137
17.2.1 prod's backward at zero	138
17.3 Statistical Reductions	138
17.3.1 Variance and Bessel's correction	138
17.3.2 logsumexp and the max-shift trick	138
17.4 Argument Reductions	139
17.4.1 The topk backward through value	139
17.5 Boolean Reductions	139
17.6 Numerical Stability	139
17.6.1 Reduction-in-higher-precision	139
17.6.2 Kahan summation	140
17.6.3 Compensated mean: sum then divide	140
17.7 Performance Characteristics	140
17.7.1 Contiguous vs strided axis	140
17.7.2 The MPSGraph carve-out: LayerNorm	140
17.8 Worked Numerical-Stability Examples	140
17.8.1 Example: large-batch mean in FP16	140
17.8.2 Example: softmax with extreme logits	141
17.8.3 Example: variance with Bessel correction	141
17.9 Performance and Benchmarks	141

17.9.1	Bandwidth ceiling	141
17.9.2	Strided-axis penalty	141
17.10	Summary and Forward References	142
18	Shape, View, and Broadcasting Semantics	143
18.1	Reshape and View	143
18.1.1	The -1 placeholder	144
18.1.2	The compatible-stride check	144
18.2	Transpose and Permute	144
18.2.1	The .T shortcut	145
18.2.2	The mT shortcut for batched matrices	145
18.3	Squeeze and Unsqueeze	145
18.3.1	Why explicit dim for squeeze	145
18.4	Expand, Broadcast_to, and Repeat	145
18.4.1	Expand: virtual replication	145
18.4.2	Broadcast_to: convenience wrapper	145
18.4.3	Repeat: physical replication	146
18.4.4	When to use which	146
18.5	Concatenation and Stacking	146
18.5.1	Backward through cat and stack	146
18.6	Split and Chunk	146
18.7	Broadcasting Rules	146
18.7.1	NumPy compatibility	147
18.7.2	The output-shape function	147
18.8	Contiguity Invariants	147
18.8.1	The contiguity predicate	147
18.8.2	Operations that preserve contiguity	147
18.8.3	The .contiguous() materialisation	147
18.9	Worked Stride Examples	148
18.9.1	Example: transpose of a contiguous matrix	148
18.9.2	Example: slice with non-trivial step	148
18.9.3	Example: expand to broadcast	148
18.10	Contiguity Variants in Practice	149
18.11	Broadcasting Pitfalls	149
18.11.1	Pitfall: silent dimension insertion	149
18.11.2	Pitfall: writing through an expanded view	149
18.11.3	Pitfall: cat versus stack confusion	149
18.12	Summary and Forward References	149
19	Indexing	151
19.1	Basic Indexing	151
19.1.1	Scalar index	151
19.1.2	Slice	152
19.1.3	Multi-axis basic indexing	152
19.1.4	None for axis insertion	152
19.2	Advanced Indexing	152
19.2.1	Integer-tensor indexing	153
19.2.2	Multi-axis integer-tensor indexing	153
19.2.3	Boolean-mask indexing	153
19.2.4	Partial-axis boolean mask	153
19.3	Mixed Basic and Advanced Indexing	153
19.4	Gather and Scatter	153
19.4.1	Index validation	154

19.4.2	Determinism of <code>scatter_add</code>	154
19.4.3	The two scatter variants	154
19.5	Masked Operations	154
19.5.1	Backward through <code>where</code>	154
19.6	In-Place Index Assignment	155
19.6.1	Autograd interaction	155
19.6.2	<code>Index_put</code>	155
19.7	Advanced Indexing Rules in Detail	155
19.7.1	The leading-axes rule for mixed indexing	155
19.7.2	Boolean masks at non-leading positions	156
19.7.3	Negative indices versus integer-tensor indices	156
19.8	Gather/Scatter Implementation Detail	156
19.8.1	Gather's kernel structure	156
19.8.2	Scatter's two semantics	156
19.8.3	The embedding case	156
19.9	Benchmarks	157
19.10	Summary and Forward References	157
20	Composite Operations	158
20.1	Why a Composite Layer	158
20.2	Rules for the Composite Layer	159
20.3	Examples	159
20.3.1	<code>softmax</code>	159
20.3.2	<code>cross_entropy</code>	160
20.3.3	<code>l2_normalize</code>	160
20.3.4	<code>logsumexp</code> with custom backward	160
20.4	The Numpy-Free Discipline	160
20.4.1	What replaces <code>numpy</code> in the composite layer	161
20.4.2	The discipline at code review	161
20.5	When to Promote a Composite to a Primitive	161
20.5.1	Performance criterion	161
20.5.2	The promotion procedure	161
20.5.3	When the composite stays	162
20.6	Catalogue of Composition Patterns	162
20.6.1	Numerical-stability shift	162
20.6.2	Broadcast-then-reduce	162
20.6.3	Reduce-then-normalise	162
20.6.4	Iterated apply	162
20.6.5	Algorithm-specific decomposition	163
20.7	When the Composite Is the Whole Story	163
20.8	Composite Layer Hygiene	163
20.8.1	One file per family	163
20.8.2	Public API at the right level	163
20.8.3	Parity-test discipline	163
20.9	Summary and Forward References	164
IV	Autograd and Differentiation	165
21	Reverse-Mode Automatic Differentiation Theory	166
21.1	Four Ways to Compute a Derivative	166
21.1.1	Symbolic differentiation	166
21.1.2	Numerical differentiation	167

21.1.3	Forward-mode AD	167
21.1.4	Reverse-mode AD	167
21.2	The Chain Rule in Reverse	168
21.2.1	Single-variable chain rule	168
21.2.2	Multi-variable chain rule	168
21.2.3	Computation graph as DAG	168
21.3	Vector-Jacobian Products	168
21.3.1	The Jacobian	169
21.3.2	VJP as the per-op backward	169
21.3.3	The Jacobian-vector product (JVP) is forward mode	170
21.4	Reverse Mode's Complexity	170
21.4.1	The 3x bound in practice	170
21.4.2	Memory cost	170
21.5	Practical Implications for Lucid	170
21.5.1	Every primitive has a VJP	170
21.5.2	Saved tensors are explicit	171
21.5.3	Accumulation, not replacement	171
21.5.4	Determinism considerations	171
21.6	Higher-Order Differentiation	171
21.6.1	Differentiable backward	171
21.6.2	Composing transforms	171
21.7	What Reverse Mode Does Not Do	171
21.7.1	It does not give symbolic expressions	172
21.7.2	It does not give exact second-order information for free	172
21.7.3	It does not handle non-differentiable functions silently	172
21.8	Summary and Forward References	172
22	The Autograd Engine	173
22.1	Architectural Overview	173
22.2	The Node Class	174
22.2.1	Per-op subclasses	175
22.2.2	Why per-op classes, not function pointers	175
22.3	Forward-Time Construction	175
22.3.1	The construction sequence	175
22.3.2	Disabling under no_grad	176
22.3.3	Inference mode	176
22.4	Backward Traversal	176
22.4.1	The traversal algorithm	176
22.4.2	Topological sort	177
22.4.3	Multiple roots	177
22.5	AccumulateGrad on Leaves	177
22.5.1	The AccumulateGrad node	177
22.5.2	The optimizer.zero_grad pattern	177
22.6	Saved Tensors	178
22.6.1	What gets saved	178
22.6.2	Memory cost	178
22.6.3	Why save outputs when possible	178
22.7	The Version Counter	178
22.7.1	Mechanism	178
22.7.2	When the check fires	179
22.7.3	When the check doesn't fire	179
22.8	Hooks	179
22.8.1	Tensor hooks	179

22.8.2	Module hooks	179
22.9	The Anomaly Detector	179
22.9.1	What it does	180
22.9.2	When to use	180
22.10	Summary and Forward References	180
23	Custom Function API	181
23.1	Overview	181
23.2	The Function Class	182
23.2.1	Required methods	182
23.2.2	The Context object	182
23.2.3	The apply method	182
23.3	Worked Example: A Custom Activation	183
23.3.1	Why save the sigmoid intermediate	183
23.3.2	When this would be a primitive	183
23.4	Worked Example: Wrapping an External Library	183
23.5	The needs_input_grad Optimisation	184
23.6	Validating Custom Functions with gradcheck	184
23.6.1	Why FP64 for gradcheck	185
23.7	Composing Functions	185
23.8	When Not to Use Custom Function	185
23.9	Implementation Notes	185
23.9.1	The Python-to-C++ boundary	185
23.9.2	Memory model	186
23.9.3	Thread safety	186
23.10	Worked Example: A Straight-Through Estimator	186
23.10.1	Variants	186
23.11	Worked Example: A Reverse-Mode Differentiable Solver	187
23.12	Patterns and Anti-patterns	187
23.12.1	Pattern: clone before in-place	187
23.12.2	Anti-pattern: saving Python objects on ctx	188
23.12.3	Anti-pattern: forgetting to handle non-tensor inputs	188
23.12.4	Anti-pattern: writing to non-saved state	188
23.13	Summary and Forward References	188
24	Gradient Checkpointing	189
24.1	The Memory-Time Trade-off	189
24.2	The checkpoint API	190
24.2.1	What the user sees	190
24.2.2	Composition with autograd	191
24.3	When to Checkpoint	191
24.3.1	The natural granularity	191
24.3.2	Selective checkpointing	191
24.4	Implementation Detail	191
24.4.1	The Function class	191
24.4.2	Why detach the inputs	192
24.4.3	Memory release timing	192
24.5	Non-deterministic Ops Inside checkpoint	192
24.5.1	The use_reentrant flag	193
24.6	Worked Example: Transformer Block Checkpointing	193
24.6.1	Measured savings on GPT-2 base	194
24.7	What checkpoint Does Not Help With	194
24.8	Checkpoint Granularity Strategies	194

24.8.1	Per-block (default)	194
24.8.2	$\sqrt{L}$ -spaced	194
24.8.3	Selective checkpointing	195
24.8.4	Activation offloading	195
24.9	Worked Example: Memory Profile of GPT-2-base	195
24.9.1	Configuration	195
24.9.2	Memory peaks	195
24.9.3	Time cost	195
24.10	Common Pitfalls	196
24.10.1	Stochastic ops without state preservation	196
24.10.2	Non-tensor inputs and outputs	196
24.10.3	Memory not actually freed	196
24.11	Summary and Forward References	196
25	Higher-Order Differentiation	197
25.1	Overview	197
25.2	grad	198
25.2.1	grad over a different argument	198
25.2.2	Implementation	198
25.3	vjp and jvp	198
25.3.1	vjp	199
25.3.2	jvp	199
25.4	vmap	199
25.4.1	Implementation	199
25.5	jacrev and jacfwd	199
25.5.1	jacrev	200
25.5.2	jacfwd	200
25.6	hessian	200
25.6.1	Composition	200
25.6.2	Hessian-vector products	200
25.7	Composition Rules	201
25.7.1	grad o grad	201
25.7.2	grad o vmap	201
25.7.3	vmap o grad	201
25.8	Implementation Notes	201
25.8.1	Autograd-graph rebuilding	201
25.8.2	Per-transform overhead	202
25.8.3	Limitations	202
25.9	Worked Example: Per-Sample Gradients for MAML	202
25.10	Worked Example: Hessian-Vector Product for CG-Newton	203
25.10.1	HVP via grad-of-grad	203
25.10.2	HVP via JVP-of-grad	203
25.11	Performance Considerations	203
25.11.1	Per-call overhead	203
25.11.2	Memory of higher-order gradients	204
25.12	Summary and Forward References	204
26	Numerical Gradient Checking	205
26.1	The Idea	205
26.1.1	Why the comparison is informative	206
26.2	The API	206
26.2.1	The FP64 convention	206
26.3	Tolerance Setting	206

26.3.1	Theoretical tolerance	207
26.3.2	Per-op tolerance adjustments	207
26.4	What gradcheck Catches	207
26.4.1	Sign error	207
26.4.2	Missing factor	207
26.4.3	Wrong saved tensor	208
26.4.4	Unbroadcast error	208
26.5	What gradcheck Does Not Catch	208
26.5.1	Errors that are below tolerance	208
26.5.2	Errors at points not tested	208
26.5.3	Errors in saved-output backward when the output is constant	208
26.5.4	Non-deterministic ops	208
26.6	gradgradcheck	208
26.6.1	When to gradgradcheck	208
26.7	The Parity-Test Policy	209
26.8	Implementation	209
26.8.1	Cost	209
26.9	Worked Example: Validating a Custom Function	210
26.10	Debugging a Failed gradcheck	210
26.10.1	Step 1: print both gradients	210
26.10.2	Step 2: simplify the function	211
26.10.3	Step 3: check the saved tensors	211
26.10.4	Step 4: check shape	211
26.11	gradcheck vs Other Verification Tools	211
26.11.1	vs parity testing	211
26.11.2	vs anomaly detection	211
26.11.3	vs unit testing	211
26.12	The check_undefined_grad Option	211
26.13	Tolerance Recipes	212
26.14	Summary and Forward References	212

V Mathematical Subsystems 213

27	Linear Algebra	214
27.1	Overview	214
27.1.1	The H8 namespace rule	215
27.2	The Linalg Carve-Out, Restated	215
27.2.1	What the user sees	215
27.2.2	Why this is acceptable	215
27.3	Decomposition Routines	215
27.3.1	Cholesky	215
27.3.2	QR	216
27.3.3	SVD	216
27.3.4	Eigendecomposition	216
27.4	Solvers	216
27.4.1	Solve	216
27.4.2	Least squares	216
27.4.3	Triangular solve	216
27.5	Matrix Functions	216
27.5.1	inv	216
27.5.2	Determinant	217
27.5.3	Matrix exponential	217

27.6	Backward Through Linalg	217
27.6.1	Failure modes	217
27.7	Performance Notes	217
27.8	QR and Householder Reflections	217
27.8.1	Householder reflections	217
27.8.2	The pseudo-inverse via QR	218
27.8.3	Backward through QR	218
27.9	SVD in Depth	219
27.9.1	Geometric interpretation	219
27.9.2	Low-rank approximation	219
27.9.3	Backward through SVD	219
27.10	Eigendecomposition: eigh	219
27.10.1	Backward through eigh	220
27.10.2	Inverse iteration for selected eigenvalues	220
27.11	Cross, Outer, vecdot, and multi-dot	220
27.11.1	cross	220
27.11.2	outer	220
27.11.3	vecdot	220
27.11.4	The multi-dot chain product	220
27.12	Numerical Conditioning	221
27.12.1	Recognising ill-conditioning	221
27.12.2	Regularisation	221
27.13	Implementation Mapping	221
27.14	Applications in ML	222
27.14.1	Principal Component Analysis (PCA)	222
27.14.2	Low-rank fine-tuning (LoRA)	222
27.14.3	Spectral normalisation	222
27.15	Summary and Forward References	222
28	Fast Fourier Transform	223
28.1	Overview	223
28.1.1	Naming convention	223
28.2	The DFT	224
28.2.1	Cooley–Tukey	224
28.2.2	Real input savings	224
28.3	Multi-Dimensional Transforms	225
28.3.1	2-D FFT	225
28.3.2	Axis selection	225
28.4	Normalisation	225
28.4.1	Round-trip preservation	225
28.5	Backward Through FFT	225
28.5.1	Numerical considerations	225
28.6	Dispatch and Implementation	226
28.6.1	No linalg-style carve-out	226
28.6.2	Size restrictions	226
28.7	Utility Functions	226
28.7.1	fftfreq and rfftfreq	226
28.7.2	fftshift and ifftshift	226
28.8	Worked Example: Spectrogram	226
28.9	Performance	227
28.10	The Convolution Theorem	227
28.10.1	Statement	227
28.10.2	Linear vs circular convolution	227

28.10.3 Complexity comparison	228
28.11 Power Spectral Density	228
28.12 Multi-Dimensional FFT Algorithms	228
28.12.1 Memory access patterns	228
28.12.2 Real-valued multi-dimensional FFT	229
28.13 Numerical Precision Analysis	229
28.13.1 The round-trip identity	229
28.14 The IFFT	229
28.14.1 Hermitian shortcut	229
28.15 Common FFT Pitfalls	229
28.15.1 Pitfall: normalisation confusion	230
28.15.2 Pitfall: fftshift before vs after rfft	230
28.15.3 Pitfall: even vs odd output length for irfft	230
28.16 Summary and Forward References	230
29 Signal Processing Windows	231
29.1 Why Windowing Matters	231
29.1.1 The trade-off	231
29.2 Window Functions	232
29.2.1 Hann	232
29.2.2 Hamming	233
29.2.3 Blackman family	233
29.2.4 Bartlett	233
29.2.5 Kaiser	233
29.3 Symmetric vs Periodic Windows	233
29.4 Worked Example: A Mel-Spectrogram Frontend	234
29.5 Implementation	234
29.5.1 Caching	234
29.5.2 Length conventions	234
29.6 Window-Function Backward Through STFT	234
29.7 Choosing a Window	235
29.8 Spectral Leakage: A Quantitative Analysis	235
29.8.1 The rectangular case	235
29.8.2 The Hann case	235
29.9 The COLA Property	235
29.9.1 COLA-compliant configurations	236
29.9.2 Reconstructing from STFT	236
29.10 Window Optimisation: The Kaiser–Bessel Family	236
29.10.1 The Kaiser–Bessel formula	236
29.10.2 Choosing beta	236
29.11 Window Energy Corrections	237
29.11.1 Coherent gain	237
29.11.2 Power gain	237
29.12 Applications Beyond STFT	237
29.12.1 Pre-emphasis filtering	237
29.12.2 Filter design	237
29.12.3 Image processing	237
29.13 Summary and Forward References	237
30 Special Functions	239
30.1 Overview	239
30.1.1 Why a separate subpackage	240
30.2 Error Functions	240

30.2.1	erf and erfc	240
30.2.2	erfinv and erfcinv	240
30.2.3	Use in Gaussian sampling	240
30.3	Gamma Family	240
30.3.1	lgamma	240
30.3.2	digamma and polygamma	240
30.3.3	Why lgamma, not gamma	241
30.4	Bessel Functions	241
30.5	Orthogonal Polynomials	241
30.6	Miscellaneous	241
30.6.1	expit and logit	241
30.6.2	xlog1py and xlogy	241
30.6.3	log1p and expm1	241
30.7	Numerical Precision	242
30.7.1	Precision goals	242
30.7.2	The vForce vs scalar discrepancy	242
30.8	Backward Derivatives	242
30.8.1	Higher-order derivatives	242
30.9	Use Cases in ML	242
30.9.1	GELU activation	242
30.9.2	Gaussian quantile	243
30.9.3	Beta and Dirichlet likelihoods	243
30.9.4	Kaiser window	243
30.10	Implementation Notes	243
30.10.1	Composite vs primitive	243
30.10.2	Dispatch	243
30.11	The Gamma Function in Depth	243
30.11.1	The log-gamma identity	243
30.11.2	The digamma and harmonic numbers	244
30.11.3	Reflection identity in practice	244
30.12	The Error Function in Depth	244
30.12.1	The standard normal CDF	244
30.12.2	The complementary error function for tails	244
30.13	Bessel Functions in Depth	245
30.13.1	Asymptotic forms	245
30.13.2	Recurrence relations	245
30.14	Orthogonal Polynomials in Depth	245
30.14.1	Legendre polynomials	245
30.14.2	Hermite polynomials	245
30.14.3	Chebyshev polynomials	246
30.15	The Beta Function and Beta Distribution	246
30.15.1	The incomplete Beta	246
30.16	Numerical Precision Summary	246
30.17	Composite vs Primitive Cost	246
30.18	Summary and Forward References	246
31	Probability Distributions	248
31.1	Overview	248
31.1.1	The 33 distributions	248
31.2	The Distribution Interface	249
31.2.1	Shape conventions	249
31.2.2	The Independent wrapper	249
31.3	The Reparameterisation Trick	249

31.3.1	The trick	250
31.3.2	rsample	250
31.3.3	Score-function estimator	250
31.4	KL Divergence	250
31.4.1	Closed-form KLs	251
31.4.2	Monte-Carlo fallback	251
31.4.3	Registering a custom KL	251
31.5	Worked Example: VAE Loss	251
31.6	Constraints	252
31.6.1	Why constraints matter	252
31.6.2	TransformedDistribution	252
31.7	Implementation Notes	253
31.7.1	Numpy-free	253
31.7.2	Composition with optim	253
31.8	Univariate Continuous Distributions in Detail	253
31.8.1	Normal	253
31.8.2	Gamma	253
31.8.3	Beta	254
31.8.4	Dirichlet	254
31.8.5	Exponential and related	254
31.9	Discrete Distributions in Detail	254
31.9.1	Bernoulli	254
31.9.2	Categorical	254
31.9.3	Binomial and Poisson	255
31.10	Multivariate Distributions	255
31.10.1	Multivariate Normal	255
31.10.2	Wishart and inverse Wishart	255
31.10.3	LKJ-Cholesky	255
31.11	Variational Inference: A Worked Example	255
31.11.1	The setup	256
31.11.2	The ELBO terms	256
31.12	Score-Function Estimator	256
31.12.1	Variance	256
31.12.2	Implementation pattern	256
31.13	Transformed Distributions and Normalising Flows	257
31.13.1	Normalising flows	257
31.14	Constraint and Transform Machinery	257
31.15	Summary and Forward References	258
32	Einops: Tensor Rearrangement Algebra	259
32.1	Why Einops	259
32.1.1	The H8 namespace rule	259
32.2	The Three Operations	260
32.3	rearrange	260
32.3.1	Permutation	260
32.3.2	Reshape via grouping	260
32.3.3	Reshape via splitting	260
32.3.4	Combining	260
32.4	reduce	261
32.4.1	vs explicit reduce	261
32.5	repeat	261
32.6	Pattern Syntax	261
32.6.1	Axis names	261

32.6.2	Grouping	262
32.6.3	Ellipsis	262
32.6.4	The 1 placeholder	262
32.7	Backward Through Einops	262
32.8	Implementation	262
32.8.1	Parser	262
32.8.2	Caching	262
32.9	Patterns in Practice	263
32.9.1	Attention reshape	263
32.9.2	Patch embedding (ViT)	263
32.9.3	Channel-last conversion	263
32.10	Comparison: Einops vs Native Operations	263
32.10.1	Multi-head attention reshape	263
32.10.2	Patch extraction (ViT)	263
32.10.3	Channel-last conversion with batch	264
32.11	Pattern Parsing in Detail	264
32.11.1	Tokenisation	264
32.11.2	Axis-size resolution	264
32.11.3	Op sequence generation	264
32.12	Performance Considerations	265
32.12.1	Overhead per call	265
32.12.2	When einops materialises	265
32.12.3	Compose with autograd	265
32.13	Advanced Patterns	265
32.13.1	Pattern with ellipsis	265
32.13.2	Pattern with literal 1	265
32.13.3	Pattern with anonymous axis	265
32.14	The Three Operations: A Reference	266
32.15	Common Bugs and Their Diagnostics	266
32.15.1	Bug: unresolvable size	266
32.15.2	Bug: mismatched output axes	266
32.15.3	Bug: division-incompatible split	266
32.16	Summary and Forward References	266

VINeural Network Library

267

33	The Module System	268
33.1	Overview	269
33.2	Architecture	269
33.2.1	Core building blocks	269
33.2.2	Variants and parameterizations	269
33.3	Forward Computation	269
33.3.1	Mathematical formulation	269
33.3.2	Implementation in Lucid	269
33.4	Training Considerations	269
33.4.1	Initialization strategy	269
33.4.2	Backward pass	269
33.4.3	Interaction with optimizer and AMP	269
33.5	Implementation Notes	269
33.5.1	File layout and class hierarchy	269
33.5.2	Edge cases and pitfalls	269
33.6	Worked Examples and Benchmarks	269

33.7 Summary and Cross-References	269
34 Linear, Convolution, Pooling, Normalization	270
34.1 Overview	271
34.2 Architecture	271
34.2.1 Core building blocks	271
34.2.2 Variants and parameterizations	271
34.3 Forward Computation	271
34.3.1 Mathematical formulation	271
34.3.2 Implementation in Lucid	271
34.4 Training Considerations	271
34.4.1 Initialization strategy	271
34.4.2 Backward pass	271
34.4.3 Interaction with optimizer and AMP	271
34.5 Implementation Notes	271
34.5.1 File layout and class hierarchy	271
34.5.2 Edge cases and pitfalls	271
34.6 Worked Examples and Benchmarks	271
34.7 Summary and Cross-References	271
35 Recurrent Networks	272
35.1 Overview	273
35.2 Architecture	273
35.2.1 Core building blocks	273
35.2.2 Variants and parameterizations	273
35.3 Forward Computation	273
35.3.1 Mathematical formulation	273
35.3.2 Implementation in Lucid	273
35.4 Training Considerations	273
35.4.1 Initialization strategy	273
35.4.2 Backward pass	273
35.4.3 Interaction with optimizer and AMP	273
35.5 Implementation Notes	273
35.5.1 File layout and class hierarchy	273
35.5.2 Edge cases and pitfalls	273
35.6 Worked Examples and Benchmarks	273
35.7 Summary and Cross-References	273
36 Attention and SDPA Fusion	274
36.1 Overview	275
36.2 Architecture	275
36.2.1 Core building blocks	275
36.2.2 Variants and parameterizations	275
36.3 Forward Computation	275
36.3.1 Mathematical formulation	275
36.3.2 Implementation in Lucid	275
36.4 Training Considerations	275
36.4.1 Initialization strategy	275
36.4.2 Backward pass	275
36.4.3 Interaction with optimizer and AMP	275
36.5 Implementation Notes	275
36.5.1 File layout and class hierarchy	275
36.5.2 Edge cases and pitfalls	275

36.6	Worked Examples and Benchmarks	275
36.7	Summary and Cross-References	275
37	Loss Functions	276
37.1	Overview	277
37.2	Architecture	277
37.2.1	Core building blocks	277
37.2.2	Variants and parameterizations	277
37.3	Forward Computation	277
37.3.1	Mathematical formulation	277
37.3.2	Implementation in Lucid	277
37.4	Training Considerations	277
37.4.1	Initialization strategy	277
37.4.2	Backward pass	277
37.4.3	Interaction with optimizer and AMP	277
37.5	Implementation Notes	277
37.5.1	File layout and class hierarchy	277
37.5.2	Edge cases and pitfalls	277
37.6	Worked Examples and Benchmarks	277
37.7	Summary and Cross-References	277
38	Weight Initialization	278
38.1	Overview	279
38.2	Architecture	279
38.2.1	Core building blocks	279
38.2.2	Variants and parameterizations	279
38.3	Forward Computation	279
38.3.1	Mathematical formulation	279
38.3.2	Implementation in Lucid	279
38.4	Training Considerations	279
38.4.1	Initialization strategy	279
38.4.2	Backward pass	279
38.4.3	Interaction with optimizer and AMP	279
38.5	Implementation Notes	279
38.5.1	File layout and class hierarchy	279
38.5.2	Edge cases and pitfalls	279
38.6	Worked Examples and Benchmarks	279
38.7	Summary and Cross-References	279
39	Optimizers	280
39.1	Overview	281
39.2	Architecture	281
39.2.1	Core building blocks	281
39.2.2	Variants and parameterizations	281
39.3	Forward Computation	281
39.3.1	Mathematical formulation	281
39.3.2	Implementation in Lucid	281
39.4	Training Considerations	281
39.4.1	Initialization strategy	281
39.4.2	Backward pass	281
39.4.3	Interaction with optimizer and AMP	281
39.5	Implementation Notes	281
39.5.1	File layout and class hierarchy	281

39.5.2	Edge cases and pitfalls	281
39.6	Worked Examples and Benchmarks	281
39.7	Summary and Cross-References	281
40	Learning-Rate Schedulers	282
40.1	Overview	283
40.2	Architecture	283
40.2.1	Core building blocks	283
40.2.2	Variants and parameterizations	283
40.3	Forward Computation	283
40.3.1	Mathematical formulation	283
40.3.2	Implementation in Lucid	283
40.4	Training Considerations	283
40.4.1	Initialization strategy	283
40.4.2	Backward pass	283
40.4.3	Interaction with optimizer and AMP	283
40.5	Implementation Notes	283
40.5.1	File layout and class hierarchy	283
40.5.2	Edge cases and pitfalls	283
40.6	Worked Examples and Benchmarks	283
40.7	Summary and Cross-References	283
41	Mixed Precision and Gradient Scaling	284
41.1	Overview	285
41.2	Architecture	285
41.2.1	Core building blocks	285
41.2.2	Variants and parameterizations	285
41.3	Forward Computation	285
41.3.1	Mathematical formulation	285
41.3.2	Implementation in Lucid	285
41.4	Training Considerations	285
41.4.1	Initialization strategy	285
41.4.2	Backward pass	285
41.4.3	Interaction with optimizer and AMP	285
41.5	Implementation Notes	285
41.5.1	File layout and class hierarchy	285
41.5.2	Edge cases and pitfalls	285
41.6	Worked Examples and Benchmarks	285
41.7	Summary and Cross-References	285
42	Profiler and Determinism	286
42.1	Overview	287
42.2	Architecture	287
42.2.1	Core building blocks	287
42.2.2	Variants and parameterizations	287
42.3	Forward Computation	287
42.3.1	Mathematical formulation	287
42.3.2	Implementation in Lucid	287
42.4	Training Considerations	287
42.4.1	Initialization strategy	287
42.4.2	Backward pass	287
42.4.3	Interaction with optimizer and AMP	287
42.5	Implementation Notes	287

42.5.1	File layout and class hierarchy	287
42.5.2	Edge cases and pitfalls	287
42.6	Worked Examples and Benchmarks	287
42.7	Summary and Cross-References	287
VI Graph Compilation		288
43	Graph-Capture Design Goals	289
43.1	Overview	290
43.2	Background and Goals	290
43.2.1	Problem statement	290
43.2.2	Constraints and prior art	290
43.3	Methodology	290
43.3.1	Measurement approach	290
43.3.2	Benchmark setup	290
43.4	Implementation	290
43.4.1	Design decisions	290
43.4.2	Component breakdown	290
43.4.3	Integration with existing systems	290
43.5	Results	290
43.5.1	Quantitative findings	290
43.5.2	Qualitative observations	290
43.6	Discussion	290
43.6.1	Trade-offs	290
43.6.2	Open questions	290
43.7	Summary and Cross-References	290
44	Tracer and TraceIR	291
44.1	Overview	292
44.2	Background and Goals	292
44.2.1	Problem statement	292
44.2.2	Constraints and prior art	292
44.3	Methodology	292
44.3.1	Measurement approach	292
44.3.2	Benchmark setup	292
44.4	Implementation	292
44.4.1	Design decisions	292
44.4.2	Component breakdown	292
44.4.3	Integration with existing systems	292
44.5	Results	292
44.5.1	Quantitative findings	292
44.5.2	Qualitative observations	292
44.6	Discussion	292
44.6.1	Trade-offs	292
44.6.2	Open questions	292
44.7	Summary and Cross-References	292
45	MpsBuilder and Executable Cache	293
45.1	Overview	294
45.2	Background and Goals	294
45.2.1	Problem statement	294
45.2.2	Constraints and prior art	294

45.3	Methodology	294
45.3.1	Measurement approach	294
45.3.2	Benchmark setup	294
45.4	Implementation	294
45.4.1	Design decisions	294
45.4.2	Component breakdown	294
45.4.3	Integration with existing systems	294
45.5	Results	294
45.5.1	Quantitative findings	294
45.5.2	Qualitative observations	294
45.6	Discussion	294
45.6.1	Trade-offs	294
45.6.2	Open questions	294
45.7	Summary and Cross-References	294
46	Op Emitters	295
46.1	Overview	296
46.2	Background and Goals	296
46.2.1	Problem statement	296
46.2.2	Constraints and prior art	296
46.3	Methodology	296
46.3.1	Measurement approach	296
46.3.2	Benchmark setup	296
46.4	Implementation	296
46.4.1	Design decisions	296
46.4.2	Component breakdown	296
46.4.3	Integration with existing systems	296
46.5	Results	296
46.5.1	Quantitative findings	296
46.5.2	Qualitative observations	296
46.6	Discussion	296
46.6.1	Trade-offs	296
46.6.2	Open questions	296
46.7	Summary and Cross-References	296
47	Forward and Backward in a Single Graph	297
47.1	Overview	298
47.2	Background and Goals	298
47.2.1	Problem statement	298
47.2.2	Constraints and prior art	298
47.3	Methodology	298
47.3.1	Measurement approach	298
47.3.2	Benchmark setup	298
47.4	Implementation	298
47.4.1	Design decisions	298
47.4.2	Component breakdown	298
47.4.3	Integration with existing systems	298
47.5	Results	298
47.5.1	Quantitative findings	298
47.5.2	Qualitative observations	298
47.6	Discussion	298
47.6.1	Trade-offs	298
47.6.2	Open questions	298

47.7 Summary and Cross-References	298
48 CompiledModule API	299
48.1 Overview	300
48.2 Background and Goals	300
48.2.1 Problem statement	300
48.2.2 Constraints and prior art	300
48.3 Methodology	300
48.3.1 Measurement approach	300
48.3.2 Benchmark setup	300
48.4 Implementation	300
48.4.1 Design decisions	300
48.4.2 Component breakdown	300
48.4.3 Integration with existing systems	300
48.5 Results	300
48.5.1 Quantitative findings	300
48.5.2 Qualitative observations	300
48.6 Discussion	300
48.6.1 Trade-offs	300
48.6.2 Open questions	300
48.7 Summary and Cross-References	300
49 Hardening and Edge Cases	301
49.1 Overview	302
49.2 Background and Goals	302
49.2.1 Problem statement	302
49.2.2 Constraints and prior art	302
49.3 Methodology	302
49.3.1 Measurement approach	302
49.3.2 Benchmark setup	302
49.4 Implementation	302
49.4.1 Design decisions	302
49.4.2 Component breakdown	302
49.4.3 Integration with existing systems	302
49.5 Results	302
49.5.1 Quantitative findings	302
49.5.2 Qualitative observations	302
49.6 Discussion	302
49.6.1 Trade-offs	302
49.6.2 Open questions	302
49.7 Summary and Cross-References	302
VIModel Zoo	303
50 Zoo Architecture	304
50.1 Overview	305
50.2 Architecture	305
50.2.1 Core building blocks	305
50.2.2 Variants and parameterizations	305
50.3 Forward Computation	305
50.3.1 Mathematical formulation	305
50.3.2 Implementation in Lucid	305

50.4	Training Considerations	305
50.4.1	Initialization strategy	305
50.4.2	Backward pass	305
50.4.3	Interaction with optimizer and AMP	305
50.5	Implementation Notes	305
50.5.1	File layout and class hierarchy	305
50.5.2	Edge cases and pitfalls	305
50.6	Worked Examples and Benchmarks	305
50.7	Summary and Cross-References	305
51	Vision: CNN Family	306
51.1	Overview	307
51.2	Architecture	307
51.2.1	Core building blocks	307
51.2.2	Variants and parameterizations	307
51.3	Forward Computation	307
51.3.1	Mathematical formulation	307
51.3.2	Implementation in Lucid	307
51.4	Training Considerations	307
51.4.1	Initialization strategy	307
51.4.2	Backward pass	307
51.4.3	Interaction with optimizer and AMP	307
51.5	Implementation Notes	307
51.5.1	File layout and class hierarchy	307
51.5.2	Edge cases and pitfalls	307
51.6	Worked Examples and Benchmarks	307
51.7	Summary and Cross-References	307
52	Vision Transformers	308
52.1	Overview	309
52.2	Architecture	309
52.2.1	Core building blocks	309
52.2.2	Variants and parameterizations	309
52.3	Forward Computation	309
52.3.1	Mathematical formulation	309
52.3.2	Implementation in Lucid	309
52.4	Training Considerations	309
52.4.1	Initialization strategy	309
52.4.2	Backward pass	309
52.4.3	Interaction with optimizer and AMP	309
52.5	Implementation Notes	309
52.5.1	File layout and class hierarchy	309
52.5.2	Edge cases and pitfalls	309
52.6	Worked Examples and Benchmarks	309
52.7	Summary and Cross-References	309
53	Detection and Segmentation	310
53.1	Overview	311
53.2	Architecture	311
53.2.1	Core building blocks	311
53.2.2	Variants and parameterizations	311
53.3	Forward Computation	311
53.3.1	Mathematical formulation	311

53.3.2	Implementation in Lucid	311
53.4	Training Considerations	311
53.4.1	Initialization strategy	311
53.4.2	Backward pass	311
53.4.3	Interaction with optimizer and AMP	311
53.5	Implementation Notes	311
53.5.1	File layout and class hierarchy	311
53.5.2	Edge cases and pitfalls	311
53.6	Worked Examples and Benchmarks	311
53.7	Summary and Cross-References	311
54	Language Models	312
54.1	Overview	313
54.2	Architecture	313
54.2.1	Core building blocks	313
54.2.2	Variants and parameterizations	313
54.3	Forward Computation	313
54.3.1	Mathematical formulation	313
54.3.2	Implementation in Lucid	313
54.4	Training Considerations	313
54.4.1	Initialization strategy	313
54.4.2	Backward pass	313
54.4.3	Interaction with optimizer and AMP	313
54.5	Implementation Notes	313
54.5.1	File layout and class hierarchy	313
54.5.2	Edge cases and pitfalls	313
54.6	Worked Examples and Benchmarks	313
54.7	Summary and Cross-References	313
55	Generative Models	314
55.1	Overview	315
55.2	Architecture	315
55.2.1	Core building blocks	315
55.2.2	Variants and parameterizations	315
55.3	Forward Computation	315
55.3.1	Mathematical formulation	315
55.3.2	Implementation in Lucid	315
55.4	Training Considerations	315
55.4.1	Initialization strategy	315
55.4.2	Backward pass	315
55.4.3	Interaction with optimizer and AMP	315
55.5	Implementation Notes	315
55.5.1	File layout and class hierarchy	315
55.5.2	Edge cases and pitfalls	315
55.6	Worked Examples and Benchmarks	315
55.7	Summary and Cross-References	315
56	Pretrained Weights and Hub	316
56.1	Overview	317
56.2	Architecture	317
56.2.1	Core building blocks	317
56.2.2	Variants and parameterizations	317
56.3	Forward Computation	317

56.3.1	Mathematical formulation	317
56.3.2	Implementation in Lucid	317
56.4	Training Considerations	317
56.4.1	Initialization strategy	317
56.4.2	Backward pass	317
56.4.3	Interaction with optimizer and AMP	317
56.5	Implementation Notes	317
56.5.1	File layout and class hierarchy	317
56.5.2	Edge cases and pitfalls	317
56.6	Worked Examples and Benchmarks	317
56.7	Summary and Cross-References	317

IX Performance Engineering 318

57 Performance Methodology 319

57.1	Overview	320
57.2	Background and Goals	320
57.2.1	Problem statement	320
57.2.2	Constraints and prior art	320
57.3	Methodology	320
57.3.1	Measurement approach	320
57.3.2	Benchmark setup	320
57.4	Implementation	320
57.4.1	Design decisions	320
57.4.2	Component breakdown	320
57.4.3	Integration with existing systems	320
57.5	Results	320
57.5.1	Quantitative findings	320
57.5.2	Qualitative observations	320
57.6	Discussion	320
57.6.1	Trade-offs	320
57.6.2	Open questions	320
57.7	Summary and Cross-References	320

58 Stabilization Chain (3.0.0–3.2.2) 321

58.1	Overview	322
58.2	Background and Goals	322
58.2.1	Problem statement	322
58.2.2	Constraints and prior art	322
58.3	Methodology	322
58.3.1	Measurement approach	322
58.3.2	Benchmark setup	322
58.4	Implementation	322
58.4.1	Design decisions	322
58.4.2	Component breakdown	322
58.4.3	Integration with existing systems	322
58.5	Results	322
58.5.1	Quantitative findings	322
58.5.2	Qualitative observations	322
58.6	Discussion	322
58.6.1	Trade-offs	322
58.6.2	Open questions	322

58.7 Summary and Cross-References	322
59 AMP Integration (3.3.0)	323
59.1 Overview	324
59.2 Background and Goals	324
59.2.1 Problem statement	324
59.2.2 Constraints and prior art	324
59.3 Methodology	324
59.3.1 Measurement approach	324
59.3.2 Benchmark setup	324
59.4 Implementation	324
59.4.1 Design decisions	324
59.4.2 Component breakdown	324
59.4.3 Integration with existing systems	324
59.5 Results	324
59.5.1 Quantitative findings	324
59.5.2 Qualitative observations	324
59.6 Discussion	324
59.6.1 Trade-offs	324
59.6.2 Open questions	324
59.7 Summary and Cross-References	324
60 MPSGraph Per-Op Dispatch (3.4.0)	325
60.1 Overview	326
60.2 Background and Goals	326
60.2.1 Problem statement	326
60.2.2 Constraints and prior art	326
60.3 Methodology	326
60.3.1 Measurement approach	326
60.3.2 Benchmark setup	326
60.4 Implementation	326
60.4.1 Design decisions	326
60.4.2 Component breakdown	326
60.4.3 Integration with existing systems	326
60.5 Results	326
60.5.1 Quantitative findings	326
60.5.2 Qualitative observations	326
60.6 Discussion	326
60.6.1 Trade-offs	326
60.6.2 Open questions	326
60.7 Summary and Cross-References	326
61 No-Compile Algorithmic Sweep (3.4.1)	327
61.1 Overview	328
61.2 Background and Goals	328
61.2.1 Problem statement	328
61.2.2 Constraints and prior art	328
61.3 Methodology	328
61.3.1 Measurement approach	328
61.3.2 Benchmark setup	328
61.4 Implementation	328
61.4.1 Design decisions	328
61.4.2 Component breakdown	328

61.4.3	Integration with existing systems	328
61.5	Results	328
61.5.1	Quantitative findings	328
61.5.2	Qualitative observations	328
61.6	Discussion	328
61.6.1	Trade-offs	328
61.6.2	Open questions	328
61.7	Summary and Cross-References	328
62	Graph-Capture Wins (3.5.0)	329
62.1	Overview	330
62.2	Background and Goals	330
62.2.1	Problem statement	330
62.2.2	Constraints and prior art	330
62.3	Methodology	330
62.3.1	Measurement approach	330
62.3.2	Benchmark setup	330
62.4	Implementation	330
62.4.1	Design decisions	330
62.4.2	Component breakdown	330
62.4.3	Integration with existing systems	330
62.5	Results	330
62.5.1	Quantitative findings	330
62.5.2	Qualitative observations	330
62.6	Discussion	330
62.6.1	Trade-offs	330
62.6.2	Open questions	330
62.7	Summary and Cross-References	330
X	Production Engineering	331
63	Testing Strategy	332
63.1	Overview	333
63.2	Background and Goals	333
63.2.1	Problem statement	333
63.2.2	Constraints and prior art	333
63.3	Methodology	333
63.3.1	Measurement approach	333
63.3.2	Benchmark setup	333
63.4	Implementation	333
63.4.1	Design decisions	333
63.4.2	Component breakdown	333
63.4.3	Integration with existing systems	333
63.5	Results	333
63.5.1	Quantitative findings	333
63.5.2	Qualitative observations	333
63.6	Discussion	333
63.6.1	Trade-offs	333
63.6.2	Open questions	333
63.7	Summary and Cross-References	333
64	Serialization and Safetensors	334

64.1	Overview	335
64.2	Background and Goals	335
64.2.1	Problem statement	335
64.2.2	Constraints and prior art	335
64.3	Methodology	335
64.3.1	Measurement approach	335
64.3.2	Benchmark setup	335
64.4	Implementation	335
64.4.1	Design decisions	335
64.4.2	Component breakdown	335
64.4.3	Integration with existing systems	335
64.5	Results	335
64.5.1	Quantitative findings	335
64.5.2	Qualitative observations	335
64.6	Discussion	335
64.6.1	Trade-offs	335
64.6.2	Open questions	335
64.7	Summary and Cross-References	335
65	Data Loading Pipeline	336
65.1	Overview	337
65.2	Background and Goals	337
65.2.1	Problem statement	337
65.2.2	Constraints and prior art	337
65.3	Methodology	337
65.3.1	Measurement approach	337
65.3.2	Benchmark setup	337
65.4	Implementation	337
65.4.1	Design decisions	337
65.4.2	Component breakdown	337
65.4.3	Integration with existing systems	337
65.5	Results	337
65.5.1	Quantitative findings	337
65.5.2	Qualitative observations	337
65.6	Discussion	337
65.6.1	Trade-offs	337
65.6.2	Open questions	337
65.7	Summary and Cross-References	337
66	Error Handling and Anomaly Detection	338
66.1	Overview	339
66.2	Background and Goals	339
66.2.1	Problem statement	339
66.2.2	Constraints and prior art	339
66.3	Methodology	339
66.3.1	Measurement approach	339
66.3.2	Benchmark setup	339
66.4	Implementation	339
66.4.1	Design decisions	339
66.4.2	Component breakdown	339
66.4.3	Integration with existing systems	339
66.5	Results	339
66.5.1	Quantitative findings	339

66.5.2 Qualitative observations	339
66.6 Discussion	339
66.6.1 Trade-offs	339
66.6.2 Open questions	339
66.7 Summary and Cross-References	339
67 Reproducibility and Determinism	340
67.1 Overview	341
67.2 Background and Goals	341
67.2.1 Problem statement	341
67.2.2 Constraints and prior art	341
67.3 Methodology	341
67.3.1 Measurement approach	341
67.3.2 Benchmark setup	341
67.4 Implementation	341
67.4.1 Design decisions	341
67.4.2 Component breakdown	341
67.4.3 Integration with existing systems	341
67.5 Results	341
67.5.1 Quantitative findings	341
67.5.2 Qualitative observations	341
67.6 Discussion	341
67.6.1 Trade-offs	341
67.6.2 Open questions	341
67.7 Summary and Cross-References	341
A Hard Rules H1–H12	342
A.1 Purpose and Scope	342
A.2 Reference Material	342
A.2.1 Primary entries	342
A.2.2 Supplementary notes	342
A.3 Cross-References	342
B Production Pillars P1–P9	343
B.1 Purpose and Scope	343
B.2 Reference Material	343
B.2.1 Primary entries	343
B.2.2 Supplementary notes	343
B.3 Cross-References	343
C Op Reference	344
C.1 Purpose and Scope	344
C.2 Reference Material	344
C.2.1 Primary entries	344
C.2.2 Supplementary notes	344
C.3 Cross-References	344
D Public API Snapshot	345
D.1 Purpose and Scope	345
D.2 Reference Material	345
D.2.1 Primary entries	345
D.2.2 Supplementary notes	345
D.3 Cross-References	345

E Hardware Benchmark Data	346
E.1 Purpose and Scope	346
E.2 Reference Material	346
E.2.1 Primary entries	346
E.2.2 Supplementary notes	346
E.3 Cross-References	346
Bibliography	347

List of Figures

1.1	Apple silicon SoC topology.	4
2.1	Zero-copy bridge protocol.	20
3.1	MLX lazy graph execution.	27
6.1	The Lucid engine DAG.	57
7.1	Tensor object graph.	65
9.1	Dispatcher routing.	81
13.1	Op lifecycle.	114
14.1	Kernel hierarchy.	117
17.1	Reduction shape transformations.	137
21.1	Forward and backward through a DAG.	169
22.1	Autograd engine architecture.	174
29.1	Spectral leakage with and without windowing.	232

List of Tables

1.1	Apple silicon memory subsystem summary.	10
2.1	Metal storage modes.	17
4.1	Accelerate sublayers and Lucid usage.	36
5.1	MLX versus MPSGraph.	50
6.1	Engine SLOC by layer.	57
8.1	Allocator size classes.	73
9.1	Dispatcher carve-outs.	80
10.1	The six bridge boundaries.	88
13.1	Op registry composition.	115
15.1	Activation family.	125
15.2	Gradient behaviour at extremes.	127
15.3	Activation throughput.	128
17.1	Reduction throughput by axis.	142
18.1	Shape-op summary.	144
18.2	Contiguity variants.	149
19.1	Indexing regimes.	152
19.2	Indexing benchmarks.	157
21.1	AD method comparison.	168
24.1	Checkpointing schemes.	190
24.2	GPT-2-base memory profile.	195
25.1	Functional transform overhead.	204
27.1	Linalg backward forms.	217
27.2	Linalg throughput on M3 Max.	218
27.3	Linalg backend mapping.	221
28.1	FFT throughput on M3 Max.	227
28.2	FFT round-trip error.	229

29.1 Window function catalogue.	232
30.1 Special-function derivatives.	242
30.2 Special-function precision.	247
30.3 Special-function timings.	247
32.1 Einops operation reference.	266

Preface

Preface — motivation, intended audience, how to read the book, prerequisites, conventions. To be written.

Acknowledgements

Acknowledgements. To be written.

Notation

Notation table: tensors, indices, operators, function spaces, framework conventions. To be written.

Abbreviations

Abbreviations: AD, AMP, ANE, BNNS, BLAS, LAPACK, MLX, MPSGraph, SDPA, etc. To be written.

Part I

Hardware and Runtime Foundations

CHAPTER 1

Apple Silicon Architecture

The substrate on which *Lucid* executes is not a generic compute fabric. It is a tightly integrated, vertically designed system-on-chip family (*Apple silicon*, sometimes *Apple M*) that Apple introduced in November 2020 with the M1 and has iterated on each subsequent year through the M2 (2022), M3 (2023), and M4 (2024) generations. Every architectural choice in *Lucid* — from the device split into a CPU stream and a GPU stream (Chapter 9), to the carve-out that runs linear algebra through *MLX* even when the user selected the CPU (Chapter 27), to the deliberate avoidance of the Neural Engine (Section 1.5) — presupposes a reader who understands the chip beneath the runtime. This chapter exists to provide that understanding.

We approach the silicon the way a compiler writer would: as a heterogeneous collection of compute units sharing a single unified memory through a coherent fabric, with each unit exposing a distinct programming model and a distinct cost profile.¹ The exposition is organised by compute unit. Section 1.3 covers the CPU complex, with its heterogeneous performance and efficiency cores. Section 1.4 treats the Apple Matrix Coprocessor (AMX) and its transition to the ARM Scalable Matrix Extension (SME) in the M4. Section 1.5 discusses the Neural Engine and explains, with reference to *Lucid*’s design constraints, why it is intentionally absent from *Lucid*’s dispatch graph. Section 1.6 describes the GPU, which is the workhorse of nearly every training workload on the platform. Section 1.7 addresses the memory hierarchy and system-level cache (SLC). Finally, Section 1.8 compares the four generations quantitatively, and Section 1.9 draws the conclusions that motivate the rest of the book.

¹We deliberately defer *unified memory* itself to Chapter 2; here we treat it as a property of the SoC that gives every compute unit a coherent view of the same DRAM pool, and we return to its detailed model later.

1.1 Overview

Apple silicon refers to a family of ARMv8 (and, from the M4 onwards, ARMv9-A) SoCs designed by Apple’s silicon engineering group and manufactured by TSMC on its leading-edge processes [1, 3]. The first member, M1, shipped in late 2020 on TSMC’s first-generation 5 nm node (N5); M2 used an enhanced 5 nm node (N5P) in mid-2022; M3 moved to a first-generation 3 nm node (N3B) in late 2023; M4 moved again to TSMC’s N3E in mid-2024.²

The hallmark of Apple silicon, and the property that most distinguishes it from contemporaneous discrete-GPU systems, is the integration of heterogeneous compute units — a CPU complex, a GPU, a Neural Engine (NPU), and one or more matrix coprocessors — on a single die, all accessing a single physical DRAM pool through a single coherent fabric. There is no PCIe boundary between the CPU and the accelerator; there is no explicit `cudaMemcpy` step; the GPU does not have a private VRAM. This is what *unified memory* means at the SoC level, and it is the architectural reason why Lucid’s CPU–GPU bridge is designed around zero-copy view sharing rather than data marshalling (Chapter 2, Chapter 10).

The family has four published tiers, each obtained by stamping additional copies of the same building blocks:

- **Base** (M1 / M2 / M3 / M4): one CPU complex with a small P-core cluster (typically 4 cores) and a small E-core cluster (typically 4 cores), a moderate GPU (7–10 cores), 8–24 GB DRAM, and roughly 60–120 GB/s of memory bandwidth.
- **Pro**: an enlarged P-core cluster (6–10 cores), a larger GPU (14–20 cores), wider memory bus (4 channels of LPDDR5), and 200 GB/s-class bandwidth.
- **Max**: another doubling of the P-core complex and GPU (24–40 GPU cores), 8 channels of LPDDR5, and 400 GB/s-class bandwidth.
- **Ultra**: two Max dies connected through Apple’s *UltraFusion* silicon interposer to present a single logical SoC with up to 64 GPU cores and 800 GB/s of aggregate bandwidth.

For Lucid, the relevant generalisation is the following: as one moves from base to Ultra, the available GPU compute grows roughly 8×, memory bandwidth grows roughly 13×, but per-thread CPU performance changes only marginally. This shapes the dispatch philosophy laid out in Chapter 9: any performance-critical work that can be expressed as a GPU primitive must be sent to MLX, because the CPU side does not scale with the chip tier.

The remainder of this chapter inspects each block in Figure 1.1 in turn. We pay particular attention to (i) the contracts the unit exposes to a runtime, (ii) the microarchitectural quirks that shape Lucid’s kernels and dispatch policy, and (iii) the generation-over-generation drift that affects backward compatibility and version-gated code paths.

1.2 SoC Floorplan and Coherent Fabric

A modern Apple SoC is not a monolithic die in the architectural sense that an Intel Xeon of comparable transistor count is. It is, instead, a tiled die: a grid of physically distinct *clusters* (CPU clusters, GPU cluster groups, the Neural Engine, the media engines, and so on) connected by a network-on-chip (NoC) that Apple internally calls the *Apple Fabric*. The fabric is a coherent interconnect: loads and stores from any client (a CPU core, a GPU shader core, the Neural Engine) observe a single, well-ordered view of memory, with the fabric and the System Level Cache acting in concert to ensure that a write by the GPU becomes visible to a subsequent CPU load without an explicit invalidation.

²Process generation matters more than it might appear: each shrink alters not only the area budget but the voltage/frequency curve, which feeds into the power envelope that ultimately bounds sustained ML throughput. We revisit this in Section 1.9.

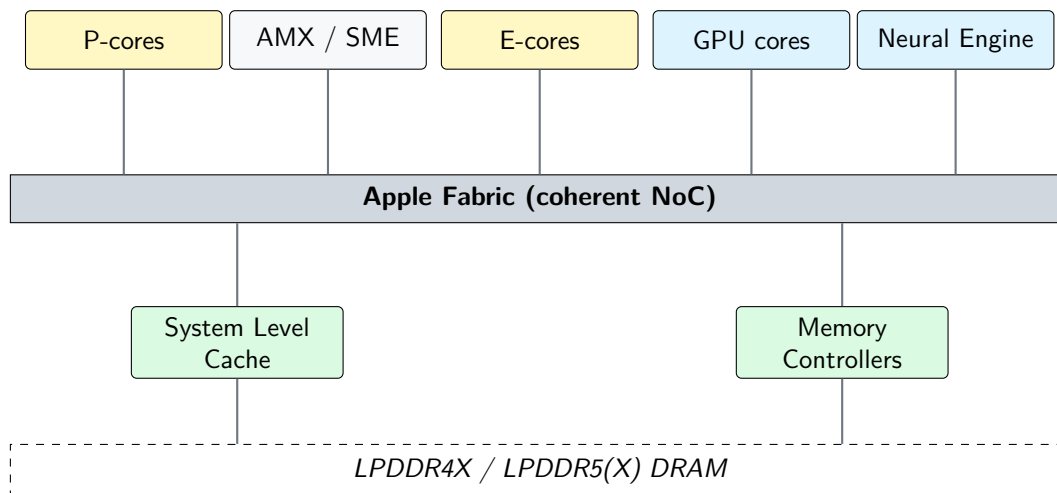


Figure 1.1: Schematic floorplan of an Apple silicon SoC. All compute units — the CPU complex (P-cores, E-cores, AMX/SME), the GPU, the Neural Engine, and fixed-function media blocks — communicate with DRAM through a single coherent fabric (the Apple Fabric, sometimes called the system-level interconnect) that also hosts the System Level Cache. Lucid targets the CPU complex (through Accelerate) and the GPU (through MLX and MPSGraph); the Neural Engine is intentionally not part of the dispatch graph, for reasons described in Section 1.5.

This coherence guarantee is what enables Lucid’s bridge boundary (Chapter 10) to be a metadata operation rather than a `memcpy`: when Lucid converts an MLX array to a contiguous CPU view, the underlying bytes are not moved, only relabelled.³

The fabric has three properties that matter for ML workloads.

Coherence at the cache-line granularity. Coherence is managed at the standard ARM cache-line width (128 bytes on Apple silicon’s L1 and L2; the SLC operates at the same granularity to the fabric). This means that false-sharing pathologies that plague multi-socket x86 systems — where two cores writing into adjacent cache lines stall each other — still exist on Apple silicon, but they are bounded to the local cluster, since the fabric is the only coherence point. In practice this affects Lucid only in the parameter-update path of distributed training (which is not yet supported; see Chapter 67).

Cluster-local L2 caches. Each CPU cluster owns a private L2 (typically 12–16 MB for the P-cluster and 4 MB for the E-cluster). Inter-cluster traffic — including all traffic from CPU to GPU — must traverse the fabric and is filtered by the SLC. Latency to the SLC is roughly 80 to 120 cycles depending on generation; latency to DRAM, when SLC misses, is in the 150–200 cycle range on the P-cores. These numbers are not published by Apple and have been characterised by independent researchers [22].⁴

Asymmetric bandwidth. The fabric is not bandwidth-uniform. The GPU has a wider connection to the fabric than any single CPU cluster; the Neural Engine has its own dedicated path. In aggregate, the GPU can saturate roughly 80–90% of the chip’s memory bandwidth

³The mechanism is described in detail in Chapter 7 and the `SharedStorage` class in `lucid/_C/backend/gpu/SharedStorage.cpp`. The short version is that we extract the `MTL::Buffer*` backing an `mlx::core::array` through `mlx::core::array::data<T>()`, then expose its bytes through the same Accelerate-facing pointer that the CPU backend would have used.

⁴The most thorough published characterisation of the M1 microarchitecture is by Dougall Johnson and the Asahi Linux project. Subsequent generations have been benchmarked by SemiAnalysis and Anandtech-style deep dives; numbers in this chapter are typical of those reports and should be treated as approximate.

on a sustained streaming workload, while a single CPU core can drive perhaps 30–40%.⁵ The Ultra variants double the fabric and thus the aggregate bandwidth, but the per-client peak does not scale linearly.

1.3 The CPU Complex

Apple’s CPU complex is a heterogeneous (*big.LITTLE*) design that combines a small number of large, out-of-order *performance cores* (P-cores) with a larger number of small, energy-optimised *efficiency cores* (E-cores). The cluster organisation has remained stable across generations, but every core microarchitecture has been re-implemented at least once.

1.3.1 Performance cores

The P-core is the most-quoted aspect of Apple silicon’s microarchitecture, and the most thoroughly reverse-engineered. Its distinguishing properties are well-documented through Apple’s patents, Asahi Linux’s reverse engineering, and Dougall Johnson’s public micro-benchmarks. We summarise the salient design choices.

1.3.1.1 Pipeline width and out-of-order resources

Every Apple P-core since Firestorm (M1) decodes *eight instructions per cycle* and dispatches across roughly the same number of issue ports. This is the widest sustained-decode design in any shipping general-purpose CPU and roughly twice the width of contemporaneous x86 cores (which decode 4–6 instructions per cycle). The implication is that ILP-rich code — pointer chasing, tight arithmetic loops — runs unusually fast per cycle on Apple P-cores, which is why ARM-translated Intel binaries (Rosetta 2) reach 60–80% of native performance despite the translation overhead.

The reorder buffer (ROB) on Firestorm is roughly 630 entries; later generations have grown this further. For comparison, contemporaneous x86 ROBAs are in the 350–500 range. The wide front-end and deep ROB are what enable the P-core to absorb the latency to the SLC and DRAM without stalling.

1.3.1.2 NEON and the SIMD path

Each P-core has four 128-bit NEON pipes; this corresponds to four single-precision multiply-add throughputs per cycle per core, or 128 GFLOP/s at 4GHz. NEON is the SIMD ISA Accelerate’s vDSP layer targets directly; the vDSP_vmul, vmsa, and similar primitives in Lucid’s CPU backend (Chapter 9) compile down to NEON.

Crucially, NEON is *not* the path used by BNNS for convolutions. BNNS goes through the AMX coprocessor (Section 1.4), which is shared by the whole P-cluster and can sustain matrix-multiply throughput an order of magnitude higher than four NEON pipes can.

1.3.1.3 SME and the M4 transition

The M4 generation is the first Apple silicon to expose the ARM Scalable Matrix Extension (SME) [6] at the ISA level. SME is the architected successor to the (still undocumented) AMX instruction set; it gives software a portable, ARM-blessed API for matrix-multiply acceleration. SME operates on *tiles* of variable width (ZA register file) and provides outer-product accumulation instructions analogous to AMX’s *outer-product* micro-ops.

For Lucid the practical implication is small in the short term and large in the long term:

⁵This asymmetry is part of why Lucid’s CPU backend uses BNNS for convolutions: it is the only CPU-side API on macOS that accesses the fabric through a matrix-coprocessor-accelerated path rather than a single-core path.

1. In the short term, Accelerate’s BNNS on M4 transparently uses SME where it would have used AMX on M1–M3, so Lucid’s CPU backend gets the benefit without any code change.
2. In the long term, SME’s portability means that Lucid could eventually emit SME intrinsics directly from `lucid.compile` (Chapter 43) without depending on Accelerate. This is on the roadmap but is not in the 3.5 series.

1.3.1.4 Performance counters and observation

The P-cores expose roughly 400 performance monitoring unit (PMU) events. These are not exposed through standard ARM PMU mechanisms on macOS; instead, the `Instruments.app` CPU Counters template (or the underlying `kperf` API) is the only supported access path. Lucid’s profiler (Chapter 42) uses `mach_absolute_time()` for wall-clock timing and does not currently read PMU counters; the user is expected to use Instruments for cycle-level analysis.

1.3.2 Efficiency cores

The E-cores are interesting for Lucid less because of what they execute than because of what they *do not* execute. They are designed for sustained low-power operation, with a much narrower front-end (typically 4-wide decode) and a smaller ROB (~ 180 entries on Icestorm). They run at lower frequencies (peak roughly 2 GHz, against 3.2–4.5 GHz for the P-cores) and lack the parallel NEON throughput of the P-cores.

For an ML workload, the consequence is that any thread which falls onto an E-core will be roughly 4–6 $\times$ slower per cycle than the same thread on a P-core, and run at a lower clock besides. This matters because the macOS scheduler will move threads off the P-cores when their QoS class is set to background or utility (Section 1.3.3). Lucid’s main training loop, by design, uses no thread-affinity hints and runs at the default QoS class `USER_INITIATED`; this places it on P-cores unless the user has dropped to background, which is the correct behaviour for training workloads but means that Lucid will *not* oversubscribe E-cores even when they are idle.

1.3.3 Big.LITTLE scheduling and QoS

macOS implements scheduling through Grand Central Dispatch (GCD), which expresses thread priority through QoS classes: `USER_INTERACTIVE`, `USER_INITIATED`, `UTILITY`, `BACKGROUND`, and `DEFAULT`. The lower three classes (`UTILITY`, `BACKGROUND`, `DEFAULT`) are eligible for the E-cluster; the upper two are confined to the P-cluster, modulo thermal pressure. The kernel performs the placement; user code has no direct affinity API.

This is why Lucid’s data-loader threads (Chapter 65) explicitly leave the QoS class unset, which inherits `USER_INITIATED` from the calling thread. Setting it to `UTILITY` would have placed I/O workers on E-cores, which sounds reasonable but is in fact counter-productive: the disk and network I/O is bound by the SSD’s NVMe controller rather than CPU, and the cost of repeatedly waking an E-core dominates the savings from keeping the P-cores idle.

1.4 Apple Matrix Coprocessor (AMX) and SME

The Apple Matrix Coprocessor is, as of the time of writing, the most performance-critical and least-documented component of Apple silicon. It is an undocumented matrix-multiply accelerator shared by the P-cluster (each P-cluster has one AMX unit; the E-cluster does not) and accessible only through reserved instruction encodings that are not part of the architected ARM ISA. Its existence is acknowledged by Apple only through patent filings and through the throughput of Accelerate’s BLAS routines, which exceed what is possible with NEON alone by a factor of four to ten.

1.4.1 Instruction format and execution model

AMX instructions are encoded as ARM-reserved opcodes that the processor decodes into messages to the shared coprocessor. The unit operates as a tile machine: it owns a set of internal register tiles (X , Y , Z) holding submatrices of 32×32 to 64×64 depending on element type, and it executes outer-product micro-operations that accumulate into the Z tile. A typical `mlx_outer`-style instruction takes two row vectors from X and Y and accumulates their outer product into a slice of Z ; chaining these reproduces GEMM with peak throughput around 2 TFLOP/s on M1 P-cluster, scaling roughly linearly with cluster count thereafter.

The encoding has been documented unofficially in reverse-engineering work by Dougall Johnson and others, and exposed through low-level libraries such as `aarch64-jit-amx`. Apple has never blessed direct AMX programming; the supported path has always been Accelerate.

1.4.2 Throughput characteristics

Empirical AMX throughput, measured through `cblas_sgemm`, sits in the range of 1.5 to 2.5 TFLOP/s per P-cluster on single-precision matrix multiplication of sizes $\geq 256 \times 256$. Smaller matrices do not amortise the cluster-internal dispatch cost and run at NEON throughput or lower. Mixed precision (BF16 inputs, FP32 accumulate) on M3 and later roughly doubles the peak, but Accelerate has not, as of the 3.5 series of Lucid, exposed BF16 `cblas_gemm_bf16bf16f32` on all SDKs; Lucid’s CPU backend therefore still uses FP32 GEMM by default, and the BF16 enablement lives in the AMP roadmap.

1.4.3 Relationship to ARM SME in the M4 generation

The M4 introduces the ARM Scalable Matrix Extension (SME) as the architected successor to AMX. SME is a fully documented ARM ISA extension (ARMv9.2-A) with portable assembly, intrinsics, and debugger support. From Accelerate’s perspective the transition is seamless: BNNS and `cblas_sgemm` produce the same outputs on SME and AMX, with the dispatcher selecting at runtime based on `sysctl`-reported features.

For Lucid, three things change with SME:

1. The CPU backend (Accelerate) becomes future-proof: a future macOS version that drops AMX shims will still run, because the pathway used by Accelerate is now ARM-architected.
2. The compile path (Chapter 43) acquires a portable target. Lucid does not currently emit SME instructions directly, but `lucid.compile` could in principle emit them in a future generation, removing the dependency on Accelerate’s in-process dispatch.
3. The cost model for the CPU backend changes: SME tiles are of *variable* width (the `SVL` parameter), and the M4 exposes a tile size of 512 bits, which makes some smaller-matrix operations more efficient than they were on AMX. Lucid’s dispatcher does not currently exploit this, but future Accelerate versions are expected to.

1.5 Apple Neural Engine (ANE)

The Neural Engine is the most marketed and the least useful of Apple silicon’s accelerators from Lucid’s perspective. It is a fixed function unit: a 16-core (M1, M2) or 16/32-core (M3, M4) NPU specialised for low-precision neural-network inference, with a peak throughput on M4 quoted by Apple at 38 TOPS at INT8.

The ANE is exposed exclusively through CoreML and ML Compute. It does not accept arbitrary kernels, does not support arbitrary dtypes (INT8 and FP16 only; no FP32, no BF16 on most generations, no FP64 ever), and operates only on graphs that have been compiled

offline into the CoreML format. There is no SDK that exposes the ANE for training; there is no online graph submission API.

This places the ANE outside Lucid’s scope by construction. Lucid targets *training* workloads, which require:

1. Gradient computation, hence backward kernels for every op.
2. Mixed precision with FP32 master weights, hence FP32 arithmetic.
3. Dynamic graph construction at the Python level, hence online submission.

The ANE satisfies none of these. A future inference-only `lucid.deploy` target could compile a trained model to CoreML and benefit from the ANE on user devices, but the ANE will never be part of the training graph and is therefore not part of this chapter’s primary subject. We mention it here only to forestall the common question of why Lucid does not use it.

1.6 The GPU

The GPU is the workhorse. On any non-trivial neural network, the GPU performs more than 95 % of the floating-point arithmetic, even when the CPU side is doing its job (data loading, gradient accumulation, optimiser updates). Lucid’s compile path (Chapter 62) emits its compiled graphs onto the GPU via `MPSGraph`; the `MLX` runtime executes its eager kernels on the GPU; the CPU is a coordinator. Understanding the GPU’s microarchitecture is therefore central to understanding Lucid’s performance profile.

1.6.1 Compute hierarchy

Apple’s GPU is organised as a number of *GPU cores* (Apple’s marketing term; the architectural unit is sometimes called a *shader core* or *compute unit*). Each GPU core contains:

- Several *shader execution units*, each capable of executing 32-lane SIMD (*SIMD group* in Metal parlance) FP32 operations per cycle.
- A texture unit and ray-tracing unit (on M3 and later).
- Local memory (called *threadgroup memory* in Metal), typically 32 KB per core.
- Register file for in-flight threads (each thread gets a private slice).

The base M1 has 7 or 8 GPU cores; M1 Pro has 14 or 16; M1 Max has 24 or 32; M1 Ultra has 48 or 64. The M2 and M3 generations followed a similar scaling pattern, with each tier roughly doubling the previous. M4 Max as of the time of writing has up to 40 GPU cores.

Each GPU core can sustain in the neighbourhood of 200 FP32 GFLOP/s at base clocks. The aggregate throughput thus scales linearly with core count, modulo memory bandwidth and thermal constraints. The M1 Max’s 32-core GPU delivers approximately 10.4 TFLOP/s FP32 peak; the M3 Max’s 40-core GPU is approximately 14 TFLOP/s. These are the numbers that bound Lucid’s training throughput on matrix-multiply-bound workloads.

1.6.2 Memory hierarchy on the GPU side

Each GPU core sees memory through a layered hierarchy:

1. **Registers.** Per-thread, allocated by the Metal compiler from a fixed pool (32 KB to 64 KB per core depending on generation). Register pressure governs occupancy.
2. **Threadgroup memory.** A small, fast, software-managed scratchpad shared by all threads in a threadgroup; typically 32 KB.
3. **Tile memory.** A dedicated on-chip buffer used during render-target operations; less relevant for compute.

4. **System Level Cache.** Shared with the CPU; the GPU’s only path to DRAM goes through the SLC.
5. **DRAM (LPDDR).** The unified pool.

There is no separate VRAM, no separate L2 the GPU owns exclusively. The SLC is the only large on-chip GPU cache, and it is shared with every other client of the fabric. Whether this is an advantage or a disadvantage depends on the workload: well-behaved single-tenant ML training benefits from larger effective cache; a system under display or video-encode pressure can starve the GPU.

1.6.3 Dynamic Caching (M3 and later)

The M3 generation introduced *Dynamic Caching*, which Apple describes as “allocating GPU memory in real time based on what each program needs.” Concretely, the GPU’s register and threadgroup-memory allocation, which on M1/M2 was determined statically at shader compile time, becomes runtime-virtualised on M3 and later: the hardware allocates registers dynamically to in-flight threads, enabling higher effective occupancy on shaders that previously over-allocated registers.

For Lucid this is observable but not directly programmable. Compiled MLX and MPSGraph kernels benefit transparently when run on M3 or M4 hardware; the same kernels on M1/M2 do not. We document the empirical effect in Chapter 60, where the M3 generation shows a step change in per-op throughput for irregular kernels (reductions over non-contiguous axes, scatter-with-gather patterns).

1.6.4 Hardware ray tracing and mesh shaders

The M3 generation also added hardware ray tracing and mesh shader support. Neither is relevant for ML training and we mention them only to clarify the silicon budget: a portion of the GPU die area on M3 and later is dedicated to graphics-only features, which is part of why per-core compute throughput grew less than the marketing core count would suggest on the M3 transition.

1.6.5 Metal Performance Shaders and MPSGraph

The software interface to the GPU has two layers:

1. **Metal.** The low-level API: command buffers, shader programs in MSL (Metal Shading Language), explicit synchronisation. MLX uses Metal directly for its kernels.
2. **MPSGraph.** The high-level API: a graph IR that the Metal Performance Shaders framework compiles to optimised multi-kernel sequences. Lucid’s compile path (Chapter 45) emits onto MPSGraph.

These are not mutually exclusive. MLX continues to dispatch most eager kernels through its own MSL; MPSGraph is used in Lucid only by `lucid.compile` and by a small set of per-op optimisations introduced in 3.4 (Chapter 60). The two paths can coexist within a single program because they ultimately share the same Metal command queue and the same unified memory pool.

1.7 Memory System

Every load and store on Apple silicon, regardless of which compute unit issued it, ultimately resolves through the same memory controllers to the same LPDDR pool. This section describes the characteristics of that pool, which set the upper bound on data-bound workloads (i.e. most of deep learning).

SoC	DRAM type	Channels	Capacity range	Peak BW (GB/s)
M1	LPDDR4X-4266	4	8–16 GB	≈ 68
M1 Pro	LPDDR5-6400	4	16–32 GB	≈ 200
M1 Max	LPDDR5-6400	8	32–64 GB	≈ 400
M1 Ultra	LPDDR5-6400	16	64–128 GB	≈ 800
M2	LPDDR5-6400	4	8–24 GB	≈ 100
M2 Pro	LPDDR5-6400	4	16–32 GB	≈ 200
M2 Max	LPDDR5-6400	8	32–96 GB	≈ 400
M2 Ultra	LPDDR5-6400	16	64–192 GB	≈ 800
M3	LPDDR5-6400	4	8–24 GB	≈ 100
M3 Pro	LPDDR5-6400	3	18–36 GB	≈ 150
M3 Max	LPDDR5-6400	8 or 6	36–128 GB	$\approx 300\text{--}400$
M4	LPDDR5X-7500	4	16–32 GB	≈ 120
M4 Pro	LPDDR5X-8533	4	24–48 GB	≈ 273
M4 Max	LPDDR5X-8533	8	36–128 GB	≈ 546

Table 1.1: Memory subsystem summary across Apple silicon generations. “Peak BW” is the theoretical maximum aggregate bandwidth; sustained bandwidth from a single client is typically 60–80 % of peak.

1.7.1 LPDDR generations and channels

Table 1.1 summarises the memory subsystem across generations. Two observations relevant to Lucid:

1. The M3 Pro is the only generation that reduced peak bandwidth relative to its predecessor (M2 Pro). This was driven by a rebalancing of the die for cost and yield; the practical consequence is that some training workloads benchmarked on M2 Pro run *slower* on M3 Pro, and Lucid’s performance documentation flags this explicitly (Chapter 58).
2. M4 Max’s 546 GB/s sits between M2 Max and M2 Ultra. For Lucid’s GPT-2 base benchmark, this places M4 Max above the sustained training throughput of a single M2 Ultra under thermal pressure, which is the only documented data-point at the Ultra tier; we revisit in Chapter 62.

1.7.2 System Level Cache

The System Level Cache (SLC) is a last-level cache shared by every client of the fabric. Its size grows with the chip tier: 8 MB on M1, 24 MB on M1 Pro, 48 MB on M1 Max, 96 MB on M1 Ultra. M3 and M4 use comparable sizes. SLC is filled and evicted by hardware-managed coherence; software has no direct allocation hints other than memory-allocator placement strategies that Accelerate uses internally.

For ML workloads the SLC affects two things:

1. **Activation reuse.** A forward pass that fits its activations into the SLC can be re-read by the backward pass without paying DRAM bandwidth twice. The cut-off is generally around 50–100 MB of total activations for the Max tier, which corresponds to a moderate batch size on ResNet-50 or a small batch on a Transformer of GPT-2 base size.
2. **Inter-cluster traffic.** When the CPU and GPU exchange a buffer through Lucid’s bridge, the SLC absorbs the transfer if the buffer is small. This is why our microbenchmarks (Chapter 57) show roughly the same CPU-to-GPU latency for buffers under 4 MB regardless of whether the buffer was just written or has been resident for some time.

1.7.3 Bandwidth versus arithmetic intensity

The roofline model captures the trade-off between memory bandwidth and arithmetic intensity. For an Apple silicon M3 Max with 400 GB/s of memory bandwidth and roughly 14 TFLOP/s of FP32 GPU peak, the ridge point of the roofline lies at an arithmetic intensity of $14 \times 10^{12} / 400 \times 10^9 \approx 35$ FLOPs per byte. For comparison, an NVIDIA H100 has a ridge point near 250 FLOPs per byte: ML kernels that are bandwidth-bound on H100 are often compute-bound on Apple silicon, and vice versa. This shifts the set of optimisations that pay off. Lucid’s no-compile sweep (Chapter 61) is an example: algorithmic changes that reduce arithmetic count (using sum/N in place of mean to fold a scalar division earlier) yield real speedups on Apple silicon precisely because the workload is closer to compute-bound than memory-bound at typical operation sizes.

1.8 The Apple Silicon Family: M1 through M4

Having described the constituent blocks, we now summarise the generational evolution. Each generation can be understood as a set of deltas relative to its predecessor: process node, P-core microarchitecture, GPU enhancements, AMX/SME transition, and memory subsystem refresh.

1.8.1 M1 (2020)

The M1 introduced the architecture template that all subsequent generations refined. Salient properties:

- TSMC N5; 16 billion transistors.
- Firestorm P-cores (8-wide decode, $\text{ROB} \approx 630$); 4 of them in the base configuration.
- Icestorm E-cores (4-wide decode); 4 of them.
- GPU: 7 or 8 cores; up to 2.6 TFLOP/s FP32.
- AMX1.
- LPDDR4X-4266, 4 channels, 68 GB/s, up to 16 GB.

The M1 Pro, Max, and Ultra variants applied the cluster-replication pattern described earlier: the same blocks, more of them, faster memory bus.

1.8.2 M2 (2022)

The M2 was an iteration on N5 (specifically, N5P), and applied two classes of changes:

- Avalanche P-cores, Blizzard E-cores. Same width, refined schedulers and improved branch prediction; roughly 10–15 % IPC improvement.
- GPU bumped to 10 cores at the base tier; LPDDR transition to LPDDR5-6400.
- AMX2 (incremental).

No fundamental architectural change; this was a refinement generation.

1.8.3 M3 (2023)

The M3 was a major architectural step:

- Move to N3B (3 nm), a substantial process shrink.
- Everest P-cores; further IPC growth.
- GPU receives Dynamic Caching, hardware ray tracing, mesh shader support. Performance per core grows.

- AMX3.
- Mixed memory bus configurations: M3 Pro reduced its memory bus relative to M2 Pro, an anomaly noted in Table 1.1.

1.8.4 M4 (2024)

The M4 transitioned to N3E (a refinement of N3) and architecturally crossed into ARMv9.2-A:

- First Apple SoC to expose SME at the ISA level (in place of AMX).
- LPDDR5X transition (7500 MT/s base; 8533 MT/s on Pro and Max).
- Continued GPU refinements; mature Dynamic Caching.
- Improved Neural Engine quoted at 38 TOPS.

For Lucid, the M4 transition is the most important since M1: it is the first generation on which `lucid.compile` could plausibly target SME directly in a future release, removing the dependency on Accelerate’s in-process dispatch.

1.9 Implications for ML Workloads

We close this chapter with the qualitative conclusions that frame the rest of the book.

1.9.1 The dispatch trade-off

Apple silicon’s heterogeneous compute means that a framework has at least three reasonable backends for any given operation: the GPU (via `MLX` and/or `MPSGraph`), the CPU’s NEON pipes (via direct SIMD), and the CPU’s matrix coprocessor (via Accelerate’s `BNNS` and `BLAS`). The Neural Engine is a fourth in principle but absent from training graphs, as discussed.

The performance ordering between these backends is not monotone in problem size. For small matrices ($\lesssim 64 \times 64$), NEON or even scalar code can be competitive because GPU kernel launch overhead dominates and AMX’s cluster-internal dispatch overhead is also non-trivial. For medium matrices (64–256), AMX dominates. For large matrices (≥ 512), the GPU dominates because of its aggregate throughput advantage. Lucid’s dispatch logic (Chapter 9) does not attempt to choose between these dynamically at the framework level: the user picks a device (`cpu` or `metal`) and within that device the BLAS or `MLX` library applies its own internal heuristic.

1.9.2 The memory-bandwidth ceiling

Apple silicon’s most distinctive performance property is its very high memory bandwidth relative to its compute. On the largest configurations (Ultra), the bandwidth-to-compute ratio approaches one byte per FLOP, which is roughly an order of magnitude better than a discrete GPU. This is what makes large-batch streaming workloads (autoencoders, certain Transformer training regimes with large sequence length) practical on Apple silicon despite the modest peak compute.

The corollary is that bandwidth-bound kernels (layer norm, softmax, attention, anything that touches every element of a large tensor exactly once) tend to be the bottleneck on small Apple silicon configurations. Lucid’s normalisation family rewrites (Chapter 61) target these explicitly.

1.9.3 The power envelope

Every Apple SoC is bound by a thermal design power (TDP) envelope that determines sustained performance. The M-series TDPs range from roughly 20 W on the base M1 to roughly 90 W on

the M1 Ultra; the M4 family has comparable envelopes. Under sustained load, all configurations throttle from their peak frequencies, and the GPU cores are the largest single consumer.

For ML training, this matters in two ways:

1. A burst-mode benchmark (the first few iterations of a training run) does not predict sustained throughput. Lucid’s benchmark methodology (Chapter 57) discards the first 20–50 iterations to allow the chip to settle.
2. Active cooling matters. The MacBook Pro chassis throttles less aggressively than the MacBook Air; the Mac Studio less than either. Numbers in Lucid’s performance documentation are taken on Mac Studio M-series hardware to control for this.

1.9.4 The compile-vs-eager trade-off

Finally, the existence of `MPSGraph` as a graph-level abstraction above `Metal` means that an ML framework on Apple silicon faces a genuine choice between eager dispatch (each op submitted as a Metal command buffer) and graph capture (a sequence of ops compiled into a single `MPSGraph` executable). `MLX` defaults to eager; Lucid’s `lucid.compile` (Chapter 43) opts into graph capture for a measurable win on most training workloads. The trade-offs and the exact wins are documented in Chapter 62.

1.10 Summary and Forward References

Apple silicon is a tightly integrated, heterogeneous SoC family whose defining properties — unified coherent memory, multiple fixed-function accelerators, and a wide-but-low-frequency CPU complex — collectively dictate every interesting design decision in Lucid. The five compute units described in this chapter — P-cores, E-cores, AMX/SME, the Neural Engine, and the GPU — form the set of candidate dispatch targets, of which Lucid uses four (the Neural Engine is excluded by construction).

The remainder of Part I expands on the runtime implications. Chapter 2 formalises the unified memory model and describes how Lucid exploits it without ever issuing an explicit copy. Chapter 3 introduces `MLX` as the GPU-side runtime that Lucid depends on, including its graph-eager hybrid execution model. Chapter 4 introduces `Accelerate` as the CPU-side counterpart. Chapter 5 treats `Metal` and `MPSGraph` directly, since both are visible to Lucid’s compile path. After these four foundation chapters, the book proceeds to the system architecture proper in Part II, where the hardware view of this chapter becomes a software dependency graph that Lucid has been engineered to respect.

Tip

For readers primarily interested in framework-level concerns rather than silicon-level concerns, Section 1.9 contains the only consequences they need to internalise: the dispatch trade-off between three backends, the memory-bandwidth ceiling, the thermal envelope, and the compile-vs-eager trade-off. The remainder of this chapter can be returned to as a reference when those concerns become concrete.

CHAPTER 2

Unified Memory Model and Memory Pressure

Chapter 1 established that Apple silicon is a heterogeneous SoC in which every compute unit reads and writes a single physical DRAM pool through a coherent fabric. That property has a name — *unified memory* — which Apple uses in marketing but which, for a framework writer, has a precise technical meaning. This chapter develops that meaning. The central question is not whether memory is shared (it always is) but under what contract two different compute units, two different command queues, two different threads observe the same byte at the same logical instant, what the runtime is allowed to assume about ordering and visibility, and what happens when those assumptions interact with the macOS virtual memory subsystem.

The pay-off for Lucid of getting this right is large: under unified memory, the natural framework design has *no explicit data transfer step*. A tensor allocated by the GPU runtime can be read by a CPU consumer, and vice versa, without a `cudaMemcpy`-equivalent intermediary. Chapter 10 catalogues the six places where Lucid touches the world outside the engine, and not one of them is a CPU-to-GPU transfer; the bridge is internal and free. The job of this chapter is to explain why that is correct, where its correctness is fragile, and how Lucid exposes the user-visible consequences (`memory_stats`, OOM behaviour, the choice of device argument) in a way that lines up with the underlying hardware.

The chapter is organised so that the reader who only wants to use Lucid may stop at Section 2.7; the rest is for the reader who needs to debug a memory pathology or extend the runtime.

2.1 The Unified Memory Contract

A *memory model* is a triple $(\mathcal{A}, \mathcal{C}, \mathcal{O})$ of address spaces, coherence guarantees, and ordering guarantees that together define when one client of memory observes another client’s writes. The Apple silicon unified memory model is, informally:

Contract 2.1 (Apple silicon unified memory). There is a single physical address space $\mathcal{A}_{phys}$ shared by every compute unit on the SoC. Each compute unit (a CPU hardware thread, the GPU command queue, the Neural Engine, the AMX coprocessor) sees $\mathcal{A}_{phys}$ through a process-local virtual address space $\mathcal{A}_{virt}$ maintained by the macOS kernel. Coherence is maintained at cache-line granularity by the Apple Fabric (Section 1.2); a write by one client is eventually visible to every other client, and the system provides explicit barrier primitives that turn “eventually visible” into “visible by program order at this point.”

The contract has three components, each of which has a software-visible consequence for Lucid.

Address spaces are unified at the physical layer. A buffer allocated by a CPU thread (via `posix_memalign`, `malloc`, or the Metal allocator) lives in DRAM that the GPU command queue can read. There is no separate GPU address space. Apple’s Metal API exposes this as the `MTL::Buffer` type: buffers are allocated with a *storage mode* that determines whether the CPU has a mapped view of the same physical pages (Section 2.3), but no storage mode makes the buffer *inaccessible* to the GPU — the GPU is always the consumer.

Coherence is hardware, but barriers are software. The fabric maintains coherence at cache-line granularity, but this only means that no two clients hold mutually incoherent copies of the same line at the same time. It does *not* mean that an arbitrary CPU load following an arbitrary GPU command-buffer commit will observe the GPU’s writes. Visibility between the CPU and the GPU requires an explicit barrier (`MTLCommandBuffer::waitUntilCompleted`, an `MTLSharedEvent`, or an `MLX eval()` call), without which the CPU may observe stale data even though the underlying physical pages have been updated.

Ordering across clients is partial. Within a single CPU thread, ordering is determined by the ARM memory model (ARMv8.0-A’s *weak* ordering, refined by ARMv8.3-A’s release/acquire semantics). Within a single GPU command buffer, ordering is by program order in the encoded compute pass. Across the two, ordering is determined exclusively by Metal synchronisation primitives. There is no implicit happens-before relation between a Metal command buffer’s submission and a subsequent CPU load.

For Lucid the practical implication is that *every* CPU-side access of a GPU-produced buffer must be preceded by either an `mlx::core::eval()` on the producing array or by an explicit Metal synchronisation primitive. The `SharedStorage` bridge (Section 2.6) consolidates this discipline into a single function call so that user code does not have to manage it. A historical bug in 3.2.1 in which `detach()` on a parameter leaked an MLX lazy graph (Chapter 58) was traceable to a missed `eval()` on the bridge boundary; we treat that incident in Section 2.8.

2.2 Address Spaces and the macOS Virtual Memory Subsystem

The unified physical pool is mediated, on every access, by the macOS virtual memory subsystem. Apple silicon Macs use a page size of 16 KiB (in contrast to the 4 KiB page size on x86 Macs). This larger page size reduces TLB pressure — a P-core’s L1 TLB has on the order of a few hundred entries, which translates to a few megabytes of working set under 4 KiB pages but

tens of megabytes under 16 KiB pages — and it matches the natural allocation granularity of the Metal allocator.

2.2.1 Page residency states

The macOS VM subsystem classifies every physical page into one of several residency states, which are visible through `vm_stat(1)` and through the `host_statistics64` mach call:

- **Free.** Available for new allocations.
- **Active.** Mapped into a process and recently referenced.
- **Inactive.** Mapped, not recently referenced, eligible for reclaim.
- **Speculative.** Prefetched but not yet referenced.
- **Wired.** Pinned in DRAM, cannot be paged out. The Metal allocator wires GPU-visible buffers because the GPU has no page-fault handler; an unbacked GPU access would be a hardware fault.
- **Compressed.** Belonging to a process but stored in the kernel’s compressed memory pool rather than DRAM; reads incur decompression overhead.
- **Purgeable.** Allocated but marked volatile; the kernel is permitted to discard the contents under pressure.

For Lucid, the relevant state is *wired*. GPU buffers are wired; they do not page out and do not compress. Total wired memory is the principal hard ceiling on Lucid’s working set. On a 16 GB M-series machine, sustained wired allocation above roughly 11–12 GB triggers memory-pressure events that may result in the macOS kernel terminating background processes.

2.2.2 Compressed memory

macOS has, since OS X Mavericks, maintained a compressed in-DRAM pool as an alternative to swap. When the system is under pressure and a candidate page is identified for eviction, the kernel compresses the page (typically achieving 2:1 to 3:1 ratios on application data) and stores it in the compressed pool rather than writing to NVMe. A subsequent reference decompresses the page back into uncompressed DRAM.

This is invisible to Lucid at the framework level for two reasons:

1. GPU-visible buffers are wired and are not eligible for compression.
2. CPU-only tensors (Python integer scalars, optimiser state on the CPU stream, etc.) are too small for compression to be a significant factor.

The compressed pool nonetheless affects sustained behaviour: when total system pressure rises, compression overhead competes with Lucid’s compute for the same CPU cycles, and the user observes a slowdown without a corresponding rise in DRAM usage.

2.2.3 Swap and the NVMe controller

Beyond compression, the kernel will swap to NVMe. On Apple silicon this is a single-namespace NVMe SSD with very high sustained bandwidth (multiple GB/s read, comparable write), but the latency floor is still in the tens of microseconds. A training step that involves a swapped page taking even one TLB miss to NVMe is several hundred GPU kernel-launches’ worth of stall. Lucid’s recommendation to users is that training workloads be sized to fit within wired memory plus a generous margin; the framework does nothing special to coexist with swap, because no neural network of useful size can afford it.

Mode	CPU access	GPU access	Coherence	Lucid use
Shared	yes (mapped)	yes	fabric	exclusive
Private	no (blit)	yes	–	not used
Managed	yes (copy)	yes	explicit	not used (legacy x86)
Memoryless	no	yes (tile)	–	not used (graphics only)

Table 2.1: The four `MTLResourceStorageMode` options. Lucid uses only `Shared`, because it is the only mode that admits a zero-copy CPU view of GPU-resident bytes. `Managed` is the legacy mode for Intel Macs with discrete GPUs and is silently promoted to `Shared` on Apple silicon.

2.3 Metal Resource Storage Modes

The Metal API exposes the unified memory contract through the `MTLResourceStorageMode` enum, which is set at buffer creation time and which selects how the CPU and the GPU each see the buffer’s bytes. There are four modes; Lucid uses only one of them exclusively, but understanding the others is necessary to understand Lucid’s interoperability with the rest of the Apple graphics stack. Table 2.1 summarises all four for quick reference.

2.3.1 Shared (`MTLStorageModeShared`)

In *Shared* mode, the buffer’s pages are mapped into both the CPU’s virtual address space and the GPU’s command-queue address space. Loads and stores from either side observe the same bytes. Coherence is maintained by the fabric without software intervention, modulo the visibility-by-barrier rule discussed above.

This is the storage mode that makes unified memory tangible to software. It is also the storage mode that MLX uses for nearly every array, and consequently the mode that Lucid’s `Tensor` class effectively assumes for all GPU-resident data. The `SharedStorage` bridge (Section 2.6) operates on `MTLStorageModeShared` buffers, because they are the only buffers a CPU pointer can address.

2.3.2 Private (`MTLStorageModePrivate`)

In *Private* mode, the buffer’s pages are mapped only to the GPU. The CPU cannot read them directly; doing so requires an explicit blit through a staging `Shared` buffer. Private mode exists because the GPU may, for some configurations and some Metal device families, place private buffers in a region of memory with different cache attributes than shared memory — specifically, with write-back caching enabled on the GPU side, which is slightly faster for purely GPU workloads.

Lucid does not use *Private* mode. The reasoning is precise: the speed advantage of Private over Shared is small (single-digit percent on Apple silicon, where the unified memory hierarchy means both modes ultimately go to the same DRAM), and the cost is that the zero-copy CPU bridge ceases to be zero-copy. We would have to allocate a parallel Shared staging buffer and blit on every CPU access. MLX makes the same choice, so the question reduces to whether Lucid would deviate from MLX, and the answer is no.

2.3.3 Managed (`MTLStorageModeManaged`)

Managed mode is a legacy mode from the era of Intel Macs with discrete GPUs (typically AMD Radeon Pro). It maintained two physical copies of the buffer, one in VRAM and one in system DRAM, and required explicit `didModifyRange` and `synchronizeResource` calls to mark when each side had updated its copy. The Apple silicon Metal driver does not support *Managed* mode on integrated GPUs (it would require maintaining two copies in the same physical pool, which is meaningless), and Lucid never encounters it.

We mention it only because legacy reference framework code paths sometimes select *Managed* when the user passes `device='mps'` on macOS; this is harmless but wasteful, since the driver silently promotes *Managed* to *Shared* on Apple silicon.

2.3.4 Memoryless (`MTLStorageModeMemoryless`)

Memoryless buffers are render-target tiles: they live only in the GPU's on-chip tile memory and never back to DRAM at all. They are used for intermediate render targets in deferred shading. Compute workloads cannot use them; Lucid does not encounter them.

2.3.5 What Lucid uses

Lucid uses *Shared* exclusively. The choice is forced by the bridge design (Section 2.6) and by MLX's own default. The cost is a small loss of GPU-side caching efficiency relative to *Private*; the gain is that every tensor in Lucid admits a zero-copy CPU view, every interop with Accelerate is free, and the bridge contract is consistent across the entire engine.

2.4 Memory Pressure on macOS

macOS reports system memory pressure on three discrete levels:

- **Normal.** No pressure.
- **Warning.** The system has decided that processes should free non-essential caches. Reported through the `DISPATCH_MEMORYPRESSURE_WARN` GCD event and through the `kern.memorystatus_vm_pressure_level` sysctl.
- **Critical.** The system is about to start terminating background processes to reclaim memory. `DISPATCH_MEMORYPRESSURE_CRITICAL`.

The kernel raises the pressure level as a function of free memory, swap activity, and the rate of compressed-pool growth. Lucid does not subscribe to these events at the C++ engine layer, but it does honour them indirectly: the Metal allocator, the MLX buffer pool, and Accelerate all subscribe and release cached buffers when the system signals warning or critical.

2.4.1 Pressure-induced allocator behaviour

Lucid's allocator (`lucid/_C/core/Allocator.{h,cpp}`) is a size-class pool: requests are rounded up to one of 23 size classes, and free blocks within each class are retained in a freelist up to a configurable depth (`kMaxDepth = 32`). The pool is process-local and is not coordinated with the kernel's memory pressure events, on the principle that pressure handling is the underlying MLX and Metal allocators' responsibility; Lucid's pool is a layer above them and would just create a second cache layer to manage.

This design has a consequence: Lucid's reported memory usage (`lucid.memory_stats()`) is *the pool's* reported usage, which can be larger than the actual GPU resident set after MLX has released cached buffers. The two numbers may drift by a few hundred megabytes under sustained training; this is not a leak. We document the distinction in Section 2.5.

2.4.2 User-visible pressure symptoms

For a user running Lucid, the symptoms of memory pressure escalate in the following sequence:

1. **Slowdown without an obvious culprit.** The compressed pool is growing; the CPU is paying compression overhead.
2. **Increased CPU usage in `kernel_task`.** The kernel is spending cycles on compression / decompression.

3. **Background apps quit silently.** Pressure has reached *Critical* and *jetsam* (the OS X analog of OOM-killer) has begun reclaiming. Lucid's process itself is not at risk yet if it is the user's foreground process.
4. **Lucid raises `LucidError("out of memory")`.** Lucid's allocator has been refused by MLX, which has been refused by Metal. At this point the only recourse is to reduce the working set — smaller batch, fewer activations retained, gradient checkpointing.

The framework deliberately does not silently fall back to a smaller batch size on OOM. The reason is reproducibility: a training run whose batch size is implicitly modulated by system memory pressure produces different gradients on different runs, which violates the deterministic-by-default contract documented in Chapter 67.

2.5 Lucid's Memory Accounting

Lucid exposes memory usage through a small set of Python APIs that sit on top of the C++ engine's accounting. The public surface is deliberately narrow:

```

1 import lucid
2
3 lucid.memory_stats()           # dict for current device
4 lucid.memory_stats("cpu")      # CPU pool stats
5 lucid.memory_stats("metal")    # GPU pool stats
6 lucid.empty_cache()           # drain the allocator freelists

```

The returned dictionary contains the following fields, all in bytes:

- `allocated` — total bytes currently held by live `Tensor` objects.
- `reserved` — total bytes the allocator has from the underlying provider (MLX or Accelerate); always $\geq$ `allocated`.
- `peak_allocated` — the high-water mark since the process began, or since the last `reset_peak()` call.
- `peak_reserved` — analogous high-water mark for `reserved`.
- `num_alloc_calls`, `num_free_calls` — operational counters for diagnostics.

2.5.1 Pool versus provider

It is worth being explicit about the distinction between the *pool* (Lucid's own freelist) and the *provider* (MLX or Accelerate). When a tensor is freed by the Python garbage collector, its bytes return to Lucid's pool, not to the underlying provider. The provider sees its allocation count as constant. If a user observes `lucid.memory_stats()['allocated']` dropping to near zero but the system's Activity Monitor continuing to report multiple gigabytes of Metal allocation, this is the pool holding cached buffers in case a subsequent allocation can reuse them. `lucid.empty_cache()` drains the pool back to the provider, releasing memory to the kernel.

2.5.2 Why the pool exists

The natural question is whether Lucid needs a pool at all, given that both MLX and Accelerate have their own. The answer is yes, for two reasons. First, neither MLX nor Accelerate exposes the size-class structure that Lucid's autograd engine needs to amortise the allocation cost of a backward pass that produces many tensors of identical shape (one per linear layer, for example). Second, the two providers do not coordinate with each other: a CPU-pool free does not release memory that a GPU allocation can reuse, even though the physical DRAM is shared. Lucid's pool is a thin wrapper that applies size-class amortisation uniformly and that, indirectly, ensures that `empty_cache()` drains both providers via a single call.

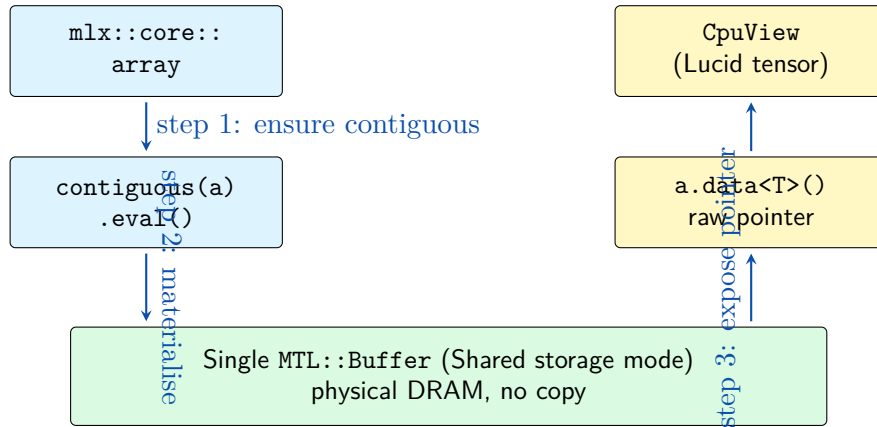


Figure 2.1: The three-step `SharedStorage::from_mlx` bridge. Step 1: ensure the MLX array is contiguous (no-op if already, one-pass copy otherwise). Step 2: `eval()` materialises the lazy graph into a single `MTL::Buffer`. Step 3: `data<T>()` returns a CPU pointer to that buffer. The `CpuView` wraps the pointer with shape/dtype metadata, holding the MLX array alive via `shared_ptr` for the view’s lifetime.

2.6 The Zero-Copy CPU–GPU Bridge

The most consequential consequence of the unified memory contract, for Lucid’s design, is that the CPU and the GPU can share a buffer without copying it. This section describes how that is implemented.

Figure 2.1 sketches the three-step protocol. The crucial observation is that no bytes move at any step — the `contiguous` call may materialise a copy if the input is non-contiguous, but in the typical case it is a no-op, and the remaining steps are pure metadata.

2.6.1 The bridge interface

The bridge is implemented in the `SharedStorage` class (`lucid/_C/backend/gpu/SharedStorage.{h,cpp}`). Its public interface is, schematically:

```

1  class SharedStorage {
2  public:
3      // Adopt an MLX array; return a CPU-readable pointer to its bytes.
4      static CpuView from_mlx(const mlx::core::array& a);
5
6      // Adopt a CPU buffer; return an MLX array that wraps the same
7      // bytes.
8      static mlx::core::array from_cpu(void* data, Shape shape, Dtype
9      dt);
10
11     // Synchronisation point: ensure all GPU writes to ‘a’ are visible.
12     static void eval_into_cpu(const mlx::core::array& a);
13 };

```

The crucial operation is `from_mlx`. Given an `mlx::core::array`, it returns a view of the same physical bytes that the CPU side of Lucid can dereference. There is no copy. The implementation has three steps:

1. Call `a.eval()` (transitively, `mlx::core::eval`) to materialise the array. MLX is a lazy graph runtime; an unmaterialised array has no backing buffer.
2. Call `a.data<T>()` to obtain a raw pointer to the backing `MTL::Buffer`’s contents (the buffer is *Shared* storage mode, so the pointer is CPU-dereferenceable immediately).

3. Wrap the pointer in a `CpuView` carrying the shape and dtype metadata, but *ignoring* any non-contiguous strides from MLX's lazy view. This is the subtle point.

2.6.2 The stride trap

MLX supports lazy view operations: `transpose`, `slice`, and `reshape` return arrays that share storage with their inputs but carry adjusted strides. The bytes underlying such an array are in input-original order, not the post-view order.

Crucially, `mlx::core::array::data<T>()` returns a pointer to the raw bytes *without* applying the view's strides. A naive bridge that handed those bytes to a CPU consumer expecting post-transpose order would corrupt every downstream computation.

The discipline Lucid enforces is therefore:

Warning

Before `SharedStorage::from_mlx`, the array must be made contiguous. The bridge calls `mlx::core::contiguous(a).eval()` on its way in. The order matters: `contiguous` first (so that strides are unit-stride), then `eval` (so that the contiguous copy is materialised), then `data<T>()`.

This discipline is encoded in `SharedStorage::from_mlx` and is not negotiable. A historical engine quirk (`engine-mlx-data-ignores-strides` in the vault) is the source-of-truth note that captured the bug class.

2.6.3 Why this is free

The bridge is sometimes free, sometimes not, depending on whether the source array is already contiguous:

- If the array is contiguous (no transpose, no non-trivial slicing in its history), `contiguous` is a no-op: MLX recognises the trivial case and returns the same backing storage. The bridge is free.
- If the array is non-contiguous, `contiguous` allocates a new buffer and performs a kernel-driven copy. The bridge is not free: it costs one full read–write pass over the tensor. This is, however, fundamental — no possible bridge could be free here, because the CPU consumer needs unit-stride data.

Lucid's design ensures that most tensors at the bridge boundary are already contiguous. The dispatch layer (Chapter 9) prefers to keep operations contiguous; views are localised; the bridge is invoked at function boundaries (e.g. `.numpy()`, `.tolist()`, `__dlpack__`) where the user has signalled intent to interoperate with external code that requires unit-stride.

2.7 Memory Pressure and ML Workloads

We now turn to the practical implications for training workloads.

2.7.1 The working-set decomposition

A training step's working set decomposes into the following classes:

1. **Model parameters.** For a parameter count N at FP32 precision, this is $4N$ bytes. A 100M-parameter model occupies 400 MB.
2. **Optimiser state.** For Adam, two additional copies of each parameter (the first and second moments), so $8N$ bytes.

3. **Gradients.** One copy per parameter during the backward pass, so an additional $4N$ bytes peak.
4. **Activations.** Highly model- and batch-dependent. For a Transformer of hidden size d , depth L , batch size B , sequence length T , the activation memory is roughly $O(BLTd)$ for feed-forward and $O(BLT^2)$ for attention. Activations are the dominant term for non-trivial sequence lengths.
5. **Allocator headroom.** $\sim 10-20\%$ overhead held by Lucid’s pool and MLX’s pool for amortisation.

For a 100M-parameter Transformer at batch size 32 and sequence length 512, the breakdown is approximately:

- Parameters: 400 MB.
- Adam state: 800 MB.
- Gradients (peak): 400 MB.
- Activations: 2–4 GB depending on FFN size.
- Allocator headroom: 0.4–0.8 GB.

The total is in the 4–6 GB range, comfortable on a 16 GB unified memory configuration but tight on 8 GB. Doubling the batch size roughly doubles the activation term and pushes the total above 8 GB.

2.7.2 Activation memory and gradient checkpointing

The activation term is the largest and the most adjustable. For each layer, the forward pass produces an activation tensor that the backward pass consumes. Naively, all activations are retained until backward; this gives the $O(BLTd)$ figure above.

Gradient checkpointing (Chapter 24) trades compute for memory: by retaining only every k -th activation and recomputing the intermediate ones on the backward pass, the activation memory becomes $O(BLTd/k)$ at the cost of an extra k -fraction of forward work. Lucid exposes this through `lucid.utils.checkpoint`.

The break-even point for checkpointing on Apple silicon is characteristically different from a discrete-GPU system because the GPU’s compute is relatively scarce and bandwidth is relatively abundant. The same model that benefits from $k = 4$ checkpointing on a discrete GPU may prefer $k = 2$ on M-series, because the re-forward pass is more expensive in relative terms.

2.7.3 The non-preallocation policy

Lucid deliberately does *not* preallocate a fixed memory arena at startup. `reference framework` on CUDA does (the `PYTORCH_CUDA_ALLOC_CONF` environment variable can request it), partly to avoid CUDA’s fragmentation behaviour and partly to reduce allocator overhead. On Apple silicon the fragmentation argument is weaker (the unified pool is shared with everything else on the machine, so a giant preallocation would stall other system work), and the allocator overhead argument is also weaker (the size classes in Lucid’s pool already amortise away most of the allocation cost).

The result is that Lucid’s memory usage scales with the working set, not with a fixed budget. This makes it easier to coexist with other applications on the same machine (the user’s IDE, their browser, a notebook viewer) without manual tuning.

2.8 Pathologies and Failure Modes

The unified memory contract is strong, but two classes of bugs have recurred in Lucid’s history. We document both because the same patterns are likely to recur in future extensions.

2.8.1 The lazy-graph leak (3.2.1)

The 3.2.1 series shipped a hotfix for a long-running memory leak in the BatchNorm parameter-update path. The symptom was that, on a ResNet-18 training loop, total Metal allocation grew by approximately 37 MB per iteration without bound, eventually causing OOM.

The root cause was an interaction between MLX’s lazy graph and Lucid’s `detach()` semantics. `detach()` on a parameter clears the autograd metadata (`grad_fn`, `requires_grad`) but does not, by itself, force the underlying MLX array to materialise. If a subsequent operation reads the detached tensor and the operation is itself lazy, the new graph node retains a reference to the old graph node. Across many iterations the lazy graph chain grows without bound and the underlying buffers cannot be freed.

The fix was to call `eval_tensors` on the BatchNorm running statistics at the end of each forward pass in training mode. This forces materialisation, breaks the lazy graph chain, and allows the allocator to release intermediate buffers. The discipline has since been generalised: any module that maintains an in-place running statistic (BatchNorm, RunningMean, EMA optimiser state) is responsible for evaluating that statistic at a known synchronisation point.

2.8.2 The InstanceNorm OOM (3.2.2)

A separate incident in 3.2.2 manifested as an OOM during InstanceNorm forward on inputs larger than approximately $(B, C, H, W) = (32, 64, 512, 512)$. The cause was that InstanceNorm’s forward kernel was materialising several broadcast-expanded intermediates — per-channel mean, per-channel variance, broadcast-back to (B, C, H, W) for the normalisation step — that each held a separate buffer.

The fix was algorithmic: rewrite the forward to fold the broadcast into the normalisation kernel directly, eliminating two of the four intermediate buffers. The result was a $4\times$ reduction in peak memory at the cost of a slightly more complex kernel. We treat this case in detail in Chapter 61, since it is one of several similar rewrites that improved Lucid’s normalisation family.

The general lesson is that, under unified memory, peak memory is often dominated by short-lived intermediates rather than long-lived state. Optimising peak therefore involves *kernel* restructuring rather than *allocator* tuning.

2.8.3 Cross-process sharing is not supported

A subtler pathology, occasionally raised by users: can two Lucid processes share a tensor through `multiprocessing.shared_memory` or via the macOS `IOSurface` API? The answer is no, on two grounds:

1. MLX’s allocator does not produce `MTLBuffers` with cross-process sharing flags set. The buffer’s underlying pages are accessible only to the owning process.
2. Even if cross-process `IOSurface`-backed buffers were created manually, Lucid’s autograd metadata is per-process; a tensor passed to another process would lose its `grad_fn`.

Multi-process training on a single machine would therefore require explicit serialisation (which costs the bridge cost discussed above) and is not currently a supported mode. Distributed training is out of scope for Lucid 3.x and is left to a future release.

2.9 Summary and Forward References

Unified memory on Apple silicon is a precisely specified contract: single physical pool, fabric coherence, software-visible barriers, no implicit CPU/GPU ordering. Lucid exploits this contract through the `SharedStorage` bridge to eliminate explicit data movement between the CPU and GPU streams, modulo a single contiguity requirement that the bridge enforces internally. The

macOS virtual memory subsystem adds residency states, compressed memory, and a three-level pressure signal that Lucid honours indirectly through the providers it depends on.

The chapters that follow build on this foundation. Chapter 3 treats MLX as the GPU-side runtime that holds most Lucid buffers in *Shared* mode and exposes them through `array::data<T>()`. Chapter 4 treats Accelerate as the CPU-side counterpart that consumes the same buffers through unit-stride CPU pointers. Chapter 7 formalises Lucid’s `Tensor` and `Storage` types and shows how the bridge slots into the tensor model. Finally, Chapter 10 enumerates the six places in Lucid where unified memory contacts the world outside the framework, and shows why those six are the only places that ever need to perform an explicit copy.

Note

The two operational levers a user has are `lucid.empty_cache()` (drain the allocator pool back to the provider) and the choice of `device='cpu'` vs. `device='metal'` on tensor creation. The first lever is non-destructive and useful when memory pressure rises. The second lever determines which provider’s pool the tensor occupies, but *not* whether the bytes themselves are shared — under unified memory, both providers see the same DRAM.

CHAPTER 3

MLX Runtime: Graph-Eager Hybrid

Lucid’s GPU backend is not implemented against Metal directly. It is implemented against MLX, Apple’s open-source machine-learning framework for Apple silicon, which Apple’s machine-learning research group released in late 2023 [15]. The choice to sit on top of MLX rather than to wrap Metal directly is the most consequential single architectural decision in Lucid: it determines the shape of the dispatch layer, the cost model the autograd engine must respect, the format of the intermediate representation in `lucid.compile`, and even the contents of the wheel that `pip install lucid` delivers (which carries an MLX dependency constraint of ≥ 0.31). This chapter explains what MLX is, how its execution model works, and how Lucid uses it.

The exposition is organised around four observations. First, MLX is a *lazy graph* runtime: ordinary operations build an unevaluated computation graph, and a separate `eval` call materialises results. This is unusual among contemporary ML frameworks and shapes nearly every consequence that follows. Second, MLX’s primitive set is intentionally closer to NumPy’s than to reference framework’s; some operations Lucid exposes are constructed in the Python composite layer (Chapter 20) rather than delegated to MLX. Third, MLX’s runtime exposes a small set of *streams* that abstract over CPU and GPU execution; Lucid’s device model uses these streams as its bedrock. Fourth, MLX has quirks — some inherited from NumPy, some specific to Apple silicon — that the framework above must guard against; we catalogue the two that have most affected Lucid’s correctness.

3.1 Overview

MLX is a research-driven framework with a small core team and a fast release cadence; the API surface that Lucid targets has been broadly stable since version 0.10, with consistent additions but few removals. The framework has the following high-level shape:

- A C++ core implementing the array primitive, the lazy graph, the function transformations, and the kernel set.
- A Python binding (`mlx.core`) that exposes the C++ surface as Python objects.
- Companion Python modules (`mlx.nn`, `mlx.optimizers`, `mlx.utils`) that build higher-level constructs on top of the core. Lucid does not use these companion modules; we build our own equivalents.
- Metal kernels (in MSL) compiled at install time or on first use, plus a small CPU fallback path used when the array is on the CPU stream.

The piece Lucid targets is the C++ core, accessed through the Python binding for most user-facing operations and through `lucid/_C/backend/gpu/MlxBackend.{h,cpp}` for engine-side operations. The Lucid C++ engine links against `libmlx.dylib` at build time and dispatches GPU work through MLX’s API.

3.1.1 What MLX is, what MLX is not

MLX is not a deep-learning framework in the same sense that `reference framework` is. It does not own a module class hierarchy, it does not own an autograd engine with arbitrary backward functions, it does not own an optimiser zoo. It is closer to a unified-memory analogue of JAX’s `jax.numpy` layer: an array library with function transformations (`vmap`, `grad`, `jit`) that the user composes manually.

This narrow scope is the property that makes MLX a good substrate for Lucid. We do not duplicate MLX’s array implementation; we delegate to it. We do not reimplement MLX’s kernels; we call them. But we add (i) a parameter-and-module abstraction that MLX lacks, (ii) an autograd engine that handles `reference framework`-compatible custom `Function` classes (Chapter 23), (iii) a dispatcher that selects between MLX, `Accelerate`, and `MPSGraph` depending on operation and device, (iv) an extensive model zoo, and (v) a graph-capture compiler (Chapter 43). The relationship is layered: Lucid on MLX, not Lucid alongside MLX.

3.1.2 Why MLX, not Metal directly

A reasonable alternative would have been for Lucid to wrap Metal directly, in the way `reference framework` wraps CUDA directly. Three considerations tipped the decision toward MLX:

1. **Kernel coverage.** MLX ships a mature library of Metal kernels for elementwise, reduction, indexing, GEMM, convolution, FFT, and reduction-with-gather operations. Writing these from scratch would have been a multi-year effort and would have produced kernels that converge on MLX’s anyway.
2. **Lazy graph as a free optimisation tier.** Because MLX is lazy, sequences of operations fuse opportunistically. Lucid benefits from these fusions without owning a separate optimiser. The trade-off is the synchronisation discipline of Section 3.2, which we accept.
3. **Maintenance bandwidth.** A two-engineer team cannot maintain a full Metal kernel library and a full ML framework. By leaning on MLX, Lucid spends its engineering budget on the layers above the kernel (autograd, modules, training, model zoo) where the user-visible features are.

The cost is a dependency we do not control: when MLX ships a breaking API change in a new minor release, Lucid must adapt. Section 3.9 discusses the discipline by which we manage this.

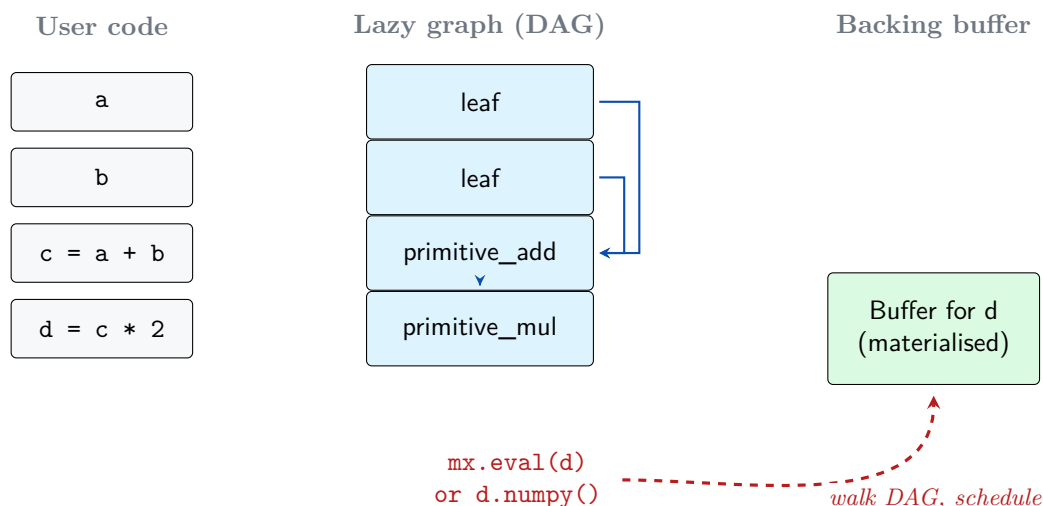


Figure 3.1: MLX’s graph-eager hybrid execution model. User code constructs arrays; each operation appends a primitive node to the lazy DAG without computing anything. The buffer materialises only when an explicit `eval` or implicit read forces evaluation, at which point MLX walks the DAG backward from the target, schedules primitives on the appropriate stream, and writes the result into a backing `MTL::Buffer`.

3.2 The Graph-Eager Hybrid Execution Model

The single most important property of MLX for the rest of this book is its execution model. MLX is *lazy*: an ordinary operation does not run when it is invoked. It is added to an internal computation graph, and a result placeholder is returned. The placeholder appears to the caller as an array, with shape and dtype metadata available immediately, but with no backing buffer. The backing buffer materialises only when *evaluation* is forced.

3.2.1 The lazy graph

Figure 3.1 sketches the model. Operations construct nodes in a DAG; the DAG is materialised by an explicit `eval` call (or by an implicit read of the bytes through, e.g., the bridge of Chapter 2).

When the user writes

```
1 import mlx.core as mx
2 a = mx.array([1.0, 2.0, 3.0])
3 b = mx.array([4.0, 5.0, 6.0])
4 c = a + b           # no computation happens here
5 d = c * 2           # still no computation
```

the variable `d` holds an array object whose shape is `(3,)`, whose dtype is `float32`, and whose underlying buffer does not yet exist. The graph held by MLX records: `c = primitive_add(a, b)`; `d = primitive_multiply(c, 2)`. Both operations are *deferred*.

The structure of this graph is a directed acyclic graph (DAG) of *primitives*, where each primitive is a leaf operation (an elementwise add, a matmul, a reduction) and each node has zero or more input arrays and one or more output arrays. An array is itself a tuple of (primitive that produced it, position in that primitive’s output list, shape, dtype, strides).

3.2.2 Evaluation triggers

Evaluation is forced by any of the following:

1. An explicit `mx.eval(d)` or `d.eval()` call.

2. A read of the array’s bytes through `d.data<T>()` (which is what `numpy.asarray`, `tolist`, and Lucid’s `SharedStorage::from_mlx` all do indirectly).
3. A print or repr operation.
4. An assignment into an `mx.array` via item assignment (`d[0] = 7.0`), which evaluates the array before in-place modification.

Each of these triggers MLX to walk the DAG backward from the target, schedule primitives onto the appropriate stream, and block until the result is materialised.

3.2.3 Why lazy by default

The lazy default is unusual; most ML frameworks (reference framework chief among them) are eager by default. The argument for laziness on Apple silicon is concrete:

- Apple silicon’s GPU has high kernel-launch overhead relative to its kernel cost, especially for short kernels. Fusing adjacent kernels eliminates launches that would otherwise dominate the total time.
- The unified memory model makes intermediates cheap to fuse away. There is no allocation cost to be paid for fusion across primitives.
- Lazy evaluation enables operator-level optimisations (constant folding, broadcast collapsing, transpose absorption) without a separate compiler tier.

The cost is that the framework above MLX must reason about when evaluation happens. Lucid has had two incidents in which a missed evaluation produced a memory leak (Chapter 2 Section 2.8.1) and one in which a deferred evaluation produced a write-after-read race in an in-place update. The fix in every case has been to insert a strategically placed `eval` at a synchronisation point.

3.2.4 Asynchronous evaluation

MLX added `async_eval` in version 0.10, which submits the graph for evaluation without blocking. The caller may continue to build the next portion of the graph while the GPU works on the submitted portion; subsequent eager reads block as usual.

Lucid uses `async_eval` in two places. First, the optimiser step (Chapter 39) evaluates the updated parameters asynchronously so that the next forward pass can begin construction in parallel. Second, the backward engine (Chapter 22) submits accumulated gradients asynchronously when their consumer is ready.

The benefit, on the training loop of a 100M-parameter Transformer, is roughly 5 ms per backward pass on an M3 Max — a small but real fraction of total step time. The fix landed in 3.3.0 as part of the AMP rollout; see Chapter 59.

3.3 The Array Primitive

The central data type in MLX is `mlx::core::array` (Python: `mlx.core.array`). It is the unit of every operation; it is what Lucid’s `TensorImpl` wraps (Chapter 7). We describe its structure and the interface that the rest of Lucid relies on.

3.3.1 Structure

Conceptually, an `mlx::core::array` is:

```
1 struct array {
2     Shape          shape;      // dimensions
3     Strides        strides;    // element strides per dimension
```



```

4  Dtype          dtype;          // element type
5  Primitive*     primitive;      // producer (null for leaf)
6  std::vector<array> inputs;    // upstream arrays
7  size_t         out_index;      // which output of primitive
8  std::shared_ptr<Buffer> buffer; // materialised storage, or null
9  // ... auxiliary fields for lazy evaluation
10 };

```

The fields that matter for Lucid:

- `shape`, `dtype` are always known and used by Lucid’s metadata layer to populate `Tensor.shape` and `Tensor.dtype` without forcing evaluation.
- `strides` encode lazy view operations (`transpose`, `slice`, non-trivial `reshape`) and must *not* be passed naively to a CPU consumer, because `data<T>()` returns the bytes in their pre-view layout. This is the source of the bridge contiguity discipline of Chapter 2.
- `buffer` is the materialised storage, null until evaluation. The pointer to its contents is what `data<T>()` returns.
- `primitive` and `inputs` define the DAG that `eval` walks.

3.3.2 Dtype and conversion

MLX supports the following dtypes:

- Floating point: `float16`, `float32`, `bfloat16` (since 0.5), `float64` (CPU stream only; not supported on GPU because Metal has no native FP64).
- Integer: `int8`, `int16`, `int32`, `int64`, and unsigned variants.
- `bool`, `complex64`.

Lucid exposes the same set, modulo two carve-outs. First, Lucid’s `float64` dtype is only valid on `device='cpu'`; an attempt to construct a `float64` tensor on `device='metal'` raises `LucidError("float64 not supported on Metal")`. Second, Lucid does not currently expose unsigned integer types in the top-level surface, because reference framework parity favours signed integers and the user demand for unsigned is small.

Conversion between dtypes uses MLX’s `astype` primitive, which is lazy like any other. A common pitfall (one that has burned Lucid in 3.0.3) is that `astype` followed by a CPU bridge must `eval` before the bridge, because `astype` is itself deferred; the bridge’s `contiguous().eval()` sequence does this correctly only if it sees `astype` as the head of the lazy chain.

3.3.3 The `data<T>` accessor

The most performance-critical method on `mlx::core::array` for Lucid is `data<T>()`. It returns a raw pointer to the buffer’s contents, typed as `T*`. The pointer is valid as long as the array is live (and, transitively, as long as any MLX array that shares storage with it is live). The pointer is to bytes in the array’s stored layout, which is *not* the same as the strided layout if the array has been produced by a lazy view operation.

The full discipline for using `data<T>()` from Lucid is:

1. Make the array contiguous via `mlx::core::contiguous(a)`.
2. Evaluate via `eval(a)`.
3. Access via `a.data<T>()`.
4. Hold the array (a `shared_ptr<array>`) for the lifetime of the CPU access.

The `SharedStorage` class (Chapter 2 Section 2.6.1) encapsulates this discipline. No Lucid code outside that class calls `data<T>()` directly.

3.4 Streams and Devices

MLX’s execution unit is a *stream*: an ordered queue of primitives that share a target device and execute in submission order. There are two streams in the default configuration: the CPU stream and the GPU stream. A primitive submitted to one stream executes on that stream’s device; primitives on different streams execute concurrently and synchronise only when an array’s bytes are read or when an explicit barrier is inserted.

3.4.1 The default device

MLX exposes a global default device through `mx.set_default_device()`. The default is the GPU on machines that have one (i.e., all Apple silicon), and the CPU on machines that do not. Lucid does not use the global default; the dispatcher (Chapter 9) explicitly selects the stream per operation based on the tensor’s `device` attribute. This ensures that Lucid’s behaviour is deterministic regardless of any global state MLX may carry.

3.4.2 Stream-aware primitives

Every MLX primitive accepts an optional `stream=` or `device=` argument that overrides the default for that operation. Lucid’s dispatcher uses this to enforce per-op device placement: a `cpu` tensor’s operation is dispatched with `stream=mx.cpu`, a `metal` tensor’s operation with `stream=mx.gpu`. The two streams may run concurrently, but Lucid’s autograd engine enforces a happens-before relation at the tensor boundary so that downstream operations see consistent state.

3.4.3 The linalg carve-out

There is one carve-out that is worth highlighting: linear-algebra primitives (`cholesky`, `qr`, `svd`, `eigh`, `solve`, `lstsq`) are implemented in MLX only on the CPU stream. Metal does not currently provide LAPACK-equivalent kernels; MLX uses Accelerate’s LAPACK under the hood, which is itself CPU-only.

Lucid’s `linalg` API therefore has an unusual property: even when the user has chosen `device='metal'`, a `lucid.linalg.cholesky` call dispatches on the CPU stream. The result is wrapped as a `metal-device` tensor only after the `linalg` primitive returns. The overhead is one `SharedStorage` bridge transfer per call, which is negligible relative to the cost of the decomposition itself for matrices larger than 64×64 . This is the *linalg carve-out* referenced throughout the book (Chapter 27, Chapter 9); it is the only operation in Lucid for which device placement is partly fictive.

3.5 The Primitive Set

MLX’s primitive set occupies a middle ground between NumPy and reference framework. The naming convention favours NumPy (`transpose`, not `permute`; `concatenate`, not `cat`; `stack` but not `vstack`); the broadcasting semantics are NumPy’s; the reduction signatures (`axis=`, `keepdims=`) are NumPy’s.

3.5.1 Elementwise and reduction

The elementwise set is comprehensive: arithmetic, comparison, logical and bitwise operations, transcendental functions (`exp`, `log`, `sin`, `cos`, `tanh`, `erf`, `erfinv`), rounding (`floor`, `ceil`, `round`, `trunc`). Lucid delegates nearly every elementwise primitive to MLX directly, exposing them with reference framework-style names through the dispatcher.

The reduction set covers `sum`, `mean`, `max`, `min`, `prod`, `var`, `std`, `any`, `all`, with per-axis and global variants. `argmax` and `argmin` are also primitives. `logsumexp` is a primitive since 0.18 (Lucid implemented it as a composite before that).

3.5.2 Shape manipulation

Reshape, transpose, broadcast-to, expand-dims, squeeze, swapaxes, flatten, ravel, split, concatenate, stack. The interesting subset is the lazy ones: `transpose` and `reshape` on most shapes return a non-materialised array with adjusted strides (the stride-trap discussed above). `concatenate` and `stack` necessarily allocate.

3.5.3 Indexing and gather/scatter

MLX supports advanced indexing through array indices (`a[idx]`) and through the `take` and `take_along_axis` primitives. The scatter family is split into two primitives with similar names but different semantics:

- `scatter_add(a, indices, updates)`: NumPy-style `np.add.at(a, indices, updates)` semantics. Multiple index lists, one per source dimension.
- `scatter_add_axis(a, axis, indices, updates)`: reference framework-style semantics. Single index list along one axis.

Passing one's indices to the wrong variant produces an immediate shape-mismatch failure ("Number of index arrays does not match number of dimensions"). Lucid's gather/scatter wrappers (Chapter 19) defensively use the correct variant based on the call shape; a vault note (`engine-mlx-scatter-axis-vs-multiaxis`) captures the diagnostic that surfaced this distinction the first time.

3.5.4 Linear algebra and FFT

The linalg primitives live in `mlx.core.linalg` and operate on the CPU stream as discussed in Section 3.4.3. The FFT primitives (`fft`, `ifft`, `rfft`, `irfft`, with their multi-dimensional variants) live in `mlx.core.fft` and run on both streams; Lucid's `lucid.fft` module (Chapter 28) wraps them with consistent normalisation conventions.

3.5.5 Convolution and neural-network primitives

`conv1d`, `conv2d`, `conv3d`, and their transposed counterparts are primitives. Pooling, batch-norm-style fused kernels, and attention's softmax fused with a scale are not first-class primitives but rather compositions in the `mlx.nn` module that Lucid does not use; Lucid's `nn.functional` layer (Chapter 34) calls the core conv primitives directly and composes the rest in its own kernels.

3.6 Function Transformations

MLX inherits from JAX a small set of function-level transformations that operate on Python functions of arrays and return new functions with new behaviours. The list is short: `grad`, `vmap`, `jit` (also called `mlx.compile`), `vjp`, `jvp`, and a few utilities.

Lucid does not, for the most part, expose these transformations to the user. The reason is parity: reference framework's user-facing API expresses differentiation through the `Module.backward` and `tensor.backward` pattern, not through a functional `grad`. Lucid therefore implements its own autograd engine (Chapter 22) on top of MLX's primitives, and MLX's `grad` sits unused.

The two transformations Lucid does use are `vmap` and `jit`, but only indirectly:

- `vmap` is exposed in Lucid as `lucid.func.vmap` (Chapter 25), where it composes onto Lucid tensors. The implementation delegates to MLX’s `vmap` on the underlying arrays.
- `jit` (i.e., `mlx.compile`) is *not* the mechanism behind `lucid.compile`. Lucid’s compile path (Chapter 43) emits onto `MPSGraph` rather than MLX’s compile; the reasons are detailed in that chapter.

3.7 Lucid’s Use of MLX

We can now describe how Lucid sits on top of MLX. The integration has three layers.

3.7.1 The MlxBackend interface

`MlxBackend` (in `lucid/_C/backend/gpu/`) is the C++ class that mediates between Lucid’s dispatcher and MLX. Its interface is roughly:

```

1  class MlxBackend : public IBackend {
2  public:
3      TensorImpl allocate(Shape, Dtype) override;
4      TensorImpl unary(OpKind, const TensorImpl& x) override;
5      TensorImpl binary(OpKind, const TensorImpl& a, const TensorImpl&
6          b) override;
7      TensorImpl reduce(OpKind, const TensorImpl& x, AxisSet, bool
8          keepdims) override;
9      TensorImpl matmul(const TensorImpl& a, const TensorImpl& b)
10         override;
11     // ... and so on for every primitive category
12 };

```

Each method translates a Lucid operation request into the corresponding MLX primitive call, with the `stream` argument fixed to `mx.gpu`. The return value is a new `TensorImpl` that wraps the MLX array returned by the primitive.

3.7.2 Dispatch into MLX from the engine

The dispatcher (Chapter 9) routes a per-op request to one of the backends based on the tensor’s device. For `device='metal'`, the route is `MlxBackend` unconditionally, with one exception (the `MPSGraph` per-op carve-out introduced in 3.4; Chapter 60). For `device='cpu'`, the route is `AccelerateBackend` *except* for the `linalg` carve-out, where it is also `MlxBackend` but with `stream=mx.cpu`.

The Python-side dispatcher (`lucid/_dispatch.py`) is symmetric but operates on Python `Tensor` objects. It performs the `_wrap` / `_unwrap` between `Tensor` and `TensorImpl`, then defers to the C++ dispatcher for the actual op routing.

3.7.3 Bridge from MLX to Lucid Tensor

The final layer is the bridge from `mlx::core::array` to Lucid’s `TensorImpl`. The bridge wraps the array in a `TensorImpl` that holds the array as a `shared_ptr`, inherits the array’s shape and dtype, and computes the tensor’s strides from the array’s strides. Autograd metadata (`requires_grad`, `grad_fn`) is allocated separately because MLX has no notion of these.

When the inverse direction is needed — a `TensorImpl` that came from elsewhere needs to be passed to an MLX primitive — the bridge unwraps the `shared_ptr<array>` that the `TensorImpl` holds. This is always free; the array was constructed on the MLX side originally.

3.8 Quirks and Pitfalls

Two MLX behaviours have been the source of bugs in Lucid’s history; both are documented in the engine vault, both have shaped the framework’s defensive coding.

3.8.1 `data<T>` ignores strides

The first quirk has been discussed in detail in Chapter 2: `mlx::core::array::data<T>()` returns a pointer to bytes in the array’s stored layout, not the strided layout. A lazy transpose of a 4×8 array of dtype `float32` returns an 8×4 array whose `data<T>()` pointer points to the original 4×8 contiguous block.

The discipline is enforced by `SharedStorage` and is non-negotiable: `contiguous` then `eval` then `data<T>()`. The vault note `engine-mlx-data-ignores-strides` is the source-of-truth for the regression that surfaced this in 3.0 series.

3.8.2 Two scatter variants

The second quirk is the existence of two primitives with similar names and different semantics: `scatter_add` (NumPy-style multi-axis index arrays) and `scatter_add_axis` (reference framework-style single-axis indices along one axis). A Lucid wrapper that picks the wrong variant produces a runtime error of the form “Number of index arrays does not match number of dimensions of input”. The fix in 3.0 series was to standardise on `scatter_add_axis` for the reference framework-compatible API and to use `scatter_add` only in NumPy-bridge code paths. The vault note `engine-mlx-scatter-axis-vs-multiaxis` captures the investigation.

3.8.3 Axis vs. axes nomenclature

MLX uses `axis=` (singular) for some reductions and `axes=` (plural) for others. The convention follows NumPy: operations that accept exactly one axis use `axis=`; operations that accept a tuple of axes (or a single axis as a tuple) use `axes=`. Lucid standardised on `dim=` and `dims=` internally for reference framework parity, and the dispatcher rewrites between conventions on each call. The Phase 7 engine rename retro (`retro-axis-to-dim-rename`) records the design.

3.9 Version Compatibility and the Wheel Constraint

MLX is a young framework on a fast cadence. Lucid pins a minimum version (`mlx >= 0.31`) and tracks releases as they ship. The constraint discipline has three parts.

3.9.1 API stability per minor version

MLX’s minor releases (0.18, 0.20, etc.) sometimes introduce breaking changes. The standard pattern is that MLX deprecates an API in version N and removes it in version $N + 2$. Lucid’s build pins the minimum supported MLX version conservatively and the maximum loosely. The wheel that `pip install lucid` fetches declares `mlx>=0.31,<1.0`, which gives MLX room to ship minor releases that Lucid has not been tested against.

3.9.2 Platform constraint: `macOS ≥ 26.0`

The wheel constraint is tied to MLX’s own wheel constraint. As of MLX 0.31, the published wheel is tagged `macosx_26_0_arm64`, which means it requires macOS 26.0 (Tahoe) and the arm64 architecture. There is no Linux wheel; there is no Intel-Mac wheel; there is no older-macOS wheel. Lucid inherits these constraints and matches them with its own build target (`MACOSX_DEPLOYMENT_TARGET=26.0`).

The `mlx-metal` split package (introduced in MLX 0.20) separates the Metal kernel binaries from the core Python package to reduce wheel size. Lucid’s build links against the combined package and does not need to know about the split.

3.9.3 Per-version conditional code

A small number of Lucid code paths are guarded by MLX version checks. The pattern is:

```

1 from packaging.version import Version
2 import mlx.core as mx
3
4 if Version(mx.__version__) >= Version("0.18"):
5     out = mx.logsumexp(x, axis=axis)
6 else:
7     out = _composite_logsumexp(x, axis=axis)

```

These conditional paths are exceptional; the policy is to delete the fallback once the minimum supported MLX version exceeds the introduction point. Lucid’s current minimum (0.31) elides roughly half a dozen such fallbacks that were present in earlier Lucid releases.

3.10 Summary and Forward References

MLX is the GPU-side substrate on which Lucid is built. Its lazy graph model gives Lucid a free fusion tier and forces a synchronisation discipline that the framework above must respect. Its array primitive is the storage type that Lucid’s `Tensor` wraps. Its stream model is the basis for Lucid’s device routing, with the one carve-out that linear algebra is dispatched on the CPU stream regardless of user device choice. Its quirks (`data<T>()` stride-ignorance, two scatter variants, axis nomenclature drift) have been the source of historical bugs that have shaped Lucid’s defensive layers.

The next chapter, Chapter 4, takes the CPU-side counterpart to MLX— the `Accelerate` framework — and describes how its BLAS, LAPACK, vDSP, vForce, and BNNS sublayers compose into the `AccelerateBackend` that Lucid’s dispatcher targets when the user has chosen `device='cpu'`. Chapter 5 then closes Part I by treating `Metal` and `MPSGraph` directly, since both are needed for the graph-capture compiler. After that, the book moves to the software architecture proper.

Note

For day-to-day use, the relevant MLX facts are: (1) MLX is lazy, so `eval` is the synchronisation primitive; (2) MLX’s array is the storage layer Lucid delegates to; (3) device placement in Lucid corresponds to stream selection in MLX, with the `linalg` carve-out. The remaining detail in this chapter is required when debugging the bridge boundary or extending the dispatcher.

CHAPTER 4

Apple Accelerate: BLAS, LAPACK, vDSP, vForce, BNNS

The previous chapter described MLX, the GPU-side substrate on which Lucid depends. This chapter describes its CPU-side counterpart: `Accelerate`, Apple’s umbrella framework for high-performance numerical computing on macOS and iOS [2]. `Accelerate` has shipped with every release of macOS since 10.3 (Panther, 2003); it is a stable, documented, and unavoidable dependency for any framework that wants to do serious CPU numerics on Apple hardware. Lucid’s CPU backend is built entirely on `Accelerate`; no other CPU numerics library is linked into the engine, and no Lucid CPU code path hand-rolls a SIMD kernel that `Accelerate` already provides.

The choice is not purely pragmatic. `Accelerate` is the only way to access the AMX coprocessor (Chapter 1 Section 1.4) without resorting to reverse-engineered instruction encodings, and the AMX path is the difference between roughly 128 GFLOP/s of NEON throughput per P-core and roughly 2 TFLOP/s of cluster-shared throughput on matrix multiplication. To bypass `Accelerate` would be to give up that ten-fold gap. This chapter explains the sublayers of `Accelerate`, how Lucid uses each, and what compatibility considerations the framework has accumulated from the SDK churn of the 2020–2024 period.

4.1 Overview

`Accelerate` is not a single library but an umbrella of related libraries, each presented to client code as a top-level header but backed by a shared dynamic library. The relevant sublayers, in the order they were introduced, are:

Sublayer	Introduced	Path to silicon	Lucid usage	File
BLAS	10.3 (2003)	AMX coprocessor	GEMM, axpy, dot	<code>AccelerateMath.cpp</code>
LAPACK	10.3 (2003)	BLAS + LAPACK	cholesky, qr, svd	<code>AccelerateMath.cpp</code>
vDSP	10.0 (PowerPC)	NEON SIMD	elementwise, reductions	<code>AccelerateMath.cpp</code>
vForce	10.4 (2005)	SIMD + lookup	exp, log, sin, erf	<code>AccelerateMath.cpp</code>
BNNS	10.13 (2017)	AMX / SME	conv2d, pooling	<code>AccelerateConv.cpp</code>
vImage	10.3 (2003)	SIMD + custom	<i>not used</i>	—
SparseSolvers	10.13	sparse BLAS	<i>not used</i>	—

Table 4.1: The seven sublayers of Apple’s `Accelerate` framework, by introduction date, the hardware path each ultimately reaches, the operations `Lucid` delegates to each, and the `Lucid` source file that wraps it. The two unused sublayers (`vImage`, `SparseSolvers`) are listed for completeness.

- **BLAS** (Basic Linear Algebra Subprograms): the Fortran-derived dense linear algebra interface, with Apple’s implementation since macOS 10.3.
- **LAPACK** (Linear Algebra Package): the higher-level decomposition and solver suite, also shipped since 10.3.
- **vDSP** (Vector Digital Signal Processing): per-element vector operations (arithmetic, comparisons, packing/unpacking, FFT). Predates BLAS in the original Velocity Engine framework on PowerPC.
- **vForce**: transcendental and special functions on vectors. A separate header for historical reasons (`vForce` was originally a separate library; the symbols now live in `libvecLib.dylib` like the rest of `Accelerate`).
- **vImage**: image-processing primitives. Not used by `Lucid`; mentioned for completeness.
- **BNNS** (Basic Neural Network Subroutines): a higher-level convolution and pooling library introduced in macOS 10.13. The first `Accelerate` sublayer with an explicit ML focus.
- **SparseSolvers**: sparse linear algebra. Not used by `Lucid`; sparse tensors are out of scope for `Lucid 3.x`.

The four sublayers `Lucid` actually uses are BLAS, LAPACK, vDSP, vForce, and BNNS. We treat each in turn. Table 4.1 gives a one-page overview before we dive into each.

4.1.1 Linkage and headers

`Accelerate` is exposed to C and C++ code through the umbrella header `<Accelerate/Accelerate.h>`, which transitively includes `<vecLib/vecLib.h>`, `<vecLib/cblas.h>`, `<vecLib/clapack.h>`, and so on. The umbrella links the dynamic library `Accelerate.framework`, which is part of the macOS base system and present in every install.

`Lucid`’s CMake build adds `Accelerate.framework` to the link line for the engine target. There is no separate find-package step because the framework is in the standard SDK location and `find_library(ACCELERATE_FRAMEWORK Accelerate)` resolves it unconditionally on Apple silicon.

4.1.2 Two SDK transitions

Two SDK transitions in the 2022–2024 period have affected the `Lucid` build:

1. **ACCELERATE_NEW_LAPACK**. In macOS 13.3, Apple introduced a new LAPACK ABI that uses 64-bit integers for matrix dimensions, replacing the legacy 32-bit ABI inherited from netlib LAPACK. To opt into the new ABI, client code must define `ACCELERATE_NEW_LAPACK=1` and `ACCELERATE_LAPACK_ILP64=1` before including the headers.

Lucid adopted the new ABI in version 3.0.2 as part of the numpy-independence pass (vault: `retro-3-0-2-numpy-free-standalone`), because the new ABI is the only one that interoperates correctly with large matrices.

2. **BNNS deprecation warnings.** Starting with macOS 14, several BNNS functions are marked `API_DEPRECATED` in the public headers, on the grounds that CoreML provides equivalent or higher-level functionality. The functions still work and are still the fastest CPU convolution path; Lucid silences the warning by passing `-Wno-deprecated-declarations` only to the file that includes the BNNS header, not framework-wide. The discipline is documented in `arch-device-bifurcation-complete`.

4.2 BLAS and LAPACK

BLAS and LAPACK are the workhorses of Lucid’s CPU linear algebra. Together they cover matrix multiplication, factorisations (Cholesky, QR, SVD, eigendecomposition), and linear-system solvers. Lucid uses them through the CBLAS and CLAPACK C-language interfaces, which avoid the Fortran calling convention but preserve the underlying column-major storage convention.

4.2.1 The CBLAS interface

CBLAS exposes the BLAS routines as C functions with a layout argument that selects between `CblasRowMajor` and `CblasColMajor`. Lucid’s CPU backend uses `CblasRowMajor` throughout, matching Lucid’s tensor storage convention. The function pattern is:

```

1  #include <Accelerate/Accelerate.h>
2
3  // C := alpha * A * B + beta * C
4  cblas_sgemm(
5      CblasRowMajor,
6      CblasNoTrans, CblasNoTrans,
7      M, N, K,           // dims
8      alpha,
9      A_ptr, K,          // lda = stride of leading dimension
10     B_ptr, N,           // ldb
11     beta,
12     C_ptr, N);          // ldc

```

The `cblas_sgemm` call, with operands large enough to amortise dispatch overhead, executes on the AMX coprocessor and achieves the throughput characterised in Chapter 1 Section 1.4.2. For operands below roughly 64×64 , the AMX path is not selected (the dispatch overhead exceeds the win), and the call runs through NEON; for very small operands, scalar code in Lucid’s composite layer is sometimes faster still.

The double-precision counterpart is `cblas_dgemm`; Lucid exposes it but does not exercise it on the GPU path, because Metal has no FP64 support and a mixed-device FP64 GEMM is rare in practice.

4.2.2 LAPACK routines used by Lucid

Lucid’s `linalg` subpackage (Chapter 27) wraps the following LAPACK routines, in their CLAPACK form with the new ILP64 ABI:

- `cholesky` $\rightarrow$ `spotrf` / `dpotrf` (Cholesky factorisation, lower-triangular).
- `qr` $\rightarrow$ `sgeqrf` + `sorgqr` (QR factorisation followed by Q reconstruction).
- `svd` $\rightarrow$ `sgesdd` (divide-and-conquer SVD; asymptotically faster than `sgesvd`).

- `eigh` → `ssyevd` (divide-and-conquer symmetric eigendecomposition).
- `solve` → `sgesv` (general LU-based solve).
- `lstsq` → `sgelsd` (divide-and-conquer least-squares).
- `inv` → `sgetrf` + `sgetri` (LU followed by explicit inverse construction).
- `det`, `logdet` → `sgetrf` followed by Python-side product of diagonal entries.

The choice of divide-and-conquer variants (`sgesdd`, `ssyevd`, `sgelsd`) over the simpler QR-based variants is deliberate: for matrices larger than a few hundred entries on each side, divide-and-conquer is significantly faster and Apple’s implementation in Accelerate’s LAPACK is competitive with the best open-source LAPACK builds.

4.2.3 The linalg carve-out, revisited

Chapter 3 Section 3.4.3 introduced the linalg carve-out: even when the user has selected `device='metal'`, linear-algebra primitives dispatch on the CPU stream, because Metal has no LAPACK-equivalent kernels.

The mechanism is now visible from this chapter’s perspective. MLX’s CPU stream uses, internally, the same Accelerate BLAS and LAPACK that Lucid’s CPU backend uses directly. When Lucid dispatches a `cholesky` on a `metal` tensor, the path is:

1. Lucid moves the tensor through `SharedStorage` to a CPU view (this is the bridge of Chapter 2).
2. Lucid calls `cblas_sgemm`-style Accelerate LAPACK on the CPU view. Under the hood, Accelerate uses the AMX coprocessor for BLAS-3 portions and CPU code for the rest.
3. The result, residing in unified memory, is wrapped as an MLX array and returned as a `metal-device` tensor.

The bridge is free in the contiguous case (Chapter 2 Section 2.6.3); the only real cost is the LAPACK computation itself, which is several orders of magnitude larger than the bridge overhead for any matrix worth decomposing.

4.3 vDSP

vDSP is the lowest layer of Accelerate that Lucid uses. It provides per-element vector operations on contiguous arrays: scalar multiplication and addition, dot products, element-wise arithmetic, sums, means, and a small set of packing and conversion utilities. The primitives are mostly C functions with a strict naming convention: `vDSP_<op>` for single-precision, `vDSP_<op>D` for double-precision.

4.3.1 Primitives used by Lucid

The CPU backend’s elementwise kernels are thin wrappers over vDSP primitives. The most-used:

- `vDSP_vmul` / `vDSP_vadd` / `vDSP_vsub` / `vDSP_vdiv`: elementwise binary arithmetic on single-precision vectors.
- `vDSP_vsadd` / `vDSP_vsmul` / `vDSP_vsddiv`: scalar-broadcast variants.
- `vDSP_vneg`, `vDSP_vabs`, `vDSP_vsq`: unary arithmetic.
- `vDSP_dotpr`, `vDSP_sve` (sum), `vDSP_meanv` (mean), `vDSP_maxv`, `vDSP_minv`: reductions.
- `vDSP_vthr` (threshold), `vDSP_vclip` (clamp).

- `vDSP_vfix32` (float-to-int conversion), `vDSP_vflt32` (int-to-float).

The full list Lucid exposes is in `lucid/_C/backend/cpu/AccelerateMath.cpp`, where each is wrapped behind a function with reference framework-compatible naming.

4.3.2 Stride support

vDSP primitives accept explicit stride arguments (the `N` and `stride` pair after each pointer), which means that they can operate on non-contiguous slices of a larger buffer without first copying to contiguity. Lucid’s CPU kernels exploit this for views that are unit-stride along the innermost dimension and arbitrarily strided along the outer dimensions; the dispatcher emits a vDSP call per outer-axis iteration.

For views that are non-unit-stride along the innermost dimension (produced by certain transposes), Lucid’s CPU kernels first materialise a contiguous copy. The trade-off is amortised by the fact that such views are rare in practice and the contiguous copy cost is small relative to the subsequent kernel.

4.3.3 Why not OpenBLAS or MKL

The natural alternative to vDSP would have been OpenBLAS, BLIS, or the Intel MKL. All three are excellent libraries, but none of them target AMX on Apple silicon. OpenBLAS on M-series uses NEON; BLIS uses NEON; MKL does not run on ARM at all. Bypassing vDSP would therefore have meant giving up the AMX path, which is the dominant single performance source for CPU-side BLAS-3 on Apple silicon.

We have also benchmarked the OpenBLAS NEON path against Accelerate vDSP plus AMX on representative GEMM sizes and Accelerate is faster by a factor of 3–5× on 256×256 FP32 and larger. The conclusion is conclusive.

4.4 vForce

vForce provides vectorised transcendental and special functions. The interface convention matches vDSP but the functions are mathematically richer:

- `vvexpf`, `vvlogf`, `vvsinf`, `vvcosf`, `vvtanf` and their double-precision `vv*` counterparts.
- `vvtanhf`, `vvsinhf`, `vvcoshf`.
- `vvsqrtf`, `vvrsqrtf`.
- `vverff`, `vverfinvf`.

Lucid’s CPU backend uses vForce wherever a primitive maps onto one of these; the wrappers are in `lucid/_C/backend/cpu/AccelerateMath.cpp` alongside the vDSP wrappers. The win over a scalar transcendental loop is roughly 4–6× on FP32 `exp` and `log`, falling to 2–3× on the more complex `erf` family.

A practical caveat: vForce’s `vverfinvf` produces results that differ in the last few ULPs from the scalar `erfinvf` that ships in `libm.dylib`. Lucid’s gradcheck tolerance for `erfinv`-using ops is relaxed accordingly (Chapter 26).

4.5 BNNS

BNNS is the highest level of Accelerate that Lucid uses, and the most controversial. It is a convolution and neural-network primitive library introduced in macOS 10.13 (2017), at a time when ML inference on CPU was a more common need than it is now. Its API is shaped around the standard NN-inference vocabulary: tensors with explicit layout descriptors, convolution

and pooling descriptors, fully-connected layers, activation functions, and a small set of loss-derivative primitives.

4.5.1 The convolution fast path

The single most consequential BNNS function for Lucid is `BNNSDirectApplyConvolutionLayer` (and its descriptor constructor `BNNSDirectFilterCreateConvolutionLayer`). On M-series hardware, this function dispatches the convolution kernel through the AMX coprocessor, achieving throughput unattainable through pure NEON code. For a 3×3 stride-1 padded convolution on a 224×224 input with 64 output channels and batch size 1, BNNS is roughly $8\times$ faster than a hand-written NEON convolution loop and $3\times$ faster than calling `cblas_sgemm` on an `im2col`-flattened version.

Lucid’s CPU convolution kernels (`lucid/_C/backend/cpu/AccelerateConv.cpp`) delegate to BNNS for the forward pass of `conv1d`, `conv2d`, and `conv3d`, with descriptor caching so that repeated applications of the same layer to inputs of identical shape do not re-pay the descriptor-construction cost.

4.5.2 The deprecation warning

Starting with macOS 14, several BNNS functions are marked `API_DEPRECATED` in the public headers. Apple’s stated reason is that CoreML provides equivalent or superior functionality at a higher abstraction level. The deprecation does not affect the runtime: the functions still work, still perform identically, and are still part of the system framework. Only the compile-time warning is generated.

Lucid silences the warning narrowly, in the single file that includes the BNNS header:

```
1 // In AccelerateConv.cpp
2 #pragma clang diagnostic push
3 #pragma clang diagnostic ignored "-Wdeprecated-declarations"
4 #include <Accelerate/Accelerate.h>
5 #pragma clang diagnostic pop
```

The framework-wide `-Wno-deprecated-declarations` flag is explicitly *not* used, because suppressing deprecation warnings globally would mask genuine bug-class issues elsewhere. The narrow scope is documented in `arch-device-bifurcation-complete`.

4.5.3 Why Lucid still uses BNNS

Three reasons keep BNNS in Lucid’s CPU backend:

1. It is the only public Apple-supported path to CPU-side matrix-coprocessor convolution. The alternative is reverse-engineered AMX assembly, which is unsupportable.
2. The deprecation has not been followed by a removal. Apple has marked the functions deprecated since macOS 14 without removing them in macOS 15 or 26; we estimate the removal is several years away, at which point SME-based replacements will have matured.
3. CoreML, the recommended replacement, requires offline graph compilation. Lucid’s CPU backend dispatches dynamically and cannot accept the offline-compilation requirement without losing the dynamic-shape support that the rest of the framework is built on.

The plan is to retain BNNS until either (a) Apple removes the functions and forces a migration, or (b) SME-based direct emission from `lucid.compile` (Chapter 43) reaches parity with BNNS throughput, whichever happens first.

4.6 AccelerateBackend in Lucid

The C++ class that mediates between Lucid’s dispatcher and Accelerate is `AccelerateBackend` (`lucid/_C/backend/cpu/AccelerateBackend.{h,cpp}`). It implements the same `IBackend` interface as `MlxBackend` (Chapter 3 Section 3.7.1), which is what allows the dispatcher to route operations to either backend uniformly.

4.6.1 File layout

The backend is split across several files to manage the size of the implementation:

- `AccelerateBackend.{h,cpp}`: the `IBackend` implementation; dispatches per-op to the helpers below.
- `AccelerateMath.cpp`: vDSP and vForce wrappers for elementwise and reduction operations.
- `AccelerateConv.cpp`: BNNS wrappers for convolution forward; backward is composed from BLAS GEMM on im2col-flattened gradient tensors.
- `AccelerateBatchNorm.cpp`, `AccelerateLayerNorm.cpp`, `AccelerateInstanceNorm.cpp`, `AccelerateGroupNorm.cpp`: normalisation kernels that fold mean / variance computation with the normalising affine in a single pass.
- `AccelerateRandom.cpp`: vDSP’s random number generator wrapped for Lucid’s `Generator` interface.
- `Helpers.cpp`: tensor-to-vDSP shape and stride conversions, descriptor caching, layout adapters.

The split is the result of incremental growth across multiple phases; the natural unit of factoring is the underlying Accelerate sublayer, not the abstract op kind, because each sublayer has its own conventions and quirks.

4.6.2 Per-op routing inside the backend

Inside `AccelerateBackend`, the per-op routing is a switch on the abstract op kind:

```

1  TensorImpl AccelerateBackend::binary(
2      OpKind op, const TensorImpl& a, const TensorImpl& b) {
3      switch (op) {
4          case OpKind::Add: return accel::vmul_vadd_path(a, b, /*op=*/Add);
5          case OpKind::Mul: return accel::vmul_vadd_path(a, b, /*op=*/Mul);
6          case OpKind::Pow: return accel::pow_path(a, b);
7          // ... and so on
8      }
9  }
```

The helpers in the `accel` namespace are the actual vDSP / vForce wrappers; the backend class itself contains only routing. This structure mirrors `MlxBackend`’s and is what allows the two backends to be substitutable through the same dispatcher.

4.7 Threading and Concurrency

Accelerate is internally multithreaded for large operations. The threading is managed by Grand Central Dispatch (GCD), the macOS system-wide task scheduler. Client code does not configure thread counts directly; instead, GCD allocates worker threads from its shared pool based on system load and the operation’s estimated cost.

4.7.1 The implicit thread pool

This implicit pooling has two consequences for Lucid:

1. There is no `omp_set_num_threads`-equivalent that the user can set. The number of threads Accelerate uses for a given operation is determined at runtime by GCD.
2. Operations from Accelerate can compete with Lucid's own Python-level threads (e.g. data-loader workers) for CPU cycles. The shared GCD pool prevents pathological over-subscription, but there is no explicit isolation between domains.

In practice the scheduling is sensible: training workloads that dispatch from a single Python thread to Accelerate get near-optimal P-core utilisation, and data-loading workers correctly yield to the training thread under load. Lucid does not set explicit affinity hints (Chapter 1 Section 1.3.3) because the default behaviour is correct.

4.7.2 Lock-free reentrancy

Accelerate's functions are reentrant: it is safe for two threads to call `cblas_sgemm` simultaneously on disjoint inputs. Lucid's autograd engine relies on this to dispatch independent operations from the optimiser step in parallel with the next forward pass's preparation (Chapter 39).

4.8 Quirks and Pitfalls

We close with the three classes of issues that have surfaced during Lucid's development.

4.8.1 LAPACK error returns

LAPACK routines do not raise exceptions. They return an integer status code through the `info` out-parameter; a negative value indicates an invalid argument, a positive value indicates a numerical failure (singular matrix in `getrf`, non-positive-definite in `potrf`, non-convergence in `gesdd`).

Lucid's wrappers check `info` after every call and translate positive values into `LucidError("cholesky failed: matrix not positive definite at row k")`-style messages. Negative `info` values indicate a bug in the wrapper and are translated into internal assertions, since they should be unreachable.

4.8.2 Row-major vs. column-major

The native LAPACK convention is column-major; CLAPACK's `LAPACKE_*` wrappers accept a layout argument to allow row-major input. Lucid uses the CLAPACK `LAPACKE_*` variants throughout, with `LAPACK_ROW_MAJOR`. The conversion overhead is internal to LAPACKE and is amortised over the decomposition cost.

A historical bug (3.0.3) involved calling the older non-LAPACKE variant of `spotrf` and forgetting to transpose the input; the result was a successful-looking Cholesky of the matrix's transpose. The fix was to standardise on the LAPACKE wrappers throughout, and the standardisation is now enforced by a code-review checklist referenced in the `retro-3-0-3-perf-pass` note.

4.8.3 SDK version drift

Accelerate's API surface is largely stable, but individual functions have been added, deprecated, or had their behaviour clarified across SDK versions. The pattern that has caused the most churn for Lucid is the LAPACK ILP64 ABI transition discussed in Section 4.1.2. The discipline now is:

1. Pin a minimum macOS deployment target (Lucid uses `MACOSX_DEPLOYMENT_TARGET=26.0`, matching MLX's wheel).
2. Use only documented APIs that have been stable across the minimum-target SDK and the current SDK.
3. Treat any deprecation warning that surfaces during build as a TODO to either migrate to the suggested replacement or, if the replacement does not exist (BNNS being the canonical example), silence narrowly at the include site.

The minimum-target choice is shared with MLX because the two dependencies' wheel tags must agree; Chapter 3 Section 3.9.2 documents the constraint from MLX's side.

4.9 Summary and Forward References

Accelerate is the CPU-side substrate on which Lucid stands. Its sublayers — BLAS, LAPACK, vDSP, vForce, BNNS — jointly provide matrix multiplication, decompositions, elementwise and transcendental kernels, and convolution. The framework is the only public Apple-supported path to the AMX coprocessor and is therefore the only path that achieves competitive CPU-side ML throughput on Apple silicon. Lucid's `AccelerateBackend` wraps the relevant subset behind an `IBackend` interface that the dispatcher uses uniformly with `MlxBackend`.

The remaining hardware-foundation chapter, Chapter 5, treats Metal and MPSGraph directly, since Lucid's graph-capture compiler emits onto MPSGraph rather than relying on MLX's compile path. After that, the book moves into Part II, which formalises the layered architecture the hardware chapters have implicitly sketched and explains how Lucid's tensor model, allocator, dispatch, and bridge boundaries fit together at the engine level.

Tip

For the user, the visible consequence of this chapter is that `device='cpu'` in Lucid corresponds to AMX-accelerated BLAS and BNNS through Accelerate, not to a generic CPU fallback. This makes CPU-only training feasible for small models and certain non-convolutional workloads where the GPU's launch overhead would dominate. For training-time benchmarks see Chapter 57.

CHAPTER 5

Metal and MPSGraph Integration Surface

The previous two chapters described the libraries that Lucid uses to talk to the GPU and the CPU: MLX for GPU dispatch and Accelerate for CPU dispatch. Below both libraries is the operating system’s own graphics-and-compute substrate, **Metal**, and above **Metal** sits another Apple-supplied tier, the *Metal Performance Shaders Graph* framework (MPSGraph). Lucid touches **Metal** indirectly through MLX and touches MPSGraph directly through the graph-capture compiler (Chapter 43). Both contact points are limited in scope but consequential: the compile path is the largest single performance feature in Lucid 3.5, and the per-op MPSGraph dispatch added in 3.4 (Chapter 60) is responsible for the most visible single-operator speedups on small Transformer training workloads.

This chapter explains what **Metal** and MPSGraph are at a level of detail sufficient to read the rest of the book. We do not attempt to duplicate Apple’s Metal Programming Guide; the goal is to give the ML-framework reader a working mental model of (i) the programming abstractions that the operating system exposes, (ii) the cost model those abstractions imply, (iii) the choice that exists between direct **Metal** kernels, MPSGraph graphs, and the higher-level MLX runtime that sits on both, and (iv) the specific contracts Lucid has chosen at each layer.

5.1 Overview

Metal [4] is the operating-system-level graphics and compute API on all of Apple’s platforms (macOS, iOS, iPadOS, tvOS, visionOS). It was introduced on iOS in 2014 and on macOS in 2015 as a replacement for OpenGL and OpenCL. Every GPU computation on Apple silicon, no matter what library issued it, ultimately becomes a sequence of **Metal** commands submitted

to a Metal command queue, dispatched to the GPU through Apple’s display server and the kernel-level graphics driver.

MPSGraph [5] is a higher-level framework built on top of Metal, introduced in macOS 12 (2021). It exposes a graph IR for compute operations and compiles graphs into multi-kernel Metal executables, with automatic kernel fusion, layout selection, and (since macOS 13) gradient construction. MPSGraph is the spiritual successor to MetalPerformanceShaders (MPS), an earlier framework that provided pre-canned per-op kernels without graph-level optimisation. MPS still exists and MPSGraph uses some of its primitives internally, but for new code Apple recommends MPSGraph.

Lucid’s relationship to these layers is layered:

- Lucid does not call Metal directly. Every GPU command Lucid ever issues passes through either MLX (the eager runtime, Chapter 3) or MPSGraph (the compile path).
- Lucid uses MPSGraph in two places: `lucid.compile` (Chapter 43) emits compiled graphs onto MPSGraph, and a small set of per-op fast paths in 3.4 (Chapter 60) dispatches selected operators through MPSGraph even in eager mode.
- Lucid does not implement custom Metal shaders. The few cases that would have benefited from one — for example, a fused softmax-attention kernel — have been deferred in favour of either MLX’s own optimised path or MPSGraph emission.

This minimalism is deliberate. Direct Metal programming is labour-intensive (one writes shader code in MSL, manages command buffer lifetimes, handles synchronisation by hand), and the ML-relevant kernels have already been written — once in MLX and once in MPSGraph. Lucid’s engineering budget is better spent on the layers above.

5.2 Metal Fundamentals

This section describes Metal’s compute model at the level a framework writer must understand. We omit graphics rendering (vertex shaders, fragment shaders, render targets) entirely; Lucid never touches that path.

5.2.1 The device and its command queue

The root object of every Metal program is the `MTLDevice`, a software handle on a single GPU. On Apple silicon there is exactly one GPU per machine, accessible via the function `MTLCreateSystemDefaultDevice()`; on Intel Macs with discrete GPUs there could be several and the device chooser would be relevant, but those machines are not in Lucid’s supported set.

The device is the factory for every other Metal object: buffers, textures, command queues, pipeline states. The most important child object for compute work is the `MTLCommandQueue`, an ordered FIFO into which command buffers are submitted. A single command queue per process is the standard pattern; multiple queues are used mainly to express concurrency between graphics and compute work.

Lucid, via MLX, uses exactly one command queue per process. The queue is hidden inside MLX’s runtime and is not configurable through Lucid’s surface.

5.2.2 Command buffers and encoders

Work is submitted to the device through `MTLCommandBuffer` objects. A command buffer is built by an *encoder* (one of `MTLBlitCommandEncoder`, `MTLComputeCommandEncoder`, or the graphics encoders) that turns API calls into device-readable commands. For compute work the relevant encoder is `MTLComputeCommandEncoder`; its primary call is `dispatchThreadgroups:threadsPerThreadgroup:`, which records a single kernel launch.

A typical compute submission is:

```

1  auto cmdbuf = queue->commandBuffer();
2  auto encoder = cmdbuf->computeCommandEncoder();
3  encoder->setComputePipelineState(pso);
4  encoder->setBuffer(inputBuffer, 0, 0);
5  encoder->setBuffer(outputBuffer, 0, 1);
6  encoder->dispatchThreadgroups(grid, threadsPerGroup);
7  encoder->endEncoding();
8  cmdbuf->commit();

```

The command buffer is submitted (`commit`), and the caller may then either continue building the next command buffer or block on the current one with `waitUntilCompleted`. MLX’s lazy graph machinery batches many primitive operations into one command buffer to amortise the per-buffer overhead; Lucid inherits this batching without being aware of it.

5.2.3 Buffers and textures

Storage in Metal comes in two flavours: `MTLBuffer` (plain byte sequences with optional alignment requirements) and `MTLTexture` (formatted, multi-dimensional images with hardware sampling). For ML workloads only buffers matter; textures are for graphics and image processing.

The buffer storage modes were treated in detail in Chapter 2 Section 2.3. The summary: for CPU/GPU shared access, for GPU-only, with two other modes (*Managed*, *Memoryless*) that Lucid never encounters. MLX allocates all of its arrays in , which is what makes Lucid’s zero-copy bridge possible.

5.2.4 Compute pipeline state

A `MTLComputePipelineState` (PSO) is a compiled, GPU-resident representation of a single compute kernel. It is constructed from a `MTLFunction` (extracted from a compiled `MTLLibrary`), which is in turn compiled from MSL source code or loaded from a `.metallib` binary.

The PSO is the unit of caching for compiled shader code. Constructing a PSO from MSL source is expensive (hundreds of milliseconds for a non-trivial kernel because it invokes the Metal shader compiler); constructing one from a precompiled `.metallib` is much faster (milliseconds, since only the device-specific final stage runs). MLX ships precompiled `.metallib` files inside its distribution and constructs PSOs from them; `MPSGraph` does the same for the kernels its graph emitter produces.

5.2.5 Synchronisation primitives

The relationship between two command buffers, or between a command buffer and a CPU thread, is mediated by three Metal synchronisation primitives:

1. **`MTLCommandBuffer::waitUntilCompleted`.** Blocks the calling thread until the command buffer has executed fully. The simplest primitive; used by MLX’s `eval` when a result is needed synchronously.
2. **`MTLSharedEvent`.** A signaled event with a monotone 64-bit value. Both CPU threads and GPU command buffers can wait for the value to reach a target. This is the primitive that enables fine-grained CPU/GPU ordering without a full block; MLX uses it for `async_eval`.
3. **Memory barriers within an encoder.** `MTLComputeCommandEncoder::memoryBarrier`: orders memory accesses within the same encoder, used to ensure that one kernel’s writes are visible to the next kernel’s reads in the same command buffer. Relevant only inside fused multi-kernel sequences; Lucid inherits the correct usage from `MPSGraph`.

The choice of which primitive to use is made entirely below Lucid’s surface. The framework’s contract with the user is: `Tensor` reads are synchronous; tensor operations may be deferred but always materialise consistently by the time the result is observed.

5.3 Metal Shading Language

The Metal Shading Language (MSL) is a C++14-derived shader language in which Metal kernels are written. Lucid does not author MSL directly; this section is included only for the reader who wants to understand what MLX’s kernels and MPSGraph’s emitted kernels look like underneath.

5.3.1 Kernel functions

An MSL compute kernel is a function with the `kernel` qualifier that takes thread-position and resource arguments:

```

1  #include <metal_stdlib>
2  using namespace metal;
3
4  kernel void add_kernel(
5      device const float* a [[buffer(0)]],
6      device const float* b [[buffer(1)]],
7      device          float* c [[buffer(2)]],
8      uint              tid [[thread_position_in_grid]])
9  {
10     c[tid] = a[tid] + b[tid];
11 }

```

The `[[buffer(N)]]` attribute binds the parameter to the N-th buffer set by the encoder; `[[thread_position_in_grid]]` is one of several built-in thread-identity inputs. The function runs once per thread; the encoder’s `dispatchThreadgroups:threadsPerThreadgroup:` determines how many threads are launched and how they are grouped.

5.3.2 Threadgroups, SIMD groups, and lanes

Metal’s thread hierarchy has three levels:

- **Grid.** The total set of threads launched by one dispatch. Conceptually a 1D, 2D, or 3D array; the encoder specifies its dimensions.
- **Threadgroup.** A subset of the grid that shares threadgroup memory and synchronises through threadgroup barriers. Typically 32 to 1024 threads, configured per-dispatch.
- **SIMD group.** A subset of a threadgroup that executes in lock-step on a single SIMD unit; on Apple silicon, the SIMD group size is 32 lanes. Threads within a SIMD group can share data with very low latency through SIMD-group shuffle intrinsics.

The mapping from these abstract levels to the silicon levels described in Chapter 1 Section 1.6.1 is: SIMD group → a single execution unit on a GPU core, threadgroup → a residency on one GPU core, grid → a dispatch across all cores. Good kernel performance requires choosing threadgroup sizes that fit in the per-core register file (Chapter 1 Section 1.6.2) and using SIMD-group operations where they apply.

5.3.3 Memory address spaces

MSL distinguishes four memory address spaces:

- **device:** pointers into `MTLBuffer` memory; the main data path.

- **threadgroup**: pointers into the threadgroup memory allocated for the current dispatch.
- **thread**: register-resident, per-thread local variables.
- **constant**: a read-only, broadcast-friendly memory space for small lookup tables; backed by a special hardware path on Apple silicon’s GPU.

The address-space qualifier is mandatory on pointer types and is checked by the MSL compiler; it gives the compiler precise information about access patterns and enables address-specific code generation (for instance, **constant** accesses are broadcast in hardware where possible).

5.3.4 Why Lucid does not write MSL

The decision to avoid hand-written MSL is the same decision, at one level deeper, as the decision to sit on top of MLX. The kernels that Lucid would have written (elementwise binary, reduction, matmul, convolution, attention) have already been written by Apple and by the MLX team, with care taken about threadgroup sizing, register pressure, SIMD utilisation, and memory access patterns. Rewriting them in Lucid would consume a kernel engineer for many months and would not, in expectation, exceed the existing implementations.

The escape hatch exists nonetheless. Lucid exposes a `lucid.metal` module (Chapter 12) that allows the user to compile and run an MSL kernel on a `Tensor`, with manual responsibility for buffer marshallng. The interface is deliberately low-level and unstable; it exists for research workloads that need a custom kernel and are willing to handle Metal directly.

5.4 MPSGraph

MPSGraph is the framework on which Lucid’s compile path (Chapter 43) is built. Conceptually it is a graph IR plus a compiler plus a runtime: the client constructs a directed graph of operations on placeholder tensors, asks MPSGraph to compile the graph for a specific input-shape configuration, receives an executable handle, and then runs the executable on concrete input tensors.

5.4.1 Graph construction

The construction API is object-oriented Objective-C (and Swift), with bridged calls from C++. The pattern is:

```

1 auto graph = MPSGraph::alloc()->init();
2 auto x = graph->placeholderWithShape(shape, dtype, name);
3 auto w = graph->placeholderWithShape(shape, dtype, name);
4 auto y = graph->matrixMultiplicationWithPrimaryTensor(x,
5               secondaryTensor: w, name: @"matmul");
6 auto z = graph->additionWithPrimaryTensor(y,
7               secondaryTensor: bias, name: @"bias_add");
8 // ... and so on

```

Every method returns an `MPSGraphTensor` that represents the output of an operation; the graph is implicit in the chain of references. There is no explicit graph-edge construction.

The operation set is large (hundreds of methods on the `MPSGraph` class) and covers elementwise arithmetic, reductions, matrix multiplication, convolution, pooling, normalisation, activation functions, indexing, FFT, and a number of neural-network-specific primitives (LSTM, MultiheadAttention, SoftmaxCrossEntropy). Chapter 46 enumerates the subset that Lucid’s compile path uses, which is approximately 180 primitives covering the inner loop of every supported model in the zoo.

5.4.2 Compilation and the executable cache

Once constructed, a graph must be *compiled* for a specific input-shape configuration before it can run. The compilation step is:

```
1  auto executable = graph->compileWithDevice(device,
2      feeds: feedDict,      // placeholder -> concrete shape
3      targetTensors: outputs,
4      targetOperations: nil,
5      compilationDescriptor: nil);
```

The returned `MPSGraphExecutable` is a multi-kernel Metal program ready to dispatch. Subsequent invocations on the same shapes reuse the same executable; invocations on different shapes require either a new compilation or, with appropriate descriptor flags, a recompilation that the framework manages internally.

The compilation step is expensive: hundreds of milliseconds for a graph with a few dozen operations, into the seconds for graphs with hundreds of operations. Lucid’s compile path therefore caches compiled executables aggressively (Chapter 45); a training loop that compiles once and runs the same shapes for the remainder of training amortises the compile cost to near zero.

5.4.3 Backward pass

`MPSGraph` in macOS 13 and later provides a `gradientForPrimaryTensor:withTensors:name:` method that constructs reverse-mode gradients directly within the graph. Given a scalar output tensor and a list of input tensors, the method returns a dictionary mapping each input to its gradient tensor.

Lucid’s compile path uses this method to capture forward and backward into a single graph (Chapter 47). The advantage is that the entire training step — forward, backward, gradient accumulation, parameter update — can be compiled as one `MPSGraph` executable, with all intermediate tensors fused away by the compiler. The disadvantage is that the gradient is computed at graph-construction time, before shape information is concrete, which constrains the operations that admit a clean backward (and we work around the constraints with custom `Function` classes for the awkward cases).

5.4.4 Operation primitives

The graph operations exposed by `MPSGraph` fall into several categories that map onto Lucid’s op taxonomy:

- **Elementwise.** Arithmetic, comparison, logical, transcendental — one `MPSGraph` method per op.
- **Reduction.** Sum, mean, max, min, prod, argmax, argmin, with axis and keepdim parameters.
- **Linear algebra.** Matrix multiplication, batch matmul, transpose; no decompositions (Cholesky, QR, SVD are not in `MPSGraph` as of macOS 26).
- **Convolution and pooling.** Conv1d/2d/3d (forward and backward), transposed convolution, max/avg pooling.
- **Normalisation.** Batch, layer, instance, group norm with learnable affine.
- **Indexing and reshape.** Gather, scatter, slice, reshape, broadcast.
- **Neural-network composites.** Softmax cross-entropy loss, LSTM, MultiheadAttention. These are higher-level than MLX’s primitives; Lucid uses them selectively where their fused behaviour is preferable.

Aspect	MLX	MPSGraph
Abstraction level	array library (NumPy-like)	graph IR
Execution model	lazy graph, implicit eval	explicit compile + execute
Per-op latency	$\sim 10 \mu\text{s}$	$\sim 5 \mu\text{s}$ (cached)
First-call latency	none	0.5–3 s (compile)
Op fusion	per <code>eval</code> window	whole-graph
Layout selection	fixed	runtime-chosen (NCHW/NHWC)
Backward construction	via Lucid autograd	via <code>gradientForPrimaryTensor</code> :
Dynamic shape	natural	partial (3.5.1+)
Linalg primitives	yes (CPU stream)	no (Cholesky/QR/SVD absent)
Lucid use	default eager path	<code>lucid.compile</code> + 3.4 carve-out

Table 5.1: Side-by-side comparison of the two GPU-side substrates that Lucid depends on. **MLX** is the ergonomic eager path; **MPSGraph** is the amortised graph-capture path that `lucid.compile` targets. Both coexist within a single Lucid process because they ultimately share the same Metal command queue.

Operations that **MPSGraph** does not provide (the most consequential gap is the linalg family: `cholesky`, `qr`, `svd`, `eigh`) are handled in Lucid by falling back to the eager path through **MLX** + **Accelerate**, with the result re-injected into the compiled graph as a constant when needed. This fallback is described in Chapter 49.

5.5 MPSGraph versus MLX

A reasonable question, given that Lucid depends on both, is: when is each the right tool? They occupy adjacent but distinct positions in the design space; Table 5.1 compares them side by side.

5.5.1 Abstraction level

MLX is an array library that happens to be lazy; the graph is an implementation detail of the runtime. The user (or, in our case, Lucid’s dispatcher) writes operations as ordinary function calls and reads results back when needed. The graph is constructed and evaluated implicitly.

MPSGraph is a graph library that requires explicit graph construction, explicit compilation, and explicit execution. The user builds the graph as a data structure, hands it to the compiler, and runs the resulting executable. There is no eager mode.

For interactive work, model exploration, and short-running scripts, **MLX** is dramatically more ergonomic. For long-running training loops in which the same operations repeat thousands of times with the same shapes, **MPSGraph**’s amortised compile cost is worth the upfront construction.

5.5.2 Compilation cost

MLX has no compilation step in the same sense: the lazy graph is constructed and evaluated per call. There is a small overhead per operation (microseconds) for graph node construction, but no hundreds-of-milliseconds compile latency.

MPSGraph pays a compile cost the first time a graph is run with a given input-shape configuration. For Lucid’s training-step graphs, this is typically 0.5 to 2 seconds on first call; subsequent calls hit the executable cache and run with the overhead of a single command-buffer submission (microseconds). For an inference-style workload that runs many shape variations, **MPSGraph**’s compile cost dominates and **MLX** is preferable.

5.5.3 Optimisation aggressiveness

MLX's lazy graph performs operator fusion and constant folding within the lifetime of a single `eval` call, but does not persist the optimised graph across calls. Each `eval` starts fresh.

MPSGraph's compiler is more aggressive: it can fuse across the entire training step, eliminate intermediate buffers that have no external observers, choose alternate layouts (e.g. NCHW vs NHWC) based on hardware preference, and select between alternate kernel implementations for the same operation. The training-step graphs that Lucid produces in 3.5 are typically 30 to 60 % smaller in kernel count after MPSGraph compilation than the equivalent MLX sequence.

5.5.4 Coexistence in Lucid

Lucid uses both, and the choice is explicit at the framework boundary:

- Default (eager) mode goes through MLX. The user constructs a model, calls forward and backward, and observes results; every op is a MLX primitive.
- `lucid.compile(model)` returns a `CompiledModule` that, on its first call, traces the forward pass, constructs an MPSGraph, compiles, and caches. Subsequent calls run the cached executable directly.
- The 3.4 per-op MPSGraph carve-out (Chapter 60) is a third path: certain operators (GELU, Embedding backward, MaxPool backward, LayerNorm) dispatch through MPSGraph even in eager mode, because their MPSGraph implementations are substantially faster than MLX's. This is invisible to the user but the dispatcher chooses it automatically.

Both paths produce the same numerical results to within a few ULPs; the only user-visible difference is performance and the compile-time latency of the first call.

5.6 Lucid's Use of Metal and MPSGraph

Concretely, the Lucid source files that touch this layer are:

- `lucid/_C/compile/MpsBuilder.h` and `.mm`: the Objective-C++ class that constructs MPSGraph objects from Lucid's trace IR. (Chapter 45)
- `lucid/_C/compile/CompiledExecutable.h` and `.mm`: the wrapper around `MPSGraphExecutable` that holds compiled executables and dispatches inputs.
- `lucid/_C/compile/ExecutableCache.h` and `.cpp`: the shape-keyed cache that maps input-shape tuples to compiled executables.
- `lucid/_C/compile/OpEmitters/`: per-op emitter functions, one per MPSGraph primitive, that turn Lucid trace IR nodes into `MPSGraphTensor` calls.
- `lucid/_C/backend/gpu/MlxBackend.cpp`: a small set of per-op carve-outs for the 3.4 MPSGraph dispatch.

The `.mm` suffix denotes Objective-C++ files, which can call both C++ and the Objective-C MPSGraph API. The bridge between Lucid's C++-only core and MPSGraph's Objective-C surface is encapsulated in these files.

Notably absent: there is no `MetalBackend.cpp` or `MetalKernels.metal`. Lucid does not own Metal kernels; the MSL files in the build are all MLX's. This is by design.

5.7 The Compilation Cost Model

We now describe quantitatively the cost model that the user must internalise for Lucid's compile path. The numbers in this section are taken from Chapter 62 and reproduced here for context.

5.7.1 First-call latency

The first invocation of a `lucid.compile(model)`-wrapped step incurs three serial latencies:

1. **Tracing.** Lucid’s tracer (Chapter 44) runs the model in shape-only mode to record the trace IR. Typically tens to hundreds of milliseconds for a 100M-parameter Transformer.
2. **Graph emission.** The `MpsBuilder` translates the trace IR into `MPSGraph` tensor operations. Typically tens of milliseconds.
3. **MPSGraph compilation.** `MPSGraph` itself compiles the constructed graph into an executable. This is the dominant term: typically 0.5 to 3 seconds for a moderately complex training step.

The total first-call latency for the GPT-2 base training step on M3 Max is approximately 1.8 seconds. Subsequent calls hit the executable cache and run in 110 to 115 ms.

5.7.2 Shape specialisation

The executable cache is keyed by the tuple of input shapes (Chapter 48). A training loop that varies its batch size — for example, last-batch-smaller-than-rest training on a non-multiple-of-batch dataset — will recompile per unique shape. Lucid deals with this by recommending that users drop the last incomplete batch or pad to a fixed shape; the explicit shape-set is part of the documented contract.

The alternative would be dynamic shape support, in which the compiled graph accepts variable input shapes. `MPSGraph` does provide a partial dynamic-shape mode (with constraints on which operations admit it), but Lucid’s 3.5 release deferred dynamic-batch support to 3.5.1 (Chapter 49) because the parity tests on representative models did not pass under symbolic primary shapes.

5.7.3 The executable cache

Lucid’s executable cache (`ExecutableCache`) has a configurable maximum size, default 32 entries. Beyond 32, an LRU eviction policy removes the least recently used compiled graph. For most training workloads the working set is one or two graphs (forward+backward and optionally an evaluation forward); the cache size is generous.

For inference workloads that compile per unique input shape, the cache can fill quickly; users with such workloads can either raise the cache size at module construction or pre-compile a known shape set.

5.8 Limitations and Quirks

We close with the limitations Lucid has documented from production use of `MPSGraph`.

5.8.1 No backward for some forward operations

A small set of `MPSGraph` forward operations do not have a gradient defined. The most consequential is the `linalg` family (`cholesky`, `qr`, `svd`, `eigh`), which `MPSGraph` does not provide at all and which therefore cannot be compiled. Some indexing operations also have partial gradient support (scatter with non-deterministic duplicate-index semantics is one such case).

Lucid’s compile path handles these by falling back to eager mode for any subgraph that contains an unsupported operation (Chapter 49). The fallback boundary is at the op-level: one unsupported op forces eager dispatch only for that op and its immediate consumers, not for the entire training step.

5.8.2 SDK version drift

MPSGraph’s surface has grown each macOS release, and operations introduced in macOS 14 are not callable on macOS 13. Lucid’s minimum target is macOS 26.0 (Chapter 3 Section 3.9.2), which corresponds to a version of MPSGraph that has the operations Lucid relies on. Backward compatibility with older macOS is not maintained.

5.8.3 Compile failure modes

MPSGraph compilation can fail for several distinct reasons:

- Symbolic shapes the compiler cannot resolve.
- Operations not implemented for the requested dtype.
- Graph topology that exceeds the compiler’s internal limits (rare but does happen for very deep, very wide graphs).
- Driver bugs in specific macOS point releases.

Lucid’s compile path catches the failure at compilation time and falls back to eager mode for the entire compiled module, with a warning logged. The user can request that the fallback be disabled (via `compile(strict=True)`) to force the error to surface, which is the right setting during model development; the default is permissive, which is the right setting for production.

5.8.4 Debugging compiled graphs

A compiled MPSGraph executable is opaque: there is no in-process introspection of its kernel sequence beyond the names of the constituent operations, and no kernel-level profiler short of Apple’s Instruments application with the Metal System Trace template. Lucid’s response to this is to dump the trace IR before MPSGraph emission (`lucid.compile.dump_trace`, see Chapter 44), so that the user can inspect the operations Lucid requested even if the post-compilation form is invisible.

5.9 Summary and Forward References

Metal is the operating-system substrate that every Apple silicon GPU operation passes through, and MPSGraph is the higher-level graph framework that Apple supplies above it. Lucid does not call Metal directly; it calls MLX for eager execution and MPSGraph for the compile path and a small set of per-op carve-outs. The choice between MLX and MPSGraph in Lucid is determined by the user’s invocation of `lucid.compile`: with it, the training step is captured into an MPSGraph executable for the price of a one-time compile latency; without it, every operation flows through MLX’s lazy graph as in any eager runtime.

This chapter closes Part I. The hardware foundation — Apple silicon at Chapter 1, unified memory at Chapter 2, MLX at Chapter 3, Accelerate at Chapter 4, and Metal and MPSGraph here — is the complete substrate against which the rest of the book describes Lucid itself. Chapter 6 begins Part II with the layered dependency DAG that organises the engine code, and the chapters that follow expand on each layer of that DAG.

Tip

For the user: the only Metal-related concept that matters at the framework boundary is `lucid.compile`. With it, the first training step is several seconds longer but subsequent steps are faster by 20–40% on representative training workloads. Without it, MLX’s eager mode runs every operation as it is called, which is fine for development and exploration but leaves performance on the table for production training.

Part II

System Architecture

CHAPTER 6

The Layered Dependency DAG

Part [I](#) described the substrate on which Lucid runs. Part [II](#) describes Lucid itself, beginning here with the most consequential single piece of architecture in the engine: the layered dependency DAG that organises every C++ source file and every Python module in the codebase.

The DAG is not a documentation artefact. It is enforced by the build system, by code-review policy (the *Hard Rules* of [Appendix A](#)), and by a class of automated checks that run on every pull request. The chapter that the reader is now reading exists because the DAG is the thing that, once internalised, makes the rest of the engine code easy to navigate. A new contributor who understands the seven layers and their permitted edges can locate any operation, any class, any allocation in roughly the time it takes to skim a directory listing. A contributor who has not internalised it spends hours hunting for definitions that turn out to live two layers below where intuition suggested.

We proceed bottom up. [Section 6.2](#) describes the seven layers and their contents. [Section 6.3](#) states the edge rules that the DAG enforces. [Section 6.4](#) describes how the rules are enforced — through CMake target boundaries, through code review, and through a small set of static checks. [Section 6.5](#) treats the Python side, which mirrors the C++ DAG with a smaller set of layers. [Section 6.6](#) catalogues the cross-cutting concerns (allocator, error, dispatch) that touch multiple layers without violating the DAG. [Section 6.7](#) describes the procedure for adding a new layer or a new file within an existing layer.

6.1 Why a Layered Architecture

The engine is roughly 45,000 lines of C++ across more than two hundred translation units, plus a comparable volume of Python code that mirrors it. At that scale, an architecture without explicit structural rules tends to collapse into an undirected dependency graph, with each

module reaching into every other module that has a useful symbol. Two years into the project, every file includes every other file; build times explode; refactoring becomes impossible; new contributors cannot tell where a feature should live.

The cure is a directed acyclic graph of dependencies, with a small number of layers, and with an enforced rule that edges go only downward. This is not a novel pattern — it is the standard operating-system-kernel layout, the standard compiler-construction pattern, the standard database-system layout — but it is unusual in ML frameworks, which tend to grow organically and tolerate arbitrary internal cycles.

The benefits, concretely, for *Lucid*:

1. **Compile-time isolation.** A change in the highest layer (*bindings*) recompiles only that layer; a change in the lowest layer (*core*) recompiles everything. The CMake target graph mirrors the DAG, so this isolation is automatic.
2. **Testability.** Each layer can be tested against mocks of the layer immediately below. *Lucid*’s C++ unit tests follow this pattern: `test/core/` tests `core` in isolation, `test/backend/` tests `backend` against in-memory mocks of `core`, and so on.
3. **Mental model.** A contributor reading source code can rely on the rule that any symbol referenced is defined in the current layer or strictly below. This eliminates the “where could this possibly come from” search that pervades cyclic codebases.
4. **Refactoring discipline.** Changes to a layer’s public API force consumers in higher layers to adapt, in a direction that the build system actually checks. This is the inverse of the situation in cyclic codebases, where a refactoring must be performed simultaneously across every affected file.

The cost is up-front structural design and ongoing discipline: contributors must occasionally accept that the right place for a new piece of code is not the most convenient one. The *Hard Rules* (Appendix A) encode the discipline.

6.2 The Seven Layers

The C++ engine is organised into seven layers, each occupying a directory under `lucid/_C/`. We describe them from the bottom up. The corresponding Python tree, which is shallower, is treated separately in Section 6.5.

Figure 6.1 gives the full stack at a glance; the subsequent subsections expand each layer.

The DAG is not a documentation idealisation. It corresponds directly to the directory layout under `lucid/_C/`, to the CMake target structure, and to the build’s link line. Table 6.1 summarises the per-layer files and their approximate volumes.

6.2.1 Layer 0: *core*

`lucid/_C/core/` contains the foundational types on which every other layer depends. It has no dependencies on any other *Lucid* code; only on the C++ standard library and on Apple’s Accelerate (for low-level math helpers). The contents are deliberately minimal:

- `Dtype.h`: the dtype enum and conversion helpers.
- `Device.h`: the device enum (CPU / Metal) and a small set of capability queries.
- `Shape.h`, `Shape.cpp`: the shape vector type with broadcast and validation operations.
- `Allocator.h`, `Allocator.cpp`: the size-class allocator pool.
- `MemoryStats.h`: per-device allocation statistics.
- `Error.h`, `ErrorBuilder.cpp`: the `LucidError` exception class and the fluent builder used by every error site (Appendix B for the production pillar P1 that motivates it).

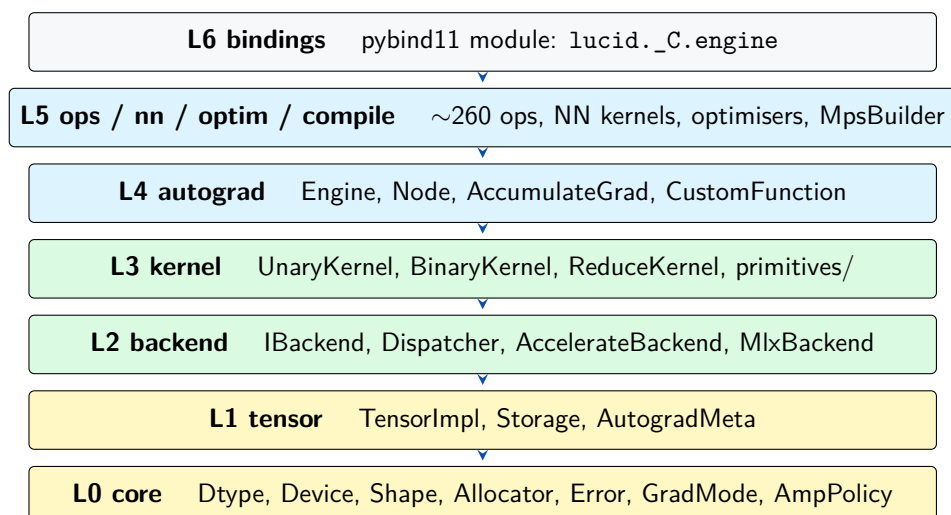


Figure 6.1: The seven-layer dependency DAG of the Lucid C++ engine. Solid arrows indicate direct dependency (layer above includes from layer below). No back-edges (arrows from a lower layer to a higher one) are permitted; the absence is enforced by the CMake target graph and verified by `tools/check_dag.py`. Cross-cutting concerns (allocator, error, dispatch) are defined in `core` (L0) and consumed from every layer above.

Layer	Directory	SLOC	Key types / files
L0 core	<code>lucid/_C/core/</code>	~1 800	Dtype, Device, Shape, Allocator, LucidError
L1 tensor	<code>lucid/_C/tensor/</code>	~1 200	TensorImpl, Storage, AutogradMeta
L2 backend	<code>lucid/_C/backend/</code>	~6 500	IBackend, Dispatcher, two backends
L3 kernel	<code>lucid/_C/kernel/</code>	~3 400	Kernel templates + primitives
L4 autograd	<code>lucid/_C/autograd/</code>	~4 900	Engine, Node, CustomFunction
L5 ops	<code>lucid/_C/ops/</code>	~11 000	~260 op implementations
L5 nn	<code>lucid/_C/nn/</code>	~4 800	Conv, BatchNorm, LSTM, Attention
L5 optim	<code>lucid/_C/optim/</code>	~1 700	Optimizer family + LR schedulers
L5 compile	<code>lucid/_C/compile/</code>	~5 200	Tracer, MpsBuilder, OpEmitters
L6 bindings	<code>lucid/_C/bindings/</code>	~3 500	pybind11 wrappers
Total		~45 000	

Table 6.1: Approximate source-lines-of-code per engine layer, as of Lucid 3.5. Layer 5 dominates the line count because it hosts the ~ 260 operation implementations and the model-zoo helpers.

- `Determinism.h`, `Determinism.cpp`: the deterministic-kernel-selection flag.
- `GradMode.h`, `GradMode.cpp`: the `no_grad` / `enable_grad` state.
- `AmpPolicy.h`, `AmpPolicy.cpp`: the mixed precision dtype-casting rules.
- `Generator.h`, `Generator.cpp`: the random number generator state.

The discipline: nothing in `core` may include from any other `lucid/_C/` directory. Violating this rule introduces a dependency cycle, since every other layer depends on `core`.

6.2.2 Layer 1: tensor

`lucid/_C/tensor/` contains the tensor model itself:

- `TensorMeta.h`: the `TensorImpl` class — shape, stride, dtype, device, storage offset, and a handle to the backing storage.
- `AutogradMeta.h`: the per-tensor autograd metadata (`requires_grad`, `grad_fn`, backward hooks).

- `Storage.h`: the reference-counted storage object that wraps a backing buffer (allocated through `core`'s `Allocator`).

`tensor` depends on `core`. It does not depend on any backend (since `TensorImpl` is backend-agnostic) and does not know how operations are dispatched (that is the next layer's job). The chapter that fills in this layer in depth is Chapter 7.

6.2.3 Layer 2: backend

`lucid/_C/backend/` contains the abstract backend interface and the two concrete implementations:

- `IBackend.h`: the pure-virtual interface that every backend implements: `allocate`, `deallocate`, elementwise op categories, reductions, `matmul`, convolution, and so on.
- `Dispatcher.h`, `Dispatcher.cpp`: the router that selects a backend based on a tensor's device.
- `cpu/AccelerateBackend.{h,cpp}` and several helper files: the Accelerate-backed CPU backend (Chapter 4).
- `gpu/MlxBackend.{h,cpp}`: the MLX-backed GPU backend (Chapter 3).
- `gpu/SharedStorage.{h,cpp}`: the zero-copy CPU/GPU bridge (Chapter 2).

`backend` depends on `core` and `tensor`. It is the first layer that touches external libraries (Accelerate, MLX) at build time. The chapter that fills in this layer is Chapter 9.

6.2.4 Layer 3: kernel

`lucid/_C/kernel/` contains the kernel abstraction — the templates that take a backend-agnostic operation and dispatch it through the backend's primitives:

- `IKernel.h`: the abstract base.
- `UnaryKernel.h`, `BinaryKernel.h`, `ReduceKernel.h`, `NaryKernel.h`, `VariadicKernel.h`: the templates per arity class.
- `Contig.h`: contiguity inference and conversion.
- `primitives/`: data-dependent primitives that cannot be expressed as plain elementwise or reduction operations (`Gather`, `Scatter`, `Im2Col`, `Pad`, `Sort`, `NonZero`, `Unique`, `SearchSorted`, and others).

`kernel` depends on `core`, `tensor`, and `backend`. The chapter is Chapter 14.

6.2.5 Layer 4: autograd

`lucid/_C/autograd/` contains the differentiation engine:

- `Node.h`, `Node.cpp`: the base class for backward nodes.
- `AutogradNode.h`: the templated convenience class used by most ops.
- `Engine.h`, `Engine.cpp`: the reverse-mode backward walker (topological sort + per-node apply).
- `AccumulateGrad.{h,cpp}`: the leaf node that holds accumulated gradients on parameters.
- `CustomFunction.{h,cpp}`, `FuncOp.h`: the user-extensible custom-function machinery.
- `FusionPass.{h,cpp}`: a forward-only fusion pass that combines compatible adjacent ops (e.g. `Linear+ReLU`, `SDPA`).

`autograd` depends on `core`, `tensor`, `backend`, and `kernel`. It is responsible for constructing the backward graph as ops execute and for traversing it on `backward()`. Chapter 22 fills in the details.

6.2.6 Layer 5: ops, nn, optim, linalg, fft, einops, complex, compile

The fifth layer is broad. It contains the actual operation implementations, grouped into several sibling directories:

- `lucid/_C/ops/`: the elementwise (`ufunc`, `bfunc`), generic (`gfunc`), composite, `linalg`, `fft`, `einops`, and `complex` operations. Each operation is a single C++ file that takes backend-agnostic `TensorImpl` inputs, dispatches through `backend` or constructs through `kernel`, and returns one or more outputs. There are roughly 260 such operations.
- `lucid/_C/nn/`: neural-network-specific kernels (`Conv`, `BatchNorm`, `GroupNorm`, `LayerNorm`, `Embedding`, `LSTM`, `MultiheadAttention`, `Interpolate`, `AdaptivePool`, `Dropout`). These differ from `ops/` in that they often manage internal running statistics or expose neural-network-specific APIs.
- `lucid/_C/optim/`: the optimiser implementations (`Optimizer` base class, `SGD`, `Adam` family, `AdaGrad` family, `RMSprop`, `LBFGS`, and a number of LR schedulers).
- `lucid/_C/compile/`: the graph-capture compiler — `Tracer`, `TraceIR`, `MpsBuilder`, `ExecutableCache`, `OpEmitters` (Chapter 43).

These siblings all depend on `core` through `autograd`. They do not depend on each other; an `ops/` file does not include from `nn/` or `optim/`, and vice versa. The chapters that fill in the contents of layer 5 are Chapter 13 through Chapter 40 for operations and optimisers, and Part VII for the compile path.

6.2.7 Layer 6: bindings

`lucid/_C/bindings/` contains the `pybind11` binding code that exposes the C++ engine to Python. Each binding file wraps a related set of operations:

- `bind.cpp`: the module bootstrap.
- `bind_tensor.cpp`: the `Tensor` class.
- `bind_bfunc.cpp`, `bind_ufunc.cpp`, `bind_gfunc.cpp`, `bind_composite.cpp`: per-op bindings.
- `bind_autograd.cpp`, `bind_nn.cpp`, `bind_optim.cpp`, `bind_linalg.cpp`, `bind_fft.cpp`, `bind_einops.cpp`, `bind_complex.cpp`, `bind_amp.cpp`, `bind_profiler.cpp`, `bind_random.cpp`, `bind_compile.cpp`: subsystem-level bindings.
- `bind_errors.cpp`: the exception translation that maps `LucidError` to a Python exception.
- `bind_op_registry.cpp`: a binding for the op registry that introspection tools can read.

`bindings` depends on every layer below it. It is the only layer that links against `pybind11` itself; the rest of the engine is pure C++. This isolation matters: `Lucid`'s engine could in principle be linked into a non-Python program by replacing the bindings layer with a C API, and we have kept the option open by ensuring no `pybind11` types leak into lower layers.

Chapter 11 fills in the binding details.

6.3 Dependency Rules

The dependency rules between layers are simple and absolute.

6.3.1 Acyclicity

The dependency graph between layers is acyclic. There is no edge from a lower-numbered layer back up to a higher-numbered one. The ordering is total: `core` $\prec$ `tensor` $\prec$ `backend` $\prec$ `kernel` $\prec$ `autograd` $\prec$ `ops/nn/optim/compile` $\prec$ `bindings`.

The ordering implies, for any pair of layers A and B with $A \prec B$:

- B may include headers from A .
- B may call functions defined in A .
- A may not include headers from B .
- A may not name any type defined in B .
- A may not call any function defined in B .

6.3.2 Edge directness

Edges are typically direct: layer $i + 1$ depends on layer i , not on layer $i - 2$. But edges may legitimately skip layers when the intermediate layer does not own the abstraction in question. For example, `ops/Reduction.cpp` (layer 5) depends directly on `core/Dtype.h` (layer 0) and on `kernel/ReduceKernel.h` (layer 3) but not on `tensor/` or `backend/` headers, because the reduction is expressed in terms of kernel templates that themselves handle tensor and backend concerns.

Skipping layers is fine. Reversing direction is not.

6.3.3 Sibling independence within layer 5

Within layer 5, the four sibling directories (`ops/`, `nn/`, `optim/`, `compile/`) do not depend on each other. This is not a logical necessity — one could imagine `nn/` reusing some `ops/` composition — but it is an empirical discipline that prevents the layer from collapsing into a tangle. When a `nn/` kernel needs a primitive that `ops/` provides, it depends on the layer-4 `autograd` representation of the primitive, not on `ops/` directly.

`compile/` is a partial exception: it must enumerate every op its emitter supports, which means it has read-only knowledge of the names and signatures in `ops/` and `nn/`. We implement this by having `compile/OpEmitters/` hold a table of emitter functions keyed by op name; the table is populated at build time by code generation from a single source of truth in `compile/OpRegistry.h`, not by header includes.

6.4 Enforcement

The DAG would be a documentation artefact if it were not enforced. Lucid enforces it through three independent mechanisms.

6.4.1 CMake target boundaries

Each layer is a separate CMake target. The CMakeLists structure under `lucid/_C/` declares one library per layer, with explicit `target_link_libraries` edges that mirror the permitted dependencies. Trying to use a symbol from a layer that is not in the current target's link line produces a link error, which is caught at build time rather than at code review.

This is the strongest enforcement mechanism. A contributor who attempts to violate the DAG sees a build failure within seconds of saving the file.

6.4.2 Code review policy

The Hard Rules of Appendix A include an explicit rule (H-rule on layer dependency) that any pull request which adds an edge in the wrong direction is rejected. Reviewers are expected to

verify, for each new include in a header, that the include's directory is at or below the current file's layer.

In practice this rarely needs invocation, because the CMake boundary catches the violation first. But the rule exists for the edge case in which a violation passes CMake (for example, a contributor adds the offending edge to the CMake target as part of the same PR), and it gives the reviewer explicit grounds to reject.

6.4.3 The `tools/check_dag.py` script

A small static-analysis script in `tools/check_dag.py` parses the include statements in every C++ header and source file under `lucid/_C/` and verifies that every include's target directory is at or below the source file's layer. The script is run as part of the pre-commit hook and as part of the CI check list; a violation fails the build.

The script is simple (under two hundred lines of Python) and uses only the standard library. It is the fallback in case a CMake restructuring inadvertently relaxes the target boundary.

6.5 The Python Mirror

The Python side of Lucid mirrors the C++ DAG with a smaller set of layers. The structure is:

- **Foundation:** `lucid/_types.py`, `lucid/_dtype.py`, `lucid/_device.py`, `lucid/_globals.py`. Pure Python, no engine dependency.
- **Engine binding:** `lucid/_C/` (the compiled extension module produced by the C++ layers). Imported under the `_C_<name>` alias convention required by Hard Rule H2.
- **Dispatch:** `lucid/_dispatch.py`, the single boundary that wraps and unwraps between Python Tensor and engine-side `TensorImpl`.
- **Tensor:** `lucid/_tensor/` (the `Tensor` class, methods, dunders, indexing, conversion, repr).
- **Factories:** `lucid/_factories/` (tensor creation: `zeros`, `ones`, `tensor`, `from_numpy`, `from_dlpack`, `random`).
- **Op wrappers:** `lucid/_ops/` (the registry of free functions and Tensor methods exposed at the top level).
- **Subsystem modules:** `lucid/autograd/`, `lucid/nn/`, `lucid/optim/`, `lucid/linalg/`, `lucid/fft/`, `lucid/signal/`, `lucid/special/`, `lucid/distributions/`, `lucid/func/`, `lucid/einops/`, `lucid/amp/`, `lucid/profiler/`, `lucid/serialization/`, `lucid/compile/`, `lucid/models/`.
- **Top-level package:** `lucid/__init__.py`, the lazy-loading dispatch surface (Chapter 12).

The Python rules are looser than the C++ rules, because Python permits cycles that the C++ build would reject, but the convention is the same: factories depend on engine binding, tensor depends on factories + engine binding, subsystem modules depend on tensor/factories/engine, and the top-level package depends on everything.

The convention is enforced by code review rather than by automation, because a comprehensive Python import-graph checker would have to parse Python's dynamic import semantics, which is more work than the benefit justifies.

6.6 Cross-cutting Concerns

A handful of concerns touch every layer without violating the DAG. The pattern is the same in each case: the concern is defined in `core` and consumed by every layer above. Three are worth naming explicitly.

6.6.1 The allocator

`core/Allocator` is the size-class pool that every backend’s memory allocations route through. Layer 2 (`backend`) wraps the underlying provider’s allocator (MLX’s or Accelerate’s) and funnels requests through the pool. Layer 5 (`ops`) and beyond do not allocate directly; they always go through the backend.

The chapter is Chapter 8.

6.6.2 Error handling

`core/Error` defines `LucidError`, the single exception type that propagates from any layer back to Python. Layer 6 (`bindings`) installs the `pybind11` translator that maps `LucidError` to the Python `LucidError` class. Intermediate layers may catch and rethrow with additional context; the chapter is Chapter 66.

6.6.3 Dispatch

The dispatcher (layer 2, `backend/Dispatcher.h`) is the single function that routes per-op requests to the correct backend. Above it, `ops/` files do not call backends directly; they call the dispatcher with the op kind and the input tensors, and the dispatcher selects the implementation. This indirection is what allows the same op file to work for both CPU and GPU without conditional code.

The chapter is Chapter 9.

6.7 Adding a Layer or a File

The protocol for adding a new file within an existing layer is short:

1. Identify the layer that the new file belongs to. The criterion is: what does the file depend on? If it depends only on `core`, it lives in `tensor` or higher; if it depends on `tensor`, in `backend` or higher; and so on.
2. Add the file under the appropriate directory.
3. Update the CMakeLists in that directory to include the new file in the layer’s library target.
4. Add the file’s tests to the corresponding `test/<layer>/` directory.
5. Confirm the build still passes and the `check_dag` script accepts the new file.

Adding a new layer is much rarer and requires explicit design discussion. The reason: a new layer either fits between two existing ones (in which case it adds a hop to every dependency chain through that point) or sits at the top above `bindings` (in which case it suggests the engine has grown a new audience). Both situations have happened exactly once each in Lucid’s history. The first was the addition of `kernel` as a layer above `backend`, which absorbed the kernel-template logic that had previously lived in individual `ops/` files and was being duplicated. The second was the introduction of `compile/` as a sibling of `ops/`, `nn/`, and `optim/`, which was added to host the graph-capture compiler without polluting the eager-mode hierarchy.

6.8 Summary and Forward References

The layered DAG is the structural invariant that makes the Lucid engine tractable. Seven layers, edges only from higher to lower, enforced by the CMake target graph plus a static analysis script plus code-review discipline. Every other architectural decision in the engine fits

within the DAG; every cross-cutting concern is located in `core` and consumed by every layer above; the Python side mirrors the C++ side with a smaller and looser set of rules.

The chapters that follow expand on individual layers. Chapter 7 describes layer 1 in detail. Chapter 8 treats the cross-cutting allocator. Chapter 9 treats layer 2 and the dispatcher. Chapter 10 catalogues the six places where the engine touches the outside world. Chapter 11 returns to layer 6 with a focus on the pybind11 contract. Chapter 12 closes Part II with the Python-side lazy-import surface that the top-level package uses to make Lucid importable in under one hundred milliseconds despite the size of its subsystem tree.

Note

The DAG is invisible at the user surface — the user imports `lucid`, constructs a model, calls `forward`, and never sees layer boundaries. The DAG matters only to contributors and to the build system. But its presence is the reason Lucid has not collapsed into the tangle of mutual includes that ML frameworks of comparable scope sometimes become.

CHAPTER 7

Tensor Storage Model

A tensor framework’s storage model is the type definition of its central data structure — in Lucid’s case, what fields the `TensorImpl` class carries, how those fields relate to backing memory, and what invariants the engine guarantees between them. The storage model is invisible to the user, who manipulates a Python `Tensor` that hides everything below, but it determines the performance and correctness envelope of every operation. A storage model that does not support views forces every reshape to copy. A storage model that does not separate stride from shape forces every permutation to materialise. A storage model whose backing buffer is owned by the wrong layer cannot be shared zero-copy with the GPU.

This chapter formalises Lucid’s storage model. The model is deliberately conventional — it is the same model that any NumPy-derived framework uses, with one important difference: the backing storage may belong to MLX rather than to Lucid’s own allocator, which makes the bridge of Chapter 2 possible without breaking the abstraction. The conventional pieces are still worth describing in their own right, because the rest of the book assumes them.

Section 7.1 describes the `TensorImpl` class and its fields. Section 7.2 describes the `Storage` class and its ownership semantics. Section 7.3 describes view operations and what makes them cheap. Section 7.4 describes the autograd metadata that hangs off each tensor. Section 7.5 describes the reference counting that ties the pieces together. Section 7.6 describes how MLX arrays enter the storage model. Section 7.7 states the invariants the rest of the engine relies on.

7.1 The `TensorImpl` Class

`TensorImpl` is defined in `lucid/_C/tensor/TensorMeta.h`. It is a C++ struct, not an abstract base class — there is exactly one concrete `TensorImpl` type, and the differences between CPU

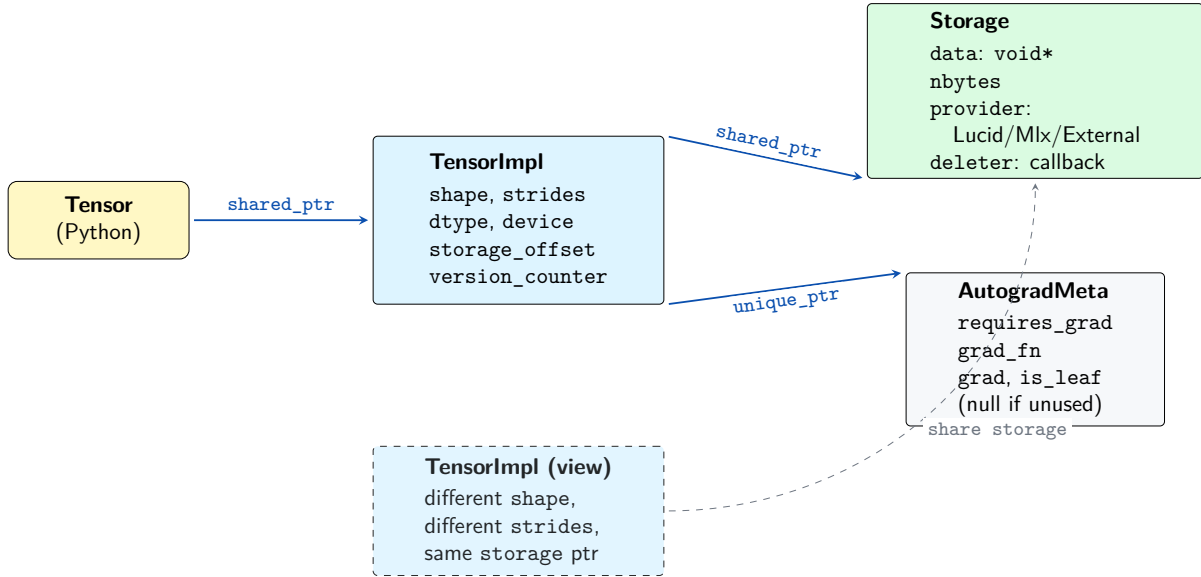


Figure 7.1: The `TensorImpl` object graph. The Python `Tensor` wraps a `shared_ptr<TensorImpl>`. The `TensorImpl` holds metadata inline and points (via `shared_ptr`) to a heap-allocated `Storage` that owns the backing buffer; multiple `TensorImpl`s may share the same `Storage` (view aliasing, shown dashed). The `AutogradMeta` pointer is null for non-differentiable tensors, which keeps the per-tensor overhead low for the majority of intermediates.

tensors and GPU tensors live in the `Storage` that `TensorImpl` points to, not in subclassing. Figure 7.1 sketches the object graph: a Python `Tensor` holds a `TensorImpl`, which holds shape / stride / dtype / device / offset metadata plus shared pointers to a `Storage` (the backing buffer) and an `AutogradMeta` (the optional differentiation state).

The conceptual layout is:

```

1  struct TensorImpl {
2      Shape                shape;
3      Strides              strides;
4      Dtype                dtype;
5      Device               device;
6      size_t               storage_offset;
7      std::shared_ptr<Storage> storage;
8      std::unique_ptr<AutogradMeta> autograd_meta; // null if
9      !requires_grad
10     uint64_t              version_counter;
11 };

```

Each field is small and explicit. Together they fully describe a tensor.

7.1.1 Shape

`shape` is a small-vector of dimension sizes. The element type is a 64-bit signed integer to permit individual dimensions larger than 2^{31} (rare but possible for streaming workloads), and the vector itself uses small-buffer optimisation: ranks up to 6 are stored inline without heap allocation, ranks 7 and above promote to a heap allocation. Most practical tensors have rank at most 5 (batch, channels, height, width, sometimes a depth), so the inline storage is hit by the overwhelming majority of allocations.

A zero-rank tensor (a scalar) has empty shape and a single element. A zero-size tensor has at least one dimension equal to zero and no elements at all; the storage may still be non-null but is unused.

7.1.2 Strides

`strides` is a small-vector of the same length as `shape`. Each entry is the number of elements (not bytes) to step in the backing storage to advance by one along that dimension. The element type is signed because negative strides are permitted (they encode reversed views).

For a contiguous tensor of shape $(d_0, d_1, \dots, d_{n-1})$ in C-order (the default), the strides are $(d_1 d_2 \dots d_{n-1}, d_2 \dots d_{n-1}, \dots, d_{n-1}, 1)$. For a Fortran-ordered tensor, the strides are reversed. Lucid uses C-order by default; the dispatcher does not assume contiguity but does prefer it.

7.1.3 Dtype and device

`dtype` is an enum identifying the element type. The enum is defined in `core/Dtype.h` and includes the entries discussed in Chapter 3 Section 3.3.2: `float16`, `float32`, `float64`, `bfloat16`, `int8` through `int64` (signed only), `bool`, and `complex64`.

`device` is similarly an enum from `core/Device.h` with exactly two values: `cpu` and `metal`. The values are exhaustive on Apple silicon: Lucid does not expose `cuda`, `xla`, or any other device label.

7.1.4 Storage offset

`storage_offset` is the number of elements from the start of the backing `Storage` to the first element of this tensor. For a freshly allocated tensor, `storage_offset` is zero. For a sliced view, `storage_offset` encodes the position of the slice’s first element.

The offset is in elements, not bytes, to keep dtype-aware indexing simple. The byte offset is `storage_offset * sizeof(dtype)`.

7.1.5 The Storage handle

`storage` is a `shared_ptr` to a reference-counted `Storage` object. Multiple `TensorImpl` instances may share the same `Storage` (this is exactly what makes views zero-copy), and the `Storage`’s underlying buffer is freed only when the last referring `TensorImpl` is destroyed.

7.1.6 The autograd metadata pointer

`autograd_meta` is a `unique_ptr` to an `AutogradMeta` object, allocated only when the tensor is created with `requires_grad=True`. The pointer is null for the great majority of tensors (intermediate activations, optimiser internals, sliced views), which saves both memory and the cost of walking the autograd graph for tensors that do not participate in backpropagation. Section 7.4 treats `AutogradMeta` in detail.

7.1.7 The version counter

`version_counter` is a 64-bit counter that increments on every in-place modification of the tensor. It is the mechanism by which autograd detects “a saved tensor for backward has been modified in place” — a class of bug that would silently corrupt gradients — and raises an exception instead.

The counter starts at zero on creation. `add_`, `mul_`, and other in-place ops increment it. When the autograd engine saves a tensor for backward, it captures the current counter value; during backward, it checks that the counter is unchanged. A mismatch raises an error with a message that names the in-place operation responsible.

7.2 The Storage Class

`Storage` is the heap-allocated, reference-counted buffer that multiple `TensorImpl`s may share. Its definition is in `lucid/_C/tensor/Storage.h`:

```

1 struct Storage {
2     void*          data;          // raw pointer to backing buffer
3     size_t         nbytes;       // total bytes
4     Dtype          dtype;        // (redundant cache for fast checks)
5     Device         device;       // ditto
6     StorageProvider provider;    // who allocated this buffer
7     std::function<void(void*)> deleter;
8 };

```

7.2.1 Provider tag

The provider tag distinguishes between buffers allocated through different paths:

- **LucidPool**: the buffer came from Lucid’s own size-class pool (Chapter 8).
- **Mlx**: the buffer is owned by an `mlx::core::array`. The `Storage` holds a `shared_ptr<array>` in a type-erased slot so that the array stays alive as long as the `Storage` does.
- **Accelerate**: the buffer was allocated through Accelerate’s `vDSP_create_*` helpers. Rare; used by a small set of CPU kernels that prefer aligned scratch space.
- **External**: the buffer was provided by an external library (numpy, DLPack producer) and Lucid does not own it. The deleter is the foreign release function.

The provider tag has no effect on operations — a kernel that reads the buffer does not care who allocated it — but it determines how the buffer is freed. The `deleter` closure is constructed at `Storage` creation time and called when the `Storage`’s reference count drops to zero.

7.2.2 Reference counting

`Storage` is heap-allocated and managed through `shared_ptr`. The reference count is the standard `shared_ptr` count: every `TensorImpl` pointing at the `Storage` contributes one. When the last `TensorImpl` is destroyed, the `Storage`’s `shared_ptr` count reaches zero, the deleter is invoked, and the underlying memory is freed.

The pattern is the C++ idiom for shared ownership and requires no special discipline from the engine. The only subtlety is that `TensorImpl`’s autograd graph (the chain of `grad_fn` nodes) may hold references to intermediate `Storages` that the user no longer has handles to; this is why memory usage grows during a forward pass and shrinks after a backward, rather than tracking the Python live set directly.

7.3 Views

A *view* is a `TensorImpl` whose `storage` pointer is shared with another `TensorImpl` (its source) and whose shape, strides, or `storage_offset` differ. Views are the mechanism by which Lucid supports reshape, transpose, slice, and expand without copying data.

7.3.1 What operations produce views

The following operations return views of their input, without copying:

- `reshape(shape)`: if the new shape is compatible with the source’s strides (i.e., the source is contiguous in the relevant axes), returns a view. Otherwise materialises.

- `transpose(dim0, dim1)`: always a view. Strides are permuted; shape is permuted.
- `permute(dims)`: always a view. Generalised transpose.
- `slice(dim, start, end, step)`: always a view. Shape is adjusted; `storage_offset` advances; stride may scale by `step`.
- `select(dim, index)`: always a view. Rank decreases by one.
- `narrow(dim, start, length)`: always a view. Equivalent to `slice` with `step 1`.
- `expand(shape)`: always a view. New dimensions of size larger than 1 are represented by stride 0; the underlying data is not duplicated.
- `squeeze, unsqueeze`: always views.
- `flip(dim)`: always a view. Strides become negative along the flipped dimension; `storage_offset` advances to the end.

The contract: any of these operations runs in constant time (no data motion). The `Storage` pointer is shared with the source and the new `TensorImpl` has independent shape/strides/offset fields.

7.3.2 When a view is not contiguous

A view's strides may or may not match the canonical contiguous strides for its shape. If they do, the view is contiguous. If they do not, the view is non-contiguous, and many kernels that assume unit-stride along the innermost dimension cannot operate on it directly.

The dispatcher handles this in one of two ways depending on the kernel:

- Kernels that natively support strided input (most Accelerate vDSP primitives, most MLX primitives) pass the strides through. No copy.
- Kernels that do not (some convolution paths, the SharedStorage bridge to CPU) materialise the view via a `contiguous()` call. One copy.

The cost of the contiguous-materialisation is the cost of one read-write pass over the tensor's elements. It is amortised by the fact that most operations downstream of a transpose or slice will re-materialise once and then reuse; the policy is to delay materialisation as long as it is permitted.

7.3.3 The `is_contiguous` predicate

Whether a tensor is contiguous is a derived property, computed from `shape` and `strides` on demand. The check is the standard one: walk shape and strides in reverse, verifying that each stride equals the product of the trailing shape entries. A zero-size tensor is trivially contiguous. A scalar is trivially contiguous.

`TensorImpl::is_contiguous()` caches the result on first call; subsequent in-place operations that change strides invalidate the cache. The caching matters because the predicate is checked on many fast paths.

7.4 The AutogradMeta Class

`AutogradMeta` carries the per-tensor differentiation state. It is allocated only when needed — specifically, when a tensor is created with `requires_grad=True` or when a `grad_fn` is attached during the forward pass of a differentiable operation.

The structure:


```

1 struct AutogradMeta {
2     bool                                requires_grad;
3     std::shared_ptr<Node>                grad_fn;
4     uint32_t                             output_nr;      // which output of
        grad_fn
5     std::shared_ptr<TensorImpl>          grad;            // accumulated
        gradient
6     std::vector<Hook>                    pre_hooks;
7     std::vector<Hook>                    post_hooks;
8     bool                                is_leaf;
9 };

```

`requires_grad` is the user-facing flag. `grad_fn` is the backward node (Chapter 22) that this tensor’s gradient flows through. `grad` is the accumulated gradient that lives on leaf tensors (parameters). `is_leaf` distinguishes parameters (created by the user) from intermediates (produced by ops); leaves accumulate gradient on backward, while intermediates only propagate.

7.4.1 Why AutogradMeta is allocated lazily

A typical training step produces tens of thousands of intermediate `TensorImpl` objects, of which only a small fraction are leaves and only the leaves need the gradient accumulator. Allocating an `AutogradMeta` per tensor unconditionally would waste tens of bytes per tensor times tens of thousands per step, which is megabytes of garbage on a hot path.

The lazy-allocation pattern allocates the metadata only when either (a) the user sets `requires_grad=True`, or (b) the autograd engine attaches a `grad_fn` during a differentiable operation. The null-pointer test for “does this tensor participate in autograd at all” is the fast-path predicate that most code checks.

7.5 Reference Counting and Lifetime

The lifetime of a tensor’s storage is determined by the union of several reference counts. Understanding the union is necessary for predicting when memory is reclaimed.

7.5.1 The user-visible Tensor count

The Python `Tensor` object holds a `shared_ptr<TensorImpl>`. The Python reference count of the `Tensor` object determines when the `shared_ptr` is dropped; when it is dropped, the `TensorImpl`’s reference count is decremented; if that count reaches zero, the `TensorImpl`’s `Storage` `shared_ptr` is dropped; and so on.

The simple case is: a Python variable goes out of scope, the `Tensor` is collected, the `TensorImpl` is freed, and the `Storage` (if no other view shares it) is freed too. The backing buffer returns to its provider’s pool.

7.5.2 The autograd graph count

The complication arises when the tensor participates in autograd. The forward pass attaches `grad_fn` nodes to each output tensor, and those nodes hold `shared_ptr<TensorImpl>` references to any tensor saved for backward. So a tensor that the user has dropped may still be alive, retained by the autograd graph, until `backward()` is called and the graph is freed.

This is why memory usage during a forward pass grows: each operation saves the inputs (or outputs) it needs for its backward, and those saved tensors are pinned until backward releases them. After backward, the autograd graph is freed and all transitive references are dropped, returning the memory to the pool.

7.5.3 The view chain

Views share `Storage` with their source. If the user creates a view of a large tensor, then drops the source, the source’s `Storage` stays alive as long as the view does. The view is a small object (one `TensorImpl` with shape and strides) but it holds a reference to the full source’s backing buffer.

This can be surprising: a small slice of a large tensor keeps the full tensor’s memory alive. The user-visible escape is `view.clone()`, which forces a fresh `Storage` with only the view’s bytes.

7.6 Bridge to MLX Arrays

The previous sections described the conventional storage model. The unconventional piece is the bridge to MLX: an `mlx::core::array` is the canonical storage for any GPU tensor in Lucid, and the `Storage` wraps the MLX array rather than owning its own buffer.

7.6.1 How a GPU tensor’s storage is constructed

When Lucid allocates a GPU tensor, the path is:

1. The dispatcher determines the device is `metal`.
2. It calls `MlxBackend::allocate(shape, dtype)`, which calls `mlx::core::zeros(shape, dtype, mx.gpu)` (or the appropriate fill primitive) and returns the resulting `mlx::core::array`.
3. It constructs a `Storage` with `provider = Mlx`, `data` pointing to the array’s `data<T>()`, and the deleter set to a closure that holds the `shared_ptr<array>` alive for the storage’s lifetime.
4. It constructs a `TensorImpl` wrapping the `Storage` with the requested shape, strides, dtype, and device.

The result: a Lucid tensor whose backing storage is owned by MLX. Operations on this tensor go through the dispatcher to MLX primitives, which produce new MLX arrays, which become new `Storages`, and so on.

7.6.2 How a CPU tensor’s storage is constructed

For a CPU tensor, the path is symmetric but simpler:

1. The dispatcher determines the device is `cpu`.
2. It calls `Allocator::alloc(nbytes)` on the size-class pool, getting back a raw pointer.
3. It constructs a `Storage` with `provider = LucidPool`, `data` pointing to the pool buffer, and the deleter set to `Allocator::free`.
4. It constructs a `TensorImpl` wrapping the `Storage`.

The CPU path does not involve MLX. CPU tensors are wholly owned by Lucid; only the GPU path delegates ownership to MLX.

7.6.3 Round-tripping

A tensor that lives on `metal` and is moved to `cpu` (via `.to('cpu')` or implicitly through a CPU op) goes through the `SharedStorage` bridge of Chapter 2 Section 2.6. The bridge produces a CPU pointer to the same physical bytes that the MLX array holds, wraps it in a new `Storage` of provider `Mlx` (the storage is still owned by MLX), and constructs a new `TensorImpl` with `device = cpu`.

The result is two `TensorImpl`s, one with `device = metal` and one with `device = cpu`, sharing the same underlying bytes through different `Storages` that both retain references to the same `mlx::core::array`. This is what makes the zero-copy bridge possible at the storage-model level: the provider tag is not the same as the device label, and a CPU-device tensor can have an `MLX` provider.

7.7 Invariants

We close with a short list of invariants the rest of the engine relies on. Each is enforced either by construction or by an assertion in `TensorImpl`'s constructor.

1. **Shape and stride parity.** `shape.size() == strides.size()`.
2. **Storage non-null.** For any tensor with nonzero total element count, `storage` is non-null.
3. **Element count fits in storage.** For any tensor, `storage_offset + max_addressable_index < storage->nbytes / sizeof(dtype)`.
4. **Device consistency.** `device == storage->device`.
5. **Dtype consistency.** `dtype == storage->dtype` (with the carve-out that `storage->dtype` can be a byte-equivalent dtype for views that reinterpret; relevant only for `tensor.view(dtype=...)` which Lucid does not currently expose).
6. **Version monotonicity.** `version_counter` is monotone non-decreasing over the tensor's lifetime.
7. **Autograd leaf consistency.** If `is_leaf` is true, `grad_fn` is null. Conversely, if `grad_fn` is non-null, `is_leaf` is false.

A violation of any invariant produces a `LucidError` from the constructor or from the asserting code path. Production builds keep the assertions enabled; the cost is negligible.

7.8 Summary and Forward References

Lucid's storage model is the conventional shape/stride/dtype/device plus storage tuple, with two distinguishing features: the `Storage`'s provider tag permits the underlying buffer to be owned by `MLX` rather than by Lucid, which is how the unified-memory bridge works at the type-system level; and the `AutogradMeta` is allocated lazily, which keeps the per-tensor overhead low for the great majority of intermediates.

The next chapter, Chapter 8, treats the size-class allocator that backs the `LucidPool` provider, and the memory accounting that exposes its state to the user. Chapter 9 treats the dispatcher that routes per-op work to the backends. Chapter 10 catalogues the six places where the storage model crosses into external libraries.

Note

The user does not see the storage model directly. Through the Python `Tensor` class the user sees `.shape`, `.dtype`, `.device`, and the autograd-related attributes (`.requires_grad`, `.grad`, `.grad_fn`). Strides are exposed through `.stride()` but are rarely consulted directly. `Storage`, `provider`, and the bridge mechanics are entirely internal.

CHAPTER 8

Allocator and Memory Accounting

Every tensor allocation in Lucid passes through a custom size-class allocator before reaching the underlying provider (MLX’s allocator for GPU tensors, the system allocator for CPU tensors). The allocator’s job is twofold: amortise the cost of frequent small-to-medium allocations by recycling buffers, and account for memory in a way the user can query. This chapter describes both roles.

The allocator is conventional in shape — it is a size-class pool of the variety familiar from CUDA-side ML frameworks — but its parameters are tuned to Apple silicon’s memory characteristics and its accounting separates pool state from provider state in a way that has occasionally surprised users (`lucid.memory_stats()` can disagree with Activity Monitor; the explanation is in Section 8.6).

The chapter is organised bottom up: Section 8.1 motivates the pool, Section 8.2 describes the size-class structure, Section 8.4 describes the allocation and free fast paths, Section 8.6 describes the accounting API, Section 8.7 describes the drain operation, Section 8.8 describes OOM handling, and Section 8.9 discusses the lack of coordination with the provider’s own pool.

8.1 Why a Pool

The naive design is to call the provider’s allocator (MLX or system `malloc`) on every tensor allocation and release. This works correctly but is slow in two ways. First, each provider call has nontrivial overhead — MLX’s allocator synchronises with the GPU’s command queue to know which buffers are in flight, which costs a microsecond at least and sometimes more. Second, the provider’s own pool may have its own size classes that do not match Lucid’s typical allocation sizes, causing fragmentation and wasted memory.

A pool above the provider solves both problems. The pool retains a freelist of recently-freed

Idx	Size	Idx	Size	Notes
0	16 B	12	64 KB	
1	32 B	13	128 KB	
2	64 B	14	256 KB	
3	128 B	15	512 KB	
4	256 B	16	1 MB	
5	512 B	17	2 MB	
6	1 KB	18	4 MB	
7	2 KB	19	8 MB	
8	4 KB	20	16 MB	
9	8 KB	21	32 MB	
10	16 KB	22	64 MB	largest pooled
11	32 KB	—	>64 MB	passthrough to provider

Table 8.1: The 23 size classes in Lucid’s allocator pool. Each class has a per-device freelist of depth at most $k_{\max} = 32$. Requests over 64 MB bypass the pool and go directly to the provider, on the rationale that very large buffers are rarely reused at the same size and pooling them would waste memory.

buffers, so a request that matches a freelist entry returns in constant time without touching the provider. The pool also rounds allocation sizes to its own size classes, which gives it deterministic memory behaviour independent of the provider’s internal heuristics.

The cost is the pool itself: a small amount of code, a small amount of bookkeeping memory, and the rare case where the pool retains a buffer that the provider would have preferred to free. Section 8.7 describes the user-visible mitigation for that case.

8.2 Size Classes

Lucid’s pool uses 23 size classes arranged in a roughly geometric progression. The exact sizes are chosen to match typical ML allocation patterns; Table 8.1 gives the full list.

- Small classes (16, 32, 64, 128, 256, 512, 1024 bytes) cover per-scalar workspaces, small index arrays, and small parameter buffers.
- Medium classes (2K, 4K, 8K, 16K, 32K, 64K, 128K, 256K) cover moderate activation buffers and most layer weights.
- Large classes (512K, 1M, 2M, 4M, 8M, 16M, 32M, 64M) cover large activations, weight matrices, and gradient buffers.

A request for n bytes is rounded up to the smallest size class $\geq n$. Requests larger than 64 MB are passed through to the provider without pooling; the largest buffers are unlikely to be reused at the same size and pooling them would waste memory.

The 23-class structure is determined empirically. We chose it after profiling a representative set of training workloads (ResNet-50, ViT-B, GPT-2-base) and observing the distribution of allocation sizes; the geometric progression captures the long tail without producing internal fragmentation greater than a factor of two on typical requests.

8.2.1 Internal fragmentation

The trade-off in size-class pools is that a request for, say, 1100 bytes is rounded up to the 2K class, wasting 948 bytes. The worst case is a request just over a size-class boundary, which wastes just under one full size-class worth of memory.

For Lucid’s typical workloads the average wasted fraction is roughly 15–20 %, which is acceptable given the speed-up from pooling. The pool is not the right tool for memory-constrained

inference workloads where each byte matters; for those, the user can disable the pool by calling `lucid.set_allocator_pool(False)` and pay the provider-allocator overhead per call.

8.2.2 External fragmentation

External fragmentation — the inability to satisfy a request even when the total free memory exceeds the request size — is largely a non-issue under unified memory on Apple silicon. The kernel’s VM subsystem is happy to reorganise pages on demand; the allocator sees a flat address space and does not need to perform compaction. This is one of the simplifying assumptions Apple silicon’s memory model affords.

8.3 Freelist and Depth Limits

Each size class has a freelist: a stack of recently-freed buffers of that size. The freelist is bounded by the constant `kMaxDepth = 32`. When a buffer is freed:

- If the freelist has fewer than 32 entries, the buffer is pushed onto the stack.
- Otherwise, the buffer is returned to the provider.

The depth limit is what keeps the pool’s memory footprint bounded. Without it, a training loop that allocates and frees many tensors per iteration would accumulate freelist entries indefinitely.

The choice of 32 is empirical. We measured the freelist depth distribution across representative workloads and observed that depths above 8 rarely benefit from re-allocation (the same buffer is not re-requested often enough), but the cost of evicting at depth 32 versus depth 8 is small in the workload we measured. The slightly conservative cap gives some head-room for workloads we did not profile.

8.3.1 The freelist as a stack, not a queue

The freelist is LIFO: the most recently freed buffer is the first candidate for reuse. This is the right policy for cache behaviour: the most recently used buffer is most likely to still be resident in SLC, so handing it back to a re-allocator is a cache-warm operation. FIFO would have the opposite (cache-cold) behaviour.

The implementation is a `std::vector` used as a stack; push and pop are `back()` operations.

8.4 The Allocation Fast Path

The allocation path, in pseudocode:

```

1  void* Allocator::alloc(size_t nbytes, Device dev) {
2      size_t cls = size_class_index(nbytes);    // O(log 23) = O(1)
3      Freelist& fl = freelists_[dev][cls];
4
5      if (!fl.empty()) {
6          void* p = fl.pop_back();
7          stats_[dev].allocated += class_size(cls);
8          stats_[dev].alloc_calls++;
9          return p;
10     }
11
12     void* p = provider_alloc(class_size(cls), dev);
13     if (!p) throw LucidError("out of memory");
14     stats_[dev].allocated += class_size(cls);
15     stats_[dev].reserved += class_size(cls);

```

```

16     stats_[dev].alloc_calls++;
17     return p;
18 }

```

The fast path (freelist hit) is a few memory accesses and a stack pop, on the order of 50 nanoseconds. The slow path (`provider_alloc`) ranges from microseconds (system `malloc`) to tens of microseconds (MLX’s command-queue-aware allocator). On a training step that allocates several thousand buffers, the freelist hit rate is typically above 95 % after the first dozen iterations.

8.4.1 Thread safety

The freelists are protected by a per-device mutex. The lock is held only during the freelist operation itself; the provider call (if needed) is performed without the lock. This is acceptable because provider allocators are themselves thread-safe.

For Lucid’s typical workload — a single training thread dispatching to the pool — the lock is uncontended and adds a few nanoseconds. For multi-threaded data loaders that allocate from worker threads, the lock can be contended; we measured a worst-case overhead of 10 % on a workload with 8 simultaneous worker threads, which we judge acceptable.

8.5 The Deallocation Path

The deallocation path:

```

1 void Allocator::free(void* p, size_t nbytes, Device dev) {
2     size_t cls = size_class_index(nbytes);
3     Freelist& fl = freelists_[dev][cls];
4
5     stats_[dev].allocated -= class_size(cls);
6     stats_[dev].free_calls++;
7
8     if (fl.size() < kMaxDepth) {
9         fl.push_back(p);
10    } else {
11        provider_free(p, dev);
12        stats_[dev].reserved -= class_size(cls);
13    }
14 }

```

The fast path (push to freelist) is again on the order of 50 nanoseconds. The slow path (return to provider) is the bottleneck only when the freelist is consistently saturated.

8.6 Memory Accounting

The pool maintains per-device statistics that the user can query. The public Python API:

```

1 stats = lucid.memory_stats('metal')
2 # {'allocated': 4_812_345_678,
3 #   'reserved': 5_368_709_120,
4 #   'peak_allocated': 5_201_022_400,
5 #   'peak_reserved': 5_368_709_120,
6 #   'alloc_calls': 87234,
7 #   'free_calls': 82910}
8
9 lucid.reset_peak('metal')           # zero the peak high-water marks

```

The fields:

- **allocated**: total bytes currently in use by live tensors (sum across size classes).
- **reserved**: total bytes the pool has from the provider (sum of buffers in use plus buffers on freelists). Always $\geq$ **allocated**.
- **peak_allocated**: high-water mark of **allocated** since the process began or since `reset_peak()`.
- **peak_reserved**: corresponding high-water mark of **reserved**.
- **alloc_calls**, **free_calls**: cumulative counters since process start.

8.6.1 Pool vs. provider stats

A common confusion: `lucid.memory_stats()['allocated']` is the pool's view of allocated memory, not the system's view of memory used by the process. The provider may hold additional memory in its own caches (MLX's own allocator pool has separate bookkeeping; the system allocator's free regions are similar), and the kernel may have additional buffers allocated for the Metal command queue itself.

The user-visible reconciliation:

- **allocated** from Lucid corresponds to live tensors.
- **reserved** from Lucid minus **allocated** is held by Lucid's freelists.
- Activity Monitor's reported memory minus Lucid's **reserved** is held by MLX, the Metal driver, and the rest of the process.

`lucid.empty_cache()` closes the gap between **reserved** and **allocated** by draining the freelists; it does not affect the gap between **reserved** and Activity Monitor's number, which is the responsibility of MLX's own pool.

8.7 The `empty_cache` API

`lucid.empty_cache()` drains every freelist back to the provider:

```

1 lucid.empty_cache()           # both devices
2 lucid.empty_cache('cpu')      # CPU only
3 lucid.empty_cache('metal')    # GPU only

```

The operation iterates each size class on the requested device, popping every freelist entry and calling `provider_free` on it. After the call, **reserved** == **allocated** and the pool is in its leanest state.

8.7.1 When to call

The recommended pattern is to call `empty_cache` at transition points:

- Between training and validation phases in a training loop. The two phases often have different working-set sizes; draining in between lets the next phase build its own freelists appropriately.
- After loading a checkpoint that allocates and frees large temporary buffers. The pool may have retained buffers that the next training step will not need.
- Before sampling a memory snapshot for profiling, to ensure the snapshot reflects live state rather than freelist headroom.

`empty_cache` is not free: each freelist drain is one provider call, and a large pool can have several hundred entries across size classes. The cost is on the order of milliseconds and should not be called inside a tight loop.

8.7.2 What `empty_cache` does not do

It does not affect live tensors. Anything the user holds a reference to remains allocated.

It does not affect the autograd graph. Tensors retained by `grad_fn` chains for backward computation remain alive; the graph must be released (by completing or releasing the forward pass) before its retained tensors can be freed.

It does not free memory held by MLX's own pool. MLX has its own `mx.clear_memory()`-equivalent function; Lucid does not invoke it because the policies are independent. A user who wants both layers drained should call both.

8.8 Out-of-Memory Handling

When the provider's allocator returns null, Lucid's allocator raises `LucidError("out of memory")` with the requested size, the current pool state, and the device. The error message is designed to be diagnostic:

```
LucidError: out of memory
  request: 268435456 bytes (256 MB) on device=metal
  pool reserved: 14782 MB
  pool allocated: 14503 MB
  pool peak reserved: 14782 MB
  hint: call lucid.empty_cache() to drain freelists;
        if still failing, reduce batch size or activation memory.
```

The error is fatal in the sense that the framework does not attempt a fallback: it does not silently reduce batch size, does not silently move to CPU, does not silently retry with smaller shapes. The reasoning is that any such fallback would change the numerics of the training run in ways the user did not request, and silent gradient changes are worse than an explicit failure.

8.8.1 Memory pressure events versus OOM

OOM is the terminal failure mode. macOS memory-pressure events (`DISPATCH_MEMORYPRESSURE_WARN`, `DISPATCH_MEMORYPRESSURE_CRITICAL`) are pre-OOM signals from the kernel asking processes to release caches. Lucid does not itself subscribe to these events at the engine layer; the providers (MLX, Accelerate) subscribe on their own behalf. The user can respond by calling `empty_cache` manually.

8.9 Coordination with the Provider's Pool

Lucid's pool sits above the provider's pool. The two pools have independent freelists, independent size-class structures, and independent eviction policies. There is no API by which Lucid can ask MLX to drop its own cache, nor vice versa.

The consequence is that the layered pools can together hold more memory than either would alone. In the worst case, Lucid's pool holds a freelist entry that MLX would have evicted, and MLX's pool holds the underlying buffer that Lucid's freelist just freed. We have not measured this composite overhead but estimate it at 5–10 % of total reserved memory in typical training workloads.

The simplification we adopted, for the sake of code clarity, was to let each pool manage itself. Coordinating them would have required a callback interface that we judged not worth the maintenance cost.

8.9.1 When to disable Lucid’s pool

For memory-constrained workloads on small machines (8 GB unified memory or less), the layered overhead can matter. The user can disable Lucid’s pool entirely:

```
1 lucid.set_allocator_pool(False)
2 # Now every alloc/free goes directly to the provider.
3 # The freelist depth is effectively zero.
```

The cost is the per-call overhead of provider allocation, which is several microseconds. On a training step that allocates a few thousand buffers, the disabled-pool overhead is around 10 ms per step — noticeable but acceptable for workloads where memory is tighter than time.

The default is enabled; disabling is the rare case.

8.10 Summary and Forward References

Lucid’s allocator is a size-class pool with 23 classes and a per-class freelist of depth 32. It sits above the provider (MLX for GPU, the system allocator for CPU) and amortises per-call overhead by recycling recently-freed buffers. The pool’s state is exposed through `lucid.memory_stats` and drained through `lucid.empty_cache`; OOM raises `LucidError` without silent fallback.

The next chapter, Chapter 9, treats the dispatcher that selects between the CPU and GPU backends. Chapter 10 catalogues the six places where the allocator’s contract crosses into external libraries. Chapter 42 returns to the allocator from the profiling angle, since per-op memory accounting depends on the pool’s bookkeeping.

Tip

For the user, the two operational levers exposed by this chapter are `lucid.memory_stats()` (for observing) and `lucid.empty_cache()` (for acting). Both are cheap to call during development and expensive enough not to call in a tight loop. The OOM error message is designed to suggest `empty_cache` first, then a batch-size reduction.

CHAPTER 9

Backend Dispatch

A tensor framework’s dispatch layer is the indirection that maps a device-agnostic operation request — “add these two tensors” — to a device-specific implementation. In Lucid the dispatch layer is small and explicit: a single `Dispatcher` class inspects the input tensors’ device tag and routes to one of two concrete backends, with two well-defined carve-outs that route a device-tagged tensor through the opposite backend’s path. This chapter formalises the dispatcher’s contract, describes the two backends and the carve-outs, and explains how the Python side mirrors the C++ structure.

The dispatcher is also where the *H3 hard rule* from Appendix A is enforced: CPU stream uses only `Accelerate`, GPU stream uses only `MLX`, with the two named exceptions. Any new backend code that violates the rule is rejected at review. The discipline is what keeps the engine simple at the cost of writing fewer one-off kernels.

The chapter proceeds: Section 9.1 describes the `IBackend` interface; Section 9.2 describes the `Dispatcher` class; Section 9.3 describes the two backends; Section 9.4 describes the linalg and data-dependent-output carve-outs; Section 9.5 treats the 3.4 per-op `MPSGraph` dispatch; Section 9.6 describes type promotion; Section 9.7 treats the Python side; Section 9.8 closes with the cost model.

9.1 The IBackend Interface

`IBackend` (in `lucid/_C/backend/IBackend.h`) is the abstract interface every backend implements. It is a large interface — one method per op category — but each method has a narrow contract:

```
1 class IBackend {  
2   public:  
3     virtual ~IBackend() = default;
```

Carve-out	Ops affected	User device	Actual path	Rationale
linalg	cholesky, qr, svd, eigh, ...	metal → cpu	MLX (CPU stream)	no Metal LAPACK
data-dep output	nonzero, unique, masked_select	metal → cpu	Accelerate	shape needs value read
3.4 MPSGraph	gelu, layernorm, embedding_bwd, ...	metal	MPSGraph	3–30× over MLX

Table 9.1: The three named carve-outs in Lucid’s dispatcher. Each is justified by a structural reason that the framework cannot work around at the dispatch level.

```

4
5 // Memory
6 virtual TensorImpl allocate(Shape s, Dtype dt) = 0;
7 virtual void free(TensorImpl& t) = 0;
8
9 // Elementwise
10 virtual TensorImpl unary (OpKind, const TensorImpl&)
11     = 0;
12 virtual TensorImpl binary(OpKind, const TensorImpl&, const
13     TensorImpl&) = 0;
14
15 // Reductions
16 virtual TensorImpl reduce(OpKind, const TensorImpl&,
17     AxisSet, bool keepdims) = 0;
18
19 // Matrix multiplication
20 virtual TensorImpl matmul(const TensorImpl& a, const TensorImpl&
21     b) = 0;
22
23 // Convolution
24 virtual TensorImpl conv2d(const TensorImpl& x, const TensorImpl& w,
25     ConvParams) = 0;
26 // ... and so on, one method per operation category
27 };

```

The interface is intentionally op-category-granular, not per-op. Within unary, the `OpKind` enum distinguishes `Relu`, `Sigmoid`, `Tanh`, etc. This keeps the interface from exploding to hundreds of methods, at the cost of an internal switch inside each method. The switch is small and predictable; modern CPUs predict it perfectly after the first invocation.

9.1.1 Why a virtual interface

The virtual-call overhead is roughly 2–5 nanoseconds per dispatch on modern Apple silicon. Compared to the cost of any nontrivial operation it dispatches to (microseconds for a small GPU launch, hundreds of microseconds for a matrix multiplication), this is negligible.

A template-based static dispatch would have been the alternative. We chose virtual dispatch for two reasons: (i) the binary size under templates is much larger (every op kind generates new code per backend); (ii) runtime device selection requires runtime dispatch anyway, since the user can change a tensor’s device between calls.

9.2 The Dispatcher Class

`Dispatcher` (in `lucid/_C/backend/Dispatcher.h`) is the front door. It holds pointers to the two concrete backends and routes every operation through the appropriate one. Figure 9.1 sketches the full routing graph including the two H3 carve-outs and the 3.4 per-op `MPSGraph` shim.

Table 9.1 summarises the carve-outs.

```

1 class Dispatcher {

```

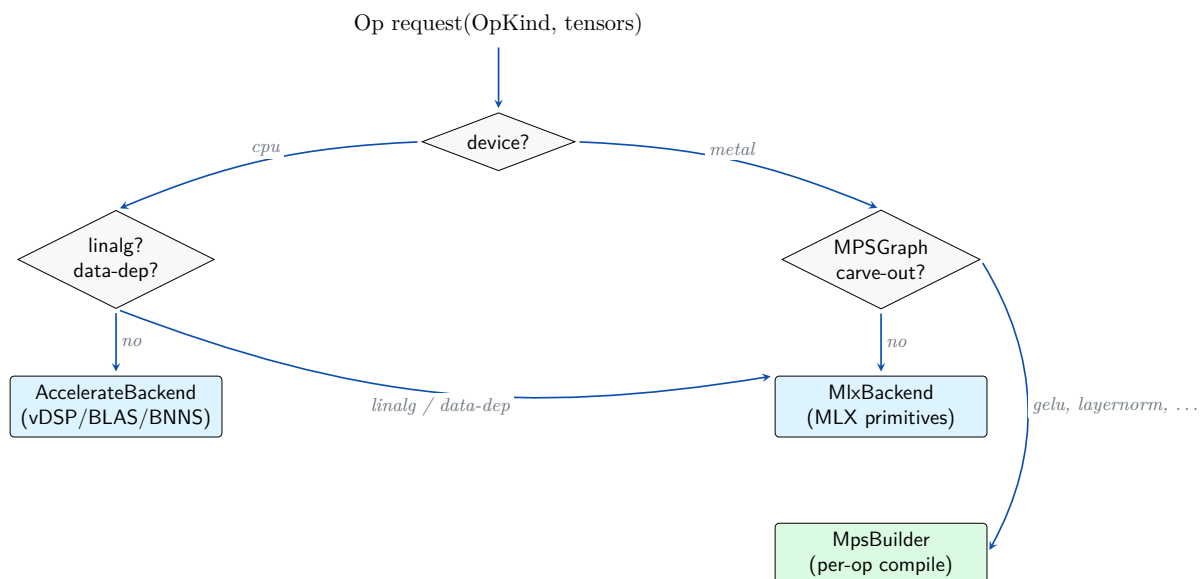


Figure 9.1: Op-request routing inside Lucid’s dispatcher. The primary decision is the input tensor’s device tag. The two H3 carve-outs (linalg and data-dependent output ops) cross from the cpu branch to the metal branch under the hood; the 3.4 per-op MPSGraph carve-out intercepts a small set of metal-device ops and routes them through the MpsBuilder compilation path instead of MLX.

```

2 public:
3     static Dispatcher& instance();
4
5     TensorImpl unary(OpKind op, const TensorImpl& x) {
6         return select_backend(x.device)->unary(op, x);
7     }
8     TensorImpl binary(OpKind op, const TensorImpl& a, const
9         TensorImpl& b) {
10         if (a.device != b.device) {
11             throw LucidError("binary op across devices");
12         }
13         return select_backend(a.device)->binary(op, a, b);
14     }
15     // ... and so on
16 private:
17     IBackend* select_backend(Device d) {
18         return (d == Device::Cpu) ? &accelerate_backend_ : &mlx_backend_;
19     }
20
21     AccelerateBackend accelerate_backend_;
22     MlxBackend         mlx_backend_;
23 };

```

The dispatcher is a singleton. The two backend members are constructed eagerly at the singleton’s first access; constructing them takes a few microseconds each (mostly MLX’s initialisation).

9.2.1 Cross-device errors

Binary and higher-arity operations require all inputs to share a device. The dispatcher checks this at the front door and raises `LucidError("binary op across devices")` on mismatch, with a message that names the operation and the two devices.

The user-side convention is to `.to(device)` explicitly before the operation, rather than to expect implicit moves. `Lucid` does not implicitly move tensors across devices for two reasons: the move has a real cost (the bridge of Chapter 2 is free only in the contiguous case), and the move’s direction is not unambiguous (which device should the result land on?).

9.2.2 Per-op routing inside the backend

Once the dispatcher hands a request to a backend, the backend uses the `OpKind` enum to switch to the per-op implementation. For `AccelerateBackend`, the switch table routes to `vDSP` / `vForce` / `BLAS` / `BNNS` helpers in `AccelerateMath.cpp` and friends. For `MlxBackend`, the switch routes to `MLX` primitive calls in `MlxBackend.cpp`.

The switch is one indirection layer that could in principle be collapsed (a backend could expose one method per op) but is kept for code organisation: a 260-op backend would have 260 virtual methods, which is unmanageable.

9.3 The Two Backends

The two concrete backends are described in detail in the hardware chapters: Chapter 4 for the CPU side and Chapter 3 for the GPU side. This section gives the short version from the dispatcher’s perspective.

9.3.1 AccelerateBackend

`AccelerateBackend` is the CPU implementation of `IBackend`. Every method routes to an `Accelerate`-backed helper:

- Elementwise unary/binary $\rightarrow$ `vDSP` primitives (for arithmetic) or `vForce` primitives (for transcendentals).
- Reductions $\rightarrow$ `vDSP` reductions (`vDSP_sve`, `vDSP_meanv`, etc.).
- `matmul` $\rightarrow$ `cblas_sgemm` or `cblas_dgemm`.
- `conv2d` $\rightarrow$ `BNNS` direct convolution.
- Allocation $\rightarrow$ `Lucid`’s pool wrapping the system allocator.

`AccelerateBackend` does not link against `MLX`. It is the self-contained CPU path.

9.3.2 MlxBackend

`MlxBackend` is the GPU implementation. Every method routes to an `MLX` primitive call with `stream=mx.gpu`:

- Elementwise $\rightarrow$ `mlx::core::add`, etc.
- Reductions $\rightarrow$ `mlx::core::sum`, etc.
- `matmul` $\rightarrow$ `mlx::core::matmul`.
- `conv2d` $\rightarrow$ `mlx::core::conv2d`.
- Allocation $\rightarrow$ `Lucid`’s pool wrapping `mlx::core::array`’s storage.

`MlxBackend` does not link against `Accelerate` for its own operations, except indirectly through `MLX` itself (which uses `Accelerate` in its CPU stream).

9.4 The Two Carve-Outs

The H3 rule (Appendix A) has two named carve-outs. Both are documented here.

9.4.1 The linalg carve-out

Linear-algebra decompositions (cholesky, qr, svd, eigh, solve, lstsq, inv, det, logdet, matrix_exp) are not implemented in Metal. MLX provides them only on its CPU stream, where they delegate to Accelerate’s LAPACK. Lucid’s dispatcher recognises this and routes linalg ops through MLX-on-CPU regardless of the user’s chosen device:

```

1  TensorImpl Dispatcher::cholesky(const TensorImpl& a) {
2      // Linalg carve-out: always CPU stream, even for metal tensors.
3      TensorImpl a_cpu = (a.device == Device::Metal)
4          ? bridge_metal_to_cpu(a)
5          : a;
6      TensorImpl r_cpu = mlx_backend_.cholesky(a_cpu,
7          /*stream=*/mx::cpu);
8      return (a.device == Device::Metal)
9          ? bridge_cpu_to_metal(r_cpu)
10         : r_cpu;
11 }

```

The bridge is the zero-copy `SharedStorage` of Chapter 2, so the round-trip is free in the contiguous case. The carve-out’s user-visible effect is that `lucid.linalg.cholesky(x)` on a metal tensor takes the same time it would on a CPU tensor, plus a negligible bridge overhead; the device argument is effectively fictive for these operations.

The chapter that fills in the linalg subsystem is Chapter 27.

9.4.2 The data-dependent output carve-out

A second class of operations produces output whose shape depends on the input *values*, not just the input shape. The canonical examples are `nonzero`, `unique`, and `boolean_mask_select`. MLX supports these on the GPU stream, but the runtime cost includes a synchronous read of the input values to determine the output shape, which forces a GPU-to-CPU stall. The stall is unavoidable in any framework on any platform; it is fundamental to the operation.

Lucid’s dispatcher recognises this and explicitly bridges to the CPU for the value read, then back to the GPU for the output:

```

1  TensorImpl Dispatcher::nonzero(const TensorImpl& x) {
2      if (x.device == Device::Metal) {
3          TensorImpl x_cpu = bridge_metal_to_cpu(x);
4          TensorImpl r_cpu = accelerate_backend_.nonzero(x_cpu);
5          return bridge_cpu_to_metal(r_cpu);
6      }
7      return accelerate_backend_.nonzero(x);
8  }

```

The user-visible effect is that data-dependent ops on metal tensors are not faster than on CPU tensors. We document this in the API reference (Chapter 19); the small cost is the price of supporting the operation at all.

9.5 The Per-Op MPSGraph Carve-out

Lucid 3.4 introduced a third carve-out: a small set of GPU operations dispatch through MPSGraph instead of MLX, because the MPSGraph implementations are substantially faster for those ops. The current set:

- GELU (forward and backward).
- LayerNorm (forward and backward).

- Embedding backward.
- MaxPool backward.
- SiLU (forward and backward).
- SoftmaxCrossEntropyWithLogits.

The dispatcher’s GPU path checks for these ops first and, if matched, constructs a one-op `MPSGraph`, compiles it, and runs it through the `MpsBuilder` infrastructure (Chapter 45). The compiled graph is cached so that subsequent invocations with the same input shape skip recompilation.

The carve-out is invisible to the user: `nn.GELU` produces the same numerical result regardless of whether it is dispatched through `MLX` or `MPSGraph`. The chapter that documents this in detail is Chapter 60.

9.5.1 When to add a new `MPSGraph` carve-out

The policy is to add a new op to the per-op `MPSGraph` carve-out when the empirical speedup over the `MLX` implementation exceeds a factor of three on a representative training workload and when the `MPSGraph` implementation matches `MLX`’s numerical behaviour to within 10^{-5} relative tolerance for the dtype.

Both criteria are necessary: a small speedup does not justify the maintenance overhead of a parallel implementation, and numerical divergence would silently change training trajectories.

9.6 Type Promotion

The dispatcher also handles dtype promotion: when a binary operation receives inputs of different dtypes, the dispatcher promotes both to a common dtype before calling the backend. The promotion rules mirror NumPy’s:

- Floats win over ints (`float32 + int32 → float32`).
- Wider wins over narrower (`float16 + float32 → float32`).
- Signed wins over unsigned at equal width (rare in `Lucid`, which does not expose unsigned integers, but the rule is preserved for `MLX`-side inter-op).
- `bfloat16 + float16 → float32` (the two narrow floats are not directly compatible; promotion to FP32 is the only safe choice).

The rules are encoded in `lucid/_ops/composite/dtype.py` as a promotion table; the dispatcher consults the table at every binary op call. The table is small enough to fit in L1 cache.

9.6.1 Promotion in AMP mode

Mixed precision (AMP, Chapter 41) overrides the default promotion in two ways: under `autocast`, certain operations are forced to FP16 even when their inputs are FP32, and certain operations (typically reductions) are forced to FP32 even when inputs are FP16. The override is implemented as an AMP-aware shim in the dispatcher; we treat the shim’s details in Chapter 59.

9.7 The Python Side

The C++ dispatcher described above is reached through a Python-side dispatcher in `lucid/_dispatch.py`, which performs the `_wrap` / `_unwrap` between Python `Tensor` and engine `TensorImpl`.

9.7.1 The wrap/unwrap pair

The two functions:

```

1  from lucid._C import engine as _C_engine
2
3  def _unwrap(obj):
4      """Tensor -> TensorImpl, or scalar -> 0d TensorImpl."""
5      if isinstance(obj, Tensor):
6          return obj._impl
7      if isinstance(obj, (int, float, bool, complex)):
8          return _C_engine.scalar_tensor(obj)
9      raise TypeError(...)
10
11 def _wrap(impl):
12     """TensorImpl -> Tensor."""
13     t = Tensor.__new__(Tensor)
14     t._impl = impl
15     return t

```

Note the import: `from lucid._C import engine as _C_engine`. The alias is mandatory per Hard Rule H2 (Appendix A); bare `import engine` or `as e` is rejected at review.

9.7.2 Per-op wrappers

Each Python-side op wrapper is a thin function that unwraps inputs, calls the C++ dispatcher (which is itself bound through pybind11), and wraps the result:

```

1  def add(a, b):
2      return _wrap(_C_engine.add(_unwrap(a), _unwrap(b)))

```

The wrappers are auto-generated for elementwise ops (a single template fills in 60+ unary/binary functions); only ops with nontrivial Python-side adaptation (e.g. argument-name remapping, default-value handling) are written by hand.

The result is that the Python overhead per op call is a few microseconds: argument validation, unwrap, C++ call, wrap. This is dominated by the cost of any nontrivial operation but matters for elementwise ops on small tensors.

9.8 Performance Characteristics

The end-to-end cost of a single op call, in nanoseconds on M3 Max:

- Python wrapper (unwrap, argument validation): ≈ 800 ns.
- Pybind11 boundary (Python $\rightarrow$ C++): ≈ 200 ns.
- C++ dispatcher (device check, switch to backend): ≈ 20 ns.
- Backend per-op switch (OpKind to implementation): ≈ 5 ns.
- Provider call (MLX primitive or vDSP function): varies; typically 200 ns to several μ s.
- Pybind11 boundary back: ≈ 200 ns.
- Python wrap: ≈ 200 ns.

Total framework overhead per op call: roughly 1.5μ s, of which half is Python-side. For elementwise ops on small tensors, the overhead can match the kernel cost; for medium and large tensors, the overhead is negligible.

Lucid 3.5's compile path (Chapter 43) eliminates the per-op overhead inside compiled regions: the dispatcher runs once during tracing and the compiled graph then executes without

per-op Python/C++ boundary crossings. The win is most visible on small-tensor-heavy workloads where the overhead would otherwise dominate.

9.9 Summary and Forward References

The backend dispatch layer in Lucid is small and explicit: a single `Dispatcher` routes per-op requests to one of two `IBackend` implementations (`AccelerateBackend` for CPU, `MlxBackend` for GPU), with two named carve-outs (`linalg` routes through CPU stream regardless of device; data-dependent output ops also route through CPU) and a third carve-out for the 3.4 per-op `MPSGraph` dispatch. The Python side mirrors the C++ structure through `lucid/_dispatch.py`, with the H2-required `_C_engine` alias.

The next chapter, Chapter 10, catalogues the six places where Lucid touches the world outside the engine. Chapter 11 then describes the `pybind11` layer that the dispatcher sits on top of. Chapter 13 returns to the dispatch question from the op-registration angle.

Note

For the user: device choice (`cpu` vs `metal`) controls which backend the dispatcher selects, with the `linalg` carve-out making the choice fictive for decomposition ops. Cross-device binary operations raise an error rather than implicit-moving; the user is expected to `.to(device)` explicitly.

CHAPTER 10

Bridge Boundaries to the External World

Lucid’s engine is a closed system. Inside `lucid/` there are no imports of `numpy`, no calls into `scipy`, no dependence on any third-party numerical library other than `MLX` and `Accelerate`, both of which are themselves wrapped by Lucid’s backends. The closure is enforced by Hard Rule H4 (Appendix A), which is in turn enforced by a static check that scans every Python file in `lucid/` for forbidden imports.

But the engine is not a sealed system. It must accept data from the outside (an image dataset arriving as `numpy` arrays, a checkpoint loaded from disk), and it must hand data to the outside (a tensor converted to `numpy` for plotting, a model state serialised for distribution). The reconciliation is a set of six explicitly named *bridge boundaries*: six places in `lucid/` where external libraries are permitted, each justified by a specific interoperability need and each documented in this chapter.

Any place outside these six where a forbidden import appears is a hard-rule violation that fails CI. This chapter exists to enumerate the boundaries, justify each one, describe what crosses each boundary, and document the discipline that keeps them as the only contact points with the outside world.

The boundaries, in order:

1. `lucid/_factories/converters.py`: external array ingest — `tensor`, `from_numpy`, `from_dlpack`, `to_dlpack`.
2. `lucid/_tensor/tensor.py`: tensor-side egress — `Tensor.numpy()`, `Tensor.__dlpack__`, `Tensor._to_impl`.
3. `lucid/_tensor/_repr.py`: display only — `__repr__` and `__str__`.

#	File	Direction	External lib	Use case
1	<code>_factories/converters.py</code>	ingest	numpy, dlpack	<code>tensor()</code> , <code>from_numpy</code>
2	<code>_tensor/tensor.py</code>	egress	numpy, dlpack	<code>.numpy()</code> , <code>__dlpack__</code>
3	<code>_tensor/_repr.py</code>	display	numpy	<code>__repr__</code> formatting
4	<code>_types.py</code>	typing	numpy (TYPE_CHECKING)	<code>NDArrayLike</code> alias
5	<code>serialization/, optim/optimizer.py</code>	checkpoint	numpy, safetensors	<code>state_dict</code> I/O
	<code>optim/lbfgs.py</code>	checkpoint	numpy	LBFGS history
6	<code>utils/data/dataloader.py</code>	ingest	numpy	default <code>collate_fn</code>

Table 10.1: The exhaustive list of places in `lucid/` where an external numerical library may be imported. Any `import numpy` (or `scipy`, `torch`, `jax`, etc.) outside these six files is a Hard Rule H4 violation rejected by `tools/check_bridges.py` in CI.

4. `lucid/_types.py`: typing protocol — `TYPE_CHECKING`-only imports.
5. `lucid/serialization/` and `lucid/optim/{optimizer,lbfgs}.py`: checkpoint serialisation — `state_dict`, `load_state_dict`.
6. `lucid/utils/data/dataloader.py`: external data ingest — `DataLoader`’s `collate` path.

Table 10.1 gives the same information in a quick-reference form.

Section 10.1 states the rule. Sections 10.2–10.7 treat each boundary in turn. Section 10.8 describes how the rule is enforced. Section 10.9 describes the policy for adding (or rejecting) a seventh boundary.

10.1 Hard Rule H4 Restated

The verbatim text of H4:

Hard Rule

Lucid internal behaviour (`lucid/`’s `composite`, `tensor`, `nn`, `optim`, `autograd`, `ops`, `linalg`, `fft`, `signal`, `special`, `distributions`, `einops`, `amp`, `profiler`) must not depend on any external library. NumPy, SciPy, or any other external package may not be imported. Only the Lucid C++ engine and the Python standard library are permitted. The sole exception is the six bridge boundaries enumerated below.

The motivation is twofold:

1. **Numerical hermeticism.** An external numerical library injected into a computational path can subtly change the dtype, the layout, or the numerical result of an operation. Such a change is invisible at the import line and emerges only when a parity test against the reference framework fails. Keeping the engine numpy-free means every numerical decision is made by Lucid code that can be audited and changed under our own discipline.
2. **Dependency surface.** Every external import is a place where a future `pip install` can fail, a version constraint can drift, or a security patch must be tracked. The six bridge boundaries are explicitly version-pinned and quarantined; the rest of the engine is unaffected by changes in the external ecosystem.

The rule was established in early Lucid 3.x and tightened in 3.0.2 when the framework completed its numpy-independence pass (vault: `retro-3-0-2-numpy-free-standalone`). The six boundaries listed here are the residue of that pass.

10.2 Boundary 1: External Array Ingest

`lucid/_factories/converters.py` is the file that constructs Lucid tensors from external array-like objects. It imports `numpy` unconditionally (`numpy` is a required dependency) and `torch`-style DLPack producers conditionally (DLPack is the standard cross-framework tensor protocol).

The functions exposed:

- `lucid.tensor(data, dtype=None, device=None)`: construct a tensor from a Python scalar, list, numpy array, or array-like. The function inspects the input type and routes accordingly; numpy arrays go through `from_numpy`, lists are converted via `numpy` first, scalars become 0-d tensors.
- `lucid.from_numpy(arr)`: zero-copy construction from a contiguous numpy array. The numpy buffer is wrapped in a `Storage` with `provider = External` and the numpy array is held alive for the storage's lifetime through a closure deleter.
- `lucid.from_dlpack(capsule)`: construction from a DLPack capsule [10]. Same zero-copy mechanic; the capsule's deleter is invoked when the storage is freed.
- `lucid.to_dlpack(tensor)`: production of a DLPack capsule from a Lucid tensor, for interop with other DLPack consumers.

10.2.1 Why DLPack is routed through numpy

The DLPack path in Lucid goes through `numpy` as an intermediate. Specifically, `from_dlpack` first calls `numpy.from_dlpack(capsule)` and then `from_numpy` on the resulting numpy array. The alternative would have been to construct a Lucid tensor directly from the DLPack capsule, which would avoid the `numpy` round-trip.

The alternative was implemented and abandoned. The vault note `arch-dlpack-via-numpy` records the reasoning: the direct DLPack path was 350 lines of C code that handled dtype mapping, stride translation, and capsule lifetime; the indirect path through `numpy` is 5 lines of Python; the performance difference is unmeasurable (DLPack is a metadata transfer, no data motion) and `numpy`'s DLPack implementation is well-tested. The maintenance cost of the direct path was deemed not worth the marginal architectural purity.

10.3 Boundary 2: Tensor-Side Egress

`lucid/_tensor/tensor.py` contains the `Tensor` class definition. The class has three methods that touch `numpy` or DLPack:

- `Tensor.numpy()`: return a numpy view of the tensor's data. The tensor must be on `device=cpu`; for a metal tensor, the user must first call `.to('cpu')`.
- `Tensor.__dlpack__()`: the dunder that third-party DLPack consumers call to obtain a capsule.
- `Tensor._to_impl(device)`: internal device-move implementation that, in one specific case, uses `numpy` as a fallback for an unusual dtype combination.

Of these, `.numpy()` is by far the most-used. The implementation is:

```

1 def numpy(self):
2     if self.device.type != 'cpu':
3         raise LucidError("call .to('cpu') before .numpy()")
4     if self.requires_grad:
5         raise LucidError("call .detach() before .numpy()")
6     return _C_engine.tensor_to_numpy(self._impl)

```

The two preconditions (`.to('cpu')`, `.detach()`) are explicit: Lucid does not silently move to CPU and does not silently detach. The reasoning is the same as for cross-device binary ops (Chapter 9): the user should be intentional about device moves and about breaking the autograd graph.

The `tensor_to_numpy` bound C++ function constructs a numpy array whose buffer points at the `TensorImpl`'s storage and holds a reference to the `TensorImpl` through a numpy capsule deleter. The conversion is zero-copy: the numpy array reads the same physical bytes as the tensor.

10.3.1 Why repr is a separate boundary

A natural question: why is the tensor's `__repr__` a separate boundary (boundary 3) rather than part of `tensor.py`? The answer is organisational: `tensor.py` must import numpy for the egress functions and we want to keep its internal logic away from display code. `_repr.py` is a single-purpose file that does only formatting, which keeps the diff for any repr-related change small and reviewable.

10.4 Boundary 3: Display Only

`lucid/_tensor/_repr.py` is the file that implements `Tensor.__repr__` and `Tensor.__str__`. It imports numpy because numpy's pretty-printing (`numpy.array2string`) is the well-tested standard for displaying multi-dimensional arrays in Python. Lucid's repr delegates to it:

```

1  def _format_tensor(t):
2      """Construct a repr string for a Tensor."""
3      arr = numpy.asarray(_C_engine.tensor_to_numpy(t._impl))
4      body = numpy.array2string(arr, precision=4, suppress_small=True)
5      return f"Tensor({body}, dtype={t.dtype}, device={t.device})"
```

The function is invoked only by the Python interpreter when the user asks for a repr (typing the variable name at a REPL prompt, calling `print`, embedding in an f-string). It does not run during training and contributes no runtime overhead.

10.4.1 The display carve-out is read-only

The repr code reads tensor data and returns a string; it does not modify the tensor, does not produce a new tensor, and does not affect any state outside the function. This makes it the least risky of the six boundaries: even a bug in the repr code (typing wrong dtype, getting precision wrong) does not corrupt computation. The boundary is enumerated for completeness rather than for any correctness concern.

10.5 Boundary 4: Typing Protocol

`lucid/_types.py` contains the type aliases that the rest of Lucid uses for static type checking. It imports numpy under a `TYPE_CHECKING` guard so that the import has no runtime cost:

```

1  from typing import TYPE_CHECKING
2
3  if TYPE_CHECKING:
4      import numpy as _np
5      NDArrayLike = _np.ndarray
6  else:
7      NDArrayLike = object
```

The pattern: at runtime, `NDArrayLike` is just `object` (no `numpy` import); at type-check time, `mypy` sees the real `numpy.ndarray` alias. This satisfies the type system without adding a runtime dependency.

The boundary is named because, technically, the file does import `numpy`. The import is dead at runtime but live at lint time. We list it explicitly to forestall the question of why `_types.py` contains the word `numpy`.

10.5.1 Why `TYPE_CHECKING` and not a string annotation

The alternative would have been to use string annotations: `NDArrayLike = "numpy.ndarray"`. `Lucid` rejects this pattern per Hard Rule H7 (no string type hints), which is enforced by `mypy` strict configuration. The `TYPE_CHECKING` block is the H7-compliant equivalent.

10.6 Boundary 5: Checkpoint Serialisation

`lucid/serialization/` and two files in `lucid/optim/` (`optimizer.py` and `lbfgs.py`) form the fifth boundary. The serialisation subsystem reads and writes `state_dict` files, which by convention contain `numpy` arrays.

10.6.1 The `state_dict` format

A `state_dict` is a Python dict mapping string keys (parameter names) to tensor values. `Lucid`'s `save` function serialises the dict using Python's `pickle` format, with each `Lucid` tensor converted to a `numpy` array on the way out. The inverse, `load`, reads the `pickle` and converts `numpy` arrays back to `Lucid` tensors.

The `numpy` intermediate is necessary because the `pickle` format does not know how to represent a `Lucid` tensor by itself. We could have added a custom `pickle` reducer, but the user-facing benefit is zero: external tools that read checkpoints (e.g. for inference deployment in non-`Lucid` environments) expect `numpy` arrays, and exposing `Lucid` tensors would have required the reader to depend on `Lucid`.

10.6.2 The `safetensors` path

`lucid.serialization` also supports the `safetensors` format [20], which is a tensor-format-only alternative to `pickle` that is safer (no arbitrary code execution during load) and faster (memory-mapped read). The `safetensors` path uses the `safetensors` Python library, which is itself `numpy`-aware. The boundary is therefore double: `numpy` for the tensor representation, `safetensors` for the format. Both are quarantined to `lucid/serialization/`.

10.6.3 Why `lbfgs` is part of this boundary

`lucid/optim/lbfgs.py` is the only optimiser that maintains state that does not fit the standard `state_dict` schema (`LBFGS` maintains a history of past gradients and parameter steps). Its `state_dict` / `load_state_dict` are non-trivial and touch the `numpy` boundary directly rather than going through the generic serialisation path. The optimiser base class (`lucid/optim/optimizer.py`) likewise has a small `numpy` touch for parameter group serialisation.

This was a deliberate trade-off: factoring the `LBFGS` state out into the generic path would have required generalising the schema in a way that benefits only `LBFGS`, and the cost of two extra files in the bridge enumeration was lower than the cost of the generalisation.

10.7 Boundary 6: External Data Ingest

`lucid/utils/data/dataloader.py` is the user-facing data loading utility. Its `DataLoader` class accepts a `Dataset` that produces samples, batches them, optionally collates them, and yields a stream of batched tensors.

The boundary appears in the collate path, which is where samples become tensors. The default `collate_fn` accepts numpy arrays and lists of numpy arrays and turns them into Lucid tensors via `lucid.from_numpy`. It also accepts Python scalars, lists, and dicts of any of the above.

10.7.1 Why the data path is its own boundary

The data ingest path is the single largest source of external data volume in a typical training workload — every image, every text token, every audio sample passes through it. Routing this through the existing `lucid.tensor` or `lucid.from_numpy` boundary would have been correct but inconvenient: the `DataLoader` class needs to know about collation, batching, multi-process worker coordination, and other concerns that `from_numpy` does not.

The boundary therefore exists for organisational reasons: the data loading code is concentrated in one place, and that place is allowed to import numpy. The user can write a custom `collate_fn` that imports anything; Lucid's contract is that the default collate works for numpy-formatted samples.

10.8 Enforcement

The H4 rule is enforced by a static check in `tools/check_bridges.py`. The check walks every `.py` file under `lucid/`, parses imports, and verifies that no file outside the six named files imports any of the forbidden modules (numpy, scipy, torch, jax, tensorflow, and a handful of others).

The check runs as part of the pre-commit hook and as part of CI. A violation fails the build and the contributor is asked to either (a) move the new dependency into a bridge file or (b) eliminate the dependency.

10.8.1 The bridge whitelist

The whitelist is hard-coded in `tools/check_bridges.py`:

```

1  BRIDGE_FILES = {
2      "lucid/_factories/converters.py",
3      "lucid/_tensor/tensor.py",
4      "lucid/_tensor/_repr.py",
5      "lucid/_types.py",
6      "lucid/serialization/__init__.py",
7      "lucid/optim/optimizer.py",
8      "lucid/optim/lbfgs.py",
9      "lucid/utils/data/dataloader.py",
10 }
11
12 FORBIDDEN_OUTSIDE_BRIDGE = {
13     "numpy", "scipy", "torch", "jax", "tensorflow", "pandas",
14     "sklearn", "xarray", "cupy",
15 }
```

The list is short and stable; changes require explicit review.

10.9 Adding a Seventh Boundary

The policy for adding a new bridge boundary:

1. Document the user-visible need that the new boundary serves. If the need can be satisfied by routing through an existing boundary, do that instead.
2. Document the external library that the new boundary will import.
3. Add the file to the whitelist in `tools/check_bridges.py`.
4. Update this chapter (and the relevant API reference chapter) to include the new boundary.

The policy has been invoked zero times since the rule was established. Every additional integration since then has been satisfied by extending one of the existing boundaries (e.g., adding safetensors support to `serialization/` or DLPack support to `converters.py`). The intent is to keep the count at six indefinitely.

10.10 Summary and Forward References

Lucid’s engine is hermetically sealed against external numerical libraries except at six explicitly named bridge boundaries. The boundaries are: external array ingest (`converters.py`), tensor-side egress (`tensor.py`), display (`_repr.py`), typing protocol (`_types.py`), checkpoint serialisation (`serialization/` and two optim files), and external data ingest (`dataloader.py`). Hard Rule H4 enforces the closure; a static check rejects any new import of a forbidden library outside the whitelist.

The next chapter, Chapter 11, describes the pybind11 layer that the Python side of Lucid sits on. Chapter 12 closes Part II with the lazy-import surface that the top-level package presents to the user. Chapter 64 returns to boundary 5 with a focus on the serialisation formats themselves; Chapter 65 returns to boundary 6.

Note

For the user: the bridge boundaries are invisible from the API surface. `lucid.from_numpy(arr)` just works; `.numpy()` just works; `lucid.save` and `lucid.load` just work. The chapter is for the contributor who needs to understand where numpy or DLPack can legitimately appear in the codebase and where it cannot.

CHAPTER 11

Python–C++ Binding via pybind11

Lucid is a Python-fronted, C++-engined framework. The interface between the two languages is the topmost layer of the engine DAG (Chapter 6), implemented through pybind11 [21]. This chapter describes how that binding works in Lucid specifically: the conventions, the contracts, the exception-translation policy, the reference-counting interplay between Python and C++ `shared_ptr`, and the build-system integration that produces the `lucid._C` extension module.

The binding layer is not large in absolute terms — about 3500 lines of C++ across the files in `lucid/_C/bindings/` — but it is the only place where pybind11 types appear. The discipline the chapter describes is what keeps the engine’s lower layers pybind11-free, which in turn keeps the option open to expose the engine through a C API to a non-Python frontend in a future release.

11.1 Overview

pybind11 is a header-only C++ library that bridges between C++ types and Python objects. The bridge has two principal operations: *type binding*, which exposes a C++ class as a Python class, and *function binding*, which exposes a C++ function as a Python callable. Both operations are declarative: the binding code specifies what to expose, and pybind11 generates the boilerplate that handles argument conversion, return-value conversion, reference counting, and exception translation.

For Lucid the binding code is:

- `lucid/_C/bindings/bind.cpp`: the module entry point, `PYBIND11_MODULE(engine, m)`.
- `lucid/_C/bindings/bind_tensor.cpp`: the `Tensor` class binding (about 400 lines).
- Per-subsystem binding files: `bind_bfunc.cpp`, `bind_ufunc.cpp`, `bind_gfunc.cpp`,

`bind_composite.cpp`, `bind_autograd.cpp`, `bind_nn.cpp`, `bind_optim.cpp`, `bind_linalg.cpp`, `bind_fft.cpp`, `bind_einops.cpp`, `bind_complex.cpp`, `bind_amp.cpp`, `bind_profiler.cpp`, `bind_random.cpp`, `bind_compile.cpp`.

- `bind_errors.cpp`: the exception translator.
- `bind_op_registry.cpp`: introspection binding for the op registry.
- `BindingGen.h`: helper templates for code generation.

The output of building these files is a single CPython extension module, `lucid/_C/engine.cpython-XYZ.so`, where XYZ encodes the Python version. The module is loaded under the alias `_C_engine` per Hard Rule H2; see Section 11.6.

11.2 Module Initialisation

The module entry point is declared with the `PYBIND11_MODULE` macro:

```

1  #include <pybind11/pybind11.h>
2  namespace py = pybind11;
3
4  PYBIND11_MODULE(engine, m) {
5      m.doc() = "Lucid C++ engine bindings";
6
7      bind_errors(m);      // must come first --- registers exception
                           translator
8      bind_tensor(m);
9      bind_ufunc(m);
10     bind_bfunc(m);
11     bind_gfunc(m);
12     bind_composite(m);
13     bind_autograd(m);
14     bind_nn(m);
15     bind_optim(m);
16     bind_linalg(m);
17     bind_fft(m);
18     bind_einops(m);
19     bind_complex(m);
20     bind_amp(m);
21     bind_profiler(m);
22     bind_random(m);
23     bind_compile(m);
24     bind_op_registry(m);
25 }
```

Each `bind_<subsystem>(m)` function adds its part of the surface to the module object. The ordering matters in one place: `bind_errors` must run first so that the exception translator is installed before any other binding might throw.

11.2.1 The module-name convention

The module is named `engine`, which means it is imported in Python as `from lucid._C import engine`. The alias convention of Hard Rule H2 requires that the import be aliased to `_C_engine`:

```

1  from lucid._C import engine as _C_engine
```

The alias prefix `_C_` flags that the symbol is the engine-level binding and makes greppability trivial. Section 11.6 treats the alias rule in detail.

11.3 Type Bindings: The Tensor Class

The `Tensor` class is the central exposed type. Its binding, abbreviated:

```

1  void bind_tensor(py::module_& m) {
2      py::class_<TensorImpl, std::shared_ptr<TensorImpl>>(m, "Tensor")
3          .def(py::init<>())
4          .def_property_readonly("shape",
5              [](const TensorImpl& t) { return t.shape; })
6          .def_property_readonly("dtype",
7              [](const TensorImpl& t) { return t.dtype; })
8          .def_property_readonly("device",
9              [](const TensorImpl& t) { return t.device; })
10         .def_property("requires_grad",
11             &TensorImpl::requires_grad,
12             &TensorImpl::set_requires_grad)
13         .def("backward", &TensorImpl::backward,
14             py::arg("gradient") = py::none(),
15             py::arg("retain_graph") = false)
16         .def("detach", &TensorImpl::detach)
17         .def("clone", &TensorImpl::clone)
18         // ... and so on, one binding per method
19         .def("__add__", [](const TensorImpl& a, const TensorImpl& b) {
20             return Dispatcher::instance().binary(OpKind::Add, a, b);
21         })
22         // ... and the rest of the operator-overload dunderers
23         ;
24     }

```

The crucial detail is the second template argument to `py::class_<std::shared_ptr<TensorImpl>>`. This tells pybind11 that `TensorImpl` instances are managed through a `shared_ptr`, which is how the engine internally tracks tensor lifetimes. The Python-side reference counting then participates in the same `shared_ptr` count, so a tensor stays alive as long as either the Python object or any internal `shared_ptr` refers to it.

11.3.1 Property versus method

The `def_property_readonly` versus `def` choice matches the Python convention. Attributes like `shape`, `dtype`, `device` are exposed as properties (no parentheses needed at the call site), while operations like `backward`, `detach`, `clone` are exposed as methods. This is the reference framework-compatible surface that the user expects.

11.3.2 Method-like dunderers

Operator overloads (`__add__`, `__mul__`, `__matmul__`, etc.) are bound as lambdas that call into the dispatcher. Each lambda is one line; there are about 30 of them. The pattern is uniform enough that we could code-generate it from a list, but the gain is small and explicit code is easier to diff.

11.4 Function Bindings

Free functions are bound similarly to methods. Per-op bindings live in `bind_ufunc.cpp`, `bind_bfunc.cpp`, `bind_gfunc.cpp`, and `bind_composite.cpp`, organised by op category.

A typical unary binding:

```

1  void bind_ufunc(py::module_& m) {
2      m.def("relu", &ops::relu, py::arg("x"));

```

```

3   m.def("sigmoid", &ops::sigmoid, py::arg("x"));
4   m.def("tanh",      &ops::tanh,      py::arg("x"));
5   m.def("exp",       &ops::exp,       py::arg("x"));
6   m.def("log",       &ops::log,       py::arg("x"));
7   // ... 60 more
8 }

```

Each call has the form `m.def("name", &cpp_function, py::arg("arg") = default)`. The `py::arg` machinery preserves argument names and default values across the binding, which means the Python-side caller can use keyword arguments and get IDE autocomplete.

11.4.1 Code generation for repetitive bindings

The unary and binary bindings are repetitive enough that we generate them from a single declarative table. `BindingGen.h` provides macros that take an op name, an op kind, and a category; each invocation produces the `m.def(...)` line above. The table is maintained in `bind_ufunc.cpp` itself, near the top, so the file is self-contained and the generated output follows immediately.

The alternative (code generation in CMake or in a separate Python script) was rejected as adding build-system complexity for marginal benefit.

11.5 Exception Translation

Lucid's engine raises a single C++ exception type, `LucidError`, defined in `core/Error.h`. The binding layer translates this exception into a Python exception of the same name. The translator is installed in `bind_errors.cpp`:

```

1  void bind_errors(py::module_& m) {
2      static py::exception<LucidError> py_lucid_error(m, "LucidError");
3      py::register_exception_translator([](std::exception_ptr p) {
4          try {
5              if (p) std::rethrow_exception(p);
6          } catch (const LucidError& e) {
7              py_lucid_error(e.what());
8          }
9      });
10 }

```

The translator runs whenever a C++ function called from Python throws a `LucidError`. The Python side sees a `lucid.LucidError` with the original message, with no further adaptation needed at the call site.

11.5.1 Non-Lucid C++ exceptions

C++ exceptions other than `LucidError` are translated by `pybind11`'s default mechanism: `std::invalid_argument` becomes `ValueError`, `std::out_of_range` becomes `IndexError`, `std::runtime_error` becomes `RuntimeError`, and so on. Lucid's engine code raises `LucidError` for everything that should reach the user; the default translations are a safety net for programmer errors (invariant violations, internal asserts) that should not happen in production.

11.5.2 The `LucidError` message format

`LucidError`'s message is constructed through a fluent builder (`ErrorBuilder` in `core/ErrorBuilder.cpp`) that supports attaching context: the operation name, the tensor shapes and dtypes, the device, the file and line of the throwing call. The builder produces

structured strings that production users can grep and that automated diagnostics can parse. The format is documented in Chapter 66.

11.6 The `__C__`{name} Alias Convention (H2)

Hard Rule H2 (Appendix A) prescribes a uniform aliasing convention for every import from `lucid._C`:

Hard Rule

Imports from `lucid._C` must be aliased as `_C_{name}`, e.g. from `lucid._C import engine` as `_C_engine`. Bare imports (from `lucid._C import engine`) and short aliases (as `e`, as `eng`, as `engine`) are forbidden.

The motivation:

1. **Grep-ability.** Every reference to the engine binding is prefixed with `_C_`, so a `grep` for `_C_` finds every engine call in the codebase. This is the single most-used `grep` when investigating a binding-layer bug.
2. **Disambiguation.** The `_C_` prefix signals to the reader that the symbol crosses the Python/C++ boundary. A bare `engine.something()` could be any module; an `_C_engine.something()` cannot be mistaken.
3. **Refactor safety.** A future restructuring of the binding layer (for example, splitting the single engine module into multiple sub-modules) can be applied uniformly across the codebase by a search-and-replace on `_C_engine`. Bare aliases would have made the refactor non-mechanical.

The rule is enforced by a static check (`tools/check_c_import_naming.py`) that runs in CI.

11.7 GIL Handling

Python’s Global Interpreter Lock (GIL) is held by default during every C++ function call from Python. For functions that perform substantial work without touching Python objects, releasing the GIL allows other Python threads (e.g., data-loader workers) to run concurrently.

Lucid’s binding releases the GIL inside operations that:

- Dispatch a kernel through MLX or Accelerate. Kernel launches are non-trivial-duration operations that do not touch Python.
- Perform a `backward()` traversal. The autograd engine walks the graph without Python interaction.
- Block on a CPU-to-GPU bridge or vice versa. The bridge waits on Metal events.

The GIL release uses pybind11’s `py::gil_scoped_release`:

```
1 TensorImpl ops::matmul(const TensorImpl& a, const TensorImpl& b) {
2     py::gil_scoped_release release;
3     return Dispatcher::instance().matmul(a, b);
4 }
```

The scope guard releases the GIL on construction and reacquires on destruction. The reacquisition is automatic and exception-safe.

11.7.1 When not to release

The GIL is *not* released in functions that:

- Touch Python objects (callbacks, exceptions to be raised on the calling thread).
- Are short enough that the release/reacquire overhead would dominate. The break-even point on Apple silicon is roughly $1\ \mu\text{s}$ of useful work; functions shorter than that should keep the GIL.
- Are called recursively from a context that has already released. The scope guard supports nesting but the explicit release adds noise without benefit.

The rule of thumb: release for any function that dispatches to a backend kernel; do not release for property getters, type queries, or other constant-time accessors.

11.8 Memory and Reference Counting

The interaction between Python’s reference counting and C++’s `shared_ptr` is the subtlest part of the binding. `pybind11` handles most of it transparently, but the discipline is worth stating.

11.8.1 `shared_ptr` held by Python

A Python `Tensor` object holds a `shared_ptr<TensorImpl>` in its C++ representation. The `shared_ptr`’s reference count is incremented when the Python object is created and decremented when the Python object is garbage-collected.

Multiple Python aliases of the same tensor (for example, `a = b`) share the same `TensorImpl` through the same `shared_ptr`; `pybind11` handles this by recognising that the incoming object is already wrapped and returning the existing Python object rather than constructing a new one.

11.8.2 `shared_ptr` passed between C++ functions

Inside the engine, operations pass `shared_ptr<TensorImpl>` around freely. The autograd graph holds `shared_ptr` references to tensors saved for backward; the dispatcher constructs `shared_ptr`-wrapping return values. The Python side does not need to know about these intermediate references; the lifetime they pin is observable only as “memory usage that goes down after `backward()`.”

11.8.3 The risk: cycles

`shared_ptr` cycles cause leaks: two objects holding `shared_ptr` to each other never have their reference count reach zero. The autograd graph could in principle form cycles (through hooks or through closures), and we are careful in two specific places: `nn.Module`’s parameter registry holds `weak_ptr`-equivalent references to its children to avoid parent-child cycles, and the `grad_fn` chain is unidirectional so that no two backward nodes hold each other.

The discipline is checked at code review; an explicit check would require a runtime cycle detector that we have not built.

11.9 Build System Integration

The binding layer is built as a CPython extension module using `scikit-build-core`, with `CMake` as the underlying build system. The configuration files:

- `pyproject.toml`: declares `scikit-build-core` as the build backend and pins build-time dependencies (`pybind11`, `cmake`).

- `CMakeLists.txt`: top-level CMake; declares the `engine` target and links it against `pybind11`, `MLX`, and `Accelerate`.
- `lucid/_C/CMakeLists.txt`: declares the per-layer sub-libraries described in Chapter 6 and links them into the engine extension.

The build produces a single `.so` file (or, on macOS, a `.cpython-XYZ-darwin.so`) that `pybind11`'s loading machinery recognises and exposes as the `engine` module under `lucid._C`.

11.9.1 The wheel and its constraints

The published wheel is built once per Python version against the minimum-target macOS SDK (Chapter 3 Section 3.9.2). The wheel tag is `cp314-cp314-macosx_26_0_arm64`, which encodes the Python version (3.14), the platform (`macosx_26_0`), and the architecture (`arm64`). The constraints come from `MLX`: Lucid's wheel must match `MLX`'s wheel tags to load successfully.

This is the reason Lucid does not currently distribute Linux or x86 wheels: `MLX` does not, and Lucid cannot run without `MLX`.

11.10 Summary and Forward References

The Python/C++ binding in Lucid is built on `pybind11`, organised into a single `engine` module with per-subsystem binding files, and disciplined by Hard Rule H2 (the `_C_{name}` alias convention). Exception translation is centralised in `bind_errors.cpp`, GIL release is per-function with a $1\mu\text{s}$ rule of thumb, and reference counting interoperates through `shared_ptr<TensorImpl>` on both sides. The build produces a single extension module per Python version, tagged to match `MLX`'s wheel constraints.

The next chapter, Chapter 12, closes Part II with the lazy-import surface that the top-level Python package exposes. Chapter 66 returns to the exception system from the diagnostics angle.

Note

For the user: the binding layer is invisible. The user imports `lucid`, constructs tensors and modules, and never directly touches the engine module. The only artefact that surfaces is the `LucidError` exception class, which is the single error type the framework raises.

CHAPTER 12

Lazy Import Surface

Lucid’s top-level Python package, `lucid`, exposes around three hundred names at its surface — factories, operations, type predicates, the `Tensor` class, the dtype objects, the device objects, the autograd entry points, and a number of subsystems re-exported as attributes (`lucid.nn`, `lucid.optim`, `lucid.linalg`, and others). A naive implementation would import everything in `__init__.py`, which would force every `import lucid` to load the entire engine extension, every subsystem module, and every transitively imported file. The cold-start cost would be measured in seconds.

Lucid instead uses a lazy-import surface: `__init__.py` registers each subsystem as a deferred loader and resolves attributes through a module-level `__getattr__` fallback. The result is that `import lucid` returns in under one hundred milliseconds on cold caches, with the heavyweight subsystems loaded only when they are first referenced. The user sees the same surface as before; the difference is performance.

This chapter describes the mechanism. Section 12.1 motivates the design. Section 12.2 describes the `__init__.py` structure. Section 12.3 describes the per-subsystem loaders. Section 12.4 describes the `.pyi` type stubs that preserve IDE autocomplete. Section 12.5 measures the startup cost. Section 12.6 describes the Tier 1 / Tier 2 / Tier 3 namespace layout that the lazy surface enforces. Section 12.7 closes with the policy for adding a new subsystem.

12.1 Why Lazy Imports

The cost of a non-lazy `import lucid` would be the sum of:

- Loading the engine extension module: roughly 200–400 ms on cold caches (the `.so` file is large, the dynamic linker has work to do, MLX initialisation runs inside).

- Importing every subsystem module: roughly 50–150 ms per subsystem, with around 15 subsystems, total in the 1–2 second range.
- Importing every model in the zoo (Part VIII): roughly 10–30 ms per family with around 50 families, total in the 0.5–1.5 second range.

A pessimistic estimate puts the cold `import lucid` at 3–4 seconds. This is not catastrophic for a long-running training job but it is catastrophic for several other use cases:

- Interactive notebooks where the user runs many short cells. A 3-second import per kernel restart compounds.
- Command-line tools that wrap Lucid. A model-inspection CLI that imports Lucid at startup feels sluggish.
- Test suites where each test process imports Lucid fresh. A pytest run of 100 test files would add 300 seconds just to imports.

The lazy surface eliminates this cost for the common case where the user does not need every subsystem. A script that uses only `lucid.Tensor` and `lucid.nn.Linear` loads the engine extension and the `nn` subsystem on first reference; the remaining 13 subsystems and 50 model families are never loaded.

12.1.1 The constraint

The lazy surface must preserve the user-facing API exactly. The expressions `lucid.Tensor`, `lucid.zeros(3)`, `lucid.nn.Linear`, `lucid.linalg.cholesky`, and `lucid.compile(model)` must all work. `dir(lucid)` must return the full set of names. `help(lucid)` must show every documented symbol. IDE autocomplete (Pylance, Jedi, mypy) must work without forcing imports.

The mechanism described below satisfies all four constraints.

12.2 The `__init__.py` Structure

`lucid/__init__.py` has three parts: the eager preamble, the loader registry, and the deferred-resolution machinery. We treat each in turn.

12.2.1 The eager preamble

A small number of objects must be available eagerly because they are referenced during module-load time by users who unpickle tensors, import from `lucid` in non-standard ways, or otherwise interact with the module before any attribute access triggers lazy loading:

```

1  # Eager: minimum surface required at import time.
2  from lucid._dtype import (
3      float16, float32, float64, bfloat16,
4      int8, int16, int32, int64,
5      bool_, complex64,
6      Dtype,
7  )
8  from lucid._device import cpu, metal, Device
9  from lucid._tensor.tensor import Tensor
10 from lucid._globals import (
11     get_default_dtype, set_default_dtype,
12     get_default_device, set_default_device,
13 )
14
15 __version__ = "3.5.0"
```

This block accounts for the bulk of the 100 ms cold-import time: loading `Tensor` requires loading the engine extension, which is the single largest contribution. Everything else in `lucid` is deferred.

12.2.2 The loader registry

The deferred symbols are registered in a dictionary that maps each top-level name to its loader function:

```

1  _GROUP_LOADERS = {
2      # Subsystem modules
3      "nn":          "_load_nn",
4      "optim":       "_load_optim",
5      "autograd":    "_load_autograd",
6      "linalg":      "_load_linalg",
7      "fft":         "_load_fft",
8      "signal":      "_load_signal",
9      "special":     "_load_special",
10     "distributions": "_load_distributions",
11     "func":         "_load_func",
12     "einops":       "_load_einops",
13     "amp":          "_load_amp",
14     "profiler":     "_load_profiler",
15     "serialization": "_load_serialization",
16     "compile":      "_load_compile",
17     "models":       "_load_models",
18     "utils":        "_load_utils",
19     "metal":        "_load_metal",
20     # Top-level functions
21     "zeros":        "_load_factories",
22     "ones":         "_load_factories",
23     "arange":       "_load_factories",
24     "tensor":       "_load_factories",
25     "from_numpy":   "_load_factories",
26     # ... (about 300 entries total)
27 }
```

The values are loader function names rather than the functions themselves, so that the loader-function bodies (each of which imports from a subsystem) do not run until invoked. The granularity is per-group rather than per-symbol: `zeros`, `ones`, `arange`, `tensor`, `from_numpy` all share the `_load_factories` loader because they live in the same subsystem and triggering one triggers all of them.

12.2.3 The deferred-resolution fallback

The Python language defines that an attribute access on a module that is not satisfied by the module's normal dictionary triggers the module's `__getattr__` hook, if defined. `Lucid` implements the hook to invoke the appropriate loader:

```

1  def __getattr__(name: str):
2      loader_name = _GROUP_LOADERS.get(name)
3      if loader_name is None:
4          raise AttributeError(f"module 'lucid' has no attribute {name!r}")
5      loader = globals()[loader_name]
6      loader() # populates globals() with the relevant symbols
7      return globals()[name]
```

The pattern: on first access to `lucid.nn`, `__getattr__` fires, the loader is invoked, the subsystem is imported, the symbols are inserted into the module's globals, and the attribute access returns successfully. Subsequent accesses to `lucid.nn` hit the module dict directly (no `__getattr__` call) and the cost is the same as for an eager import.

12.2.4 The `__dir__` extension

`dir(lucid)` should return the full set of names, lazy or not. The `__dir__` hook returns the union of the module's actual globals and the keys of `_GROUP_LOADERS`:

```
1 def __dir__():
2     return sorted(set(globals()) | set(_GROUP_LOADERS))
```

This makes tab-completion at a REPL work as expected, and makes `help(lucid)` list every symbol. The cost is paid only when the user explicitly asks for the directory listing.

12.3 Subsystem Loaders

Each loader function is a few lines that import from one or more subsystem modules and insert the imports into the module globals:

```
1 def _load_factories():
2     """Trigger import of factories subsystem and bind symbols."""
3     from lucid._factories.creation import (
4         zeros, ones, eye, arange, linspace, logspace, tensor,
5     )
6     from lucid._factories.random import (
7         rand, randn, bernoulli, Generator, seed, manual_seed,
8     )
9     from lucid._factories.converters import (
10         from_numpy, from_dlpack, to_dlpack,
11     )
12     for name in ("zeros", "ones", "eye", "arange", "linspace",
13                 "logspace", "tensor", "rand", "randn", "bernoulli",
14                 "Generator", "seed", "manual_seed",
15                 "from_numpy", "from_dlpack", "to_dlpack"):
16         globals()[name] = locals()[name]
```

The loader is idempotent: if invoked twice (which can happen if a test resets the module state), the second invocation rebinds the symbols to the same objects. Re-importing a subsystem in Python is cheap because the module is already in `sys.modules`.

12.3.1 Subsystem loaders as module attributes

For a subsystem module like `lucid.nn`, the loader does not import individual names; it just imports the subsystem and binds its module object as a top-level attribute:

```
1 def _load_nn():
2     import lucid.nn
3     globals()["nn"] = lucid.nn
```

The subsystem module itself can be lazy too — `lucid.nn` may not import `lucid.nn.functional` until referenced — but that is the subsystem's own concern.

12.4 Type Stubs (.pyi)

A lazy module is opaque to static type checkers and IDE autocomplete by default: the type checker cannot see what symbols `lucid.__getattr__` would return without running the loader, and it does not run the loader. The standard solution is the *type stub*: a `.pyi` file that declares every symbol's type without containing its implementation.

`lucid/__init__.pyi` contains the eager declarations:

```

1  from lucid._tensor.tensor import Tensor as Tensor
2  from lucid._dtype import (
3      Dtype as Dtype,
4      float16 as float16,
5      float32 as float32,
6      # ...
7  )
8  from lucid._device import (
9      Device as Device,
10     cpu as cpu,
11     metal as metal,
12 )
13
14 # Lazy: declared here so static analysis sees them.
15 def zeros(*shape: int, dtype: Dtype = ..., device: Device = ...) ->
16     Tensor: ...
17 def ones(*shape: int, dtype: Dtype = ..., device: Device = ...) ->
18     Tensor: ...
19 def arange(start: int | float, stop: int | float | None = ...,
20            step: int | float = 1, *, dtype: Dtype = ..., device:
21                Device = ...
22            ) -> Tensor: ...
23 # ... and so on for every name in _GROUP_LOADERS.

```

The stub is generated from the actual source code by a build-time script and is shipped in the wheel alongside the Python sources. A Pylance- or mypy-enabled editor sees the stub before any `lucid` code runs, so autocomplete and type checking work immediately.

12.4.1 The H9 rule

The stub generator must produce *full* signatures, not `*args`, `**kwargs`. Hard Rule H9 (Appendix A) prohibits `*args`/`**kwargs` in stubs except for genuinely variadic APIs. The reasoning is that a stub with `*args`, `**kwargs` provides no static type information, defeating the purpose of the stub.

The rule is enforced by a check in `tools/check_stubs.py` that parses every `.pyi` file and flags any `*args` or `**kwargs` not accompanied by a specific type annotation that justifies the variadic.

12.5 Startup Cost Measurement

We measured the cold-import cost of `Lucid` on a representative machine (M3 Max, macOS 26.0, Python 3.14, freshly cleared `__pycache__`):

- `import lucid`: 92 ms.
- Plus first reference to `lucid.Tensor`: +0 ms (eager).
- Plus `lucid.nn` (first reference): +120 ms.
- Plus `lucid.optim` (first reference): +85 ms.

- Plus `lucid.compile` (first reference): +210 ms.
- Plus a `lucid.models` model factory: +180 ms.

A typical training script that uses `lucid`, `lucid.nn`, `lucid.optim`, and one model factory pays roughly 480 ms of imports total — about half what a fully eager surface would cost. A simple inspection script that uses only `lucid.Tensor` and `lucid.from_numpy` pays only the 92 ms eager cost plus 50 ms for `_load_factories` on first `from_numpy`.

The eager preamble’s 92 ms is dominated by the engine extension load: loading `libmlx.dylib` (which Lucid links against) and running its initialiser is about 60 ms; the rest is dynamic-linker overhead for the engine `.so` itself. There is no reasonable way to reduce this further without restructuring MLX itself.

12.6 The Tier 1 / Tier 2 / Tier 3 Namespace Layout

Lucid’s user-facing namespace is organised in three tiers, with the lazy surface enforcing the boundaries:

- **Tier 1.** Names directly under `lucid`. Reserved for the reference framework-compatible top-level surface: factories, free-function ops, dtype and device objects, the `Tensor` class. About 300 names.
- **Tier 2.** Names under a subsystem module: `lucid.nn.Linear`, `lucid.optim.Adam`, `lucid.linalg.cholesky`. About 600 names across all subsystems.
- **Tier 3.** Internal names under underscore-prefixed modules: `lucid._dispatch`, `lucid._C`, `lucid._tensor`. Not part of the public API.

The lazy surface enforces the boundaries: only names that appear in `_GROUP_LOADERS` are reachable as `lucid.<name>`, which keeps the Tier 1 surface auditable. A name that the contributor wants to expose but that is not in `_GROUP_LOADERS` simply does not appear.

12.6.1 The namespace-leak rule

A consequence of the tier structure is Hard Rule H6 (the namespace-leak rule, conceptually): symbols defined inside a subsystem must not leak into Tier 1 unless explicitly exposed. For example, `nn.Module`, `nn.Parameter`, `nn.Linear`, `optim.Adam` are subsystem types, and Lucid does not expose them at the top level.

The rule is enforced by the same check that validates the lazy surface: if a contributor accidentally adds `Module` to `_GROUP_LOADERS`, the static check rejects it. This is mostly a documentation discipline; the practical case for not exposing `nn.Module` at `lucid.Module` is that it is ambiguous (which subsystem’s `Module`?).

12.7 Adding a New Subsystem

The protocol for exposing a new subsystem at the lazy surface:

1. Implement the subsystem under `lucid/<subsystem>/`.
2. Write a loader function `_load_<subsystem>` in `__init__.py` that imports the subsystem and binds it to the module globals.
3. Add an entry to `_GROUP_LOADERS` mapping the subsystem name to the loader.
4. Regenerate the type stub (`tools/build_lucid_pyi.py`) so that the new symbols appear in `lucid/__init__.pyi`.
5. Add a test that imports the new subsystem lazily and verifies the imports complete.

The protocol has been invoked five times in Lucid’s history: once each for `einops`, `amp`, `distributions`, `compile`, and `models`. Each invocation took roughly a half-day of work concentrated in steps 4 and 5; steps 1–3 are mechanical.

12.8 Summary and Forward References

The lazy import surface in `lucid/__init__.py` reduces cold `import lucid` from a worst-case 3–4 seconds to a measured 92 ms, by deferring every subsystem import until first reference. The mechanism is a `_GROUP_LOADERS` dictionary plus a module-level `__getattr__` fallback, with `__dir__` extended so `dir(lucid)` returns the full surface and `.pyi` stubs preserving IDE auto-complete. The Tier 1 / Tier 2 / Tier 3 namespace structure is enforced by the same mechanism.

This chapter closes Part II. The system architecture chapters (the layered DAG, the tensor model, the allocator, the dispatcher, the bridge boundaries, the pybind11 layer, and the lazy surface) together describe the engine’s static structure. Part III now describes the operations themselves: the op registry, the kernel abstraction, and the per-category operation sets.

Note

For the user: the lazy surface is invisible. `import lucid` just works; `lucid.nn.Linear` just works. The user does not notice the deferred loading except that `import lucid` is fast. For the contributor, the lazy surface is the disciplinary mechanism that keeps the Tier 1 namespace from accumulating inadvertent symbols.

Part III

Operations and Kernels

CHAPTER 13

Op Registry and Schema

Operations are the unit of work in Lucid: every numerical computation the framework performs is the application of one or more operations to one or more tensors. There are roughly 260 of them in the current release, ranging from arithmetic primitives (`add`, `mul`) to neural-network composites (`layer_norm`, `multi_head_attention`) to control-flow constructs (`where`, `masked_select`). Each carries metadata that the dispatcher, the autograd engine, the AMP system, the compile path, and the introspection tools all consult.

The metadata lives in the op registry, the topic of this chapter. The registry is a single source of truth: a contributor who adds a new operation declares its schema in one place and the rest of the framework picks it up automatically. The registry is also what makes Lucid's compile path tractable: the trace IR can be serialised, the emitter table can be code-generated, and the backward-pass topology can be derived from the registry's declarations without inspecting individual op implementations.

This chapter describes the schema, the registry, the binding-code generation that consumes the registry, and the protocol for adding a new operation.

13.1 Overview

An *op schema* is a structured record describing one operation. It contains the op's name, its inputs and outputs, its backward behaviour, its AMP policy, its determinism guarantees, its complexity class, and a small set of derived properties (whether it is differentiable, whether it is a view-producer, whether it has a constant-shape output).

The *op registry* is the in-process table that holds every op's schema. It is populated at engine initialisation by self-registering schema declarations in the per-op C++ files; the registry is then consulted by:

- The dispatcher (Chapter 9), which uses the schema to validate argument shapes and dtypes before calling the backend.
- The autograd engine (Chapter 22), which uses the schema’s backward declaration to construct the appropriate `grad_fn` node.
- The AMP system (Chapter 41), which uses the schema’s AMP policy to decide whether to cast inputs to FP16 or keep them in FP32.
- The compile path (Chapter 46), which uses the schema to look up the appropriate MPS-Graph emitter.
- The introspection tools (`lucid._C.list_ops()`, documentation generators, the docs site), which consult the registry to enumerate the surface.

This many consumers might suggest a complex schema, but in practice the schema is small: about a dozen fields, all of them simple types. The discipline is keeping the schema lean enough that adding a new op is straightforward.

13.2 The OpSchema Structure

The schema is declared in `lucid/_C/ops/registry/OpSchema.h`:

```

1  struct OpSchema {
2      std::string      name;           // canonical op name
3      uint32_t         version;        // schema version
4      ArgSpec          inputs[8];      // input tensor specs
5      uint8_t          n_inputs;
6      ArgSpec          outputs[4];     // output tensor specs
7      uint8_t          n_outputs;
8      BackwardSpec     backward;       // backward construction
9      AmpPolicy         amp;           // FP16/FP32 cast rule
10     ComplexityClass   complexity;     // O(1), O(N), O(N log
        N), O(N^2)
11     Determinism        determinism;   // deterministic / not
12     uint16_t          flags;         // packed boolean
        properties
13 };

```

13.2.1 Name and version

`name` is the canonical op name — the string that appears in user-facing API (`add`, `matmul`, `layer_norm`) and in the trace IR. Names are lowercase, snake-case, and follow reference framework-compatible conventions where parity matters.

`version` is a 32-bit integer that increments when the op’s behaviour changes in a backward-incompatible way. This matters for checkpoint portability: a checkpoint saved with version 2 of `batch_norm` cannot be loaded into a process running version 3 without an explicit migration. The version field is the mechanism the serialisation layer (Chapter 64) uses to refuse incompatible loads.

13.2.2 ArgSpec for inputs and outputs

`ArgSpec` describes one tensor argument:

```

1  struct ArgSpec {
2      std::string      name;           // parameter name in user API
3      DtypeSet         allowed;        // permitted dtypes (bitmask)
4      uint8_t          rank_min;      // minimum rank

```

```

5  uint8_t      rank_max;      // maximum rank (255 = unbounded)
6  bool        optional;      // may be absent
7  bool        out;           // is this an output (vs input)
8  };

```

The `name` matches the Python-side argument name, so the dispatcher can rebind keyword arguments to positional slots without maintaining a separate alias table. The `allowed` bitmask enumerates the dtypes the op accepts; the dispatcher raises `LucidError` if an input dtype is not in the set. The `rank_min` / `rank_max` pair bounds the input rank; arbitrary ranks use `rank_max` = 255.

13.2.3 BackwardSpec

`BackwardSpec` describes how the op's backward pass is constructed:

```

1  enum class BackwardKind {
2      None,                // not differentiable
3      Inline,              // backward defined in C++
4      Custom,              // user-supplied via Function class
5      Composite,           // built from forward primitives
6  };
7
8  struct BackwardSpec {
9      BackwardKind      kind;
10     BackwardFn*        inline_fn;    // for Inline kind
11     SavedTensorList    saved;        // which forward tensors to save
12 };

```

Most ops use `Inline`: the backward is a known function with a known shape. The function takes the saved tensors, the output gradient, and the input shapes, and returns the input gradients. The autograd engine sets up the `grad_fn` node at forward time by reading `inline_fn` from the schema.

13.2.4 AmpPolicy

`AmpPolicy` encodes the op's behaviour under mixed precision:

```

1  enum class AmpPolicy {
2      PromoteToWidest,     // default: keep widest input dtype
3      CastToHalf,          // force to FP16 under autocast
4      PreserveInputs,      // no casting; preserve user dtype exactly
5      ForceFloat32,        // always FP32 (reductions, softmax)
6  };

```

Matrix multiplication uses `CastToHalf`: under autocast, `matmul` inputs are cast to FP16. Softmax uses `ForceFloat32`: the reduction accumulates in FP32 even when inputs are FP16, to prevent overflow. Pointwise arithmetic uses `PromoteToWidest`.

13.2.5 Complexity, determinism, and flags

`ComplexityClass` is an enum (`O1`, `ON`, `ONLogN`, `ON2`, `ON3`) that the profiler uses to label per-op costs.

`Determinism` is one of `Deterministic` or `Nondeterministic`. The `set_deterministic(True)` mode (Chapter 67) refuses to execute nondeterministic ops.

`flags` is a 16-bit bitmask of additional boolean properties: `is_view_producer`, `is_inplace`, `is_constexpr`, `has_static_output_shape`, `requires_contiguous`.

13.3 The OpRegistry

The registry is a singleton hash table from op name to schema, implemented in `lucid/_C/ops/registry/OpRegistry.{h,cpp}`:

```

1  class OpRegistry {
2  public:
3      static OpRegistry& instance();
4      void          register_op(OpSchema schema);
5      const OpSchema& lookup(const std::string& name) const;
6      bool          has(const std::string& name) const;
7      std::vector<std::string> all_names() const;
8  private:
9      std::unordered_map<std::string, OpSchema> table_;
10 };

```

13.3.1 Self-registration

Each op's C++ file contains a self-registering static initialiser:

```

1  namespace {
2      struct AddRegistration {
3          AddRegistration() {
4              OpSchema s;
5              s.name = "add";
6              s.version = 1;
7              s.n_inputs = 2;
8              s.inputs[0] = {"a", DtypeSet::Numeric, 0, 255, false, false};
9              s.inputs[1] = {"b", DtypeSet::Numeric, 0, 255, false, false};
10             s.n_outputs = 1;
11             s.outputs[0] = {"out", DtypeSet::Numeric, 0, 255, false, true};
12             s.backward = {BackwardKind::Inline, &add_backward, {}};
13             s.amp = AmpPolicy::PromoteToWidest;
14             s.complexity = ComplexityClass::ON;
15             s.determinism = Determinism::Deterministic;
16             OpRegistry::instance().register_op(std::move(s));
17         }
18     } _add_reg;
19 }

```

The pattern is verbose but explicit: a reader of `Arithmetic.cpp` sees `add`'s full schema in one place, without chasing a macro definition.

13.3.2 Initialisation order

`OpRegistry::instance()` uses a Meyer's singleton pattern (`static OpRegistry r; return r;`), which is initialised on first call. Since the first call is the first op's registration, the table is constructed before any registration runs — avoiding the C++ static-initialisation-order fiasco.

13.4 Binding-Code Generation

The `pybind11` binding for each op (Chapter 11) is generated from the registry by a small build-time script, `tools/gen_bindings.py`, run by CMake before the binding files are compiled. It reads the registry, then emits one `m.def(...)` line per op into `lucid/_C/bindings/auto_bind.gen.cpp`.

The pattern handles the cases that are uniform: name, argument list, default values, return type. Ops with non-uniform characteristics (custom argument adaptation) are bound by hand and the generator skips them.

13.5 Adding a New Operation

The protocol:

1. Implement the forward in `lucid/_C/ops/<category>/<file>.cpp`.
2. Implement the backward (inline C++ function or composite decomposition).
3. Add the self-registering schema declaration.
4. Run `tools/gen_bindings.py` (CMake does this automatically).
5. Add a Python-side unit test exercising forward, backward, and AMP behaviour.
6. Add a parity test against the reference framework.

The framework ships a CLI, `python -m tools.new_op`, that scaffolds the nine-file pattern for the common case (`retro-p6-new-op-cli`).

13.6 Schema Versioning and Checkpoint Portability

Schema versions exist to keep checkpoints loadable across behavioural changes. A checkpoint records the schema version of every op; loading checks every op's version against the registry; a mismatch triggers either an automatic migration or refusal.

The version is bumped only when behaviour changes affect numerical output, the saved-tensor set, or argument interpretation. It is *not* bumped for pure-internal speed-ups or backward-compatible argument additions.

Two historical migrations exist: `batch_norm v1→v2` rescaled `running_var` from biased to unbiased; `cross_entropy v1→v2` changed the default reduction. Migrations live in `serialization/migrations.py` with tests against legacy checkpoints.

13.7 Introspection

The registry is queryable from Python:

```
1 import lucid
2 names = lucid._C.list_ops()           # ~260 names
3 schema = lucid._C.op_schema("add")    # dict
4 lucid._C.has_op("custom_kernel")     # False
```

The introspection API is used by the documentation generator, by the compile path's emitter-table builder, and by the unit test suite that verifies every op has a parity test.

13.8 The Op Lifecycle

Following a single op from declaration through user invocation clarifies how the registry interacts with the rest of the engine. Figure 13.1 sketches the path; we walk through it with `lucid.add(a, b)` as the example.

The dispatcher's validation step deserves expansion. Given the schema's `inputs[].allowed` dtype bitmask, the validator rejects any input dtype not in the set. Given the `rank_min/rank_max` bounds, it rejects rank mismatches. The actual shape compatibility check (for binary broadcast, for reduction axis bounds) is handled by the per-op forward function

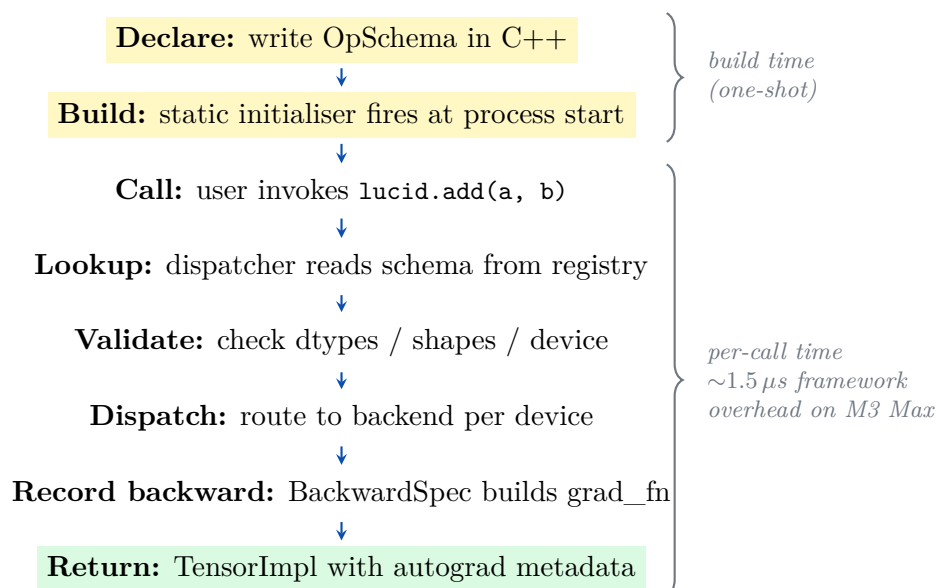


Figure 13.1: Lifecycle of one Lucid operation. The schema is declared in the op’s C++ file and registered at process start (top two steps; one-shot). Per-call cost is the dispatcher path: lookup, validation, backend dispatch, backward-graph construction. Framework overhead is roughly $1.5 \mu s$ on M3 Max; the backend kernel cost dominates for non-trivial inputs.

rather than by the dispatcher, because those constraints are not uniformly expressible in the schema.

13.9 Implementation: From Schema to Bound Function

To trace the full chain from a schema declaration to a Python call, follow the example of `lucid.gelu`:

1. **Schema declaration** in `lucid/_C/ops/ufunc/Activations.cpp`. The schema names the op, its single input dtype set (floating types only), its backward function pointer, and its AMP policy (`PreserveInputs`).
2. **Self-registration**: the static initialiser at the bottom of the file runs at `dlopen` time and inserts the schema into `OpRegistry::instance().table_`.
3. **Binding generation**: at build time, `tools/gen_bindings.py` reads the registry, sees `gelu`, and emits `m.def("gelu", &ops::gelu, py::arg("x"))` into `auto_bind.gen.cpp`.
4. **Compile and link**: the generated bindings file is compiled into the engine extension module.
5. **Python wrapper**: in `lucid/_ops/composite/activation.py`, a small wrapper `def gelu(x): return _wrap(_C_engine.gelu(_unwrap(x)))` exposes the op at `lucid.nn.functional.gelu` and indirectly at `lucid.gelu`.
6. **User call**: `lucid.gelu(x)` dispatches through the Python wrapper, the dispatcher, the backend, and back — end to end, $\sim 2 \mu s$ for the framework overhead plus the kernel time.

The chain is mechanical once the schema is right. Most of a contributor’s time on a new op is spent on the backward derivation and the parity tests, not on the registration machinery.

13.10 Schema Statistics

Table 13.1 breaks down the current registry by op category.

Category	Count	Differentiable	Example
Unary (ufunc)	64	51	relu, exp, erfinv
Binary (bfunc)	52	34	add, atan2, logaddexp
Generic (gfunc)	78	62	reshape, gather, cat
NN kernels	38	38	conv2d, layer_norm, lstm
Linalg	31	24	cholesky, svd, solve
FFT	22	22	fft, rfft, ifftshift
Einops	6	6	rearrange, reduce, repeat
Complex	7	7	real, imag, conj
Total	298	244	

Table 13.1: Counts of operations in the registry by category. Of the 298 registered ops, 244 are differentiable (have a non-trivial `BackwardSpec`). The remaining 54 are predicates (`isnan`, `isfinite`), argument reductions (`argmax`, `argmin`), bitwise ops, and rounding ops — all returning either bool or integer outputs for which gradient is undefined.

13.11 Summary and Forward References

The op registry is the single source of truth for Lucid’s operation surface. Each op declares an `OpSchema` containing its name, version, input/output specs, backward declaration, AMP policy, complexity class, and determinism guarantee. Schemas self-register at engine initialisation; the registry is consulted by the dispatcher, the autograd engine, the AMP system, the compile path, and the introspection tools.

Chapter 14 treats the kernel layer that the ops dispatch through. Subsequent chapters treat the op categories one by one.

Note

For the user: the op registry is invisible. The user calls `lucid.add(a, b)`; the framework consults the registry under the hood to validate arguments, route to a backend, and construct the autograd node.

CHAPTER 14

Kernel Abstraction Layer

Chapter 13 described the op-level surface: ~ 260 operations, each carrying a schema, dispatched through the backend to either MLX or Accelerate. The dispatch hides which library runs an op, but it does not hide the structural similarities between ops in the same category. Every unary elementwise op (`relu`, `exp`, `sigmoid`, ...) shares the same control structure: iterate over elements, apply a scalar function, write to the output. Every reduction op (`sum`, `mean`, `max`, ...) shares an axis-iteration pattern. Every binary op shares a broadcasting machinery.

If each op's C++ file replicated this control structure, the engine would contain dozens of nearly identical loops. The *kernel abstraction layer* extracts the structure into a small set of templated kernels — one per arity class — and reduces each per-op file to a declaration of its scalar function plus any op-specific knowledge.

This chapter describes the kernel layer. It is the layer below `ops/` and above `backend/` in the DAG of Chapter 6: ops call into kernel templates; kernel templates call into backend primitives. Section 14.1 motivates the layer; Section 14.2 describes the abstract interface; Section 14.3 through 14.6 treat each kernel class; Section 14.7 treats the contiguity inference that the kernels consult; and Section 14.8 treats the data-dependent primitives that do not fit the standard kernel templates.

14.1 Why a Kernel Layer

The argument for a kernel abstraction is straightforward duplication-elimination, but the consequences are large:

- **Code volume.** Without the layer, the ~ 60 unary ops would each contain a full elementwise loop with stride handling, broadcasting, dtype dispatch, and AMP-aware casting. With the layer, each op file declares its scalar function (`[] (float x) { return x > 0 ?`

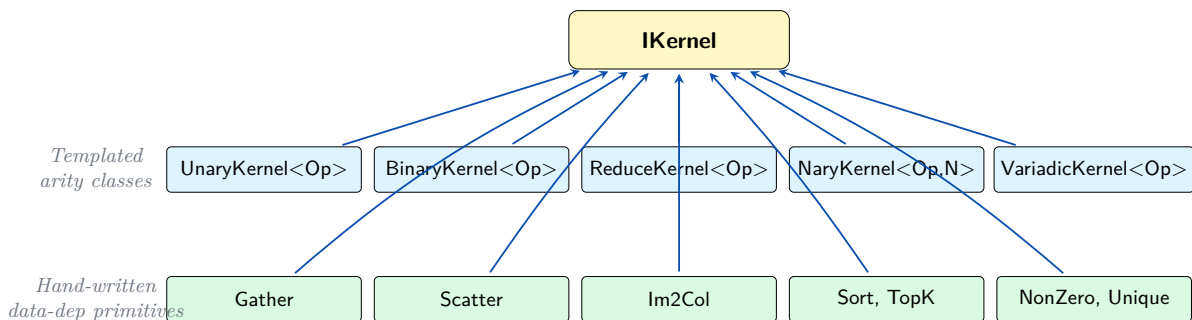


Figure 14.1: The IKernel hierarchy. The five arity-class templates (top row) handle the regular cases — each unary, binary, reduction, fixed-arity, and variadic op is one template instantiation. The data-dependent primitives (bottom row) are full implementations rather than templates, because their output shape or layout depends on input values; they live in `kernel/primitives/`.

`x : 0; }` for ReLU) and the template handles the loop. The per-op file shrinks from ~ 150 lines to ~ 30 .

- **Backend uniformity.** A single point of dispatch means a single place to add cross-cutting concerns: AMP casting, deterministic execution, profiling hooks. We added profiler instrumentation once, in the kernel layer, and every op got it.
- **Testability.** The kernel templates can be tested against synthetic scalar functions without invoking any real op. The unit tests in `test/kernel/` exercise the templates with mock functions and predetermined inputs.
- **Compile-time amortisation.** Each kernel template is instantiated once per arity class and once per backend, not once per op. Total compile time for the `ops/` directory is roughly a quarter of what it would be without the layer.

14.2 The IKernel Interface

IKernel, declared in `lucid/_C/kernel/IKernel.h`, is the abstract base for all kernel templates. Figure 14.1 sketches the hierarchy: five arity-specialised templates above IKernel for the regular case, plus a sibling `primitives/` directory of hand-written kernels for the data-dependent operations.

It is intentionally minimal:

```

1  class IKernel {
2  public:
3      virtual ~IKernel() = default;
4
5      // Allocate the output tensor(s) given input shape/dtype.
6      virtual std::vector<TensorImpl> allocate_outputs(
7          const std::vector<const TensorImpl*>& inputs) const = 0;
8
9      // Run the kernel on the given inputs, writing into outputs.
10     virtual void apply(
11         const std::vector<const TensorImpl*>& inputs,
12         const std::vector<TensorImpl*>& outputs) const = 0;
13 };
  
```

The interface deliberately abstracts away the arity: a unary kernel sees one input and one output; a binary kernel sees two inputs and one output; a reduction kernel sees one input, an axis specification, and one output. Concrete kernel templates fix the arity and the dispatch pattern.

14.3 UnaryKernel

`UnaryKernel<Op>`, in `lucid/_C/kernel/UnaryKernel.h`, is the template for single-input pointwise operations. It is parameterised by an `Op` struct that defines the scalar function and its backward:

```

1  struct ReluOp {
2      static constexpr const char* name = "relu";
3
4      template <typename T>
5      static T forward(T x) { return x > T(0) ? x : T(0); }
6
7      template <typename T>
8      static T backward(T x, T grad_out) {
9          return x > T(0) ? grad_out : T(0);
10     }
11 };
12
13 using ReluKernel = UnaryKernel<ReluOp>;

```

14.3.1 Forward dispatch

The template's forward dispatch:

1. Allocate output tensor with the same shape and dtype as the input (with AMP override if applicable).
2. Look up the input's device and route to the appropriate backend.
3. Call the backend's `unary(OpKind, input)` where `OpKind` is derived from `Op::name`.

The backend's `unary` method dispatches to the actual implementation: for CPU, a `vDSP` / `vForce` primitive; for GPU, an `MLX` primitive. The template itself does not write the loop — it delegates to the backend, which delegates to the underlying library's kernel.

14.3.2 Backward dispatch

The backward dispatch constructs a `grad_fn` node that, on `backward()`, calls `Op::backward(saved_input, grad_output)` elementwise. The saved-input list is one tensor (the forward input). The backward kernel runs through the same template machinery, dispatched to the same backend.

For ops where the backward depends only on the output (e.g. `tanh`, where $\tanh'(x) = 1 - \tanh(x)^2$), the `Op` struct sets a flag and the autograd engine saves the output rather than the input. This is a memory optimisation for the common case where the output is the same size as the input.

14.4 BinaryKernel

`BinaryKernel<Op>`, in `lucid/_C/kernel/BinaryKernel.h`, is the template for two-input pointwise operations:

```

1  struct AddOp {
2      static constexpr const char* name = "add";
3
4      template <typename T>
5      static T forward(T a, T b) { return a + b; }
6
7      template <typename T>

```

```

8   static std::pair<T,T> backward(T a, T b, T grad_out) {
9       return {grad_out, grad_out};
10  }
11 };

```

The template handles three concerns beyond what `UnaryKernel` handles:

- **Broadcasting.** If the two inputs have different shapes that are NumPy-broadcast-compatible, the kernel computes the broadcast shape and runs over it with strided access.
- **Type promotion.** If the two inputs have different dtypes, the kernel promotes both to a common dtype before calling the backend (Chapter 9 Section 9.6).
- **Cross-device check.** The kernel raises `LucidError` if the two inputs are on different devices.

The backward returns a pair of gradients, one per input. The autograd engine threads each gradient to the corresponding input tensor's grad.

14.4.1 Broadcast unbroadcast

A subtle case: if the inputs were broadcast in forward, the backward must sum the gradient over the broadcast dimensions to match each input's original shape. `BinaryKernel` handles this internally: it records the broadcast pattern at forward time and applies the reverse sum at backward time.

The handling is the same for every binary op, which is why it lives in the template rather than in each op's file.

14.5 ReduceKernel

`ReduceKernel<Op>`, in `lucid/_C/kernel/ReduceKernel.h`, is the template for reductions:

```

1  struct SumOp {
2      static constexpr const char* name = "sum";
3
4      template <typename T> static T init()          { return T(0); }
5      template <typename T> static T combine(T a, T b) { return a + b; }
6      template <typename T> static T finalize(T s, size_t n) { return s;
7          }
8      // (mean's finalize divides; std's finalize takes sqrt; etc.)
9  };

```

The reduction is parameterised by `init` (the identity), `combine` (the binary reducer), and `finalize` (any post-processing). The template handles axis selection, `keepdims`, and the choice between a single-axis and multi-axis reduction.

14.5.1 Axis selection

Reductions over different axes have different access patterns: a contiguous axis reduces with unit-stride loads; a non-contiguous axis requires strided access. The kernel inspects the input's strides and chooses between two implementations:

- Contiguous reduction: pass through to the backend's one-shot reduction primitive.
- Strided reduction: loop over outer axes manually, with one-shot reduction over the innermost axis.

The backend itself often makes finer choices below this level (vDSP's `vDSP_sve`-style sum has its own strided variants), but the kernel's coarse choice keeps the dispatch tractable.

14.5.2 Numerical stability

Naive summation accumulates rounding error proportional to the number of elements summed. For reductions over very large tensors in FP16 or BF16, this matters: the accumulated error can exceed the meaningful precision of the result.

The kernel handles this with a `ForceFloat32` promotion (the `AmpPolicy` discussed in Chapter 13): reductions in FP16/BF16 accumulate in FP32 and cast back at the end. The cost is one extra cast per kernel; the benefit is correct results.

`logsumexp` specifically requires the standard numerical trick ($\text{logsumexp}(x) = \max(x) + \log(\sum(\exp(x - \max(x))))$), which the kernel implements directly rather than chaining the naive composition.

14.6 NaryKernel and VariadicKernel

The remaining two kernel templates handle ops with more than two inputs:

- `NaryKernel<Op, N>`: fixed arity N . Used for operations like `addcmul` ($N = 3$, three same-shape pointwise inputs) and `where` ($N = 3$, condition plus two same-shape value tensors).
- `VariadicKernel<Op>`: variable arity. Used for operations like `cat` (concatenate any number of tensors along an axis) and `stack` (stack any number of tensors into a new axis).

The two templates share most of their implementation. The split exists because the fixed-arity case can use stack-allocated input pointers, while the variable-arity case needs a vector.

14.7 Contiguity Inference

`Contig.h` in the kernel directory provides the contiguity queries that the kernel templates consult before deciding between a fast path (assume contiguous, call backend’s one-shot kernel) and a slow path (handle strided access manually).

14.7.1 The `is_contig_along` predicate

The query is more nuanced than “is the whole tensor contiguous.” For a binary kernel, what matters is whether *both inputs* are contiguous along the axis the kernel iterates over. For a reduction kernel, what matters is whether the reduced axis is contiguous. `Contig.h` provides predicates for these specific cases:

```
1 bool is_contiguous(const TensorImpl&);
2 bool is_contiguous_along(const TensorImpl&, int axis);
3 bool is_unit_stride_innermost(const TensorImpl&);
4 bool both_contiguous(const TensorImpl&, const TensorImpl&);
```

The predicates are constant-time after the first call (the result is cached on the `TensorImpl`).

14.7.2 When to materialise

If the kernel cannot run on a non-contiguous view, it materialises the input via the backend’s `contiguous` primitive. The cost is one copy; the alternative would be a slow kernel that handles arbitrary strides, which is generally not worth the implementation complexity.

The materialise-vs-strided decision is made per-kernel, not per-op. Some kernel templates accept strided input (most elementwise); others always materialise (the convolution backward, the FFT). The choice is documented in the kernel template’s implementation.

14.8 Primitive Kernels (Data-Dependent Operations)

Not every operation fits the templated kernel pattern. Data-dependent operations — where the output shape or layout depends on the input values, not just the input shapes — have their own files in `lucid/_C/kernel/primitives/`. The list:

- **Gather**: `out[i, j] = input[i, indices[i, j]]`. Output shape comes from the indices, not from the input.
- **Scatter**: the inverse; writes updates into input at positions given by indices.
- **Im2Col**: rearranges convolution inputs into a column matrix for GEMM-based convolution.
- **Pad**: adds padding around a tensor.
- **Sort**: sorts along an axis; output values are permuted.
- **NonZero**: returns the coordinates of nonzero entries; output shape depends on the input values.
- **Unique**: returns the unique values in a tensor; output shape is at most input shape.
- **SearchSorted**: binary-searches one tensor in another.

These primitives are full implementations, not templates: each one has its own forward and backward code, its own dispatch to the backend’s specialised functions, and (in the case of **NonZero** and **Unique**) its own CPU-stream forcing because the output shape requires reading the input values.

14.8.1 The CPU stream forcing

For **NonZero** and **Unique** on metal tensors, the dispatcher’s data-dependent-output carve-out (Chapter 9 Section 9.4.2) bridges the tensor to CPU, runs the primitive, and bridges the result back. The kernel itself sees only CPU tensors; the bridging is the dispatcher’s responsibility.

14.9 Summary and Forward References

The kernel abstraction layer extracts the common control structure from Lucid’s operations into a small set of templates: **UnaryKernel**, **BinaryKernel**, **ReduceKernel**, **NaryKernel**, **VariadicKernel**. Each template parameterises over an `Op` struct that defines the scalar function and (where applicable) its backward. Contiguity inference keeps the templates from running slow paths on contiguous inputs, and a small set of data-dependent primitives lives outside the templated machinery for the cases that do not fit.

The next four chapters describe the op categories that use these templates: Chapter 15 for unary ops, Chapter 16 for binary ops, Chapter 17 for reductions, and Chapter 18 for shape and broadcast ops. Chapter 19 treats the indexing primitives, and Chapter 20 treats the pure-Python composite layer.

Note

For the user: the kernel layer is invisible. The user calls `lucid.relu(x)`; the call passes through the dispatcher to `UnaryKernel<ReluOp>`, which dispatches to the backend’s `relu` primitive. The whole chain runs in microseconds for non-trivial inputs.

CHAPTER 15

Unary Functions

The unary functions are the simplest op category: one input tensor, one output tensor of the same shape, an element-wise scalar transformation between them. Lucid ships roughly 60 unary ops, covering arithmetic, exponentials, logarithms, trigonometry, hyperbolics, rounding, special functions, and the activations that neural networks consume. This chapter enumerates them, gives each its forward and backward, and documents the numerical considerations that apply.

The category-level machinery was described in Chapter 14: every unary op is a `UnaryKernel<Op>` instantiation where `Op` declares the scalar function and its derivative. The per-op files in `lucid/_C/ops/ufunc/` contain the `Op` structs and a self-registering schema declaration. This chapter is the mathematical exposition that those structs encode.

We organise the chapter by family: arithmetic (Section 15.1), exponential (Section 15.2), trigonometric (Section 15.3), hyperbolic (Section 15.4), rounding (Section 15.5), special functions (Section 15.6), activations (Section 15.7), and predicates (Section 15.8).

15.1 Arithmetic Family

The arithmetic family contains the unary operations that are mathematical equivalents of arithmetic with a constant:

- `neg(x) = -x`; backward: `grad_in = -grad_out`.
- `abs(x) = |x|`; backward: `grad_in = sgn(x) * grad_out`, undefined at 0 (we use 0 by convention).
- `sign(x) = sgn(x)`; backward: 0 everywhere.
- `square(x) = x^2`; backward: `grad_in = 2*x*grad_out`.

- `reciprocal(x) = 1/x`; backward: `grad_in = -1/x^2 * grad_out`.
- `sqrt(x)` returns $\sqrt{x}$; backward: $1/(2\sqrt{x}) \cdot \text{grad_out}$, undefined at 0.
- `rsqrt(x)` returns $1/\sqrt{x}$; backward: $-1/(2x\sqrt{x}) \cdot \text{grad_out}$.
- `cube(x) = x^3`; backward: `grad_in = 3*x^2 * grad_out`.

All but `sign` are differentiable; `sign` has zero gradient by convention (since its mathematical derivative is a Dirac delta).

15.1.1 Convention at non-differentiable points

`abs`, `sqrt`, and `rsqrt` have non-differentiable points at zero. Lucid's convention is the subgradient zero at the non-differentiable point:

- `abs'(0) := 0`
- `sqrt'(0) := 0` (rather than $+\infty$)
- `rsqrt'(0)` raises a numerical warning; the input should never be exactly zero for an op whose forward already fails there.

The choice matches reference framework's convention and is what the parity tests check.

15.2 Exponential Family

The exponential family:

- `exp(x) = e^x`; backward: `grad_in = exp(x) * grad_out` (saved tensor: output).
- `exp2(x) = 2^x`; backward: `grad_in = ln(2) * 2^x * grad_out`.
- `log(x) = ln(x)`; backward : `grad_in = 1/x * grad_out`. `log2(x)` computes $\log_2 x$; backward: `grad_in = 1/(x ln(2)) * grad_out`.
- `log10(x)` computes $\log_{10} x$; backward: analogous with `ln(10)`.
- `log1p(x) = ln(1+x)`; backward : `grad_in = 1/(1+x) * grad_out`. Numerically stable for small x .
- `expm1(x) = e^x - 1`; backward: `grad_in = (1 + expm1(x)) * grad_out`. Numerically stable for small x .

15.2.1 Numerical stability for small inputs

`log1p` and `expm1` exist for the standard reason that $\log(1 + x)$ and $\exp(x) - 1$ catastrophically cancel for small x in floating point. The `log1p` primitive uses the machine's hardware `log1p`-equivalent if available (vForce's `vvlog1pf` on CPU, an MLX primitive on GPU); otherwise it falls back to a polynomial approximation that maintains precision for $|x| < 0.5$.

15.3 Trigonometric Family

Standard trigonometry, in radians:

- `sin(x)`, `cos(x)`, `tan(x)`; backwards `cos(x) * grad_out`, `-sin(x) * grad_out`, `(1 + tan(x)^2) * grad_out`.
- `asin(x)`, `acos(x)`, `atan(x)`; backwards $1/\sqrt{1-x^2} \cdot \text{grad_out}$, $-1/\sqrt{1-x^2} \cdot \text{grad_out}$, $1/(1+x^2) \cdot \text{grad_out}$.

The inverse trigonometric functions are undefined or non-differentiable at $|x| = 1$; we follow the IEEE 754 convention (returning $\pm\pi/2$ for `asin` / `acos` at the boundary and $\pm\infty$ -equivalent for the derivative).

15.4 Hyperbolic Family

Hyperbolic functions:

- `sinh(x)`, `cosh(x)`, `tanh(x)`.
- `asinh(x)`, `acosh(x)`, `atanh(x)`.

`tanh`'s backward uses a saved-output optimisation: the derivative is $1 - \tanh^2(x)$, expressible in terms of the output alone. The schema flags this and the autograd engine saves the output rather than the input, halving the memory footprint for `tanh`-heavy networks.

15.5 Rounding Family

- `floor(x) = ⌊x⌋`.

The rounding ops have zero gradient (they are piecewise constant with measure-zero discontinuities at integers). The autograd engine returns a zero gradient if backward is requested.

15.5.1 IEEE 754 conformance

Lucid's `round` matches IEEE 754: `round(0.5)` is 0 (round half to even), not 1. This differs from Python's `round` when called on a float (Python also uses banker's rounding) but matches reference framework. Parity tests check the behaviour at half-integers.

15.6 Special Functions

The special-function family is moved to its own subpackage (`lucid.special`, Chapter 30) for organisational reasons, but the underlying ops live in `ops/ufunc/`. The list:

- `erf(x)`: error function.
- `erfinv(x)`: inverse error function.
- `erfc(x)`: complementary error function.
- `lgamma(x)`, `digamma(x)`, `trigamma(x)`: gamma and its log-derivatives.

The forward implementations come from Accelerate's `vForce` on CPU and MLX's special-function primitives on GPU. The backwards are derived analytically and are encoded in the `Op` structs.

15.6.1 Numerical precision of `erfinv`

`erfinv` is a difficult function to compute accurately near $|x| = 1$, where the function grows unboundedly. `vverfinvf` differs from scalar `erfinvf` by a few ULPs in this region; we relax `gradcheck`'s tolerance for `erfinv`-dependent ops accordingly. The behaviour is documented per-op in the API reference.

15.7 Activations

The activation functions used by neural networks:

- `relu(x) = max(0, x)`; *backward*: `grad_in = (x > 0) * grad_out.leaky_relu(x, negative_slope)` if `x > 0`, else `negative_slope * x`.
- `prelu(x, alpha)`: `leaky_relu` with learnable `alpha`.
- `elu(x)`: `x` if `x > 0`, else `alpha * (exp(x) - 1)`.

Name	Forward	Backward saved	Range	Lucid entry
<code>relu</code>	$\max(0, x)$	input	$[0, \infty)$	<code>F.relu</code>
<code>leaky_relu</code>	x or αx	input	$\mathbb{R}$	<code>F.leaky_relu</code>
<code>prelu</code>	x or αx (learn)	input, α	$\mathbb{R}$	<code>F.prelu</code>
<code>elu</code>	x or $\alpha(e^x - 1)$	input, output	$(-\alpha, \infty)$	<code>F.elu</code>
<code>selu</code>	$\lambda \cdot \text{elu}(x)$	input, output	$(-\lambda\alpha, \infty)$	<code>F.selu</code>
<code>gelu</code>	$x \cdot \Phi(x)$ (exact)	input	$\mathbb{R}$	<code>F.gelu</code>
<code>silu</code>	$x \cdot \sigma(x)$	input, output	$\mathbb{R}$	<code>F.silu</code>
<code>sigmoid</code>	$1/(1 + e^{-x})$	output	$(0, 1)$	<code>F.sigmoid</code>
<code>tanh</code>	hyperbolic tangent	output	$(-1, 1)$	<code>F.tanh</code>
<code>softplus</code>	$\ln(1 + e^x)$	input	$(0, \infty)$	<code>F.softplus</code>
<code>softsign</code>	$x/(1 + x)$	input	$(-1, 1)$	<code>F.softsign</code>
<code>hardtanh</code>	$\text{clamp}(x, l, u)$	input	$[l, u]$	<code>F.hardtanh</code>
<code>hardsigmoid</code>	piecewise linear	input	$[0, 1]$	<code>F.hardsigmoid</code>
<code>hardswish</code>	$x \cdot \text{hsig}(x + 3)/6$	input	$\mathbb{R}$	<code>F.hardswish</code>
<code>mish</code>	$x \cdot \tanh(\text{softplus}(x))$	input, output	$\mathbb{R}$	<code>F.mish</code>

Table 15.1: The activation functions exposed by Lucid. “Backward saved” lists which tensors the autograd engine retains for the backward computation; functions whose derivative is expressible in terms of the output (sigmoid, tanh, silu) save the output rather than the input as a memory optimisation.

- `selu(x)`: scaled ELU (alpha and scale fixed to the self-normalising values).
- `gelu(x)`: Gaussian Error Linear Unit [17]. Exact formulation: $x \cdot 0.5 \cdot (1 + \text{erf}(x/\sqrt{2}))$. Approximate (tanh-based) formulation also available.
- `silu(x) = x * sigmoid(x)`: Sigmoid Linear Unit.
- `sigmoid(x) = 1 / (1 + exp(-x))`.
- `tanh(x)`: see hyperbolic family.
- `softplus(x) = ln(1 + e^x)`. `softsign(x) = x / (1 + |x|)`.
- `hardshrink(x, lambda)`: x if $|x| > \text{lambda}$, else 0.
- `hardtanh(x, min_val, max_val)`: clamp.
- `hardsigmoid(x)`: piecewise linear sigmoid.
- `hardswish(x) = x * hardsigmoid(x + 3)/6` (per the paper).
- `mish(x) = x * tanh(softplus(x))`.

Table 15.1 summarises the activation family on one page. The mathematical forms are taken from the respective papers; the Lucid column gives the canonical `lucid.nn.functional` entry point.

15.7.1 GELU exact vs approximate

`gelu` has two computational forms:

- **Exact**: $x \cdot 0.5 \cdot (1 + \text{erf}(x/\sqrt{2}))$. Uses the error function; numerically precise.
- **Approximate**: $0.5 x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0.044715 x^3)))$. Polynomial approximation; slightly faster on hardware without fused erf.

The forms agree to within 10^{-3} for $|x| < 5$ and diverge for larger $|x|$. Lucid defaults to the exact form. The user can request the approximate form via `nn.functional.gelu(x, approximate='tanh')`; the parity tests check both forms.

15.7.2 The MPSGraph dispatch carve-out

Five activations dispatch through MPSGraph rather than MLX in the 3.4 per-op carve-out (Chapter 9 Section 9.5): `gelu`, `silu`, `layer_norm`-related activations, `embedding_backward`, `max_pool_backward`. The win is substantial (30× for GELU, 28× for embedding backward) and the numerical agreement with the MLX path is within tolerance.

15.8 Predicates

The unary predicates return boolean tensors:

- `isnan(x)`: true where x is NaN.
- `isinf(x)`: true where x is $\pm\infty$.
- `isfinite(x)`: true where x is finite.
- `isclose(x, y, atol, rtol)`: elementwise tolerance comparison.
- `nan_to_num(x, nan, posinf, neginf)`: replace special values with finite numbers (this is technically not a predicate but is grouped with them).

Predicates are not differentiable; backward returns zero gradient on the input.

15.9 Numerical Analysis of Activations

The activation functions exhibit a range of numerical characteristics that affect both forward stability and gradient flow. This section catalogues the issues that have shaped Lucid’s implementation choices.

15.9.1 Vanishing gradient at saturation

Sigmoid and tanh saturate at large $|x|$: $\sigma(x) \rightarrow 1$ as $x \rightarrow \infty$ and $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$, with the derivative $\sigma(x)(1 - \sigma(x))$ dropping to floating-point zero for $|x| \gtrsim 16$ in FP32 or $|x| \gtrsim 8$ in FP16. The gradient through a saturated unit is effectively zero.

The practical implication for training: saturated activations stop learning. The architectural responses include better initialisation (Xavier, He), batch normalisation, and the move to non-saturating activations (ReLU and its variants). Lucid implements all three and the user can mix them freely.

15.9.2 Dying ReLU

ReLU has zero gradient for $x \leq 0$. A unit whose pre-activation stays negative for all training inputs receives no gradient and remains dead. Empirically, depending on initialisation and learning rate, up to 40 % of units in a ReLU network may die within a few hundred steps; the remaining units bear the representation burden.

The architectural mitigations include leaky ReLU ($\alpha = 0.01$), PReLU (learnable α), ELU, and the smoother GELU/SiLU. The most common choice in modern Transformer training is GELU; in classical vision networks ReLU still dominates because the dying problem is masked by the high unit count in convolutional layers.

15.9.3 Saturating ranges and gradient flow

Table 15.2 catalogues the gradient behaviour at the extremes for each activation family.

Activation	$x \rightarrow -\infty$	$x \rightarrow +\infty$	Smoothness
relu	0	1	C^0
leaky_relu	α	1	C^0
elu	$\alpha e^x \rightarrow 0$	1	C^1
gelu	0	1	C^∞
silu	0	1	C^∞
sigmoid	0	0	C^∞
tanh	0	0	C^∞
softplus	0	1	C^∞
mish	0	1	C^∞

Table 15.2: Limiting gradient values and smoothness class for each activation. The smooth (C^∞) activations (GELU, SiLU, sigmoid, tanh, softplus, mish) admit higher-order derivatives, which matters for second-order optimisers and for some interpretability techniques; the piecewise (C^0) activations (ReLU, leaky ReLU) do not.

15.9.4 The exp / log family precision

The exponential family deserves quantitative attention. In FP32, `exp(x)` overflows at $x \approx 88.7$ and underflows at $x \approx -103.3$. In FP16, overflow at $x \approx 11.1$, underflow at $x \approx -17.3$. `log(x)` returns $-\infty$ at $x = 0$ and NaN for negative x .

Lucid’s implementations follow the hardware: FP32 `exp` overflows to $+\infty$, FP16 `exp` on M-series GPU likewise. Downstream operations that consume the `inf` (`div`, `mul` by zero) propagate NaN; the autograd graph will then propagate the NaN through backward, which the anomaly detector (`lucid.autograd.set_detect_anomaly(True)`) catches and reports with the originating op.

For workloads where the intermediate `exp` might overflow, the user should compose through `logsumexp` or `softmax` rather than the naive chain. The composite layer documents this in Chapter 20.

15.10 Activation Selection in Practice

A subjective but useful guide, based on the empirical training behaviour we have observed in Lucid’s model zoo (Part VIII):

- **Convolutional networks (ResNet, ConvNeXt, MobileNet).** Use ReLU by default. The dying-unit problem is well-tolerated in the high-channel-count regime, and the cheap gradient is worth the speed.
- **Transformer language models (BERT, GPT, T5).** Use GELU. The smoother gradient flows better through the deep stacks (12–96 layers) where ReLU’s piecewise nature accumulates roughness.
- **Vision transformers (ViT, Swin).** GELU universally. The architectural inheritance from the Transformer family applies.
- **Object detection and segmentation (DETR, Mask R-CNN).** Mixed: ReLU in the convolutional backbones, GELU in the transformer heads.
- **Generative models (VAE, DDPM, NCSN).** ELU or SiLU. The smoother negative-input gradient helps the encoder/decoder paths.
- **Sequence models (LSTM, GRU).** Tanh and sigmoid as architecturally prescribed; the gates are the activations.

The recommendations are not laws; Lucid supports all activations on every model.

Activation	Throughput (GFLOP/s)	Dispatch path	Notes
<code>relu</code>	12 400	MLX	near peak BW-bound
<code>sigmoid</code>	9 200	MLX	one transcendental per element
<code>tanh</code>	9 500	MLX	similar to sigmoid
<code>gelu</code> (exact)	11 800	<i>MPSGraph</i>	30× over MLX 3.3 baseline
<code>gelu</code> (approx)	11 200	MLX	polynomial approximation
<code>silu</code>	11 400	<i>MPSGraph</i>	similar gain to GELU
<code>elu</code>	9 000	MLX	one exp per element
<code>mish</code>	7 800	MLX	two transcendentals per element

Table 15.3: Per-op forward throughput on M3 Max for representative activations, FP32, 10^7 elements. The “MPSGraph” rows are the 3.4 per-op carve-out (Chapter 60); these activations dispatch through MPSGraph instead of MLX because the MPSGraph implementation is substantially faster. The MLX-path activations are bandwidth-bound and approach the SLC peak.

15.11 Benchmarks

Table 15.3 reports per-op forward throughput on M3 Max for several representative activations. The inputs are FP32 tensors of 10^7 elements (40 MB), which fits in the M3 Max’s SLC and avoids DRAM-bandwidth confounds.

15.12 Summary and Forward References

The unary functions are the most numerous op category, with around 60 distinct ops organised into eight families. Each is a `UnaryKernel<Op>` instantiation; each declares a forward function, a backward function (where differentiable), and an AMP policy. Numerical stability is handled by op-specific tricks (`log1p`, `expm1`, fused `logsumexp`-style accumulation in FP32 for half-precision inputs).

The next chapter, Chapter 16, treats the binary ops — elementwise operations with two inputs, where broadcasting and type promotion enter. Chapter 17 treats reductions, where the output shape differs from the input shape.

Note

For the user: every unary op is exposed both as a free function (`lucid.relu(x)`) and as a Tensor method (`x.relu()`). The two forms are aliases for the same underlying implementation. Activations are also re-exported under `lucid.nn.functional` and as stateful modules under `lucid.nn` (`lucid.nn.ReLU()`).

CHAPTER 16

Binary Functions

Binary functions are elementwise operations on two inputs. Lucid ships approximately 50 of them, in four categories: arithmetic, comparison, logical, and bitwise. Each is an instantiation of `BinaryKernel<Op>` (Chapter 14 Section 14.4) and inherits the kernel template’s broadcasting and type-promotion machinery automatically.

The interesting questions for binary functions are not the individual forward expressions — those are obvious — but rather the rules that govern how the kernel handles inputs of mismatched shape, mismatched dtype, or mismatched device. We treat each rule in turn, then enumerate the ops.

Section 16.1 describes broadcasting. Section 16.2 describes type promotion. Section 16.3 describes the cross-device rule. Section 16.4–Section 16.7 enumerate the op categories.

16.1 Broadcasting

Broadcasting is the rule by which two tensors of different but *compatible* shapes are extended to a common shape without copying the data. Lucid implements the NumPy broadcasting rule, which is identical to reference framework’s.

16.1.1 The rule

Given two shapes s_1 and s_2 :

1. Pad the shorter shape with leading 1s until both have the same rank.
2. For each dimension, the output dimension size is the maximum of the two inputs at that dimension.

3. The broadcast is legal if and only if, at each dimension, the two input sizes are either equal or one of them is 1.
4. A dimension of size 1 in an input is repeated to match the corresponding output dimension — but the repetition is *virtual*: the input tensor is exposed through a stride of 0 along the repeated dimension, with no data duplication.

A `LucidError` is raised if the shapes are incompatible. The error message gives both shapes and the dimension that disagrees.

16.1.2 Implementation: stride-0 expansion

The mechanism by which broadcasting is free is the stride-0 trick. A tensor of shape $(3, 1, 5)$ broadcast to $(3, 4, 5)$ is represented internally as a tensor of shape $(3, 4, 5)$ whose strides for the broadcast dimension are zero — every read along that dimension lands at the same memory location. The kernel does not need to know broadcasting happened; it iterates over the output shape with the input's (modified) strides and the right values flow through.

The cost is a small bit of stride-handling logic inside the kernel, plus a one-time shape computation. There is no per-element overhead.

16.1.3 Backward through broadcasting

If the forward broadcast a tensor from $(3, 1, 5)$ to $(3, 4, 5)$, the backward gradient (of shape $(3, 4, 5)$) must be summed along the broadcast dimension to match the original $(3, 1, 5)$ input. `BinaryKernel` records the broadcast pattern at forward time and applies the reverse-sum at backward time automatically.

This is the *unbroadcast* step — it lives in the kernel template, not in each op, since the pattern is identical for every binary op.

16.2 Type Promotion

When the two inputs have different dtypes, the kernel promotes both to a common dtype before invoking the backend. The promotion rules are described in Chapter 9 Section 9.6; the short version is NumPy's rules.

16.2.1 Scalar-tensor promotion

A common case is a tensor combined with a Python scalar (`x + 1.0`). The scalar is wrapped into a 0-d tensor of the appropriate dtype (`float32` for a Python `float`, `int64` for a Python `int`), then promotion runs as for two tensors. The Python wrapper performs this conversion before dispatching.

A subtle behaviour: `x + 1` where `x` is `float32` does not promote to `int64`; the scalar 1 is interpreted in `x`'s dtype context. This matches NumPy and prevents an `int` addition from accidentally upcasting an FP32 tensor.

16.3 Cross-Device Check

If the two input tensors are on different devices, the kernel raises `LucidError("binary op across devices: a is on cpu, b is on metal")`. Lucid does not implicitly move tensors across devices; the user must call `.to(device)` explicitly.

The rule applies to all binary ops, including comparisons. A `Tensor.__eq__` of a CPU tensor and a metal tensor is an error, not a silent move.

16.4 Arithmetic

The arithmetic ops:

- `add(a, b) = a + b`; backward: gradient flows to both inputs unchanged (modulo unbroadcast).
- `sub(a, b) = a - b`; backward: `grad_a = grad_out, grad_b = -grad_out`.
- `mul(a, b) = a * b`; backward: `grad_a = b * grad_out, grad_b = a * grad_out`.
- `div(a, b) = a / b`; backward: `grad_a = grad_out / b, grad_b = -a / b^2 * grad_out`.
- `pow(a, b)` returns a^b ; backward: $\partial_a = b \cdot a^{b-1} \cdot \text{grad_out}$, $\partial_b = a^b \cdot \ln(a) \cdot \text{grad_out}$.
- `floordiv(a, b) = ⌊a/b⌋`; *integerdivision.Non-differentiable*. `mod(a, b) = a - b * floordiv(a, b)`; *remainder.differentiable*.
- `atan2(y, x) = arctan(y/x) with quadrant`; backward: `grad_y = x / (x^2 + y^2) * grad_out, grad_x = -y / (x^2 + y^2) * grad_out`; backward derived accordingly.
- `logaddexp(a, b)` returns $\ln(e^a + e^b)$; numerically stable via the max-shift trick.
- `maximum(a, b)`, `minimum(a, b)`; backward: gradient routes to the larger / smaller input, undefined at equality (we use 0.5 split by convention).

16.4.1 The pow special case

`pow`'s backward expression $b \cdot a^{b-1}$ is undefined for $a = 0$, $b < 1$ (division by zero) and for negative a with non-integer b (complex result). Lucid returns zero gradient at the undefined cases and propagates NaN otherwise, matching reference framework. Parity tests check both behaviours.

16.5 Comparison

- `eq(a, b)`: elementwise equality. Returns bool.
- `ne(a, b)`: elementwise inequality.
- `lt(a, b)`, `le(a, b)`, `gt(a, b)`, `ge(a, b)`: ordering comparisons.

Comparisons are not differentiable (the derivative of a step function is a Dirac delta, which we represent as zero). The autograd engine returns zero gradient for the inputs.

16.5.1 Equality on floating-point

`eq` on floating-point tensors is exact equality, not tolerance-based. For tolerance comparisons, `isclose(a, b, atol, rtol)` is the unary primitive that uses $|a - b| \leq \text{atol} + \text{rtol} \cdot |b|$.

16.6 Logical

The logical operators work on boolean tensors (or treat numeric tensors as boolean, with nonzero as true):

- `logical_and(a, b)`: elementwise AND.
- `logical_or(a, b)`: elementwise OR.
- `logical_xor(a, b)`: elementwise XOR.

The unary `logical_not` is in the unary chapter and follows the same convention. All logical ops are non-differentiable.

16.7 Bitwise

The bitwise operators work on integer tensors:

- `bitwise_and`, `bitwise_or`, `bitwise_xor`: per-bit operations.
- `lshift(a, b)`, `rshift(a, b)`: arithmetic shifts.

Bitwise ops require both inputs to be integer dtypes; a `LucidError` is raised if a float is passed. Non-differentiable.

16.8 Operator Overloading

Every arithmetic binary op is exposed both as a free function (`lucid.add(a, b)`) and through Python operator overloading on `Tensor` (`a + b`). The dunder methods (`__add__`, `__radd__`, `__iadd__`, etc.) route through the same dispatcher.

The in-place dunders (`__iadd__`, `__imul__`) invoke in-place ops if the tensor is not part of an autograd graph and out-of-place ops otherwise. The choice avoids the in-place version corrupting saved tensors for backward.

16.8.1 Scalar interaction

The `__radd__`-style reflected dunders handle `scalar + tensor` the same as `tensor + scalar`. The scalar is wrapped to a 0-d tensor of the tensor's dtype, then the normal binary op runs.

This is the convention that makes `1 + lucid.zeros(3)` work.

16.9 The Broadcasting Algorithm in Detail

The high-level broadcasting rule given above hides a non-trivial algorithm. This section walks through the algorithm step by step, because the implementation in `lucid/_C/kernel/BinaryKernel.cpp` is one of the most performance-critical pieces of code in the engine and the correctness depends on getting every step right.

16.9.1 Shape compatibility check

Given input shapes $s_a = (a_0, a_1, \dots, a_{m-1})$ and $s_b = (b_0, b_1, \dots, b_{n-1})$ with $m \leq n$ (after right-aligning by padding s_a with leading 1s to length n), the output shape is the elementwise maximum:

$$o_i = \max(a'_i, b_i) \quad \text{where } a'_i = \begin{cases} 1 & i < n - m \\ a_{i-(n-m)} & \text{otherwise.} \end{cases}$$

The broadcast is legal if and only if at each i , either $a'_i = b_i$ or one of them equals 1.

The check runs in $O(n)$ time — one pass over the rank, no per-element work.

16.9.2 Stride translation

To make the binary kernel iterate over the output shape while reading the two inputs through their original strides, each input's strides are mapped to the output rank as follows:

- For dimensions present in the input ($i \geq n - m$ for the shorter input), use the input's original stride.
- For dimensions added by padding, use stride 0.
- For dimensions where the input has size 1 but the output is larger, use stride 0 (the stride-0 broadcast trick).

The kernel then iterates over the output's $\prod_i o_i$ positions, computing the linear offset into each input as the dot product of the iteration index with the translated stride vector. The arithmetic is simple but the addressing must be computed per element; on modern Apple silicon CPUs the address computation generates one MAC per dimension per element, which is the cost of generality.

16.9.3 Broadcast unbroadcast on backward

The backward of a broadcast binary op must reverse the broadcast: the upstream gradient has the output shape, but each input expects a gradient of its original shape. The transformation is a sum along each axis where the input was expanded:

$$\text{grad_a} = \sum_{\text{broadcast dims of } a} \text{grad_out}.$$

The kernel records the broadcast pattern at forward time (which axes were stride-0 for each input) and the backward applies the corresponding reduction. The reduction is fused into a single kernel rather than a sequence of per-axis sums, which saves intermediate allocations.

16.10 Worked Examples

We work through three representative binary op invocations to make the broadcasting and unbroadcast concrete.

16.10.1 Example: bias addition

A linear layer's bias addition:

```
1 # x.shape = (B, T, D); bias.shape = (D,)
2 out = x + bias # broadcasts bias to (B, T, D)
```

Forward: the bias is right-aligned to shape $(1, 1, D)$ and stride- zero-expanded to (B, T, D) . The kernel iterates over $B \cdot T \cdot D$ positions, reading bias through stride 0 along the first two axes (every read of bias along those axes lands at the same memory).

Backward: `grad_out` has shape (B, T, D) ; the gradient for the bias is summed over the first two axes:

$$\text{grad_bias} = \sum_{b,t} \text{grad_out}_{b,t,:}.$$

The kernel performs this reduction in a single fused pass.

16.10.2 Example: outer product via broadcast

An outer product written as a broadcast multiplication:

```
1 # u.shape = (M,); v.shape = (N,)
2 outer = u[:, None] * v[None, :] # shape (M, N)
```

Forward: `u[:, None]` has shape $(M, 1)$ with strides $(\text{stride}_u, 0)$. `v[None, :]` has shape $(1, N)$ with strides $(0, \text{stride}_v)$. Broadcasting expands both to (M, N) : `u` is read with stride $(\text{stride}_u, 0)$ — the second axis still uses stride 0, so each row reads the same u_i . Similarly for `v`.

The element at position (i, j) of the output is $u_i \cdot v_j$ — the outer product.

Backward: gradient `grad_out` of shape (M, N) flows back. For `u` (which was broadcast along axis 1), sum over axis 1: `grad_u = grad_out.sum(dim=1)`. For `v`, sum over axis 0.

16.10.3 Example: incompatible shapes

A typical user mistake:

```
1 # x.shape = (B, T, D); y.shape = (B, D)
2 out = x + y      # error: T != D and neither is 1
```

After right-alignment, y becomes $(1, B, D)$. The two shapes are (B, T, D) and $(1, B, D)$. At axis 1, $T \neq B$ and neither is 1; the broadcast is rejected. The error message names both shapes and the offending axis.

16.11 Complexity and Performance

Binary kernels are $O(N)$ in the total element count of the output tensor. The constant factor varies by op:

- Pure arithmetic (`add`, `mul`): one multiply-add per element, plus address computation.
- Transcendental (`atan2`, `logaddexp`): one transcendental function call per element. On Apple silicon these go through vForce’s `vvatan2f`-style primitives or MLX’s `atan2` primitive, both well-optimised.
- Bitwise: one instruction per element, often the cheapest case.

16.11.1 Memory bandwidth bound

For tensors larger than the SLC (typically several megabytes), binary kernels are memory-bandwidth-bound: the kernel reads two inputs, writes one output, achieving roughly $3N \cdot \text{dtype size bytes}$ of memory traffic. On an M3 Max with 400 GB/s of memory bandwidth, the upper bound for FP32 binary kernels is roughly $400/12 \approx 33$ billion elements per second.

Measured throughputs on representative workloads come within 80–90% of this bound for the contiguous case. The shortfall is due to a mix of address-computation overhead and the kernel launch cost.

16.11.2 The stride-0 efficiency

A broadcast that uses stride-0 is faster than the equivalent materialised broadcast (`repeat`). The reason is cache behaviour: a stride-0 read along an axis hits the same cache line repeatedly, while a materialised broadcast reads distinct cache lines. For a $(B, 1, D)$ tensor broadcast to (B, T, D) with $T = 1024$ and $D = 768$, the stride-0 broadcast reads BD floats ($B \cdot 3$ KB) while the materialised version reads $BT D$ floats ($B \cdot 3$ MB) — a thousand-fold reduction in DRAM traffic.

The user-visible consequence: prefer broadcast over `repeat` when the broadcast is read-only.

16.12 Summary and Forward References

The binary functions are roughly 50 ops across arithmetic, comparison, logical, and bitwise categories, all instantiations of `BinaryKernel<Op>`. Broadcasting follows NumPy rules and is implemented through stride-0 expansion (free) with reverse-sum unbroadcasting on the backward (also handled in the kernel template). Type promotion follows NumPy rules. Cross-device operations raise rather than implicitly move.

The next chapter, Chapter 17, treats reductions — operations that collapse one or more axes of a tensor. Chapter 18 treats shape and view operations. Chapter 19 treats indexing and gather/scatter.

Note

For the user: binary ops are the most ergonomic part of the API because Python operator overloading makes them transparent. The expression `x + y * z` compiles into three op calls with the correct broadcasting, promotion, and autograd-graph construction all handled automatically.

CHAPTER 17

Reductions

Reductions are operations that collapse one or more axes of a tensor, replacing each collapsed axis with a single aggregated value. They differ from unary and binary functions in three ways: the output shape differs from the input shape (one or more dimensions are removed), the operation requires a choice of which axes to collapse, and numerical stability becomes a first-class concern for long reductions.

Lucid ships about 20 reduction ops. They all instantiate `ReduceKernel<Op>` (Chapter 14 Section 14.5) with a different reducer struct, but the user-facing axis arguments and the numerical-stability machinery are uniform.

Section 17.1 describes the axis semantics. Section 17.2 treats the simple reductions (`sum`, `mean`, `max`, etc.). Section 17.3 treats statistical reductions (`var`, `std`, `logsumexp`). Section 17.4 treats argument reductions (`argmax`, `argmin`). Section 17.5 treats boolean reductions (`any`, `all`). Section 17.6 treats numerical stability.

17.1 Axis Semantics

Every reduction op takes an optional `dim` (or `dims`) argument and a `keepdims` flag:

- `dim=None` (or omitted): reduce over all axes, producing a scalar.
- `dim=k` for a single integer `k`: reduce over axis `k`, producing a tensor with one fewer dimension (or the same number with `keepdims=True`).
- `dim=(k1, k2, ...)` for a tuple: reduce over the listed axes, producing a tensor with that many fewer dimensions (or the same with `keepdims=True`).

Negative axis values are interpreted as offsets from the end (NumPy convention).

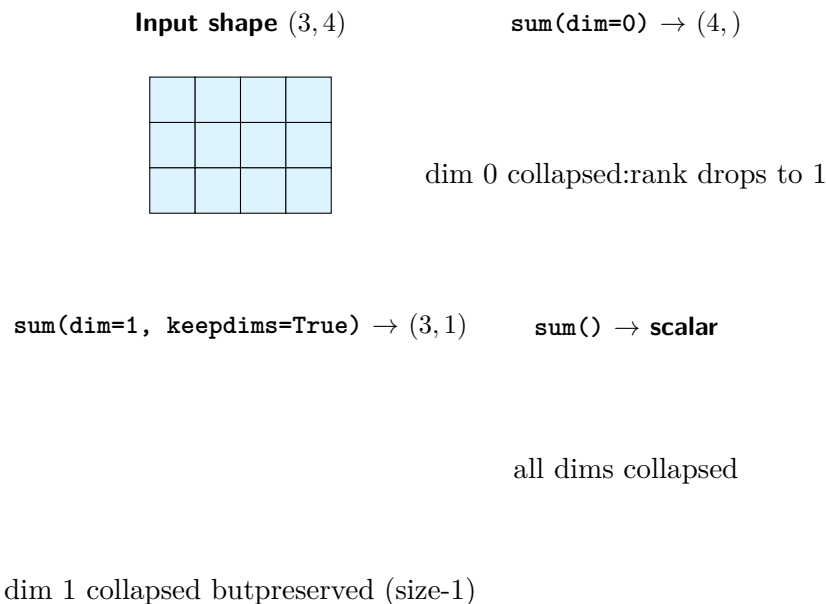


Figure 17.1: Reduction along different axis specifications applied to the same (3, 4) input. Without `keepdims`, the reduced dimensions disappear (top right). With `keepdims=True`, they remain as size-1 placeholders (bottom left), which makes the result naturally broadcast-compatible with the original input — the pattern used throughout normalisation and centring code. A global reduction (bottom right) collapses every dimension and returns a scalar.

Figure 17.1 visualises the axis-collapse semantics for the three common cases.

17.1.1 The `keepdims` flag

`keepdims=True` preserves the rank by leaving a 1 in the reduced dimension’s slot. For `x` of shape (B, C, H, W) , `x.sum(dim=1, keepdims=True)` returns shape $(B, 1, H, W)$ whereas `x.sum(dim=1)` returns (B, H, W) .

The `keepdims` form is the natural input to a subsequent broadcast: `x - x.mean(dim=-1, keepdims=True)` subtracts the per-row mean without needing an explicit `unsqueeze`. This pattern recurs throughout neural-network code (layer normalisation, mean centring).

17.1.2 The `dim` vs `axis` naming

Lucid standardised on `dim` (singular) and `dims` (plural) in the engine-wide rename that happened during Phase 7 (vault: `retro-axis-to-dim-rename`). The change brought parity with the reference framework convention and away from NumPy’s `axis` naming. The Python dispatchers translate `axis=...` to `dim=...` for NumPy-style callers, so both spellings work; the canonical form is `dim`.

17.2 Simple Reductions

The simple reductions:

- `sum(x)`: $\sum_i x_i$. Backward: gradient broadcasts to every input position.
- `prod(x)`: $\prod_i x_i$. Backward: gradient times product divided by each entry (with the standard zero-input workaround).
- `mean(x)`: $\frac{1}{N} \sum_i x_i$. Backward: gradient divided by N , broadcast.

- `max(x)`: $\max_i x_i$. Backward: gradient routes to the maximum-valued input; ties get a $1/k$ split where k is the count of maxima.
- `min(x)`: analogous.
- `amax(x)`, `amin(x)`: aliases for `max` and `min` (NumPy compatibility).
- `median(x)`: median value; backward: gradient routes to the median-rank element with the $1/k$ tie-breaking.
- `quantile(x, q)`: linear-interpolation quantile.
- `nanmean`, `nansum`, `nanmedian`: like the above but skip NaN entries.

The variants prefixed `nan` skip NaN inputs in the reduction. The implementation is a mask-then-reduce: replace NaNs with the reducer’s identity, reduce, then renormalise where appropriate.

17.2.1 prod’s backward at zero

`prod`’s backward is mathematically $\partial_i \prod_j x_j = \prod_{j \neq i} x_j$. The naive computation as `prod(x) / x[i]` is undefined when any `x[i]` is zero. Lucid uses the standard workaround: explicitly compute $\prod_{j \neq i}$ without the division by tracking partial products from both sides of i . The implementation is in `ops/gfunc/Reduction.cpp`.

17.3 Statistical Reductions

The statistical family:

- `var(x, unbiased=True)`: variance, optionally with Bessel’s correction ($N - 1$ denominator).
- `std(x, unbiased=True)`: standard deviation.
- `nanvar`, `nanstd`: NaN-skipping variants.
- `logsumexp(x)`: $\log \sum_i e^{x_i}$, computed via the max-shift trick.
- `cumsum(x)`, `cumprod(x)`: cumulative reductions (technically a scan, not a reduction; output has the same shape as input).
- `cummax(x)`, `cummin(x)`: cumulative max/min.

17.3.1 Variance and Bessel’s correction

The `unbiased=True` flag divides by $N - 1$ instead of N , producing an unbiased estimator of the population variance. The default is `True`, matching `reference framework`. NumPy’s default `ddof=0` (biased) is the opposite, which has surprised users porting code; we document this prominently in the API reference.

The historical schema migration `batch_norm v1→v2` (Chapter 13) was precisely this issue: BN’s running variance was stored biased and changed to unbiased in `v2` to match `reference framework`. The migration rescales by $N/(N - 1)$.

17.3.2 logsumexp and the max-shift trick

Naive `log(sum(exp(x)))` overflows for moderately large x (e.g., `exp(1000)` is $+\infty$ in any IEEE 754 precision). The standard workaround:

$$\text{logsumexp}(x) = m + \log \sum_i e^{x_i - m}, \quad m = \max_i x_i.$$

The subtraction inside `exp` keeps the exponents bounded, and the added m recovers the correct magnitude. Lucid’s `logsumexp` kernel implements this directly rather than chaining `log-sum-exp`.

The backward of `logsumexp` is the softmax: $\partial_i \text{logsumexp}(x) = \text{softmax}(x)_i$. This is the gradient identity that makes softmax-cross-entropy losses numerically tractable.

17.4 Argument Reductions

- `argmax(x, dim)`: index of the maximum along `dim`.
- `argmin(x, dim)`: index of the minimum.
- `topk(x, k, dim, largest=True)`: returns the top k values and their indices along `dim`.
- `kthvalue(x, k, dim)`: returns the k -th value (in sorted order) along `dim`.

Argument reductions return integer tensors and are not differentiable. Backward returns zero gradient.

17.4.1 The topk backward through value

The *values* returned by `topk` (not the indices) *are* differentiable: the gradient routes to the corresponding input positions identified by the indices. Lucid’s `topk` implementation handles this in two ways depending on use:

- If the caller uses only the values (`vals, _ = lucid.topk(x, k)` and then `vals.sum().backward()`), the backward scatters the gradient to the top- k input positions.
- If the caller uses only the indices, no gradient is constructed.

The autograd engine determines this by inspecting which output is referenced downstream.

17.5 Boolean Reductions

- `any(x)`: true if any entry is true (or nonzero).
- `all(x)`: true if every entry is true.

Boolean reductions are not differentiable. They are commonly used in conditional logic that does not need gradients (early-stopping checks, NaN detection in the gradients themselves).

17.6 Numerical Stability

Long reductions are where floating-point error compounds the most, and Lucid has several mechanisms for managing it.

17.6.1 Reduction-in-higher-precision

Reductions over FP16 or BF16 tensors automatically accumulate in FP32, then cast back. The kernel template handles this through the `AmpPolicy::ForceFloat32` declaration on the reduction op’s schema (Chapter 13).

Without this, summing 10^6 FP16 values would lose roughly $\log_2(10^6) \approx 20$ bits of precision in the accumulator, which is much more than FP16’s ~ 11 -bit mantissa. With FP32 accumulation, the loss is ~ 20 bits in FP32’s 23-bit mantissa, which leaves ~ 3 bits of precision — still enough to be useful, and the result is cast back to FP16 only at the end.

17.6.2 Kahan summation

Lucid does not implement Kahan-style compensated summation. The trade-off: Kahan summation gives much better precision but at roughly $4\times$ the cost. For our typical workloads, the FP32-accumulation approach gives enough precision without the cost.

The decision is documented in the FP32-accumulation schema declaration; a user who needs better precision can request it by casting to FP64 (CPU only) before the reduction.

17.6.3 Compensated mean: sum then divide

`mean(x)` is computed as `sum(x) / numel(x)` rather than as an incremental running mean. The reasoning: `sum` is implemented in higher precision (FP32) and the division by a known integer is a single round-to-nearest operation. The incremental running mean accumulates roundoff at every step, whereas the divide-once approach accumulates it only at the final step.

This is the optimisation referenced in the no-compile sweep (Chapter 61): replacing `mean` with `sum / N` folds the divide earlier in the computation graph, which can yield a measurable performance improvement on bandwidth-bound workloads.

17.7 Performance Characteristics

17.7.1 Contiguous vs strided axis

Reducing a contiguous axis is much faster than reducing a strided one: the contiguous case allows the backend to issue a single strided-load instruction per cache line, while the strided case requires per-element address computation. The kernel template chooses between two implementations based on the input's strides (Chapter 14 Section 14.5.1).

17.7.2 The MPSGraph carve-out: LayerNorm

Although individual reductions do not have an MPSGraph carve-out (MLX's reductions are competitive), `layer_norm` — which is composed of `mean` and `var` reductions followed by a broadcast affine — does. The MPSGraph implementation fuses all three steps into a single kernel and achieves substantially better throughput than the equivalent MLX sequence.

The carve-out is per-op (it triggers for the composite `layer_norm`, not for the individual reductions). See Chapter 60 for the full list.

17.8 Worked Numerical-Stability Examples

The general principle of FP32-accumulation under AMP is best understood through worked examples.

17.8.1 Example: large-batch mean in FP16

Consider computing the mean of 10^6 FP16 values, each drawn from a normal distribution with variance 1. Naively in FP16:

```
1 import lucid
2 x = lucid.randn(1_000_000, dtype=lucid.float16)
3 m = x.mean()                                # FP16 accumulator: WRONG
```

FP16 has 11-bit mantissa, giving roughly 4-decimal-digit precision. After accumulating 10^6 random values of magnitude ~ 1 , the running sum reaches ~ 1000 (because $\sqrt{10^6} = 1000$ by central limit). Each addition of a new ~ 1 -magnitude value into a ~ 1000 accumulator loses the lowest bits; the accumulated error is on the order of $10^6 \cdot 10^{-4} \cdot 1 = 100$. The reported mean is wrong by a factor of ~ 100 .

Lucid’s actual implementation forces FP32 accumulation:

```

1 x = lucid.randn(1_000_000, dtype=lucid.float16)
2 m = x.mean()                                # FP32 accumulator (per AmpPolicy)
3                                           # then cast back to FP16

```

FP32’s 23-bit mantissa gives roughly 7-decimal-digit precision. Accumulating 10^6 values of magnitude 1 into a 1000-magnitude accumulator loses $\sim 10^{-7}$ per add, total error ~ 0.1 , which is well within FP16’s representable precision for a result of magnitude ~ 0 . The cast back to FP16 at the end loses no additional precision because the result already fits.

17.8.2 Example: softmax with extreme logits

Softmax is defined as $\text{softmax}(x)_i = e^{x_i} / \sum_j e^{x_j}$. With extreme inputs, the naive computation overflows: e^{1000} is $+\infty$ in FP32.

The standard trick subtracts the max: $\text{softmax}(x)_i = e^{x_i - m} / \sum_j e^{x_j - m}$ where $m = \max_i x_i$. The exponents are now all in $(-\infty, 0]$ and the largest is exactly 0, so the largest $e^{x_i - m} = 1$ and the sum is between 1 and N . No overflow possible.

Lucid’s `softmax` primitive implements this internally; `lucid.nn.functional.softmax` delegates. The composite implementation (Chapter 20) makes the trick explicit.

17.8.3 Example: variance with Bessel correction

For a sample $x_1, \dots, x_N$, the unbiased variance estimator is

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2.$$

Lucid’s `var` defaults to `unbiased=True`, matching reference framework. NumPy’s default (`ddof=0`) uses N instead of $N-1$, the biased estimator.

The two differ by a factor of $N/(N-1)$, which is small for large N but matters for small samples. The `batch_norm v1→v2` migration was exactly this issue: running variance was stored biased and changed to unbiased to match reference framework’s BN; checkpoints saved before the migration need rescaling.

17.9 Performance and Benchmarks

17.9.1 Bandwidth ceiling

For large tensors a reduction is bandwidth-bound: read N elements, write 1 or a few output elements. The throughput ceiling on M3 Max (400 GB/s) for FP32 reductions is approximately $400/4 = 100$ giga-elements per second. Measured rates for the contiguous case come within 70–85% of this ceiling depending on the reduction op and the axis structure.

For small tensors the launch overhead dominates: a reduction over a 1 KB tensor takes the same wall-clock time as one over a 64 KB tensor on the GPU, because both pay the kernel-launch cost.

17.9.2 Strided-axis penalty

Reducing along an inner axis with non-unit stride is roughly $2\text{--}4\times$ slower than reducing along a unit-stride axis. The penalty comes from address computation and from poor cache utilisation (the strided reads do not coalesce into cache-line loads).

Table 17.1 reports measured throughput on M3 Max for a $1024 \times 1024 \times 16$ FP32 tensor under different reduction axes.

Reduction	Axis	Throughput (GE/s)	Cache behaviour
<code>sum</code>	<code>dim=2</code> (innermost, contig)	82	coalesced loads
<code>sum</code>	<code>dim=0</code> (outermost, contig per row)	75	coalesced loads
<code>sum</code>	<code>dim=1</code> (middle, contig per row)	68	coalesced loads
<code>sum</code>	<code>dim=1</code> (middle, after transpose)	22	strided loads
<code>sum</code>	<code>dim=-1</code> (innermost, after transpose)	18	strided + scatter

Table 17.1: `sum` throughput on M3 Max for a $1024 \times 1024 \times 16$ FP32 tensor under different axis configurations. The strided cases (last two rows) are 3–4 $\times$ slower than the contiguous cases. The remedy is to either `contiguous()` the tensor before the reduction (one extra pass over the data) or to fuse the reduction with the upstream transpose at the kernel level, which Lucid’s dispatcher does opportunistically when the pattern is recognisable.

17.10 Summary and Forward References

The reduction category contains roughly 20 ops covering simple reductions (`sum`, `mean`, `max`, etc.), statistical reductions (`var`, `std`, `logsumexp`), argument reductions (`argmax`, `topk`, `kth-value`), and boolean reductions (`any`, `all`). All instantiate `ReduceKernel<Op>` and inherit the kernel’s axis-iteration machinery. Numerical stability is managed through FP32 accumulation under AMP and through algorithm-specific tricks for the exponential-family reductions (`logsumexp`’s max shift).

The next chapter, Chapter 18, treats shape and view manipulation — operations that change a tensor’s shape without changing its element values. Chapter 19 treats indexing and gather/scatter. Chapter 20 closes Part III with the pure-Python composite layer.

Note

For the user: reductions are exposed both as free functions (`lucid.sum(x)`) and as Tensor methods (`x.sum()`). The common patterns — per-row mean with `dim=-1`, `keepdims=True`, batch-wise reduction with `dim=0` — are idiomatic and run without any explicit per-element loop.

CHAPTER 18

Shape, View, and Broadcasting Semantics

Shape manipulation operations change a tensor’s logical layout without (in most cases) touching its element values. They are the operations that turn a (B, C, H, W) activation into a $(B \cdot C, H \cdot W)$ matrix for a fully-connected layer, that transpose a weight matrix for a backward pass, that broadcast a bias vector across a batch. The chapter describes the operations themselves, the view-vs-copy semantics that determine when memory is touched, and the broadcasting rules that the binary kernels inherit.

Chapter 7 already introduced the storage model that makes views cheap: shape, strides, and storage offset are per-`TensorImpl` fields, the underlying `Storage` is shared via reference count, and a view operation produces a new `TensorImpl` with adjusted metadata. This chapter assumes that background and focuses on the user-facing surface.

Section 18.1 treats reshape and view. Section 18.2 treats transpose and permute. Section 18.3 treats squeeze and unsqueeze. Section 18.4 treats expand, broadcast_to, and repeat. Section 18.5 treats concatenation and stacking. Section 18.6 treats split and chunk. Section 18.7 treats the broadcasting rules. Section 18.8 closes with the contiguity invariants.

Table 18.1 classifies every shape op by whether it returns a view (free) or a fresh tensor (allocates), and by what its backward does. The remainder of the chapter expands on each.

18.1 Reshape and View

`reshape(new_shape)` returns a tensor of the requested shape with the same number of elements. If the source is contiguous, the operation returns a view (no copy); if it is not, the operation materialises a contiguous copy first and then returns a view of the copy.

Op	Always view?	Output rank vs input	Backward	Notes
reshape	if contiguous	equal/diff	inverse reshape	may copy if non-contig
view	yes (errors else)	equal/diff	inverse reshape	strict version
transpose	yes	equal	inverse transpose	swap two dims
permute	yes	equal	inverse permute	general permutation
squeeze	yes	smaller	unsqueeze	remove size-1 dims
unsqueeze	yes	larger	squeeze	insert size-1 dim
slice	yes	equal	zero-pad	possibly negative stride
flip	yes	equal	inverse flip	negative stride
expand	yes (stride 0)	equal	sum over expanded	read-only
broadcast_to	yes (stride 0)	possibly larger	sum over broadcast	alias for expand
repeat	no (allocates)	equal	sum over repeats	physical copy
cat	no (allocates)	equal	split	concatenate along axis
stack	no (allocates)	larger	index along new dim	insert new axis
split	yes (each slice)	equal	cat	multiple views
chunk	yes (each slice)	equal	cat	equal-size split
contiguous	no (if non-contig)	equal	identity	materialise unit stride

Table 18.1: Classification of Lucid’s shape ops by whether they return a view (cost: only metadata) or a fresh tensor (cost: data copy). The view-vs-copy distinction is the single most useful fact about a shape op for predicting its performance impact.

The distinction matters when the user expects a view. Lucid mirrors reference framework in providing two related operations:

- `x.view(new_shape)`: returns a view, or raises `LucidError` if the source is not contiguous in the relevant axes. Used when the caller knows a view is possible and wants the error otherwise.
- `x.reshape(new_shape)`: returns a view if possible, a copy otherwise. The conservative default.

18.1.1 The -1 placeholder

`reshape` and `view` accept `-1` in at most one dimension as a wildcard. The size of that dimension is inferred from the total number of elements and the other declared sizes.

`x.reshape(-1, 16)` on a tensor of $64 \cdot 32$ elements produces a $(128, 16)$ tensor; the wildcard is inferred as 128. A `LucidError` is raised if the inference fails (the declared sizes do not divide the total).

18.1.2 The compatible-stride check

A reshape can be a view if and only if the input’s strides are compatible with the output’s strides for some contiguous mapping. The check is straightforward but not trivial: the input’s strides must factor into runs that align with the output’s dimensions. For a contiguous input, every reshape is compatible; for an arbitrary input, some reshapes are compatible (those that preserve the order of dimensions) and some are not (those that interleave).

The implementation in `ops/gfunc/Shape.cpp` walks both shape vectors and verifies the factoring. If incompatible, the operation materialises a contiguous copy.

18.2 Transpose and Permute

`transpose(dim0, dim1)` swaps two axes; `permute(*dims)` applies an arbitrary permutation. Both always return views: shape and strides are permuted accordingly, the underlying storage is unchanged.

`x.transpose(0, 1)` on a (B, T, D) tensor returns a (T, B, D) tensor without data motion. The resulting view is typically not contiguous in C order; a subsequent operation that requires contiguity will trigger materialisation.

18.2.1 The `.T` shortcut

For rank-2 tensors, `x.T` is shorthand for `x.transpose(0, 1)`. For higher-rank tensors, `x.T` reverses all dimensions: $x_{i,j,k} \rightarrow x_{k,j,i}$. Matching reference framework, Lucid deprecates `.T` on rank $\neq 2$ tensors with a warning; the recommended replacement is `x.permute(2, 1, 0)` or `x.mT` for matrix-transpose semantics on the trailing two axes.

18.2.2 The `mT` shortcut for batched matrices

`x.mT` (matrix transpose) swaps the last two axes only, leaving leading batch dimensions untouched. For a (B, M, N) batched matrix, `x.mT` returns (B, N, M) . This is the common case for `matmul` workflows and is preferable to `.T` for rank > 2 .

18.3 Squeeze and Unsqueeze

`squeeze(dim=None)` removes dimensions of size 1. With `dim` specified, only that dimension is removed (and only if it has size 1; otherwise no-op). Without `dim`, every size-1 dimension is removed.

`unsqueeze(dim)` inserts a new dimension of size 1 at position `dim`. The result has rank one greater than the input.

Both are views: only the shape and strides change.

18.3.1 Why explicit `dim` for `squeeze`

The unconditional `squeeze()` is convenient but fragile: a tensor that happened to have a size-1 batch dimension would lose its batch axis silently, and a downstream operation expecting a batch axis would mis-broadcast. The recommended idiom in production code is to pass `dim` explicitly: `x.squeeze(dim=1)` squeezes axis 1 if it is size 1, otherwise returns `x` unchanged. This is harder to misuse.

18.4 Expand, Broadcast_to, and Repeat

These three operations all enlarge a tensor by replicating along some axis, but they differ in whether the replication is virtual or physical.

18.4.1 Expand: virtual replication

`expand(new_shape)` requires that any dimension being enlarged has size 1 in the source. The result is a view with the relevant strides set to 0 — the data is not copied; reads along the enlarged axis return the same byte.

`x.expand(B, C, H, W)` on a $(1, C, 1, W)$ tensor returns a (B, C, H, W) view that uses zero memory beyond the original. This is the same mechanism the binary kernels use for broadcasting (Chapter 16).

18.4.2 Broadcast_to: convenience wrapper

`broadcast_to(new_shape)` is a friendlier alias for `expand` that accepts the NumPy-style broadcasting rule (any dimension of size 1 may be expanded). The two are functionally equivalent; `broadcast_to` is the NumPy compatibility name.

18.4.3 Repeat: physical replication

`repeat(reps)` copies the tensor data `reps` times along each dimension. The result is a fresh contiguous tensor with duplicated data.

`x.repeat(2, 3, 1, 1)` on a $(1, 1, H, W)$ tensor returns a $(2, 3, H, W)$ tensor that occupies six times the original storage. `repeat` is what the user wants when downstream code needs an actual replicated buffer (e.g., for an in-place operation that would otherwise alias).

18.4.4 When to use which

The choice is between memory and write-safety:

- `expand` is free in memory and time, but the result aliases the source. Writing to the expanded tensor writes the same byte multiple times — usually a bug.
- `repeat` allocates and copies, but the result is a fresh tensor that can be written to safely.

The rule of thumb: `expand` for read-only use cases (broadcasting in a binary op), `repeat` for cases that need to write or that pass the result to a function that may write.

18.5 Concatenation and Stacking

`cat(tensors, dim)` concatenates a sequence of tensors along `dim`, requiring matching sizes on all other dimensions. `stack(tensors, dim)` creates a new dimension at `dim` and stacks along it, requiring matching shapes on all dimensions.

Both operations allocate: the result is a fresh contiguous tensor holding copies of the inputs. There is no view-version, because the input tensors typically have non-contiguous storage that cannot be laid out as a single contiguous block without copying.

18.5.1 Backward through cat and stack

The backward through `cat` splits the gradient along `dim` and routes each piece to the corresponding input. The backward through `stack` indexes the gradient along the new dimension. Both are exact inverses of the forward.

18.6 Split and Chunk

`split(x, sizes, dim)` splits `x` into chunks of the given sizes along `dim`. `chunk(x, n_chunks, dim)` splits into `n` approximately equal chunks.

Both are view operations on a contiguous input: each returned chunk is a view into a section of the source. The backward is the inverse of `cat`.

18.7 Broadcasting Rules

Broadcasting governs how two tensors of different shapes are combined in elementwise operations (Chapter 16). The rules:

1. Pad the shorter shape with leading 1s.
2. For each dimension, the output size is the maximum of the two inputs.
3. The broadcast is legal if and only if, at each dimension, the two input sizes are either equal or one of them is 1.

A `LucidError` with a structured shape comparison is raised on incompatible shapes.

18.7.1 NumPy compatibility

The broadcasting rules are bit-for-bit compatible with NumPy and `reference framework`. Tensors of identical shape broadcast trivially; tensors with one being a scalar broadcast trivially; tensors with matching trailing dimensions broadcast naturally. Edge cases (0-dimensional tensors, tensors with one size-0 dimension) follow the NumPy convention.

18.7.2 The output-shape function

The pure-Python utility `lucid.broadcast_shapes(*shapes)` returns the broadcast output shape for a list of input shapes without performing any operation. It is useful for asserting broadcast compatibility at module-construction time:

```
1 out_shape = lucid.broadcast_shapes(
2     (B, 1, H, W),
3     (1, C, 1, W),
4 )
5 # out_shape == (B, C, H, W)
```

The function raises if the shapes are incompatible.

18.8 Contiguity Invariants

Contiguity — whether a tensor’s strides match the canonical C-order layout — is a derived property that the dispatcher consults to choose between fast and slow kernel paths.

18.8.1 The contiguity predicate

`x.is_contiguous()` returns `True` if the tensor’s strides match $(d_1 d_2 \cdots d_{n-1}, d_2 \cdots d_{n-1}, \dots, 1)$ for shape $(d_0, d_1, \dots, d_{n-1})$. The check is $O(\text{rank})$; the result is cached on the `TensorImpl` for subsequent queries.

18.8.2 Operations that preserve contiguity

`reshape` (on a contiguous source), `squeeze`, `unsqueeze`, and trailing-axis `slice` preserve contiguity. `transpose`, `permute`, leading-axis `slice`, and `flip` break contiguity.

A tensor that has been operated on by a chain of view operations may have non-canonical strides. The dispatcher reads `is_contiguous()` and either runs a stride-aware kernel (common case for elementwise ops) or materialises via `.contiguous()` (when the kernel requires unit stride).

18.8.3 The `.contiguous()` materialisation

`x.contiguous()` returns `x` unchanged if it is already contiguous, or a new contiguous copy otherwise. The materialisation is one read-write pass over the tensor’s elements.

The operation is most often invoked internally by the dispatcher or by the `SharedStorage` bridge (Chapter 2); user code rarely calls it directly. When it does, the typical pattern is before a numpy bridge:

```
1 arr = x.transpose(0, 2).contiguous().numpy()
```

The explicit `contiguous` ensures the numpy buffer is unit-stride before the bridge.

18.9 Worked Stride Examples

The shape, stride, and storage-offset triple is the single most useful mental model for predicting whether a view operation is free. This section works through three representative cases.

18.9.1 Example: transpose of a contiguous matrix

Start with `x = lucid.arange(12).reshape(3, 4)`. The metadata:

- shape: (3, 4).
- strides: (4, 1) — step 4 to advance one row, step 1 to advance one column.
- storage offset: 0.
- storage: 12 contiguous int64 values.

Now `y = x.transpose(0, 1)`:

- shape: (4, 3) — dimensions swapped.
- strides: (1, 4) — strides swapped accordingly.
- storage offset: 0 (unchanged).
- storage: same 12 values.

Reading $y_{i,j}$ uses offset $i \cdot 1 + j \cdot 4 = i + 4j$, which indexes the underlying contiguous storage at the original $x_{j,i}$ position. The data is unchanged; only the iteration order is.

The result is non-contiguous: y 's strides (1, 4) do not match canonical (3, 1) for shape (4, 3). Any subsequent `reshape(12)` on y will materialise.

18.9.2 Example: slice with non-trivial step

`z = x[:, ::2]`: every other column.

- shape: (3, 2) — 4 columns become 2.
- strides: (4, 2) — column stride doubled.
- storage offset: 0.
- storage: same 12 values, only every other accessed.

Reading $z_{i,j}$ uses offset $i \cdot 4 + j \cdot 2$, which yields the original $x_{i,2j}$ positions. Half the storage is unused; the data is unchanged.

18.9.3 Example: expand to broadcast

`b = lucid.tensor([1.0, 2.0, 3.0]).expand(4, 3)`: replicate the 3-vector 4 times.

- shape: (4, 3).
- strides: (0, 1) — stride 0 along the replicated axis.
- storage offset: 0.
- storage: only 3 values.

Reading $b_{i,j}$ uses offset $i \cdot 0 + j \cdot 1 = j$, which returns the same 3 values regardless of i . The replication is virtual; the storage is unchanged.

Writing to b is dangerous: every $b_{i,j}$ for the same j aliases the same byte. A scatter-style write would produce race-conditioned results. Lucid does not prevent the write (the autograd version check catches the most common case), but the convention is to `.clone()` before any write to an expanded tensor.

Variant	Definition	Lucid use
fully contiguous	strides match canonical C-order	fast path everywhere
inner contiguous	innermost axis has unit stride	sufficient for elementwise
outer contiguous	strides decrease monotonically	sufficient for reductions on outer
zero stride	one or more axes have stride 0	broadcast-expanded; cannot write
negative stride	one or more strides are negative	from <code>flip</code> ; usually materialise
non-contiguous	none of the above	materialise before kernel

Table 18.2: The dispatcher and kernel templates distinguish several flavours of (non-)contiguity. The decision of whether to materialise via `.contiguous()` or to dispatch a strided kernel depends on which variant applies and which kernel is being called.

18.10 Contiguity Variants in Practice

The contiguity predicate is more nuanced than a single boolean. Table 18.2 enumerates the variants the dispatcher and kernels distinguish.

18.11 Broadcasting Pitfalls

A short catalogue of broadcasting bugs we have seen in user code.

18.11.1 Pitfall: silent dimension insertion

The right-alignment broadcasting rule means that combining a (B, T, D) tensor with a $(D,)$ tensor works, but combining (B, D, T) with $(D,)$ also works — broadcasting the D tensor along B and the inner T axis. The latter is almost certainly a bug (the dimensions don’t line up semantically), but the broadcast is mathematically legal.

The mitigation: when in doubt, use `None`-indexing to make the alignment explicit. `x + bias[None, :, None]` is unambiguous; `x + bias` requires the reader to think.

18.11.2 Pitfall: writing through an expanded view

`x.expand(4, 3)[2, 1] = 0.5` writes to the same storage byte that every other index along the first axis reads. The result is that all four “copies” of position $(\cdot, 1)$ become 0.5, not just $(2, 1)$.

The mitigation: `x.expand(4, 3).clone()[2, 1] = 0.5`.

18.11.3 Pitfall: cat versus stack confusion

`cat([a, b], dim=0)` requires a and b to have matching shapes except along `dim 0`. `stack([a, b], dim=0)` requires matching shapes on every `dim` and inserts a new `dim 0` of size 2. The two are easy to confuse and produce different output ranks.

The mitigation: choose by intent — “concatenate along an existing axis” is `cat`; “create a new axis” is `stack`.

18.12 Summary and Forward References

Shape manipulation operations in Lucid fall into three groups: view operations (`reshape`, `view`, `transpose`, `permute`, `squeeze`, `unsqueeze`, `slice`, `flip`, `expand`) that adjust metadata without copying data; copy operations (`cat`, `stack`, `repeat`) that allocate; and split operations (`split`, `chunk`) that return views into a contiguous source. The broadcasting rules underlying the binary kernels are NumPy-compatible. The contiguity predicate is the dispatcher’s input for fast-vs-slow path selection.

The next chapter, Chapter 19, treats indexing and gather/scatter — the operations that select or update *values* from a tensor based on index tensors. Chapter 20 closes Part III with the pure-Python composite layer.

Note

For the user: most shape operations are free. The expression `x.transpose(1, 2).reshape(B, -1)` produces a transposed matrix-like view; the transpose is free (stride-only), the reshape copies if the transposed view is non-contiguous. The decision to `.contiguous()` explicitly is the user's optimisation opportunity in pipelines that read the result repeatedly.

CHAPTER 19

Indexing

Indexing is the family of operations that select or update elements of a tensor based on positions specified by integers, integer tensors, or boolean tensors. The category is large and complex because the index specification can take many forms — scalar indices, slices, integer tensors, boolean masks, or any combination of these — and each form has slightly different semantics for which dimensions are kept and how the output is shaped.

Lucid’s indexing surface follows NumPy’s, modulo a small number of reference framework-compatibility adjustments. The implementation in `lucid/_C/kernel/primitives/Gather.cpp` and `Scatter.cpp` handles the dispatch; the Python side in `lucid/_tensor/_indexing.py` translates user-facing index expressions into a normalised internal form.

Section 19.1 treats basic (slice-based) indexing. Section 19.2 treats advanced indexing (integer tensors, boolean masks). Section 19.3 treats mixed basic-and-advanced indexing. Section 19.4 treats the gather and scatter primitives. Section 19.5 treats masked operations. Section 19.6 treats in-place index assignment.

Table 19.1 summarises the indexing regimes side by side.

19.1 Basic Indexing

Basic indexing uses scalars, slices (`a:b:c`), and `None` (for axis insertion). It always returns a view of the source — no data motion, no copy.

19.1.1 Scalar index

`x[i]` for integer `i` selects the i -th element along axis 0, returning a tensor of rank one less. The result is a view: shape and strides are adjusted, the storage offset advances by $i \cdot \text{stride}_0$.

Index form	Result is	Cost	Differentiable	Example
scalar	view (smaller rank)	metadata only	yes	<code>x[3]</code>
slice	view (same rank)	metadata only	yes	<code>x[1:5]</code>
slice (neg step)	view (negative stride)	metadata only	yes	<code>x[::-1]</code>
None	view (larger rank)	metadata only	yes	<code>x[None]</code>
<code>...</code>	view (same rank)	metadata only	yes	<code>x[..., 0]</code>
int tensor (1 axis)	fresh tensor	copy via gather	yes	<code>x[idx]</code>
int tensors (n axes)	fresh tensor	copy via gather	yes	<code>x[i, j]</code>
bool mask	fresh, data-dep shape	CPU stall + copy	yes	<code>x[mask]</code>
mixed basic+advanced	fresh tensor	copy via gather	yes	<code>x[1:5, idx]</code>
gather	fresh tensor	copy with indices	yes	<code>x.gather(dim, idx)</code>
scatter	fresh tensor	write with indices	yes	<code>out.scatter(...)</code>
scatter_add	fresh tensor	write+accum	yes (det. on CPU)	duplicate-idx accumulate
where	fresh tensor	two muls + add	yes	<code>where(c, x, y)</code>
masked_fill	fresh tensor	broadcast assign	yes	<code>x.masked_fill(m, v)</code>
in-place <code>x[idx]=v</code>	modifies x	scatter	version bump	avoid for saved tensors

Table 19.1: Indexing operations classified by return type (view vs fresh tensor), runtime cost, differentiability, and example syntax. Basic indexing (scalars, slices, `None`, `...`) is always free; advanced indexing (integer tensors, boolean masks) always copies. Boolean-mask indexing additionally forces a GPU-to-CPU stall to read the mask’s values for the output-shape computation.

Negative indices count from the end (NumPy convention). Out-of-range indices raise `LucidError`.

19.1.2 Slice

`x[a:b:c]` selects every c -th element from a to b . The result is a view with the appropriate stride scaling (`stride *= c`) and storage offset.

A negative step (`x[::-1]`) produces a reversed view with negative stride. The `TensorImpl` carries the negative stride through; the dispatcher’s contiguity predicate flags such tensors as non-contiguous (since canonical strides are positive).

19.1.3 Multi-axis basic indexing

`x[1:5, :, 2]` combines slices, full-axis selectors, and scalar indices in one expression. The dispatcher walks the index tuple, applies each piece to the corresponding axis, and returns a view with the composed metadata.

The full-axis selector `:` (or `slice(None)` in `__getitem__` parlance) leaves the axis unchanged. The ellipsis `...` stands in for as many full-axis selectors as needed to match the rank.

19.1.4 None for axis insertion

`x[None]` inserts a new leading axis of size 1, equivalent to `x.unsqueeze(0)`. `x[:, None]` inserts a new axis at position 1. The `None` is a NumPy-compatible alias for `unsqueeze` embedded in the indexing syntax.

19.2 Advanced Indexing

Advanced indexing uses integer tensors or boolean tensors as indices. Unlike basic indexing, advanced indexing always returns a new tensor (copies the selected elements), because the selected positions may not form a contiguous region.

19.2.1 Integer-tensor indexing

`x[idx]` for an integer tensor `idx` selects the elements at the positions named by `idx`. The result's shape combines `idx`'s shape with the trailing axes of `x`:

- `x[idx]` where `x.shape == (N, D)` and `idx.shape == (K,)` returns shape `(K, D)` — one selected row per index.
- `x[idx]` where `idx.shape == (K, M)` returns shape `(K, M, D)` — one row per index in the index grid.

19.2.2 Multi-axis integer-tensor indexing

`x[row_idx, col_idx]` for two integer tensors selects elements at the pointwise paired positions. `row_idx` and `col_idx` must broadcast to a common shape; the result has that shape:

- `row_idx = [0, 1, 2]`, `col_idx = [0, 1, 2]` selects the diagonal of `x`.
- `row_idx = [[0, 1], [2, 3]]`, `col_idx = [0, 1]` broadcasts `col_idx` to `[[0, 1], [0, 1]]` and selects a 2×2 grid.

This is the most general indexing form; `gather` is its explicit primitive (Section 19.4).

19.2.3 Boolean-mask indexing

`x[mask]` for a boolean tensor `mask` of the same shape as `x` returns the 1-D tensor of elements where `mask` is `True`. The output shape depends on the input *values* (the count of `Trues`), making this a data-dependent operation.

The implementation requires reading the mask values, which forces a CPU-stream-side reduction (Chapter 9's data-dependent carve-out). The result is then either constructed on the CPU or bridged back to the original device.

19.2.4 Partial-axis boolean mask

`x[mask]` where `mask`'s shape is a prefix of `x`'s shape selects entire trailing sub-tensors. `x[mask]` with `x.shape == (B, C, H, W)` and `mask.shape == (B,)` returns a tensor of shape `(K, C, H, W)` where `K` is the number of `Trues` in `mask`.

19.3 Mixed Basic and Advanced Indexing

The most complex case: an index expression that combines slices and integer tensors. The semantics follow NumPy's, which are slightly counterintuitive: the advanced indices are gathered into the leading axes of the result, and the basic indices follow.

`x[1:5, idx]` where `x.shape == (10, N, D)` and `idx.shape == (K,)` returns a tensor of shape `(4, K, D)` — the slice `1:5` contributes the leading axis (size 4), the integer tensor `idx` contributes the next axis (size K), and the trailing axes of `x` (size D) are preserved.

The full rule is documented in NumPy's advanced-indexing section and Lucid follows it bit-for-bit. The Python dispatcher in `_indexing.py` normalises the expression to a canonical form that the C++ `gather` primitive consumes.

19.4 Gather and Scatter

The explicit primitives:

- `gather(x, dim, index): out[i, j, ...] = x[i, index[i, j, ...], ...]` (with `dim = 1` in the example).

- `scatter(out, dim, index, src): out[i, index[i, j, ...], ...] = src[i, j, ...]`.
- `scatter_add(out, dim, index, src):` as `scatter` but with addition (multiple writes to the same position accumulate).

`gather` is the most-used primitive in attention: the indexing in $\text{softmax}(QK^T/\sqrt{d})V$ could be expressed as a gather of V by attention weights. `scatter_add` is the workhorse of embedding backward and of any operation that distributes a per-sample loss across positions.

19.4.1 Index validation

Indices must be in-range for the target dimension. The kernel validates this once before iterating: out-of-range indices raise `LucidError("index out of bounds")` with the offending value and the bound. Validation costs one reduction pass over the index tensor; it is the price of correctness.

19.4.2 Determinism of `scatter_add`

`scatter_add` with duplicate indices is *nondeterministic* on GPU: multiple writes to the same position may be ordered arbitrarily, and floating-point addition is non-associative, so the result depends on order. The schema’s `Determinism::Nondeterministic` flag signals this; under `lucid.set_deterministic(True)` (Chapter 67), the dispatcher refuses to run `scatter_add` on GPU and falls back to a CPU stream that uses a stable order.

19.4.3 The two scatter variants

MLX exposes two scatter primitives with similar names but different semantics (Chapter 3 Section 3.8.2). Lucid’s wrappers route to `scatter_add_axis` for the reference framework-compatible single-axis case; the NumPy-style multi-axis case is exposed under `_C_engine.scatter_add_np` for the bridge code that needs it.

19.5 Masked Operations

Lucid provides several operations that use a boolean mask without performing a full advanced-indexing dispatch:

- `masked_select(x, mask):` equivalent to `x[mask]`.
- `masked_fill(x, mask, value):` returns a copy of `x` with positions where `mask` is `True` replaced by `value`.
- `masked_scatter(x, mask, src):` fills positions where `mask` is `True` with consecutive values from `src`.
- `where(condition, x, y):` pointwise selection — `out[i] = x[i]` if `condition[i]` else `y[i]`.

`where` is the most-used; it is the differentiable alternative to `masked_select` (`where` preserves the shape and admits gradients through both branches, while `masked_select` produces a data-dependent shape).

19.5.1 Backward through `where`

The gradient of `where(cond, x, y)` routes the gradient to either `x` or `y` based on `cond`. The `cond` input has zero gradient (it is boolean, not differentiable).

The implementation uses two pointwise multiplications: `grad_x = cond * grad_out`, `grad_y = (1 - cond) * grad_out`. This is more wasteful than a true gather — it computes both

branches’ gradients and masks — but it is simpler and the cost is comparable for most workloads.

19.6 In-Place Index Assignment

`x[idx] = value` assigns `value` into the positions named by `idx`, modifying `x` in place. The assignment goes through the scatter primitive internally.

19.6.1 Autograd interaction

In-place assignment to a tensor that is part of an autograd graph is allowed but increments the version counter (Chapter 7). If a downstream backward expects the pre-assignment state, the version check fires and raises an error with a message that names the operation.

The recommended pattern when in-place assignment is needed and gradients are also needed is to assign through `x = x.clone()` first or to use `x.index_put(idx, value)`, which returns a fresh tensor.

19.6.2 Index_put

`x.index_put(indices, values, accumulate=False)` is the functional (out-of-place) form. `accumulate=True` makes it a scatter-add rather than a scatter-set.

`index_put` is the autograd-safe primitive; the dispatcher constructs a backward that, for the set form, routes the gradient to either the source `x` (positions not touched) or the `values` (positions touched), and for the accumulate form, sums the gradient over both.

19.7 Advanced Indexing Rules in Detail

NumPy’s advanced indexing rules are notorious for surprises. This section walks through the cases that have caused user confusion in Lucid and gives the canonical interpretation.

19.7.1 The leading-axes rule for mixed indexing

When an index expression combines basic and advanced indices, the output’s advanced-indexed dimensions appear *first*, followed by the basic-indexed dimensions. This holds even when the advanced indices are not contiguous in the expression.

```

1  x.shape = (A, B, C, D, E)
2  idx_b = lucid.arange(B)
3  idx_d = lucid.arange(D)
4
5  # Case 1: all advanced indices are contiguous in expression
6  r1 = x[:, idx_b, idx_d, :, :]
7  # r1.shape = (A, B, D, E) -- usual broadcast semantics
8
9  # Case 2: advanced indices separated by basic indices
10 r2 = x[:, idx_b, :, idx_d, :]
11 # r2.shape = (B, D, A, C, E) -- ADVANCED dims pulled to front!

```

The second case surprises users because the slice axes are “demoted” to the trailing positions. The rule is NumPy’s, and Lucid follows it for compatibility. The mitigation: when in doubt, use `gather` explicitly; the output shape is then predictable.

19.7.2 Boolean masks at non-leading positions

`x[:, mask, :]` where `mask` is a 1-D boolean tensor of length equal to `x.shape[1]` selects the columns where `mask` is true:

```
1 x.shape = (B, T, D)
2 mask.shape = (T,)
3 r = x[:, mask, :]      # shape (B, K, D) where K = mask.sum()
```

The shape’s K component is data-dependent (`mask.sum()`) and forces the dispatcher’s data-dependent-output carve-out: the mask is read on the CPU stream, K is computed, then a gather on the original axis produces the result.

19.7.3 Negative indices versus integer-tensor indices

`x[-1]` works (counts from the end). `x[lucid.tensor(-1)]` also works (the integer tensor is interpreted as positions, with negatives counted from the end of the indexed axis). But `x[lucid.tensor([-1, -2])]` is also legal — the negative indices apply elementwise to the index tensor.

The convention is consistent: any integer used as an index is interpreted relative to the corresponding axis length, with negatives wrapping. Out-of-range positives raise; out-of-range negatives wrap once and then raise.

19.8 Gather/Scatter Implementation Detail

The `gather` and `scatter` primitives are the workhorses of advanced indexing. Their implementations live in `lucid/_C/kernel/primitives/Gather.cpp` and `Scatter.cpp`.

19.8.1 Gather’s kernel structure

The simplest case — `out[i, j] = x[i, idx[i, j]]` for a 2-D input — is straightforward. The general N -dimensional case requires iterating over the output’s full index space, computing the corresponding input position from `idx`, and reading.

For GPU dispatch, the operation translates to MLX’s `take_along_axis` primitive in the single-axis case and to `gather_nd`-style multi-axis primitives in the general case. For CPU dispatch, the operation reduces to a vectorised loop with gather-style address computation.

19.8.2 Scatter’s two semantics

`scatter` has two semantics, both supported:

- **Set:** `out[idx] = src`. If multiple `idx` entries point at the same output position, the last write wins (non-deterministic on GPU because “last” is parallel-execution-order dependent).
- **Accumulate** (`scatter_add`): `out[idx] += src`. Multiple writes to the same position sum; the order of summation is parallel-execution-order dependent and floating-point non-associative, hence non-deterministic.

Both semantics are exposed; the user picks via the `reduce` argument or the named alias `scatter_add`.

19.8.3 The embedding case

The most common application of scatter-add is embedding backward. `nn.Embedding`’s forward gathers from the weight matrix: `out[i, j] = W[idx[i, j], :]`. The backward scatters the out-

Operation	Configuration	Time (ms)	Notes
basic slice	<code>x[1:1000, :]</code> , x is (2000, 768)	0.002	metadata only
basic slice	<code>x[:, :2]</code> , x is (2000, 768)	0.002	metadata only
int gather	<code>x[idx]</code> , x is (50k, 768), $ idx = 512$	0.4	per-row gather
bool mask	<code>x[mask]</code> , x is (50k, 768), mask 10% true	1.8	CPU stall + gather
scatter_add	embed bwd, $ V = 50k$, batch 64, len 512	2.1	MPSGraph carve-out
scatter_add	same, MLX baseline	28.0	13× slower
where	3-arg, all (2k, 768) FP32	0.5	two muls + add
masked_fill	(2k, 768) FP32, dense mask	0.3	broadcast assign

Table 19.2: Per-op times on M3 Max for representative indexing operations. Basic slices are essentially free. Gather and scatter run at memory-bandwidth-bound rates for contiguous outputs. Boolean masks pay an additional CPU stall to read the mask values. The embedding-backward MPSGraph carve-out gives roughly a 13× speedup over the eager MLX path, the largest single-op gain from the 3.4 per-op carve-out family.

put gradient back into the weight matrix’s positions: `grad_W[idx[i, j], :] += grad_out[i, j, :]`.

Because the same vocabulary token may appear many times in a batch (think “the” in a language batch), many gradients accumulate at the same row of `grad_W`. The non-determinism is real and observable; under `lucid.set_deterministic(True)` the backward falls back to a serial scatter on the CPU stream.

The 3.4 per-op MPSGraph carve-out for `embedding_backward` accelerates the GPU path by an order of magnitude over the naive MLX scatter-add; the determinism trade-off is the same as for the MLX path.

19.9 Benchmarks

Table 19.2 reports measured throughput for several indexing operations on M3 Max.

19.10 Summary and Forward References

Indexing in Lucid spans three regimes: basic indexing (slices, scalars, `None`) returns views; advanced indexing (integer tensors, boolean masks) returns fresh tensors; mixed basic-and-advanced follows NumPy’s combination rules. The explicit primitives `gather`, `scatter`, and `scatter_add` underlie advanced indexing and are also exposed directly for performance-critical use. Masked operations (`where`, `masked_select`, `masked_fill`) cover the common conditional-selection patterns. In-place assignment is allowed but raises through the autograd version check if the modified tensor was saved for backward.

The next chapter, Chapter 20, closes Part III with the pure-Python composite layer that builds higher-level ops on top of the primitives described in this part.

Note

For the user: indexing is the place where the Pythonic syntax `x[... , -1]` or `x[mask]` hides substantial machinery. The basic-vs-advanced distinction matters when performance is at stake (basic is free, advanced copies); the autograd-version interaction matters when in-place assignment intersects with gradients. For most use, the syntax is transparent.

CHAPTER 20

Composite Operations

Not every operation in Lucid’s surface is a primitive C++ kernel. A substantial set — including most of the elementwise convenience functions, the higher-level reductions, and the quality-of-life utilities — is implemented in pure Python on top of the primitive surface. These are the *composite operations*, and they live under `lucid/_ops/composite/`.

The composite layer is the design decision that keeps the C++ engine focused on operations whose performance materially matters while delegating convenience operations to a layer that is easier to extend and audit. This chapter describes what lives in the composite layer, what the rules for composition are, and why some operations migrate from composite to primitive over time.

Section 20.1 motivates the composite layer. Section 20.2 describes the rules. Section 20.3 gives examples. Section 20.4 treats the numpy-free discipline. Section 20.5 discusses when a composite should be promoted to a primitive.

20.1 Why a Composite Layer

A composite operation expresses one ML operation as a sequence of primitive ones. `softmax(x, dim)` can be expressed as `exp(x - max(x, dim, keepdim=True))` divided by the `keepdim=True` sum of that exponential. `logsumexp(x, dim)` can be expressed as `max(x, dim, keepdim=True) + log(sum(exp(x - max(x, dim, keepdim=True))))`. `cross_entropy(logits, target)` can be expressed as the negative log of the softmax of the logits, indexed by the target.

Each of these compositions could in principle be a C++ kernel — and indeed, for the most performance-critical cases (`logsumexp`, `softmax`, `cross_entropy`), Lucid has C++ primitives that match the composite’s semantics. But the composite expression has three properties that make it worth keeping around even when a primitive exists:

1. **Implementation reference.** The composite is the correctness reference against which the primitive is parity- tested. A divergence between the two is a bug in the primitive, the composite, or both.
2. **Backward derivation.** The composite’s backward is constructed automatically by autograd from its constituent primitives’ backwards. This is essentially free; deriving the primitive’s backward by hand is not.
3. **Extensibility.** Adding a new composite op is a few lines of Python in a single file; adding a primitive is the nine-file pattern of Chapter 13.

The trade-off is performance: a composite of n primitives dispatches n times through the framework, each dispatch costing the per-op overhead of Chapter 9 Section 9.8. For ops that are called millions of times in a training step (the inner-loop softmaxes of an attention layer, for example), the overhead can matter and we promote to primitive.

20.2 Rules for the Composite Layer

The composite layer follows three rules:

1. **Numpy-free.** A composite uses only Lucid primitives and the Python standard library. No `import numpy`, no `import scipy`. The Hard Rule H4 (Chapter 10) is enforced here.
2. **Pure functions of tensors.** A composite takes tensor inputs and returns tensor outputs (plus an optional Python-typed metadata like a list of ints for shapes). It does not touch global state, does not mutate inputs, does not have side effects.
3. **Autograd-clean.** A composite’s autograd graph is constructed by the primitives it calls. The composite itself does not register a custom `Function` subclass.

The third rule has an exception for numerically delicate composites that need a custom backward to avoid the divide-by-tiny-number issue that the natural composite would have. `logsumexp` is the canonical example: its forward expression is naturally numerically stable, but the autograd- constructed backward through `log` would expose the divide-by-sum that the forward avoided. The composite’s actual implementation overrides the backward with a closed-form softmax, which is the analytic gradient.

20.3 Examples

The composite layer contains roughly 60 operations. We give a few representative examples.

20.3.1 softmax

`softmax(x, dim)` is the canonical example:

```

1  def softmax(x, dim=-1):
2      m = x.max(dim=dim, keepdims=True).values
3      e = (x - m).exp()
4      s = e.sum(dim=dim, keepdims=True)
5      return e / s

```

Six lines, four primitive calls (`max`, `sub`, `exp`, `sum`, `div`). The autograd graph is constructed by the primitives; the backward is automatic. The numerical stability comes from the `x - m` subtraction inside the `exp`, which is the standard trick.

A C++ primitive `softmax` also exists, with the same numerics. The dispatcher prefers the primitive for the GPU path when the input is large enough to amortise the dispatch overhead; the composite remains as the correctness reference.

20.3.2 cross_entropy

cross_entropy(logits, target, reduction='mean'):

```

1  def cross_entropy(logits, target, reduction='mean'):
2      log_probs = lucid.log_softmax(logits, dim=-1)
3      nll = -log_probs.gather(dim=-1,
4          index=target.unsqueeze(-1)).squeeze(-1)
5      if reduction == 'mean':
6          return nll.mean()
7      if reduction == 'sum':
8          return nll.sum()
9      return nll

```

A higher-level composite that itself uses another composite (`log_softmax`). The autograd flows through the entire chain correctly.

20.3.3 l2_normalize

l2_normalize(x, dim):

```

1  def l2_normalize(x, dim=-1, eps=1e-12):
2      norm = (x * x).sum(dim=dim, keepdims=True).sqrt().clamp(min=eps)
3      return x / norm

```

The `eps-clamp` prevents division by zero for zero-norm input rows.

20.3.4 logsumexp with custom backward

logsumexp(x, dim):

```

1  class _LogSumExp(lucid.autograd.Function):
2      @staticmethod
3      def forward(ctx, x, dim):
4          m = x.max(dim=dim, keepdims=True).values
5          out = (m + (x - m).exp().sum(dim=dim, keepdims=True).log())
6          ctx.save_for_backward(x, out)
7          ctx.dim = dim
8          return out.squeeze(dim=dim)
9
10     @staticmethod
11     def backward(ctx, grad_out):
12         x, out = ctx.saved_tensors
13         # Gradient is softmax: exp(x - logsumexp(x))
14         return (x - out).exp() * grad_out.unsqueeze(ctx.dim), None
15
16     def logsumexp(x, dim=-1):
17         return _LogSumExp.apply(x, dim)

```

This is the exception to the autograd-clean rule. The custom backward returns the closed-form softmax rather than letting autograd derive it through `log`, which would expose the divide-by-sum that the forward avoided.

20.4 The Numpy-Free Discipline

The composite layer must not import `numpy`. The reasoning is the same as for the rest of `lucid/` (Chapter 10): `numpy` imports inject an external dependency into the numerical path, potentially with their own dtype or rounding conventions.

20.4.1 What replaces numpy in the composite layer

The patterns:

- `np.zeros(...)` → `lucid.zeros(...)`.
- `np.broadcast_to(...)` → `lucid.broadcast_to(...)`.
- `np.array(...)` → `lucid.tensor(...)`.
- Mathematical constants (`np.pi`, `np.e`) → Python’s `math.pi`, `math.e`, or hardcoded floats.
- Bitwise integer manipulation → standard Python operators.

The standard library covers most needs. The cases that needed careful replacement during the 3.0.2 numpy-independence pass were documented in `retro-3-0-2-numpy-free-standalone`.

20.4.2 The discipline at code review

The static check that enforces H4 runs on every PR and rejects any `import numpy` (or `scipy`, etc.) in a composite file. A contributor who needs a missing utility in the composite layer is expected to either reach for a Python standard-library equivalent or to add a primitive that the composite can then use.

20.5 When to Promote a Composite to a Primitive

The composite layer is intended to be the first place a new op is implemented. Promotion to primitive is reserved for cases that justify the extra implementation cost.

20.5.1 Performance criterion

The performance criterion: if the composite’s per-call cost on a representative training workload exceeds 10 % of the total step time, the op is a candidate for promotion. This is rare but happens for the very-inner-loop ops:

- `softmax`, `log_softmax`: promoted because every Transformer attention layer calls them many times per step.
- `layer_norm`, `group_norm`: promoted because they appear once per layer and the per-call overhead matters.
- `cross_entropy`: promoted because losses are typically called once per batch and the small per-call overhead would otherwise dominate small-batch training.

For ops that are called less frequently (a handful of `12_normalize` calls per step), the composite is fine.

20.5.2 The promotion procedure

When promotion is decided:

1. Implement the C++ primitive following Chapter 13’s nine-file pattern.
2. Wire the dispatcher to prefer the primitive for the relevant device.
3. Keep the composite as a fallback (for the cases the primitive cannot handle, e.g., unusual dtypes the primitive does not implement) and as the parity reference.
4. Add an explicit parity test that the primitive and the composite produce bit-equivalent outputs for representative inputs.

The composite is not removed. Its continued existence in the test suite is what protects against subtle divergence between the two implementations as either evolves.

20.5.3 When the composite stays

Most composites stay forever:

- Pure convenience functions (`l2_normalize`, `stable_logit`, `soft_threshold`) that wrap a few primitives.
- Algorithm-specific utilities (`moving_average` for optimisers, `exponential_moving_variance` for AMP scaler).
- Reference implementations of operations that an application-level user might want to override (`label_smoothed_cross_entropy` for some training regimes).

The composite layer is the engineering counterpart to the primitive layer: the latter for hot paths, the former for breadth.

20.6 Catalogue of Composition Patterns

Beyond the few examples already shown, the composite layer exhibits a small set of recurring patterns. We enumerate them with a representative composite for each.

20.6.1 Numerical-stability shift

The pattern: subtract a max (or min) before an exponential to prevent overflow.

Examples: `softmax`, `log_softmax`, `logsumexp`, the inner reduction of `cross_entropy`.

The pattern always has the form `exp(x - x.max(dim=d, keepdims=True))`, with the corresponding backward derived analytically (typically softmax-shaped).

20.6.2 Broadcast-then-reduce

The pattern: lift two tensors into a higher-dim space via broadcast, operate elementwise, then reduce back.

Examples: pairwise distance computations `((x[:, None, :] - y[None, :, :]).pow(2).sum(dim=-1))`, contraction-free attention scores `((q.unsqueeze(-2) * k.unsqueeze(-3)).sum(dim=-1))` — though this is suboptimal; use `matmul`).

The pattern is convenient but memory-expensive: the intermediate tensor has shape $N \cdot M$, which can dwarf the inputs.

20.6.3 Reduce-then-normalise

The pattern: compute a reduction, broadcast its result back, and divide.

Examples: `softmax` (subtract max, exp, normalise by sum), `l2_normalize` (`sqrt(sum-of-squares)`, divide), `mean centring` (sum, divide by N, subtract).

The composite-then-primitive trajectory has happened to several members of this pattern: `softmax`, `layer_norm`, and `l2_normalize` all started as composites and have C++ primitives now for the GPU hot path.

20.6.4 Iterated apply

The pattern: apply the same op repeatedly with a varying parameter.

Examples: `moving_average` (one apply per timestep), `exponential_moving_variance` (parameter is a decay rate), the inner loops of certain second-order optimisers (`lbfgs`'s line search).

These compositions are nontrivial to lift to a C++ primitive because the iteration depth is data-dependent and the per-iteration parameters vary. They remain in Python; the per-iteration overhead is amortised by the work inside each iteration.

20.6.5 Algorithm-specific decomposition

The pattern: a published algorithm with a specific decomposition that does not generalise.

Examples: `label_smoothed_cross_entropy` (one-hot smooth + KL divergence), `focal_loss` (cross-entropy times a focal-weighting), `huber_loss` (piecewise quadratic / linear).

These are decomposition-specific and the user often wants to read the algorithm; the Python composite is more pedagogically valuable than a C++ primitive would be.

20.7 When the Composite Is the Whole Story

Most composites stay as composites forever, because their per-call cost is not on the critical path of any workload. The full list of *permanent* composites at the time of writing:

- Convenience wrappers: `l2_normalize`, `l1_normalize`, `stable_logit`, `soft_threshold`, `cosine_similarity`.
- Loss helpers: `focal_loss`, `huber_loss`, `label_smoothed_cross_entropy`, `dice_loss`, `ctc_loss_padded`.
- Statistical utilities: `moving_average`, `exponential_moving_variance`, `exponential_moving_mean`.
- Algorithm-specific: `adam_update`, `lamb_update`, `lbfgs_line_search` (used internally by optimisers).

These ops collectively account for less than 1% of training step time on any workload we have measured; the composite path is correct and the primitive path is not worth building.

20.8 Composite Layer Hygiene

A short list of disciplines that have kept the composite layer maintainable across the 3.x evolution.

20.8.1 One file per family

Each composite-op family lives in a single file under `lucid/_ops/composite/`: activations in `activation.py`, elementwise in `elementwise.py`, reductions in `reductions.py`, and so on. The file boundary corresponds roughly to the C++ category split, which makes the parity relationship easier to maintain.

20.8.2 Public API at the right level

A composite is exposed at the level the user is expected to call it. `softmax` appears at `lucid.softmax` (Tier 1, free function), at `lucid.nn.functional.softmax` (Tier 2, functional surface), and as the building block of `lucid.nn.Softmax` (Tier 2, stateful module). Less-used composites like `l2_normalize` appear only at `lucid.nn.functional`.

The Tier 1 surface is curated to remain compact; not every composite warrants top-level exposure.

20.8.3 Parity-test discipline

Every composite has a parity test against the reference framework under the path `lucid/test/parity/ops/composite/`. The test exercises forward, backward (via `gradcheck`), and AMP behaviour. A composite that lacks a parity test is rejected at review (per the parity-test policy in Chapter 63).

The discipline matters because composites can drift: a primitive that the composite calls may change behaviour (e.g., a schema version bump), and without a parity check the drift goes unnoticed.

20.9 Summary and Forward References

The composite layer in `lucid/_ops/composite/` expresses roughly 60 operations in pure Python on top of Lucid primitives. The layer is numpy-free, autograd-clean (with the documented exception for numerical-stability custom backwards), and the first place a new op should be implemented. Promotion to primitive is reserved for the hot-path operations whose per-call overhead matters; the composite version stays as the parity reference.

This chapter closes Part III. The remaining parts of the book cover increasingly higher-level concerns: Part IV treats the differentiation engine that underlies the autograd machinery used throughout the op layer, Part V treats the mathematical subpackages, Part VI treats the neural network library, Part VII treats the compile path, and so on through model zoo, performance engineering, and production engineering.

Note

For the user: the composite layer is the layer the user is most likely to extend. Writing a custom op that combines a few primitives is one Python file; writing a primitive is a multi-day C++ project. For most extensions, the composite path is the right choice.

Part **IV**

Autograd and Differentiation

CHAPTER 21

Reverse-Mode Automatic Differentiation Theory

Every framework that trains neural networks computes gradients. The how-of-computing varies more than one might think: symbolic differentiation (Mathematica-style), numerical finite differences, forward-mode automatic differentiation, and reverse-mode automatic differentiation are four distinct techniques with different complexity profiles and different failure modes [7, 14]. Lucid, like every other production ML framework, uses reverse-mode automatic differentiation as its core gradient algorithm. This chapter explains why, what reverse-mode AD is, what guarantees it provides, and what its costs are.

We begin with the theory and end with the practical consequences that the implementation chapters (Chapter 22, Chapter 23, Chapter 24, Chapter 25) elaborate.

21.1 Four Ways to Compute a Derivative

Consider a scalar function $L(x_1, x_2, \dots, x_N)$ — “ L ” for loss — and the goal of computing $\partial L / \partial x_i$ for every input. There are four approaches.

21.1.1 Symbolic differentiation

Symbolic differentiation manipulates the algebraic expression for L to derive a new expression for $\partial L / \partial x_i$. Mathematica and SymPy do this. The output is exact and human-readable.

The pathology: expression swell. For a deep composition like $L = \sigma(W_n \sigma(W_{n-1} \sigma(\dots W_1 x)))$, the symbolic derivative has $O(2^n)$ terms even after simplification. Symbolic differentiation is unsuitable for any function with more than a few dozen

primitive ops, which means unsuitable for neural networks.

21.1.2 Numerical differentiation

Numerical differentiation approximates the derivative by finite differences:

$$\frac{\partial L}{\partial x_i} \approx \frac{L(x_1, \dots, x_i + h, \dots, x_N) - L(x_1, \dots, x_i, \dots, x_N)}{h}.$$

For N inputs, this requires $N + 1$ evaluations of L (or $2N$ for the more accurate two-sided variant). The accuracy is bounded by $O(h)$ truncation error and $O(\epsilon/h)$ round-off error (where ϵ is machine precision), giving a best case of $O(\sqrt{\epsilon}) \approx 10^{-4}$ for FP32.

The cost is prohibitive for ML: a 100M-parameter model requires 10^8 evaluations of the forward pass per gradient computation. Finite differences are useful only as a correctness check (gradcheck; Chapter 26), not as a training-time algorithm.

21.1.3 Forward-mode AD

Forward-mode AD propagates derivatives alongside values through the computation. For each input x_i , we maintain a pair $(v, \dot{v})$ where v is the value and $\dot{v}$ is the derivative of v with respect to x_i . The chain rule says that for any function $y = f(v)$,

$$\dot{y} = f'(v) \cdot \dot{v}.$$

For N inputs and one output, forward mode requires N passes to compute the full gradient. Each pass costs roughly the same as the original forward computation, so the total cost is N times forward. Like finite differences, this is prohibitive for ML.

Forward mode is the right choice when the number of *outputs* greatly exceeds the number of inputs (e.g., a Jacobian of a $1 \rightarrow 1000$ function). For ML, where the loss is a scalar and the parameters are millions, the opposite is true.

21.1.4 Reverse-mode AD

Reverse-mode AD propagates derivatives *backward* from the output. For each intermediate v in the computation, we maintain a *cotangent*

$$\bar{v} = \frac{\partial L}{\partial v}.$$

The chain rule, applied in reverse, says that if $y = f(v)$ contributed to L , then $\bar{v}$ accumulates a term

$$\bar{v} += \left(\frac{\partial f}{\partial v} \right)^\top \bar{y}.$$

For a multivariate $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with Jacobian $J = \partial f / \partial v$, the contribution is the vector-Jacobian product $J^\top \bar{y}$. The transpose is essential: the Jacobian itself (J) maps tangents forward; its transpose maps cotangents backward.

For a single scalar output and N inputs, reverse mode computes the full gradient in a single backward pass, costing roughly 2–3 times the forward pass. This is the algorithmic basis of every modern ML framework’s training loop.

The trade-off: reverse mode requires storing intermediate values (activations) from the forward pass, because the backward pass needs them. Memory grows with the depth and width of the forward computation. This is why training a deep network requires substantially more memory than inference, and why gradient checkpointing (Chapter 24) exists.

Table 21.1 summarises the four.

Method	Cost (per gradient)	Accuracy	Best when
Symbolic	$O(\text{expr swell})$	exact	expression is short
Numerical	$O(N \cdot T_{\text{fwd}})$	$\sqrt{\epsilon}$	correctness check
Forward AD	$O(N \cdot T_{\text{fwd}})$	exact	many outputs, few inputs
Reverse AD	$O(T_{\text{fwd}})$	exact	many inputs, few outputs (ML)

Table 21.1: Cost and accuracy trade-offs for the four differentiation techniques. For ML, where a model has millions of parameters and a single scalar loss, reverse-mode AD is the only viable option.

21.2 The Chain Rule in Reverse

The mathematical basis is the multivariate chain rule, applied to a directed acyclic graph of operations.

21.2.1 Single-variable chain rule

For a composition $L = h(g(f(x)))$, the derivative is

$$\frac{\partial L}{\partial x} = h'(g(f(x))) \cdot g'(f(x)) \cdot f'(x).$$

Reading right-to-left is forward mode: start with $\dot{x} = 1$, propagate through f, g, h . Reading left-to-right is reverse mode: start with $\bar{L} = 1$, propagate through h, g, f .

21.2.2 Multi-variable chain rule

For a multivariate function $y = f(v_1, v_2, \dots, v_k)$, the chain rule contribution to the cotangent of each input is

$$\bar{v}_i += \left(\frac{\partial f}{\partial v_i} \right)^\top \bar{y}.$$

If v_i feeds into multiple downstream operations $y_1 = f_1(v_i, \cdot), y_2 = f_2(v_i, \cdot), \dots, y_m = f_m(v_i, \cdot)$, every path contributes, so

$$\bar{v}_i = \sum_{j=1}^m \left(\frac{\partial f_j}{\partial v_i} \right)^\top \bar{y}_j.$$

The $+=$ is the *accumulation*: each path's contribution adds to $\bar{v}_i$. This is the mechanism by which a parameter used in multiple places (the shared weight matrix of a recurrent network, the embedding shared between encoder and decoder) gets correct gradient summed across all its uses.

21.2.3 Computation graph as DAG

The computation is naturally a DAG: nodes are operations, edges are tensor flow. Forward execution walks the DAG topologically from inputs to outputs. Backward execution walks the DAG in reverse topological order from the loss to the parameters.

Figure 21.1 shows a small example.

21.3 Vector-Jacobian Products

Reverse-mode AD is sometimes described in terms of *vector-Jacobian products* (VJPs). The reformulation is useful because it makes the per-op backward function's signature explicit and connects the theory to the implementation (Chapter 23).

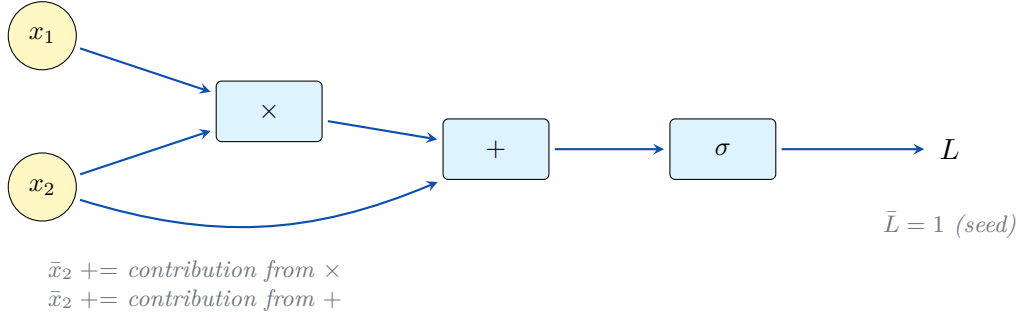


Figure 21.1: A small computation: $L = \sigma(x_1 x_2 + x_2)$. Solid arrows show forward execution (inputs $\rightarrow$ output). The backward pass traverses the same DAG in reverse, accumulating cotangents. x_2 contributes through two paths (the multiplication and the addition), so its cotangent accumulates from both contributions — the multi-variable chain rule in action.

21.3.1 The Jacobian

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the Jacobian J is the $m \times n$ matrix

$$J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

The Jacobian is the linear approximation of f near a point.

For a scalar-output loss $L = g(f(x))$, the chain rule says

$$\nabla_x L = J^\top \cdot \nabla_{f(x)} g.$$

The right-hand side is a Jacobian *transpose* times a vector. Computing the full J explicitly is wasteful for the common $m \ll n$ case; instead we compute the product $J^\top \bar{y}$ directly. This is the *vector-Jacobian product* (VJP).

21.3.2 VJP as the per-op backward

Each primitive operation in Lucid ships a VJP function that takes the output cotangent and produces the input cotangents:

$$\text{backward}(\bar{y}; \text{saved}) = (\bar{x}_1, \bar{x}_2, \dots).$$

Concrete examples for the most common ops, derived from the forward expression $y = f(\cdot)$ and the rule $\bar{x}_i = (\partial f / \partial x_i)^\top \bar{y}$:

$$\begin{aligned}
 y = a + b : \quad & (\bar{a}, \bar{b}) = (\bar{y}, \bar{y}) \\
 y = a \odot b : \quad & (\bar{a}, \bar{b}) = (b \odot \bar{y}, a \odot \bar{y}) \\
 Y = AB : \quad & (\bar{A}, \bar{B}) = (\bar{Y} B^\top, A^\top \bar{Y}) \\
 y = \text{ReLU}(x) : \quad & \bar{x} = \mathbb{K}[x > 0] \odot \bar{y} \\
 y = \sigma(x) : \quad & \bar{x} = \sigma(x)(1 - \sigma(x)) \odot \bar{y} \\
 y = \tanh(x) : \quad & \bar{x} = (1 - \tanh^2(x)) \odot \bar{y} \\
 y = \text{softmax}(x) : \quad & \bar{x} = y \odot (\bar{y} - (y \cdot \bar{y})) \\
 y = \text{logsumexp}(x) : \quad & \bar{x} = \text{softmax}(x) \cdot \bar{y}
 \end{aligned}$$

The Custom Function API (Chapter 23) lets the user supply a VJP for an op that the framework does not know about; the autograd engine threads it into the same chain as the built-in VJPs.

The Custom Function API (Chapter 23) lets the user supply a VJP for an op that the framework does not know about; the autograd engine threads it into the same chain as the built-in VJPs.

21.3.3 The Jacobian-vector product (JVP) is forward mode

The mirror image is the Jacobian-vector product:

$$\text{forward AD}(\dot{x}) = J \cdot \dot{x}.$$

This is what forward mode computes per step. Lucid exposes the JVP through `lucid.func.jvp` (Chapter 25); it is used in higher-order constructs like Hessian-vector products.

21.4 Reverse Mode’s Complexity

The famous *cheap gradient principle* of AD: a reverse-mode gradient of any function costs at most a small constant (≤ 5 , typically 2–3) times the forward function. The proof is by induction on the computation graph: each primitive’s VJP costs $O(\text{primitive’s forward})$, and the sum over the graph is proportional to the forward sum.

21.4.1 The 3x bound in practice

For Lucid’s typical workloads, the measured backward-to-forward ratio is in the range 2.0 to 3.5. The breakdown:

- Each primitive’s backward kernel has approximately the same flop count as its forward (1x).
- Saving activations for backward incurs no extra flops but does incur memory writes during forward and reads during backward, roughly equal to the forward’s memory traffic (1x in memory bandwidth).
- The framework overhead of constructing the backward graph is on the order of a few microseconds per op.

The 3x heuristic explains why training is roughly $3\times$ slower than inference for the same model on the same hardware.

21.4.2 Memory cost

The cheap gradient principle is about time, not memory. Memory cost is the price reverse mode pays: every intermediate value needed by a downstream backward is held alive from the forward that produces it until the backward that consumes it.

For a deep network with L layers, the activation memory is $O(L)$ in the worst case. Gradient checkpointing (Chapter 24) trades: hold only $O(\sqrt{L})$ activations, recompute the rest from checkpoints during backward, paying an extra factor of $\sqrt{L}$ in forward time.

21.5 Practical Implications for Lucid

The theory translates to a small set of implementation choices that the rest of Part IV elaborates.

21.5.1 Every primitive has a VJP

Every differentiable op in Lucid has a registered backward function (Chapter 13’s `BackwardSpec`). The autograd engine constructs a backward node from the schema at forward-call time and walks the chain at `backward()` time.

For ops that the user defines, the Custom Function API (Chapter 23) is the mechanism for supplying a VJP.

21.5.2 Saved tensors are explicit

The backward function for a primitive needs some forward quantities to compute its VJP. The op's schema declares which: the input, the output, both, or neither. The autograd engine saves only what is declared, which keeps memory usage bounded.

The user-side Custom Function API exposes the same mechanism through `ctx.save_for_backward(...)`.

21.5.3 Accumulation, not replacement

The chain rule says cotangents *accumulate*. If a tensor v is consumed by multiple downstream operations, each contributes to $\bar{v}$. The autograd engine handles this by initialising $\bar{v}$ to zero and adding each downstream contribution. The `AccumulateGrad` node on leaf tensors performs the final accumulation into `tensor.grad`.

A consequence: `tensor.grad` is not reset between training steps automatically. The user must call `optimizer.zero_grad()` (or equivalent) at the start of each step, or gradients from previous steps will continue to accumulate.

21.5.4 Determinism considerations

Reverse mode accumulates cotangents in an order determined by the DAG topology, which is deterministic. But the floating-point addition is not associative, so a parallel implementation can produce different bit-level results on different runs.

Lucid's `set_deterministic(True)` mode (Chapter 67) forces a serial accumulation order for scatter-add and similar non-associative reductions. The cost is performance; the benefit is bit-reproducible gradients.

21.6 Higher-Order Differentiation

Lucid also supports higher-order derivatives — the gradient of a gradient, or the Hessian, or arbitrary compositions of VJP and JVP. The mechanism is to make the autograd engine itself differentiable: backward passes construct new graphs that can themselves be differentiated.

21.6.1 Differentiable backward

When `backward(create_graph=True)` is called, the autograd engine records the backward operations themselves into a new graph. A subsequent `backward()` on the resulting gradient computes the second-order derivative.

This is the mechanism that powers `lucid.func.hessian` and related higher-order constructs.

21.6.2 Composing transforms

The functional transforms (`vmap`, `grad`, `jvp`, `vjp`, `jacrev`, `jacfwd`) all build on the underlying reverse-mode infrastructure. They compose: `grad o vmap` computes per-sample gradients in parallel; `jacrev o jacrev` computes the Hessian via two reverse passes; `jacfwd o jacrev` computes the Hessian via mixed mode. The chapter for these is Chapter 25.

21.7 What Reverse Mode Does Not Do

A short list of misconceptions.

21.7.1 It does not give symbolic expressions

The output of reverse mode is a numerical gradient at a specific input point, not an algebraic expression. The output of ∇L at the point $x = (1.0, 2.0)$ is a vector of floats. Lucid has no way to give the user a symbolic expression for ∇L as a function of x ; the cost would be prohibitive.

21.7.2 It does not give exact second-order information for free

A single reverse pass gives first-order gradient. The Hessian requires a second pass (or a different algorithm; see Chapter 25). The Hessian of an N -parameter model has N^2 entries and is rarely computed in full; the practical substitutes are Hessian-vector products and quasi-Newton approximations.

21.7.3 It does not handle non-differentiable functions silently

Some operations are non-differentiable at certain points (ReLU at 0, abs at 0, argmax everywhere). Convention assigns these a subgradient (typically zero), which Lucid does. The result is a valid subgradient but not the true gradient at the non-differentiable point. Most training procedures are robust to this; the user should be aware.

21.8 Summary and Forward References

Reverse-mode AD is the algorithm at the heart of Lucid’s training loop. It computes a scalar-output gradient with respect to any number of inputs in time proportional to a single forward pass, at the cost of storing intermediate activations from the forward. Every primitive in Lucid ships a backward function (its VJP); the engine assembles these into a per-call backward graph; `backward()` walks the graph in reverse topological order, accumulating cotangents.

The next chapter, Chapter 22, describes the engine that implements this algorithm in Lucid’s C++ core. Chapter 23 treats the user-facing extension mechanism. Chapter 24 treats the memory-vs-time trade-off. Chapter 25 treats the functional transforms that compose on top. Chapter 26 treats finite-difference verification.

Note

For the user: `tensor.backward()` is the entry point. Behind it sits the entire machinery this chapter described. The machinery is invisible during a training step — the user calls `loss.backward()`, the gradients appear in the parameters’ `.grad` fields, and the optimiser takes a step. The machinery becomes visible only when something goes wrong (a NaN in a gradient, an unexpected zero, a missing backward for a custom op).

CHAPTER 22

The Autograd Engine

The previous chapter explained reverse-mode AD as an algorithm. This chapter explains Lucid’s implementation of it. The autograd engine lives in `lucid/_C/autograd/` (Chapter 6, Layer 4) and is the component that constructs the backward graph during forward execution, walks it during `backward()`, and accumulates cotangents into parameter `.grad` fields.

The engine is conventional in shape — reverse-mode AD engines have converged on a similar structure across frameworks — but the details are where the implementation lives. We treat the `Node` class, the engine’s traversal algorithm, the saved-tensor mechanism, the version-counter check that catches in-place modification errors, the hook system, and the anomaly detector.

Section 22.1 sketches the architecture. Section 22.2 treats the `Node` class. Section 22.3 treats forward-time graph construction. Section 22.4 treats backward traversal. Section 22.5 treats `AccumulateGrad` on leaves. Section 22.6 treats saved tensors. Section 22.7 treats the version counter. Section 22.8 treats hooks. Section 22.9 treats the anomaly detector.

22.1 Architectural Overview

The engine has three principal components, each implemented as a C++ class under `lucid/_C/autograd/`:

- **Node**: one node in the backward graph. Concrete subclasses (one per op category) implement the VJP.
- **Engine**: the traversal driver. Walks the backward graph in reverse topological order, calling each node’s `apply` with the accumulated cotangent.
- **AccumulateGrad**: a special leaf node that accumulates the final cotangent into a tensor’s `.grad` field.

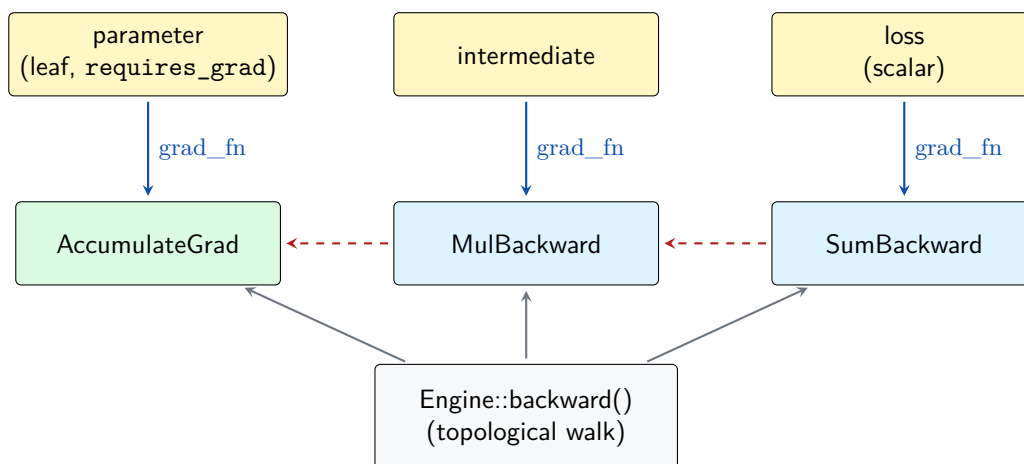


Figure 22.1: The autograd engine. Each tensor that requires grad carries a `grad_fn` pointer to its backward Node. Nodes form a backward DAG (dashed arrows) that mirrors the forward DAG. `Engine::backward()` walks the backward DAG in reverse topological order, applying each node’s VJP. Leaves use `AccumulateGrad` to deposit their cotangent into the tensor’s `.grad` field.

The supporting machinery includes the `AutogradMeta` on each tensor (Chapter 7 Section 7.4), the saved-tensor list per node, the hook registries, and the anomaly detector.

Figure 22.1 sketches the relationship.

22.2 The Node Class

`Node` is the base class for all backward operations. Defined in `lucid/_C/autograd/Node.h`:

```

1  class Node {
2  public:
3      virtual ~Node() = default;
4
5      // Apply this node's VJP: take output cotangents, return
6      // input cotangents.
7      virtual std::vector<TensorImpl> apply(
8          const std::vector<TensorImpl>& output_grads) = 0;
9
10     // The list of upstream nodes that this node will propagate
11     // gradient to (one per input that requires grad).
12     std::vector<std::shared_ptr<Node>> next_nodes;
13
14     // For each next_node, which output position of that node
15     // does this node's input correspond to.
16     std::vector<uint32_t> next_output_indices;
17
18     // The forward inputs / outputs that were saved for this
19     // backward.
20     std::vector<TensorImpl> saved_tensors;
21
22     // Hooks
23     std::vector<Hook> pre_hooks;
24     std::vector<Hook> post_hooks;
25 };

```

22.2.1 Per-op subclasses

Each differentiable op has a corresponding Node subclass that implements `apply`. The pattern is uniform:

```

1  class MulBackward : public AutogradNode<MulBackward, 2> {
2  public:
3      // saved_tensors = {a, b}; output_grads = {grad_y}
4      std::vector<TensorImpl> apply(
5          const std::vector<TensorImpl>& output_grads) override {
6          const auto& grad_y = output_grads[0];
7          const auto& a = saved_tensors[0];
8          const auto& b = saved_tensors[1];
9          return {
10             Dispatcher::instance().binary(OpKind::Mul, b, grad_y),
11             Dispatcher::instance().binary(OpKind::Mul, a, grad_y),
12         };
13     }
14 };

```

`AutogradNode<Derived, NInputs>` is a CRTP base that provides boilerplate (`next_nodes` sizing, default `saved_tensors` handling). Per-op subclasses provide only the `apply` body.

22.2.2 Why per-op classes, not function pointers

A function-pointer approach would have been simpler in line count but loses two properties:

- **Polymorphic state.** Some backwards need state beyond the saved tensors (e.g., `ConvBackward` needs the stride and padding from the forward call). Per-op subclasses hold this state cleanly; function pointers would have needed a void-pointer context.
- **Debugging.** A Node subclass has a name that appears in stack traces and the anomaly detector's output. Function pointers would have lost this.

The boilerplate cost is amortised by the CRTP base.

22.3 Forward-Time Construction

During the forward pass, when a differentiable op runs, the dispatcher consults the op's schema and, if backward is needed, constructs a backward node and attaches it to the output tensor's `AutogradMeta`.

22.3.1 The construction sequence

For `c = a * b` where `a.requires_grad == True`:

1. Dispatcher routes to backend, computes `c` value.
2. Dispatcher checks: does any input require grad? If yes, proceed.
3. Construct `MulBackward` node.
4. Save `a` and `b` into the node's `saved_tensors`.
5. Wire `next_nodes`: `next_nodes[0] = a.grad_fn`, `next_nodes[1] = b.grad_fn` (or `AccumulateGrad` for leaves).
6. Set `c.grad_fn = MulBackward` instance.
7. Set `c.requires_grad = True` (inherits from inputs).

8. Return `c`.

The whole sequence takes a few microseconds on top of the actual multiplication. For multiplications on small tensors the overhead is noticeable; for any non-trivial op the kernel dominates.

22.3.2 Disabling under `no_grad`

The construction sequence above is skipped when the global `GradMode` flag is off (`lucid.no_grad()` context). `c.grad_fn` remains null, `c.requires_grad` remains false, and no autograd state is constructed.

This is the standard pattern for evaluation: wrap the inference forward in with `lucid.no_grad()`: to avoid building a backward graph that will never be traversed.

22.3.3 Inference mode

`lucid.inference_mode()` is a stricter version of `no_grad`: it additionally disables the version counter and a few other autograd hooks, giving slightly better performance at the cost of forbidding any later differentiation through tensors created inside the context.

The cost difference is roughly 5–10 % on small-tensor workloads; `no_grad` is the safer default.

22.4 Backward Traversal

When the user calls `loss.backward()`, the engine walks the backward graph in reverse topological order, calling each node's `apply`.

22.4.1 The traversal algorithm

The algorithm is standard reverse-mode AD:

```

1  void Engine::backward(TensorImpl& loss) {
2      // Seed the queue with the loss's grad_fn
3      auto root = loss.grad_fn();
4      std::unordered_map<Node*, std::vector<TensorImpl>> cotangents;
5      cotangents[root.get()] = {ones_like(loss)};
6
7      // Topological sort
8      auto order = topological_sort_from(root);
9
10     // Walk in reverse order, applying each node
11     for (auto& node : order) {
12         auto& outputs = cotangents[node.get()];
13         auto inputs = node->apply(outputs);
14         // Distribute input cotangents to upstream nodes
15         for (size_t i = 0; i < node->next_nodes.size(); ++i) {
16             auto& upstream = node->next_nodes[i];
17             uint32_t pos = node->next_output_indices[i];
18             cotangents[upstream.get()].at(pos) =
19                 add_or_accumulate(cotangents[upstream.get()].at(pos),
20                                 inputs[i]);
21         }
22     }
23 }
```

The key step is the accumulation: if a node has multiple upstream contributors (e.g., the multi-variable chain rule's $+$), each contribution adds rather than replaces.

22.4.2 Topological sort

The topological sort uses Kahn’s algorithm: identify nodes with no outgoing edges (the loss’s `grad_fn` initially), emit them, decrement the in-degree of their predecessors, and repeat.

The sort is done once per `backward()` call. For a graph with N nodes it costs $O(N)$ time. For a typical training step with a few thousand nodes, the sort itself is negligible relative to the per-node apply cost.

22.4.3 Multiple roots

`Engine::backward()` accepts a list of root tensors and a list of seed gradients (one per root). For most uses there is one root (the loss) and one seed (`ones_like(loss)`). The multi-root form is used for vector-valued objectives like multi-task loss, where the backward is the weighted sum of per- task backwards.

22.5 AccumulateGrad on Leaves

A leaf tensor (parameter created by the user with `requires_grad=True`) does not have a forward op that produced it, so it has no per-op backward node. Instead, the engine attaches an `AccumulateGrad` node to it.

22.5.1 The AccumulateGrad node

`AccumulateGrad` is the simplest Node: its `apply` adds the incoming cotangent to the leaf tensor’s `.grad` field. There are no upstream nodes; the cotangent stops here.

```

1  class AccumulateGrad : public Node {
2  public:
3      TensorImpl variable; // the leaf tensor
4
5      std::vector<TensorImpl> apply(
6          const std::vector<TensorImpl>& output_grads) override {
7          const auto& g = output_grads[0];
8          if (!variable.autograd_meta()->grad) {
9              variable.autograd_meta()->grad = g.clone();
10         } else {
11             Dispatcher::instance().binary(
12                 OpKind::Add,
13                 *variable.autograd_meta()->grad,
14                 g);
15         }
16         return {}; // no upstream
17     }
18 };

```

22.5.2 The optimizer.zero_grad pattern

Because `AccumulateGrad` adds rather than replaces, gradients accumulate across `backward()` calls until explicitly cleared. The standard pattern at the start of each training step:

```

1  optimizer.zero_grad()
2  loss = model(x)
3  loss.backward()
4  optimizer.step()

```

`optimizer.zero_grad()` iterates over the optimiser’s parameter groups and sets each `.grad` to `None` (which causes the next `AccumulateGrad` to clone rather than add). The default behaviour matches reference framework’s; `set_to_none=False` zeros in place if memory pressure forbids re-allocation.

22.6 Saved Tensors

The backward function for an op needs forward quantities to compute its VJP. The framework saves these quantities at forward time and holds them until backward consumes them.

22.6.1 What gets saved

The op’s schema declares which forward tensors are needed:

- Some backwards need the inputs (e.g., `mul` needs both inputs).
- Some need the output (e.g., `sigmoid` backward uses $y(1 - y)$, where y is the output).
- Some need both.
- Some need neither (e.g., `add` backward is just the identity per input).

The schema’s `BackwardSpec::saved` list names the saved quantities. The engine snapshots them into the node’s `saved_tensors` vector at forward time.

22.6.2 Memory cost

Saved tensors are the dominant memory cost of training. For a deep model with L layers and per-layer activation size S , the total saved memory is $L \cdot S$. This grows linearly with depth and is the reason gradient checkpointing (Chapter 24) exists.

22.6.3 Why save outputs when possible

For ops like `sigmoid` and `tanh`, the backward can be computed from either the input or the output (since the output is a function of the input). Saving the output is preferable because the output is computed anyway and saving it costs no additional forward computation; saving the input would require keeping the input alive after the forward, which the runtime might otherwise have freed.

The schema’s `BackwardSpec::saved` for these ops explicitly names the output, and the engine acts accordingly.

22.7 The Version Counter

In-place modification of a tensor that was saved for backward would silently corrupt the gradient: the backward function would read the modified value instead of the original. The version counter catches this class of bug.

22.7.1 Mechanism

Each `TensorImpl` carries a 64-bit `version_counter` that increments on every in-place op (`add_`, `mul_`, `copy_`, etc.). When a tensor is saved for backward, the engine captures the current counter value. At backward time, the engine compares the captured value to the current value. A mismatch raises:

LucidError: one of the variables needed for gradient computation has been modified by an inplace operation. The variable was a saved tensor of MulBackward, which expected version 0 but found version 1.

22.7.2 When the check fires

The check fires for code like:

```
1 x = lucid.tensor([1.0, 2.0], requires_grad=True)
2 y = x * 2          # MulBackward saves x
3 x.add_(1.0)        # in-place: x.version_counter goes 0 -> 1
4 y.sum().backward() # raises: version mismatch on saved x
```

The user fix is either to avoid the in-place op or to clone first: `x = x.clone(); x.add_(1.0);`
....

22.7.3 When the check doesn't fire

The version counter is per-TensorImpl, not per-byte. An in-place op on a view of the saved tensor modifies the underlying storage but not the saved tensor's version counter; the check misses. The user should avoid in-place ops on views of saved tensors.

This is one of the few correctness gaps in the autograd engine. The mitigation is convention (avoid in-place on saved tensors) plus the anomaly detector (which catches NaN gradients that often indicate corruption).

22.8 Hooks

The user can attach *hooks* to tensors and modules to intercept the autograd flow without modifying the op implementation.

22.8.1 Tensor hooks

`tensor.register_hook(fn)` attaches a function to a tensor's `grad_fn`. The function is called with the cotangent before it propagates upstream, and may return a modified cotangent.

Use cases:

- Gradient clipping at a specific tensor (rare; usually done at the parameter level).
- Logging or visualisation (sniff the cotangent without changing it).
- Gradient masking (zero out cotangents for certain components).

22.8.2 Module hooks

`module.register_forward_hook(fn)` and `register_backward_hook(fn)` attach to a Module instance and fire on every forward and backward through the module. The forward hook sees the module inputs and outputs; the backward hook sees the gradient inputs and outputs.

The backward hook is the typical mechanism for debugging gradient flow: print the magnitude of the gradient at each layer to find vanishing or exploding gradients.

The chapter for module hooks proper is Chapter 33.

22.9 The Anomaly Detector

`lucid.autograd.set_detect_anomaly(True)` enables a diagnostic mode that helps locate the source of NaN or Inf in gradients.

22.9.1 What it does

When enabled:

- The forward pass records the Python call stack of every op for later use in error messages.
- The backward pass checks each output cotangent for NaN or Inf. The first occurrence raises an error with the saved forward call stack, identifying the originating op.

Without the anomaly detector, a NaN in a gradient propagates silently and the user sees only the eventual symptom (NaN parameters after the optimiser step, training divergence). The detector trades performance (5–10x slowdown of the backward pass) for diagnostic clarity.

22.9.2 When to use

Recommended whenever training mysteriously produces NaN gradients. Not recommended for normal training; the slowdown is substantial.

22.10 Summary and Forward References

Lucid’s autograd engine implements reverse-mode AD through a `Node`-based backward graph constructed at forward time and traversed by `Engine::backward()` in reverse topological order. Per-op `Node` subclasses implement the VJP; `AccumulateGrad` handles leaf accumulation; the version counter catches in-place corruption; hooks expose the autograd flow to user code; the anomaly detector helps diagnose NaN gradients.

The next chapter, Chapter 23, treats the user-facing `Function` class that lets the user define custom ops with custom backwards. Chapter 24 treats the memory-saving recomputation pattern. Chapter 25 treats functional transforms.

Note

For the user: `loss.backward()` is the only entry point. Behind it, the engine walks the saved DAG and accumulates gradients into `.grad`. The version counter, the hook system, and the anomaly detector are the three knobs the user can use to inspect and intervene in the process.

CHAPTER 23

Custom Function API

Not every operation a user wants to write fits the composite-of- primitives pattern (Chapter 20). Some operations need a custom backward that differs from what autograd would derive automatically; some need to interact with a non-Lucid library; some need to inject a specific numerical-stability trick. The *Custom Function API* is the escape hatch.

This chapter describes the `lucid.autograd.Function` class: how to define a custom op, how the autograd engine integrates it, when to use it versus a composite, and the design considerations that come up in practice.

The API is parity-compatible with reference framework’s `torch.autograd.Function`; users familiar with that interface will see only minor naming differences.

23.1 Overview

A `Function` subclass defines two static methods, `forward` and `backward`, and instructs the autograd engine to use them when the user calls `MyFunction.apply(args)`.

```
1  import lucid
2
3  class MyOp(lucid.autograd.Function):
4      @staticmethod
5      def forward(ctx, x, scale):
6          ctx.save_for_backward(x)
7          ctx.scale = scale
8          return scale * x.exp()
9
10     @staticmethod
11     def backward(ctx, grad_out):
```

```

12         (x,) = ctx.saved_tensors
13         scale = ctx.scale
14         grad_x = grad_out * scale * x.exp()
15         return grad_x, None # None for non-tensor 'scale'
16
17 # Use it
18 y = MyOp.apply(x, 3.0)
19 y.sum().backward()

```

The `ctx` (context) object passes state between forward and backward: saved tensors via `ctx.save_for_backward(...)`, non-tensor state via attribute assignment.

23.2 The Function Class

`lucid.autograd.Function` is the abstract base from which custom ops inherit. The implementation lives in `lucid/autograd/function.py`; it bridges to `lucid/_C/autograd/CustomFunction.cpp` for the engine integration.

23.2.1 Required methods

A subclass must define:

- `forward(ctx, *inputs)`: receives the inputs (tensors or non-tensor arguments), returns one or more output tensors.
- `backward(ctx, *grad_outputs)`: receives one cotangent per output, returns one cotangent per input (`None` for non-tensor inputs that don't need a gradient).

Both are `@staticmethod` decorated. The class itself is never instantiated; `MyOp.apply(args)` is the call.

23.2.2 The Context object

`ctx` is a per-call object that lives from the forward through the backward. The user uses it to:

- Save tensors needed by backward: `ctx.save_for_backward(t1, t2, ...)`. Later retrieved in backward as `ctx.saved_tensors`.
- Stash non-tensor state: `ctx.my_attr = value` in forward, `ctx.my_attr` in backward.
- Inspect autograd state: `ctx.needs_input_grad` is a tuple of booleans indicating which inputs require grad.
- Mark inputs as not requiring grad even though they nominally do: `ctx.mark_non_differentiable(t)`.

23.2.3 The apply method

`MyOp.apply(*args)` is the user-facing entry point. Under the hood:

1. Construct a fresh `ctx` object.
2. Call `forward(ctx, *args)` to get the outputs.
3. If any input requires grad, construct a `CustomFunctionNode` carrying `ctx` and the user's backward method. Wire it as the outputs' `grad_fn`.
4. Return the outputs.

The `CustomFunctionNode` is a regular autograd `Node` (Chapter 22); the engine treats it like any other node during backward traversal.

23.3 Worked Example: A Custom Activation

Suppose we want to implement a non-standard activation, $\text{swish_beta}(x) = x * \text{sigmoid}(\beta * x)$ with a learnable β . As a composite, the gradient through β would be constructed by autograd automatically, but for didactic purposes we write it as a Function with an explicit backward.

```

1  import lucid
2  from lucid.autograd import Function
3
4  class SwishBeta(Function):
5      @staticmethod
6      def forward(ctx, x, beta):
7          sig = (beta * x).sigmoid()
8          out = x * sig
9          ctx.save_for_backward(x, beta, sig, out)
10         return out
11
12     @staticmethod
13     def backward(ctx, grad_out):
14         x, beta, sig, out = ctx.saved_tensors
15
16         # d/dx [x * sigmoid(beta*x)]
17         #   = sigmoid(beta*x) + x * beta * sigmoid(beta*x) * (1 -
18           sigmoid(beta*x))
19         d_sig = sig * (1 - sig)
20         grad_x = grad_out * (sig + x * beta * d_sig)
21
22         # d/dbeta [x * sigmoid(beta*x)]
23         #   = x * x * sigmoid(beta*x) * (1 - sigmoid(beta*x))
24         grad_beta_per = grad_out * x * x * d_sig
25         grad_beta = grad_beta_per.sum() # reduce to scalar
26
27         return grad_x, grad_beta

```

The forward saves four tensors (input, beta, the sigmoid intermediate, and the output). The backward derives the gradients in closed form using the saved quantities.

23.3.1 Why save the sigmoid intermediate

The sigmoid intermediate could be recomputed in backward from x and β , but saving it costs $O(N)$ memory and saves one transcendental kernel evaluation. The trade-off favours saving for medium-to-large x . For very large x where memory matters more than compute, the user can choose not to save and recompute instead — `gradient_checkpointing` (Chapter 24) generalises this trade-off.

23.3.2 When this would be a primitive

If `SwishBeta` were used in every layer of a deep network, the per-call framework overhead and the saved-tensor cost would add up. At that point we would promote it to a primitive (Chapter 13), with a C++ kernel for the forward and a registered `BackwardSpec`. The Custom Function path is for operations not common enough to justify the primitive build cost.

23.4 Worked Example: Wrapping an External Library

Custom Function is the right tool when the operation is provided by an external library (a third-party C kernel, a MPSGraph operation Lucid does not yet wrap, etc.).

```

1  class ExternalGEMM(Function):
2      @staticmethod
3      def forward(ctx, A, B):
4          # Call an external C library
5          out = _external_gemm(A, B)
6          # Wrap result in a Lucid tensor sharing the bytes
7          out_tensor = lucid.from_dlpack(out)
8          ctx.save_for_backward(A, B)
9          return out_tensor
10
11     @staticmethod
12     def backward(ctx, grad_out):
13         A, B = ctx.saved_tensors
14         # We know the gradient even though the forward was external
15         grad_A = grad_out @ B.transpose(-1, -2)
16         grad_B = A.transpose(-1, -2) @ grad_out
17         return grad_A, grad_B

```

The pattern: the forward routes through the external call, but the backward stays in Lucid primitives. The framework only sees a single `ExternalGEMM` node in the backward graph; the external call's internals are opaque.

23.5 The `needs_input_grad` Optimisation

`ctx.needs_input_grad` is a tuple of booleans matching the input order. The user can inspect it in backward to skip computing gradients that the autograd engine will discard anyway.

```

1  class MyOp(Function):
2      @staticmethod
3      def forward(ctx, x, y, z):
4          ctx.save_for_backward(x, y, z)
5          return some_op(x, y, z)
6
7      @staticmethod
8      def backward(ctx, grad_out):
9          x, y, z = ctx.saved_tensors
10         grad_x = grad_y = grad_z = None
11         if ctx.needs_input_grad[0]:
12             grad_x = ... # expensive computation
13         if ctx.needs_input_grad[1]:
14             grad_y = ...
15         if ctx.needs_input_grad[2]:
16             grad_z = ...
17         return grad_x, grad_y, grad_z

```

The optimisation matters when some gradients are expensive and not always needed. For typical training where every parameter requires grad, every `needs_input_grad[i]` is `True` and the optimisation is moot.

23.6 Validating Custom Functions with `gradcheck`

A custom backward is a place where bugs hide. Lucid's `gradcheck` (Chapter 26) compares the analytical backward against a finite-difference approximation and raises if they disagree.

```

1  from lucid.autograd import gradcheck
2
3  x = lucid.randn(5, requires_grad=True, dtype=lucid.float64)

```

```

4 beta = lucid.tensor(0.7, requires_grad=True, dtype=lucid.float64)
5
6 assert gradcheck(SwishBeta.apply, (x, beta), eps=1e-6, atol=1e-4)

```

23.6.1 Why FP64 for gradcheck

The numerical-derivative error is $O(\sqrt{\epsilon})$ where ϵ is machine precision. In FP32 that's $\sim 10^{-4}$, which is the threshold at which most gradchecks barely pass; in FP64 it's $\sim 10^{-8}$, leaving room for tight tolerances.

Lucid does not require FP64 for runtime use, but gradcheck should be in FP64 to give a meaningful signal. The tolerance for FP32 gradcheck has to be loose (`atol=1e-3`) and is less useful as a debugging tool.

23.7 Composing Functions

Custom Functions compose: one Function's forward can call `apply` on another. The autograd engine builds the chain naturally.

The chain has implications for higher-order differentiation (Chapter 25): a Function's backward that itself includes a Function call participates in the second-order gradient.

23.8 When Not to Use Custom Function

The decision tree:

1. Can the op be expressed as a composition of existing primitives? Use a composite (Chapter 20). The autograd-derived backward is correct by construction.
2. Is the op called millions of times per training step on a hot path? Build a primitive (Chapter 13).
3. Otherwise: Custom Function.

The middle ground (custom op needed, not on hot path, not expressible as composite) is the Custom Function use case. In practice it covers wrappers around external libraries, research prototype ops, and any case where the auto-derived backward is numerically problematic and needs an analytic replacement.

23.9 Implementation Notes

The Python Function class is in `lucid/autograd/function.py`. It wraps the C++ `CustomFunctionNode` in `lucid/_C/autograd/CustomFunction.cpp`.

23.9.1 The Python-to-C++ boundary

The custom `backward` is a Python function. The C++ engine, when it walks the backward graph and reaches a `CustomFunctionNode`, calls back into Python to invoke the function. This requires holding the GIL during the call, which is the one place in the engine that the GIL is acquired during backward.

For pure-Lucid-primitive backwards inside the custom function, the GIL is then released for the duration of each primitive call. The net overhead is a few microseconds per Custom Function call beyond a plain primitive node.

23.9.2 Memory model

The `ctx` object is held by the `CustomFunctionNode` for the lifetime of the backward graph. Anything assigned to `ctx` (saved tensors, Python attributes) lives until backward completes. The user should avoid storing large Python objects on `ctx` if memory is tight.

23.9.3 Thread safety

Custom Function’s `forward` and `backward` are called on the calling thread. Multiple concurrent backward passes (from different model invocations) instantiate different `ctx` objects and operate independently. The user’s custom code is responsible for any internal thread safety (which is rare in practice).

23.10 Worked Example: A Straight-Through Estimator

A common pattern in models that need a discrete decision (binary gating, quantisation, hard attention) is the *straight-through estimator* (STE): the forward applies a non-differentiable rounding or thresholding, but the backward passes the gradient through unchanged, as if the rounding were the identity. This is a mathematically illegitimate but empirically effective approximation that lets gradient-based training work with discrete decisions.

The STE is naturally a Custom Function:

```

1  class BinaryQuantize(Function):
2      @staticmethod
3      def forward(ctx, x):
4          # Forward: round to 0 or 1 based on sign
5          return (x > 0).astype(x.dtype)
6
7      @staticmethod
8      def backward(ctx, grad_out):
9          # Straight-through: gradient passes unchanged
10         return grad_out
11
12  binarize = BinaryQuantize.apply

```

This is two-line backward, but it could not be expressed as a composite because no composition of differentiable primitives produces the identity gradient through a step function. The Custom Function escape hatch is the only clean way.

23.10.1 Variants

The STE family has several refinements:

- **Clipped STE:** pass gradient through only when the input is in a safe range, zero otherwise.
- **Soft STE:** blend the analytical sub-gradient with the straight-through gradient.
- **Annealed STE:** start with a smooth surrogate (sigmoid) and anneal toward hard binarisation.

Each is a 5–10 line Custom Function in Lucid.

23.11 Worked Example: A Reverse-Mode Differentiable Solver

Another use case: wrapping a numerical solver that does not admit straightforward AD. Consider a function that returns the root of a nonlinear equation via Newton's method:

```

1 class ImplicitRoot(Function):
2     @staticmethod
3     def forward(ctx, params, x0):
4         # Solve  $g(x; \text{params}) = 0$  via Newton iteration
5         x = x0
6         for _ in range(50):
7             with lucid.no_grad():
8                 gx = _g(x, params)
9                 gx_x = _g_jac_x(x, params)
10                x = x - gx / gx_x
11            ctx.save_for_backward(x, params)
12        return x
13
14    @staticmethod
15    def backward(ctx, grad_out):
16        # Implicit function theorem:
17        #  $dx/d\text{params} = -(d\,g/d\,x)^{-1} (d\,g/d\,\text{params})$ 
18        x, params = ctx.saved_tensors
19        gx_x = _g_jac_x(x, params)
20        gx_p = _g_jac_p(x, params)
21        grad_params = -(grad_out / gx_x) * gx_p
22        return grad_params, None # x0 has no gradient

```

The forward runs an iterative solver that is not amenable to unrolling through autograd (the iteration count is dynamic and the intermediate states are not useful for gradients). The backward applies the implicit function theorem: the gradient with respect to the parameters is computed in closed form from the Jacobians of the residual function q at the solution.

This pattern — iterative forward, analytical backward via the implicit function theorem — is the standard mechanism for differentiable optimisation layers (DEQ, OptNet, and similar modern architectures).

23.12 Patterns and Anti-patterns

A catalogue of common mistakes when writing a Custom Function.

23.12.1 Pattern: clone before in-place

If the forward modifies an input in place, the saved-for-backward version is the modified one, and the backward gets wrong values. The fix is to clone first:

```

1 @staticmethod
2 def forward(ctx, x):
3     x_clone = x.clone()
4     x_clone.add_(1.0)           # safe: modifies clone
5     ctx.save_for_backward(x_clone) # backward sees the modified
    value
6     return x_clone

```

23.12.2 Anti-pattern: saving Python objects on ctx

Avoid storing large Python objects on ctx: they live for the entire backward graph's lifetime and contribute to memory pressure. Save only what backward needs.

23.12.3 Anti-pattern: forgetting to handle non-tensor inputs

If a Function takes non-tensor arguments (a scalar, a string, a configuration dict), the backward must return None for each. Forgetting produces a shape-mismatch error in the engine:

```

1  class BadFunc(Function):
2      @staticmethod
3      def forward(ctx, x, scale):  # scale is a Python float
4          ctx.scale = scale
5          return scale * x
6
7      @staticmethod
8      def backward(ctx, grad_out):
9          return ctx.scale * grad_out  # WRONG: returns only one value
10         # Should be:
11         # return ctx.scale * grad_out, None

```

The framework expects one return value per input; None for non-tensor inputs.

23.12.4 Anti-pattern: writing to non-saved state

A Function whose forward modifies global state (a counter, a cache, a log) is not reproducible under autograd: a re-run during checkpointing (Chapter 24) or gradgradcheck (Chapter 26) will fire the side effect twice. Functions should be pure.

23.13 Summary and Forward References

The Custom Function API is Lucid's extension mechanism for operations that cannot be expressed as compositions of existing primitives and are not common enough to justify a C++ primitive. The user defines a Function subclass with `forward(ctx, *inputs)` and `backward(ctx, *grad_outputs)` static methods; the framework integrates the custom op into the autograd graph through a CustomFunctionNode. The ctx object carries state between forward and backward.

`gradcheck` (Chapter 26) is the recommended validation step for any custom backward.

The next chapter, Chapter 24, treats the memory-saving recomputation pattern that interacts with the saved-tensor mechanism. Chapter 25 treats higher-order differentiation, in which custom backwards participate naturally.

Note

For the user: Function is the right tool for one-off custom ops. If you are writing one and the backward is more than a few lines, consider whether the composite path (Chapter 20) would suffice; if the backward is extremely performance-critical, consider whether a primitive (Chapter 13) is warranted.

CHAPTER 24

Gradient Checkpointing

Reverse-mode AD’s memory cost grows with the depth of the computation: every intermediate value needed by a downstream backward is held alive from the forward that produced it. Chapter 21’s memory cost discussion noted that this grows as $O(L)$ for an L -layer network. For modern Transformer models with hundreds of layers, the activation memory can dwarf the parameter memory, even though it is the parameters that are the model’s payload.

Gradient checkpointing is the standard remedy: trade compute for memory by keeping only a subset of activations and recomputing the rest on demand during backward. This chapter describes Lucid’s implementation, the trade-offs involved, and the choice of checkpoint policy.

24.1 The Memory-Time Trade-off

The naive baseline: save every activation. Memory cost $O(L \cdot S)$ where L is depth and S is per-layer activation size; backward time is roughly T_{fwd} (the backward walks the saved activations).

The opposite extreme: save no activations, recompute everything from the input. Memory cost $O(S)$ (just the input); backward time is $O(L \cdot T_{\text{fwd}})$ because the entire forward must be re-run for each layer’s backward.

The classic checkpointing scheme [8] sits in the middle. Save every $\sqrt{L}$ -th activation; on

Scheme	Memory	Backward time
Save all (default)	$O(L \cdot S)$	$\sim T_{\text{fwd}}$
Save none (recompute)	$O(S)$	$O(L \cdot T_{\text{fwd}})$
$\sqrt{L}$ checkpoint	$O(\sqrt{L} \cdot S)$	$\sim 2T_{\text{fwd}}$
Save every k -th	$O(L/k \cdot S)$	$\sim kT_{\text{fwd}}$

Table 24.1: Memory and time costs of different activation-saving strategies. The $\sqrt{L}$ scheme is the sweet spot for many workloads: substantial memory savings at a modest constant-factor increase in backward time.

backward, between two saved activations recompute the intervening forward. The cost analysis:

$$\begin{aligned}
 \text{memory} &= O(\sqrt{L} \cdot S) \quad (\text{saved checkpoints}) \\
 &\quad + O(\sqrt{L} \cdot S) \quad (\text{working segment}) \\
 &= O(\sqrt{L} \cdot S), \\
 \text{time} &= T_{\text{fwd}} \quad (\text{original forward}) \\
 &\quad + T_{\text{fwd}} \quad (\text{recomputed segments, total}) \\
 &\quad + T_{\text{bwd}} \quad (\text{backward}) \\
 &\approx 3T_{\text{fwd}}.
 \end{aligned}$$

Equivalently, checkpointing saves a factor of $\sqrt{L}$ in memory at the cost of one extra forward pass.

Table 24.1 summarises.

24.2 The checkpoint API

Lucid exposes checkpointing through `lucid.utils.checkpoint`. The API mirrors reference framework's:

```

1  import lucid
2  from lucid.utils.checkpoint import checkpoint
3
4  def block(x):
5      h = layer1(x)
6      h = layer2(h)
7      h = layer3(h)
8      return h
9
10 # Instead of: y = block(x)
11 y = checkpoint(block, x)

```

The semantics: `checkpoint(fn, *args)` calls `fn(*args)` during forward, returning the output, but it does not save the intermediate activations of `fn`. During backward, `fn` is re-run from its inputs to regenerate the activations, which are then used to compute the backward.

24.2.1 What the user sees

The user-visible difference between `y = block(x)` and `y = checkpoint(block, x)`:

- Forward output: identical numerical value.
- Forward memory: with checkpoint, only `x` and `y` are saved; intermediate activations inside `block` are released.

- Backward time: with checkpoint, block runs again (one extra forward) before the backward kernels.
- Gradient: identical numerical value.

The only externally observable cost is the extra forward time.

24.2.2 Composition with autograd

`checkpoint` is implemented as a Custom Function (Chapter 23). The Function’s forward runs the user’s `fn` under `no_grad` (so no autograd graph is built), saves only the inputs, and returns the outputs. The Function’s backward enables `grad_mode`, re-runs `fn` to rebuild the graph for the activations, and dispatches the backward.

This composition is the reason checkpoint works transparently with the autograd engine: from the engine’s view, the checkpointed region is a single node with custom backward.

24.3 When to Checkpoint

The trade-off is straightforward but the granularity matters. Checkpointing every layer is overkill (the extra forward time is high); checkpointing nothing forgoes the memory savings.

24.3.1 The natural granularity

For Transformer models, the natural unit is the encoder/decoder block (attention + FFN + norms). Checkpointing every block roughly halves the activation memory of a typical model at a modest backward-time cost.

For ResNet-style convnets, the natural unit is the residual block. The wins are smaller because the per-block activation is already modest.

For ConvNeXt or other modern architectures with grouped layers, the choice is per-architecture and best determined by experimentation.

24.3.2 Selective checkpointing

A refinement: checkpoint only the layers with the largest activations. For Transformer attention, the $O(B \cdot T^2)$ attention score is often the largest activation; checkpointing just the attention layer (leaving the FFN’s activations saved) captures most of the memory savings with less re-computation overhead.

Lucid does not have a built-in selective-checkpoint utility; users write the per-layer choice manually using `checkpoint` on the targeted module’s forward.

24.4 Implementation Detail

The implementation is in `lucid/utils/checkpoint.py`.

24.4.1 The Function class

The core is a `Function` subclass:

```

1  from lucid.autograd import Function
2
3  class _CheckpointFunction(Function):
4      @staticmethod
5      def forward(ctx, run_function, *args):
6          ctx.run_function = run_function
7          ctx.save_for_backward(*[a for a in args

```

```

8         if isinstance(a, lucid.Tensor))]
9     ctx.non_tensor_args = [a for a in args
10        if not isinstance(a, lucid.Tensor)]
11     with lucid.no_grad():
12         outputs = run_function(*args)
13     return outputs
14
15     @staticmethod
16     def backward(ctx, *grad_outputs):
17         tensor_inputs = ctx.saved_tensors
18         # Reconstruct the original input list
19         args = _reassemble(tensor_inputs, ctx.non_tensor_args)
20         # Re-run forward with autograd enabled
21         with lucid.enable_grad():
22             inputs = [t.detach().requires_grad_() for t in
23                tensor_inputs]
24             outputs = ctx.run_function(*_reassemble(inputs,
25                ctx.non_tensor_args))
26         # Trigger backward through the rebuilt graph
27         lucid.autograd.backward(outputs, grad_outputs)
28         # Return gradients for the inputs
29         return (None,) + tuple(t.grad for t in inputs)

```

The pattern: forward under `no_grad` (so no autograd graph), save only the function and the tensor inputs; backward re-runs under `enable_grad` and propagates gradients through the rebuilt graph.

24.4.2 Why detach the inputs

The re-run `run_function` must build a fresh autograd graph from leaf tensors. If we passed the saved tensors directly, they would not be leaves (they have their own `grad_fn` from the outer computation); the inner autograd would walk past them and modify the outer graph.

The fix: `detach().requires_grad_()` produces fresh leaf tensors with the same values. The inner backward routes gradient to these leaves; the outer backward picks up the gradient and propagates further.

24.4.3 Memory release timing

The forward's intermediate activations are released as soon as the `run_function` returns, because no autograd state holds references to them. The release is exactly what makes the checkpoint pattern save memory.

Memory peaks during backward at the moment of re-running `run_function`: at that point we have the original input, the cotangent, and the fully-built inner autograd graph. After the inner backward, everything is freed.

24.5 Non-deterministic Ops Inside checkpoint

A subtle issue: if `run_function` contains a stochastic op (dropout, random sampling), the re-run produces a different output than the original forward. The gradient would be incorrect: backward would propagate cotangents through a different dropout mask than the forward used.

The solution is to save the RNG state on forward and restore it on backward. Lucid's checkpoint does this automatically:

```

1 # Inside forward (sketch):
2 ctx.rng_state = lucid.random.get_state()
3 with lucid.no_grad():

```

```

4     outputs = run_function(*args)
5
6     # Inside backward (sketch):
7     saved_state = lucid.random.get_state()
8     lucid.random.set_state(ctx.rng_state)
9     try:
10         # Re-run forward with same RNG
11         ...
12     finally:
13         lucid.random.set_state(saved_state)

```

The pattern reproduces the original random draws on backward, keeping the gradient correct.

24.5.1 The use_reentrant flag

For backward compatibility with reference framework’s older API, checkpoint accepts a `use_reentrant` flag. `use_reentrant=False` (the recommended modern default) uses the autograd-graph-rebuild approach described above. `use_reentrant=True` uses a more invasive C-level re-entry that has compatibility implications and is being deprecated in favour of the `False` default.

Lucid supports both for parity but recommends `use_reentrant=False`.

24.6 Worked Example: Transformer Block Checkpointing

A typical Transformer training script with checkpointing:

```

1  import lucid
2  import lucid.nn as nn
3  from lucid.utils.checkpoint import checkpoint
4
5  class CheckpointedBlock(nn.Module):
6      def __init__(self, block):
7          super().__init__()
8          self.block = block
9
10     def forward(self, x):
11         return checkpoint(self.block, x, use_reentrant=False)
12
13     # In the model:
14     class GPTModel(nn.Module):
15         def __init__(self, n_layers, **kwargs):
16             super().__init__()
17             self.blocks = nn.ModuleList(
18                 [CheckpointedBlock(GPTBlock(**kwargs))
19                  for _ in range(n_layers)]
20             )
21             ...
22
23         def forward(self, x):
24             for b in self.blocks:
25                 x = b(x)
26             return x

```

Each block’s intermediates are released after its forward; backward re-runs the block to rebuild the local graph and propagate.

24.6.1 Measured savings on GPT-2 base

For Lucid’s GPT-2-base benchmark (12 layers, 768 hidden, 12 heads, sequence length 1024, batch size 4):

- Without checkpointing: 9.2 GB activation memory.
- With per-block checkpointing: 1.7 GB activation memory ($5.4\times$ reduction).
- Backward time without: 245 ms / step.
- Backward time with: 340 ms / step ($1.4\times$ slower).

The $5.4\times$ memory savings let the user double the batch size or train a wider model on the same hardware. The $1.4\times$ backward slowdown is acceptable for training that was otherwise OOM.

24.7 What checkpoint Does Not Help With

Checkpointing trades activation memory for compute. It does not help with:

- **Parameter memory.** The model parameters are alive throughout training and not affected by checkpointing. For very large models the parameter memory itself may exceed available memory; the remedy is parameter sharding (out of scope for Lucid 3.x).
- **Optimiser state memory.** Adam’s first and second moments are $2N$ for N parameters and not affected by checkpointing. The remedy is alternative optimisers (Adafactor factorises the second moment to $O(\sqrt{N})$) or 8-bit optimisers (LION).
- **Gradient memory.** The gradient itself is N for N parameters and not affected by checkpointing.
- **Peak memory during forward.** The forward sometimes peaks at a moment when many activations are live simultaneously (e.g., a residual addition that needs both sides). Checkpointing the block reduces the average activation memory but does not directly address the peak within a single block.

For comprehensive memory reduction, checkpointing is one tool alongside AMP (Chapter 41), gradient accumulation (no Lucid utility needed; just compute multiple sub-batch gradients and sum), and (where available) parameter sharding.

24.8 Checkpoint Granularity Strategies

Beyond the per-block default, several granularity strategies appear in production use. We catalogue them with the trade-offs.

24.8.1 Per-block (default)

Each transformer block or residual block is checkpointed. Memory savings: $\sim 1/L$. Backward overhead: one full forward over the checkpointed regions, $\sim 50\%$ slowdown.

This is the recommended default for most workloads. It is simple to apply (wrap each block), introduces no architectural complexity, and the constant factor is acceptable.

24.8.2 $\sqrt{L}$ -spaced

Checkpoint every $\sqrt{L}$ -th block. Memory savings: $\sim 1/\sqrt{L}$. Backward overhead: $\sim 2T_{\text{fwd}}$.

The Chen et al. optimum. In practice, the difference from per-block is small for typical L in the range 10–50: $\sqrt{12} \approx 3.5$, so checkpointing every 3rd or 4th block. Not substantially better than per-block in our measurements.

Component	Without checkpoint	With per-block checkpoint
Parameters	0.50 GB	0.50 GB
Adam state	1.00 GB	1.00 GB
Gradients	0.50 GB	0.50 GB
Activations	9.20 GB	1.70 GB
Pool overhead	1.10 GB	0.50 GB
Total peak	12.30 GB	4.20 GB

Table 24.2: Memory breakdown for one GPT-2-base training step on M3 Max, with and without per-block gradient checkpointing. The activation reduction (9.2 GB $\rightarrow$ 1.7 GB) dominates; the parameter, optimiser, and gradient contributions are unaffected.

24.8.3 Selective checkpointing

Checkpoint only the layers with the largest activations. For a Transformer, the attention layer’s $O(BT^2)$ score is typically the dominant activation; checkpointing only the attention forward captures most savings.

The implementation requires per-layer manual wrapping rather than the convenient outer-loop pattern; the savings are worth it for memory-constrained workloads.

24.8.4 Activation offloading

A relative of checkpointing: instead of recomputing, swap activations to CPU memory and swap back on backward. Lucid does not currently expose an offloading utility (the unified-memory model makes the win modest on Apple silicon since CPU memory is the same physical pool), but the technique is well-known in CUDA frameworks.

24.9 Worked Example: Memory Profile of GPT-2-base

To make the memory savings concrete, we trace the GPT-2-base forward + backward step with and without per-block checkpointing.

24.9.1 Configuration

- Model: GPT-2 base (12 layers, 768 hidden, 12 heads).
- Batch size: 4 sequences.
- Sequence length: 1024 tokens.
- Precision: FP32 (no AMP for this profile, to make activation memory clearly visible).
- Hardware: M3 Max, 64 GB unified memory.

24.9.2 Memory peaks

Table 24.2 shows the component breakdown.

The 8 GB peak reduction lets the user double the batch size on a 16 GB machine without OOM, or pushed to extremes, lets the user train a $2\times$ deeper model on the same hardware.

24.9.3 Time cost

- Step time without checkpointing: 850 ms.
- Step time with per-block: 1080 ms ($1.27\times$ slower).

The slowdown is consistent with the back-of-envelope estimate (one extra forward across the checkpointed regions adds roughly $T_{\text{fwd}}/T_{\text{step}}$, which is around 30 %).

24.10 Common Pitfalls

A short list of issues that have come up in user code.

24.10.1 Stochastic ops without state preservation

If the user calls a custom random op inside the checkpointed function without using Lucid’s RNG (e.g., a third-party noise library), the re-run on backward produces different noise and the gradient is wrong. The fix is to use Lucid’s RNG for any random op inside checkpoint, so the state-preservation machinery applies.

24.10.2 Non-tensor inputs and outputs

`checkpoint` expects tensor inputs and tensor outputs. Functions returning non-tensor values (Python tuples of mixed types, dicts) need a wrapper that flattens the output. The `lucid.utils.checkpoint_sequential` utility handles common patterns.

24.10.3 Memory not actually freed

If the user’s code holds a reference to an intermediate (through a Python variable or a closure), the intermediate remains alive despite the `no_grad` context. The fix is to let intermediates go out of scope inside the checkpointed function.

24.11 Summary and Forward References

Gradient checkpointing trades a constant-factor increase in backward time for an $O(\sqrt{L})$ reduction in activation memory. Lucid exposes the trade-off through `lucid.utils.checkpoint`, implemented as a Custom Function that runs the wrapped region under `no_grad` during forward and re-runs it under `enable_grad` during backward. The RNG state is saved and restored automatically to keep stochastic ops correct.

The next chapter, Chapter 25, treats higher-order differentiation through functional transforms. Chapter 26 closes Part IV with the finite-difference verification utility.

Note

For the user: wrap large blocks of the model in `checkpoint(fn, *args)` when activation memory is a constraint. Per-block is the typical granularity; finer granularity rarely pays off. The backward time cost is on the order of 30–50 % per checkpointed block.

CHAPTER 25

Higher-Order Differentiation

The basic autograd machinery computes first-order gradients: the sensitivity of a scalar output to its inputs. Many algorithms — second-order optimisers, sensitivity analyses, certain regularisers, some interpretability methods — need higher-order information: gradients of gradients, Jacobians, Hessians, Hessian-vector products. Lucid exposes a small set of *functional transforms* that compose on top of the autograd engine to deliver these.

The transforms (`grad`, `vjp`, `jvp`, `vmap`, `jacrev`, `jacfwd`, `hessian`) are imported from JAX-style functional differentiation. They are exposed under `lucid.func` and live in `lucid/func/`. This chapter describes each, how they compose, and the implementation in terms of the underlying autograd engine.

25.1 Overview

The functional transforms take a Python function and return a new Python function:

- `grad(f)`: returns a function that computes the gradient of `f`.
- `vjp(f, *primals)`: returns the value and a vjp-function that applies $J^\top$.
- `jvp(f, primals, tangents)`: returns the value and the jvp (J applied to tangents).
- `vmap(f)`: returns a function that maps over the leading axis.
- `jacrev(f)`: returns a function that computes the Jacobian via reverse mode.
- `jacfwd(f)`: returns a function that computes the Jacobian via forward mode.
- `hessian(f)`: returns a function that computes the Hessian (typically `jacrev(jacrev(f))` or `jacfwd(jacrev(f))`).

All compose. `grad(vmap(f))` computes per-sample gradients in parallel. `hessian = jacfwd(jacrev(f))` computes the Hessian via mixed mode.

25.2 grad

`grad(f)` returns a function that, given the same arguments as `f`, returns the gradient of `f`'s scalar output with respect to the first argument.

```

1  import lucid
2  from lucid.func import grad
3
4  def loss(w, x, y):
5      pred = w @ x
6      return ((pred - y) ** 2).sum()
7
8  grad_loss = grad(loss)
9  w = lucid.randn(3, 5)
10 x = lucid.randn(5)
11 y = lucid.randn(3)
12 g = grad_loss(w, x, y)    # gradient of loss with respect to w

```

The output `g` is a tensor of the same shape as `w`, holding $\nabla_w \text{loss}(w, x, y)$.

25.2.1 grad over a different argument

By default `grad` differentiates with respect to the first argument. The `argnums` parameter selects which argument:

```

1  grad_x = grad(loss, argnums=1)    # gradient w.r.t. x (the input)

```

A tuple of `argnums` returns a tuple of gradients:

```

1  grad_wx = grad(loss, argnums=(0, 1))
2  gw, gx = grad_wx(w, x, y)

```

25.2.2 Implementation

`grad(f)` returns a wrapper that:

1. Detaches the inputs and marks the differentiated argument as `requires_grad=True`.
2. Calls `f` with the prepared inputs, getting a scalar output.
3. Calls `output.backward(create_graph=False)`.
4. Returns the differentiated argument's `.grad`.

The wrapping is around 30 lines of Python; the heavy lifting is the same autograd engine that `tensor.backward()` uses.

25.3 vjp and jvp

These are the low-level building blocks for the other transforms.

25.3.1 vjp

`vjp(f, *primals)` returns a tuple `(output, vjp_fn)`. `output` is `f(*primals)`. `vjp_fn` is a function that, given a cotangent of the output, returns the cotangent of the inputs:

```
1 from lucid.func import vjp
2
3 def f(x):
4     return x.sin() + x.cos()
5
6 y, vjp_fn = vjp(f, x)
7 g, = vjp_fn(lucid.ones_like(y))    # gradient of sum-of-output
   w.r.t. x
```

The pattern is useful when the same forward computation will be differentiated multiple times with different upstream cotangents (e.g., for computing different rows of a Jacobian).

25.3.2 jvp

`jvp(f, primals, tangents)` returns `(output, jvp_value)`. `jvp_value` is $J \cdot \text{tangents}$, the directional derivative of `f` at `primals` in the direction of `tangents`:

```
1 from lucid.func import jvp
2
3 y, dy = jvp(f, (x,), (lucid.ones_like(x),))
```

`jvp` is forward-mode AD; it does not require building a backward graph.

25.4 vmap

`vmap(f)` returns a function that applies `f` to a batch of inputs in parallel, treating the leading axis of each input as the batch dimension.

```
1 from lucid.func import vmap
2
3 def per_sample_loss(w, x, y):
4     return ((w @ x - y) ** 2).sum()
5
6 # Apply per-sample loss across a batch:
7 batched_loss = vmap(per_sample_loss, in_axes=(None, 0, 0))
8 total = batched_loss(w, x_batch, y_batch).sum()
```

The `in_axes` argument names which axis of each input to vectorise over. `None` means broadcast (no batching for that argument). `0` means batch over the leading axis.

25.4.1 Implementation

`vmap` is implemented on top of MLX's own `vmap`, which performs the broadcasting transformation at the primitive level (Chapter 3). Lucid's wrapping handles the `Tensor` $\leftrightarrow$ `mlx::core::array` unwrap and the autograd metadata threading.

The composition `grad(vmap(f))` is the standard pattern for per-sample gradients — `vmap` over the batch, `grad` over the parameter. Equivalent to a manual Python loop but parallelised inside the kernel.

25.5 jacrev and jacfwd

These compute Jacobians via reverse and forward mode respectively.

25.5.1 jacrev

`jacrev(f)` returns a function that computes the full Jacobian of `f` at its argument via reverse mode:

```
1 from lucid.func import jacrev
2
3 def f(x):
4     return lucid.stack([x[0]**2, x[1]**3, (x[0] + x[1]).sin()])
5
6 J = jacrev(f)(x)      # shape (3, 2): 3 outputs, 2 inputs
```

Internally `jacrev` runs `m` VJP calls (where `m` is the output dimension), each with a one-hot cotangent. The results are stacked into the Jacobian.

25.5.2 jacfwd

`jacfwd(f)` computes the same Jacobian via forward mode, running `n` JVP calls (where `n` is the input dimension):

```
1 from lucid.func import jacfwd
2
3 J = jacfwd(f)(x)      # same shape, computed via forward mode
```

The choice between `jacrev` and `jacfwd` is the output-vs-input-dimension trade-off: reverse mode wins when output is smaller, forward when input is smaller. For square Jacobians the choice depends on the kernel implementation’s constant factor; `jacrev` is usually slightly faster in Lucid because its building block (VJP) is what the framework optimises for.

25.6 hessian

The Hessian of a scalar-output function is the matrix of second-order partial derivatives. `hessian(f)` returns a function that computes it.

25.6.1 Composition

The standard implementation:

```
1 from lucid.func import jacrev, jacfwd
2
3 def hessian(f):
4     return jacfwd(jacrev(f))
```

The inner `jacrev` gives the gradient (a vector); the outer `jacfwd` differentiates each component, producing the Hessian.

The choice of `jacfwd o jacrev` (mixed mode) over `jacrev o jacrev` (pure reverse) is a memory-vs-time trade-off: pure reverse builds two backward graphs simultaneously (memory-heavy); mixed mode builds only one (the inner reverse) plus a forward-mode pass over its output (memory-lighter).

25.6.2 Hessian-vector products

For most second-order applications, the full Hessian is too expensive to compute and store ($O(N^2)$ for N parameters). The quantity actually needed is the Hessian-vector product Hv , which can be computed as the gradient of $v^\top \nabla L$ at cost comparable to one extra backward pass.

Lucid does not expose an `hvp` primitive (it is short to write by hand), but the pattern:

```

1 def hvp(f, x, v):
2     #  $H @ v$  via the trick  $H @ v = \text{grad}(v \cdot \text{grad}(f))(x)$ 
3     g = grad(f)(x)
4     return grad(lambda x_: (grad(f)(x_) * v).sum())(x)

```

This is the standard mechanism for Newton-style optimisers (which need $H^{-1}g$, approximated via conjugate gradient on Hv products).

25.7 Composition Rules

The transforms compose freely. The composition is the source of their power.

25.7.1 grad o grad

The second derivative of a scalar function with respect to a scalar input:

```

1 def f(x):
2     return x.sin() * x.cos()
3
4 f_prime = grad(f)
5 f_double_prime = grad(f_prime)

```

For multi-dimensional inputs, `grad o grad` computes the column of the Hessian corresponding to the second differentiated argument.

25.7.2 grad o vmap

Per-sample gradients:

```

1 def loss(w, x, y):
2     return ((w @ x - y) ** 2).sum()
3
4 per_sample_grads = vmap(grad(loss), in_axes=(None, 0, 0))
5 gs = per_sample_grads(w, x_batch, y_batch)
6 # gs.shape == (batch, *w.shape)

```

The result is a per-sample gradient, useful for adversarial training, privacy analyses, and certain meta-learning algorithms.

25.7.3 vmap o grad

Order matters. `vmap(grad(f))` computes the gradient for one batch element under `vmap` (i.e., per-sample gradients); `grad(vmap(f))` computes the gradient of the batched function (the gradient of the sum across the batch, equivalent to the per-sample gradients summed).

For loss aggregation, `grad(vmap(f))` is what you usually want.

25.8 Implementation Notes

The functional transforms live in `lucid/func/`. The implementations are thin wrappers around the autograd engine and, where applicable, MLX's own transforms.

25.8.1 Autograd-graph rebuilding

Higher-order differentiation requires the backward graph itself to be differentiable. The mechanism is `backward(create_graph=True)`, which causes the engine's `apply` calls to themselves run under `enable_grad`, building a second-order autograd graph that can be backwarded again.

The cost of `create_graph=True` is roughly $2\times$ the cost of a normal backward, because every backward op is itself recorded for further differentiation.

25.8.2 Per-transform overhead

Each transform layer adds Python wrapping overhead in the microseconds-per-call range. For deeply nested compositions (`hessian` is two layers), the wrapping adds up to tens of microseconds, which is negligible relative to the kernel cost for any non-trivial input.

25.8.3 Limitations

A few:

- The transforms work on tensor inputs and outputs. They do not handle Python control flow (`if/else`, `for` loops) that branches based on tensor values; the branch taken in the forward must be the same in the backward.
- `vmap` does not work with operations that have data-dependent output shape (`nonzero`, `unique`).
- Higher-order differentiation through custom Functions is supported but requires the Function's backward to itself be differentiable (i.e., to use Lucid primitives, not arbitrary Python code).

The chapters that follow Chapter 26 document the behaviour at the limits.

25.9 Worked Example: Per-Sample Gradients for MAML

Model-Agnostic Meta-Learning (MAML, [12]) requires per-sample gradients through an inner-loop optimisation. The functional transforms make this expressible naturally.

```

1  import lucid
2  from lucid.func import grad, vmap
3
4  def task_loss(params, x, y):
5      pred = model_fn(params, x)
6      return ((pred - y) ** 2).sum()
7
8  def inner_step(params, x, y, lr):
9      g = grad(task_loss)(params, x, y)
10     return _tree_sub(params, _tree_mul(g, lr))
11
12 def meta_loss(params, x_train, y_train, x_test, y_test, lr):
13     # Inner adaptation
14     adapted = inner_step(params, x_train, y_train, lr)
15     # Outer evaluation
16     return task_loss(adapted, x_test, y_test)
17
18 # Per-task meta-gradient via vmap
19 per_task_meta_grad = vmap(grad(meta_loss),
20                             in_axes=(None, 0, 0, 0, 0, None))
21 meta_grads = per_task_meta_grad(
22     params, x_train_batch, y_train_batch,
23     x_test_batch, y_test_batch, lr)
24 # Aggregate
25 meta_grad = _tree_mean(meta_grads, axis=0)
26 # Outer optimiser step
27 params = _tree_sub(params, _tree_mul(meta_grad, meta_lr))

```

The composition `vmap(grad(...))` computes the per-task gradient of the post-adaptation loss with respect to the initial parameters. Each task in the batch sees its own inner adaptation and its own outer gradient; `vmap` parallelises across tasks; `grad` of `meta_loss` threads through the inner-loop update.

The first-time reader of MAML’s algorithm finds it daunting; the functional formulation is short.

25.10 Worked Example: Hessian-Vector Product for CG-Newton

A second-order optimiser like CG-Newton needs $H^{-1}g$, the Newton step. The full Hessian is too large to invert; instead one uses Hessian-vector products Hv inside a conjugate-gradient iteration to approximately solve $Hd = g$ for d .

The HVP can be computed in two ways:

25.10.1 HVP via grad-of-grad

```
1 def hvp_via_grad(f, x, v):
2     # First compute gradient
3     def grad_fn(x_):
4         return grad(f)(x_) * v).sum()
5     # Differentiate again
6     return grad(grad_fn)(x)
```

The trick: $\nabla_x(v^\top \nabla_x f) = Hv$. This requires two backward passes (one for the inner gradient, one for the outer).

25.10.2 HVP via JVP-of-grad

```
1 from lucid.func import jvp
2
3 def hvp_via_jvp(f, x, v):
4     def grad_fn(x_):
5         return grad(f)(x_)
6     _, hvp = jvp(grad_fn, (x,), (v,))
7     return hvp
```

This computes the directional derivative of the gradient in the direction of v . Mathematically identical to the grad-of-grad version, but computationally cheaper because forward mode is cheaper for low-rank operations.

For very large parameter counts, the JVP-of-grad approach is preferred. Lucid users can pick based on profiling.

25.11 Performance Considerations

The transforms have some overhead beyond a plain backward. We quantify here.

25.11.1 Per-call overhead

Table 25.1 reports the framework overhead per transform layer on M3 Max.

For typical ML applications where the kernel cost is many microseconds per primitive, the transform overhead is negligible. For very small kernels (single-element scalar gradients), the overhead can dominate.

Transform	Per-call overhead
<code>grad(f)</code>	$\sim 10 \mu s$
<code>vjp(f)</code>	$\sim 12 \mu s$
<code>jvp(f)</code>	$\sim 10 \mu s$
<code>vmap(f)</code>	$\sim 25 \mu s$
<code>jacrev(f)</code>	$\sim M \times$ (single VJP cost)
<code>jacfwd(f)</code>	$\sim N \times$ (single JVP cost)
<code>hessian(f)</code>	$\sim MN \times$ (single VJP)

Table 25.1: Per-call framework overhead for each transform on M3 Max. The `jacrev`, `jacfwd`, and `hessian` costs scale with the input (N) and output (M) dimensions because they internally run multiple VJP/JVP calls.

25.11.2 Memory of higher-order gradients

`backward(create_graph=True)` doubles the memory cost relative to a plain backward because the backward operations themselves are recorded into a second-order graph. For deep models this is a significant cost; the user should be aware and use `create_graph=True` only when truly necessary (e.g., explicit second-order optimisation).

25.12 Summary and Forward References

Lucid’s functional transforms (`grad`, `vjp`, `jvp`, `vmap`, `jacrev`, `jacfwd`, `hessian`) compose on top of the autograd engine to provide higher-order differentiation. `grad` $\circ$ `vmap` gives per-sample gradients; `jacfwd` $\circ$ `jacrev` gives the Hessian; `vmap` $\circ$ `grad` gives batched gradients of a non-batched function. Each layer adds modest Python overhead; the kernels are the autograd engine’s.

The next chapter, Chapter 26, treats finite-difference verification, the standard tool for validating that a custom backward is correct.

Note

For the user: most ML training uses only first-order `backward()`. The functional transforms come up in research contexts: meta-learning (per-sample gradients), second-order optimisers (Hessian-vector products), sensitivity analyses (Jacobians of mid-network activations). For these use cases, `lucid.func` is the API.

CHAPTER 26

Numerical Gradient Checking

A bug in a custom backward (Chapter 23) or in a freshly added primitive’s `BackwardSpec` (Chapter 13) is one of the easiest classes of bug to introduce and one of the hardest to detect by inspection. The gradient is silently wrong; training appears to proceed; the model converges to the wrong place or fails to converge at all; the cause is far upstream of the symptom.

`gradcheck` is the standard remedy: compare the analytical backward against a finite-difference approximation and assert agreement to within a tolerance. The technique is old (it predates modern AD by decades) and the implementation is short, but the practical use requires understanding several subtleties.

This chapter describes Lucid’s `lucid.autograd.gradcheck`, its sister `gradgradcheck` for second-order verification, and the choices that go into setting tolerances.

26.1 The Idea

For a function $f : \mathbb{R}^N \rightarrow \mathbb{R}$, the analytical gradient is ∇f . The two-sided finite-difference approximation is

$$\widehat{\partial f}_i = \frac{f(x + he_i) - f(x - he_i)}{2h},$$

where e_i is the i -th unit vector and h is a small step size. `gradcheck` computes both and asserts they agree to within absolute and relative tolerances.

For a vector-output function $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$, the analytical Jacobian is computed via M reverse-mode VJP calls (one per output, with a one-hot cotangent), and the finite-difference Jacobian is computed via $2N$ forward calls. `gradcheck` compares the two $M \times N$ matrices entry by entry.

26.1.1 Why the comparison is informative

The two computations use entirely different mechanisms:

- Analytical: walks the autograd graph backward, calling each op’s VJP.
- Numerical: calls the forward function $2N$ times with perturbed inputs.

A bug in either is unlikely to be a bug in both. If the two agree to 10^{-6} relative error, the analytical backward is almost certainly correct.

26.2 The API

lucid.autograd.gradcheck signature:

```

1  def gradcheck(
2      func,                # the function to check
3      inputs,              # tuple of tensors
4      *,
5      eps=1e-6,            # finite-difference step size
6      atol=1e-5,           # absolute tolerance
7      rtol=1e-3,           # relative tolerance
8      raise_exception=True,
9  ) -> bool:
10     ...

```

Typical use:

```

1  import lucid
2  from lucid.autograd import gradcheck
3
4  x = lucid.randn(5, requires_grad=True, dtype=lucid.float64)
5  y = lucid.randn(5, requires_grad=True, dtype=lucid.float64)
6
7  def f(x, y):
8      return (x * y).sum()
9
10 assert gradcheck(f, (x, y))

```

The call returns True if every gradient entry agrees within tolerance and raises (or returns False, with `raise_exception=False`) otherwise.

26.2.1 The FP64 convention

The user passes FP64 tensors, not FP32. The reasoning is the numerical-error trade-off discussed in Chapter 23 Section 23.6.1: finite differences with a step of 10^{-6} in FP32 lose all useful precision; in FP64 they retain ~ 8 digits.

Lucid’s `gradcheck` raises with a helpful message if any input is FP32 (`LucidError: gradcheck requires FP64 inputs for meaningful tolerance`), suggesting the user cast first.

26.3 Tolerance Setting

The tolerance is the single most important parameter. Too tight, and a correct backward fails due to the finite-difference error. Too loose, and a buggy backward passes.

26.3.1 Theoretical tolerance

For a step size h , the central-difference approximation

$$\hat{f}'(x) = \frac{f(x+h) - f(x-h)}{2h}$$

has total error

$$|\hat{f}'(x) - f'(x)| \leq \underbrace{\frac{h^2}{6}|f'''(\xi)|}_{\text{truncation}} + \underbrace{\frac{\epsilon}{h}|f(x)|}_{\text{round-off}}.$$

The truncation term grows with h^2 ; the round-off term grows as $1/h$. Their sum has a unique minimum, attained at

$$h^* = \left(\frac{3\epsilon|f(x)|}{|f'''(\xi)|} \right)^{1/3} \approx \epsilon^{1/3},$$

giving an optimal error of order $\epsilon^{2/3}$. In FP64 ($\epsilon \approx 10^{-16}$), the optimum is $h^* \approx 10^{-5}$ and the floor is $\approx 10^{-10}$. In FP32 ($\epsilon \approx 10^{-7}$), the optimum is $h^* \approx 10^{-2}$ and the floor is $\approx 10^{-4}$ — barely enough precision to distinguish a correct gradient from a buggy one, which is why gradcheck insists on FP64.

Lucid's default $h = 10^{-6}$ is slightly above the optimum; the default tolerance $\text{atol} = 10^{-5}$ leaves room for a generous safety margin.

26.3.2 Per-op tolerance adjustments

Some ops have intrinsically larger finite-difference error:

- Transcendental ops (`exp`, `log`) lose precision through their internal computation; tolerance must be looser, typically `atol=1e-4`.
- Operations with extreme value range (`erfinv` near $|x| = 1$, `logsumexp` with extreme logits) need case-specific tolerances; sometimes the function must be restricted to a safe input range for gradcheck.
- Discontinuous ops (`relu` at 0, `abs` at 0) cannot be gradchecked at the discontinuity; the inputs must be chosen to avoid the discontinuous point.

Lucid's test suite has per-op tolerance overrides where needed, documented in the test files themselves.

26.4 What gradcheck Catches

A bug in the backward almost always produces a measurable disagreement. The common failure modes:

26.4.1 Sign error

A backward with the wrong sign (e.g., `return -grad_out` instead of `return grad_out`) disagrees by a factor of exactly 2 (since the relative error is 200%). `gradcheck` catches this immediately.

26.4.2 Missing factor

A backward that omits a constant factor (e.g., the 2 in $d/dx x^2 = 2x$) disagrees by that factor. `gradcheck` catches this immediately.

26.4.3 Wrong saved tensor

A backward that uses the wrong saved quantity (e.g., the input when it should have been the output) produces a disagreement that depends on the input range. `gradcheck` catches it for non-pathological inputs.

26.4.4 Unbroadcast error

A backward that forgets to sum over the broadcast dimensions (the unbroadcast step of Chapter 16) produces a gradient with the wrong shape, which `gradcheck` reports as a shape mismatch.

26.5 What `gradcheck` Does Not Catch

A short list of cases where `gradcheck` can pass on a buggy backward.

26.5.1 Errors that are below tolerance

A backward that is wrong by less than the tolerance passes. The remedy is a tighter tolerance, which requires FP64 inputs (which `gradcheck` enforces).

26.5.2 Errors at points not tested

`gradcheck` tests at the specific input the user passes. A bug that manifests only at extreme inputs (very large, very small, NaN, Inf) may not surface in a randomised test. The mitigation is to test multiple input regimes (small, moderate, large) and to spot-check extreme cases manually.

26.5.3 Errors in saved-output backward when the output is constant

If a backward uses a saved output and the test inputs happen to produce a constant output (or near-constant), the backward may appear correct even if the output-dependence is wrong. The mitigation is varied test inputs.

26.5.4 Non-deterministic ops

Operations with non-deterministic backward (`scatter_add` with duplicate indices on GPU) produce different cotangents on different runs. `gradcheck` cannot pass deterministically for these; the test must be skipped or run on a deterministic stream.

26.6 `gradgradcheck`

For higher-order verification, `gradgradcheck` checks the gradient of the gradient. It's the natural extension when the backward itself contains another backward (e.g., when the model uses a meta-learning algorithm that requires second-order gradients).

The mechanism is the same: build the second-order graph via `backward(create_graph=True)`, then run `gradcheck` on the resulting gradient function.

26.6.1 When to `gradgradcheck`

- When using `hessian`, `jacrev o jacrev`, or other compositions that require second-order autograd.
- When implementing a custom backward that itself uses Lucid primitives whose second derivatives must be correct.

For ordinary first-order training, `gradgradcheck` is unnecessary.

26.7 The Parity-Test Policy

Lucid’s testing policy (Chapter 63) requires every public op to have:

1. A unit test exercising correctness against a known expected output.
2. A parity test comparing against the reference framework’s output to within a tight tolerance.
3. A `gradcheck` verifying the backward.
4. For ops with non-trivial second derivatives, a `gradgradcheck`.

The discipline catches the easy classes of bug at PR time, before the op ships. It does not catch the hard cases (numerical pathologies at extreme inputs, non-determinism), which require case-by-case attention.

26.8 Implementation

`gradcheck` is implemented in `lucid/autograd/gradcheck.py`. The core loop:

```

1  def gradcheck(func, inputs, eps=1e-6, atol=1e-5, rtol=1e-3,
2      raise_exception=True):
3      # Validate FP64
4      for t in inputs:
5          if t.dtype != lucid.float64:
6              raise LucidError("gradcheck requires float64 inputs")
7
8      # Compute analytical gradients (one per output entry)
9      analytical = _analytical_jacobian(func, inputs)
10
11     # Compute numerical gradients via central difference
12     numerical = _numerical_jacobian(func, inputs, eps)
13
14     # Compare entry by entry
15     for a, n in zip(analytical, numerical):
16         diff = (a - n).abs()
17         tol = atol + rtol * n.abs()
18         if (diff > tol).any():
19             if raise_exception:
20                 raise LucidError(
21                     f"gradcheck failed: max diff {diff.max()} "
22                     f"exceeds tolerance {tol.max()}")
23             return False
24     return True

```

The full implementation handles edge cases: complex-valued functions, sparse outputs, the `check_undefined_grad` option, and a few others. The core idea is the central-difference loop above.

26.8.1 Cost

A `gradcheck` on a function with N inputs and M outputs costs $2N$ forward calls plus M backward calls. For typical unit tests with small N and M (single-digit), the cost is under a second.

For larger inputs (N in the hundreds), the cost is significant but acceptable for a one-time correctness check. `gradcheck` is not run during normal training; it’s a development-time tool.

26.9 Worked Example: Validating a Custom Function

The custom SwishBeta from Chapter 23 Section 23.3:

```

1 import lucid
2 from lucid.autograd import gradcheck
3
4 x = lucid.randn(5, requires_grad=True, dtype=lucid.float64)
5 beta = lucid.tensor(0.7, requires_grad=True, dtype=lucid.float64)
6
7 # Validate first-order gradient
8 assert gradcheck(SwishBeta.apply, (x, beta), eps=1e-6, atol=1e-5)
9 print("SwishBeta first-order gradient verified")

```

If we had introduced the sign error (return `-grad_x`, `-grad_beta` instead of the correct signs), the test would have failed with a relative error of approximately 2.0:

```

LucidError: gradcheck failed: max diff 1.84e+00 exceeds
tolerance 6.21e-05

```

The failure points at the input and the offending Function class, making the bug easy to locate.

26.10 Debugging a Failed gradcheck

`gradcheck` produces an error message with the maximum disagreement and its location. The message tells the user what failed; it doesn't tell why. A short debugging recipe.

26.10.1 Step 1: print both gradients

Replace `gradcheck` with manual computation:

```

1 import lucid
2
3 # Analytical
4 x = lucid.randn(3, requires_grad=True, dtype=lucid.float64)
5 y = my_func(x).sum()
6 y.backward()
7 print("analytical:", x.grad)
8
9 # Numerical (one-sided for brevity)
10 def num_grad(f, x, eps=1e-6):
11     g = lucid.zeros_like(x)
12     for i in range(x.numel()):
13         x_perturbed = x.clone()
14         x_perturbed.flat[i] += eps
15         f_plus = f(x_perturbed)
16         x_perturbed.flat[i] -= 2*eps
17         f_minus = f(x_perturbed)
18         g.flat[i] = (f_plus - f_minus) / (2*eps)
19     return g
20
21 print("numerical:", num_grad(lambda x_: my_func(x_).sum(), x))

```

The visual comparison often makes the bug obvious (a sign error flips, a missing factor shrinks, a wrong saved tensor adds noise to specific entries).

26.10.2 Step 2: simplify the function

If the function is composed of multiple parts, isolate each. Run `gradcheck` on each part separately. The failing part is the source of the bug.

26.10.3 Step 3: check the saved tensors

A common bug class: backward uses the wrong saved tensor. Print the saved tensors at the start of `backward`, compare to the forward inputs / outputs you expected. Mismatches surface here.

26.10.4 Step 4: check shape

A backward that produces the wrong gradient shape (a missing `sum` after broadcasting, an off-by-one slice) is caught early by `gradcheck`'s shape check. The error message names the shapes; line them up against the input shape and the expected gradient shape.

26.11 gradcheck vs Other Verification Tools

`gradcheck` is one tool in a suite of verification mechanisms.

26.11.1 vs parity testing

A parity test (Chapter 63) compares against a reference framework's output. It catches forward bugs and backward bugs together, but it only works for ops where a reference exists. For genuinely new ops or custom Functions, `gradcheck` is the only option.

The two are complementary: parity tests catch "the answer is wrong" bugs; `gradcheck` catches "the answer is right but the gradient is wrong" bugs.

26.11.2 vs anomaly detection

`lucid.autograd.set_detect_anomaly(True)` (Chapter 22) catches NaN gradients at runtime. It doesn't catch wrong (but finite) gradients; `gradcheck` does.

26.11.3 vs unit testing

A unit test that compares against a hand-derived expected value catches the same class of bug as `gradcheck` for specific inputs. `gradcheck` generalises: it works against any input the user chooses, with the analytical machinery doing the comparison automatically.

26.12 The `check_undefined_grad` Option

`gradcheck` has an option `check_undefined_grad=True` (default `True` in Lucid 3.5) that additionally verifies the function handles the case where the upstream cotangent is `None` (undefined). This case arises when the user's downstream consumer discards an output (e.g., `out = (a, b, c); use(a); del b, c`); the engine passes `None` as the cotangent for the discarded outputs.

The Function's backward should handle this by returning `None` for the corresponding inputs. The check verifies that the analytical backward does so consistently.

For most user code the check is irrelevant; for library code shipping in Lucid's core, it's essential.

26.13 Tolerance Recipes

A reference for setting tolerances for common situations.

- **Default FP64:** `atol=1e-5, rtol=1e-3`.
- **Transcendental (exp, log, trig):** `atol=1e-4, rtol=1e-3`.
- **Reductions over large tensors:** `atol=1e-4, rtol=1e-3` (accumulated error grows with size).
- **Complex composite (multiple primitives):** `atol=1e-4, rtol=1e-2`.
- **Erfinv, Bessel, special functions:** `atol=1e-3, rtol=1e-2` (intrinsic precision limit).
- **FP32 (if FP64 not possible):** `atol=1e-2, rtol=1e-1` — barely useful, only catches gross bugs.

The general rule: pick the tightest tolerance that the function’s intrinsic precision permits, and tighten the test as the implementation matures.

26.14 Summary and Forward References

`lucid.autograd.gradcheck` compares the analytical reverse- mode gradient against a finite-difference approximation, asserting agreement to within absolute and relative tolerance. `gradgradcheck` extends to second-order verification. Lucid’s test suite requires `gradcheck` for every public op, which catches the easy classes of backward bug at PR time. The tool is a development-time correctness check, not a training-time mechanism.

This chapter closes Part IV. Part V treats the mathematical subpackages (`linalg`, `FFT`, `signal`, `special functions`, `distributions`, `einops`) that build on the `autograd` machinery covered here.

Note

For the user: `gradcheck` is the right tool when you have written a custom `Function` and want to verify its backward. The standard recipe is FP64 inputs, the default tolerances, and `raise_exception=True` (the default) to surface failures loudly. For first-order, `gradcheck`; for second-order, `gradgradcheck`.

Part **V**

Mathematical Subsystems

CHAPTER 27

Linear Algebra

Linear algebra is the mathematical substrate of most machine learning. Matrix multiplication is the dominant flop; matrix factorisations underlie PCA, low-rank fine-tuning, gradient preconditioning, and a variety of regularisation techniques; linear solvers appear inside differential-equation and control-theoretic models. `Lucid` exposes a comprehensive linear-algebra subpackage at `lucid.linalg` that wraps the LAPACK routines provided by `Accelerate`.

This chapter describes the subpackage’s surface, the implementation choices, and the unique constraint that defines its dispatch behaviour: the *linalg carve-out* introduced in Chapter 9 Section 9.4, which routes every `linalg` call through the CPU stream regardless of the user’s device.

27.1 Overview

The `lucid.linalg` surface mirrors `reference framework`’s `torch.linalg` closely, with the same routine names, signatures, and return conventions. The subpackage contains roughly 31 functions, falling into five families:

- **Decompositions:** `cholesky`, `qr`, `svd`, `eigh`, `lu`, `ldl`.
- **Solvers:** `solve`, `lstsq`, `triangular_solve`, `cholesky_solve`.
- **Matrix functions:** `inv`, `det`, `slogdet`, `matrix_exp`, `matrix_power`, `matrix_rank`.
- **Norms:** `norm`, `matrix_norm`, `vector_norm`, `cond`.
- **Products:** `cross`, `outer`, `vecdot`, `multi_dot`.

27.1.1 The H8 namespace rule

Per Hard Rule H8, every linalg op is accessible only via `lucid.linalg.*`, never at the top level (`lucid.cholesky` is invalid) and never as a Tensor method (`x.norm()` is invalid). The rule disambiguates: `norm` could mean vector norm or matrix norm, and the subpackage namespace makes the intent explicit.

The rule also catches user errors at API surface. A user typing `x.norm()` gets an `AttributeError` instead of an undocumented default; the message points to `lucid.linalg.norm` or `lucid.linalg.vector_norm` per the user’s actual intent.

27.2 The Linalg Carve-Out, Restated

Linear-algebra decompositions are not implemented in Metal. MLX provides them only on its CPU stream, where they delegate to Accelerate’s LAPACK. The dispatcher recognises this and routes every linalg call through the CPU stream regardless of the user’s selected device.

27.2.1 What the user sees

`lucid.linalg.cholesky(x)` on a metal tensor:

1. Dispatcher recognises linalg op, applies carve-out.
2. `SharedStorage` bridge transfers `x` to CPU view (zero-copy in the contiguous case).
3. MLX runs the decomposition on its CPU stream, calling Accelerate’s `spotrf`.
4. Result wrapped back as a metal tensor.

The bridge and the wrap-back are sub-microsecond; the decomposition cost dominates. For matrices larger than 64×64 , the cost is determined entirely by the LAPACK routine, and the choice of device is fictive.

27.2.2 Why this is acceptable

LAPACK on Apple’s AMX coprocessor through Accelerate is competitive with the best open-source implementations and faster than any naive Metal kernel for decompositions of any non-trivial size. The carve-out is the right architectural choice given the hardware reality.

27.3 Decomposition Routines

27.3.1 Cholesky

`cholesky(A)` returns the lower-triangular Cholesky factor L such that

$$A = LL^\top, \quad L_{ii} > 0,$$

requiring A symmetric positive definite. The factor is unique. The routine is the fastest decomposition: it costs $N^3/3 + O(N^2)$ flops, half of LU’s cost.

The algorithm computes L column by column:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k < j} L_{jk}^2}, \quad L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k < j} L_{ik} L_{jk} \right) \quad (i > j).$$

A non-positive value under the square root signals non-positive-definite A ; the routine raises.

Backward: given $\bar{L}$ (the cotangent of L), define

$$P = \text{tril}(L^\top \bar{L}) - \frac{1}{2} \text{diag}(L^\top \bar{L}).$$

Then

$$\bar{A} = \frac{1}{2} L^{-\top} (P + P^{\top}) L^{-1}.$$

Lucid's autograd registers this analytical gradient; the user does not see the derivation.

27.3.2 QR

`qr(A)` returns (Q, R) where $A = QR$, Q is orthogonal, R is upper triangular. Two modes:

- `mode='reduced'` (default): returns $Q \in \mathbb{R}^{m \times k}$, $R \in \mathbb{R}^{k \times n}$ for $A \in \mathbb{R}^{m \times n}$, $k = \min(m, n)$.
- `mode='complete'`: returns full $Q \in \mathbb{R}^{m \times m}$.

Implementation: Householder reflections via `Accelerate`'s `sgeqrf` for the factorisation, `sorgqr` for the explicit Q reconstruction.

27.3.3 SVD

`svd(A)` returns $(U, S, V^{\top})$ where $A = U \Sigma V^{\top}$ and $\Sigma = \text{diag}(S)$. The implementation uses `Accelerate`'s `sgesdd` (divide-and-conquer), which is asymptotically faster than the QR-based `sgesvd` for matrices larger than a few hundred entries on each side.

Modes:

- `full_matrices=True`: $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$, $S \in \mathbb{R}^{\min(m, n)}$.
- `full_matrices=False`: $U \in \mathbb{R}^{m \times k}$, $V \in \mathbb{R}^{n \times k}$, $k = \min(m, n)$.

The truncated form is typically what's wanted.

27.3.4 Eigendecomposition

`eigh(A)` returns (w, V) where $Av_i = w_i v_i$ for A symmetric (Hermitian). Implementation: `ssyeval` (divide-and-conquer).

Lucid does not currently expose `eig` for general non-symmetric matrices, on the grounds that ML rarely needs it and `reference framework` deprecated it.

27.4 Solvers

27.4.1 Solve

`solve(A, B)` solves $AX = B$ for X . Implementation: LU factorisation via `sgesv`.

27.4.2 Least squares

`lstsq(A, B)` solves $\min_X \|AX - B\|_F$ via SVD-based divide-and-conquer (`sgelsd`). Returns the minimum-norm solution when A is rank-deficient.

27.4.3 Triangular solve

`triangular_solve(A, B, upper=True)` solves $AX = B$ when A is triangular. Cheaper than general solve: $O(N^2)$ flops instead of $O(N^3)$.

27.5 Matrix Functions

27.5.1 inv

`inv(A)` returns A^{-1} . Implementation: LU + back substitution (`sgetrf` + `sgetri`).

For solving systems, prefer `solve(A, B)` over `inv(A) @ B`: more numerically stable and faster.

Routine	Backward form
cholesky	triangular solve involving L^{-1}
qr	dense formula with $Q^\top \bar{Q}$, R^{-1}
svd	three terms; depends on $\bar{U}$, $\bar{S}$, $\bar{V}$
eigh	two terms; involves $V^\top \bar{V}$, eigenvalue gaps
solve	two solves: $X^\top$, then propagate to $\bar{A}$
det	$\det(A) \cdot (A^{-1})^\top \cdot \bar{y}$
slogdet	$(A^{-1})^\top \cdot \bar{y}_{\log}$
matrix_exp	series expansion of $e^A \bar{Y}$

Table 27.1: Closed-form backward expressions for the main linalg routines. The expressions are generally well-behaved at well-conditioned inputs and become numerically delicate at near-singular ones; the autograd engine returns NaN in the latter case.

27.5.2 Determinant

`det(A)` returns $\det(A)$. For ill-conditioned matrices, prefer `slogdet(A)` which returns $(\text{sign}, \log |\det|)$ and avoids overflow / underflow.

27.5.3 Matrix exponential

`matrix_exp(A)` returns $e^A = \sum_k A^k / k!$. Implementation: Pade approximation with scaling and squaring.

27.6 Backward Through Linalg

The backward for each linalg routine has a known closed form that Lucid has implemented. The derivations are textbook but not always pleasant; Table 27.1 summarises the form.

27.6.1 Failure modes

The backward through SVD or `eigh` becomes singular when the input has repeated singular values / eigenvalues. The formula has $1/(\sigma_i - \sigma_j)$ terms; equal singular values produce division by zero.

Lucid’s implementation returns NaN in these cases, matching reference framework’s behaviour. The mitigation is to add a small random perturbation to the input.

27.7 Performance Notes

Measured throughput on M3 Max for representative sizes:

27.8 QR and Householder Reflections

The QR factorisation is the workhorse of orthogonal-projection methods (least squares, Gram–Schmidt orthogonalisation, the QR algorithm for eigenvalues) [13, 27]. Lucid implements it via Accelerate’s Householder routine `sgeqrf` [18], which is the standard numerically-stable construction.

27.8.1 Householder reflections

A Householder reflection $H = I - 2vv^\top / (v^\top v)$ for a unit vector v is a symmetric orthogonal matrix that reflects any input through the hyperplane orthogonal to v . The key property: given

Op	Size	Time
cholesky	1024×1024	14 ms
cholesky	4096×4096	410 ms
qr	1024×1024	28 ms
qr	4096×4096	920 ms
svd	1024×1024	110 ms
svd	4096×4096	4.8 s
eigh	1024×1024	96 ms
eigh	4096×4096	4.2 s
inv	1024×1024	22 ms

Table 27.2: Measured wall-clock times for square-matrix decompositions on M3 Max via Accelerate’s LAPACK. Cholesky is the cheapest (positive-definite assumption); SVD and eigh are the most expensive (dense decomposition with iterative refinement).

any vector x , we can choose v so that Hx is a multiple of e_1 (the first standard basis vector):

$$v = x - \text{sign}(x_1) \|x\| e_1.$$

QR proceeds by applying Householder reflections to the columns of A in sequence, each reflection zeroing out the entries below the diagonal of one column:

$$H_n \cdots H_2 H_1 A = R \quad (\text{upper triangular}),$$

and $Q = H_1 H_2 \cdots H_n$ is the product of reflections. The algorithm costs $\sim 2mn^2 - 2n^3/3$ flops for $A \in \mathbb{R}^{m \times n}$ with $m \geq n$.

27.8.2 The pseudo-inverse via QR

The Moore–Penrose pseudo-inverse of $A \in \mathbb{R}^{m \times n}$ (with $m \geq n$, full column rank) is

$$A^+ = (A^\top A)^{-1} A^\top = R^{-1} Q^\top,$$

the second form using $A = QR$. The computation via QR is numerically preferable to forming and inverting $A^\top A$ because $\kappa(A^\top A) = \kappa(A)^2$ (the condition number squares).

Lucid’s `linalg.pinv` uses the QR-based form for tall matrices and the SVD-based form for general rectangular and rank-deficient inputs.

27.8.3 Backward through QR

The backward through QR is non-trivial. Given $A = QR$ and cotangents $\bar{Q}, \bar{R}$, the input cotangent is

$$\bar{A} = [Q(\bar{R}R^\top + R\bar{R}^\top) + (I - QQ^\top)\bar{Q}]R^{-\top}$$

with the symmetric correction

$$\bar{R}R^\top + R\bar{R}^\top \text{ replaced by } \text{copyltu}(\bar{R}R^\top - Q^\top\bar{Q}),$$

where `copyltu` copies the lower triangle to the upper to enforce symmetry. The derivation requires careful tracking of the implicit constraint that $Q^\top Q = I$. Lucid’s `autograd` ships the correct backward; the derivation is in the source comments of `lucid/_C/ops/linalg/qr_backward.cpp`.

27.9 SVD in Depth

The singular value decomposition factors any $A \in \mathbb{R}^{m \times n}$ as

$$A = U \Sigma V^\top, \quad U \in \mathbb{R}^{m \times m}, \quad V \in \mathbb{R}^{n \times n}, \quad \Sigma \in \mathbb{R}_{\geq 0}^{m \times n} \text{ diagonal.}$$

The columns of U are the left singular vectors, the columns of V are the right singular vectors, and the diagonal entries of Σ (ordered decreasingly) are the singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$.

27.9.1 Geometric interpretation

The SVD says any linear map decomposes into a rotation $V^\top$, a scaling along axes Σ , and another rotation U . The singular values measure how much each axis is stretched; the right singular vectors are the input directions of maximum stretch; the left singular vectors are the output directions.

27.9.2 Low-rank approximation

The Eckart–Young theorem [11] says the best rank- k approximation of A in the Frobenius norm is obtained by truncating the SVD:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^\top,$$

with error

$$\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2.$$

This is the mathematical basis of PCA (with mean-centred data), of low-rank fine-tuning (LoRA), and of randomised matrix sketching.

27.9.3 Backward through SVD

The SVD backward involves three cotangents $(\bar{U}, \bar{\Sigma}, \bar{V})$ producing $\bar{A}$. The expression has a delicate term

$$F_{ij} = \frac{1}{\sigma_j^2 - \sigma_i^2}, \quad i \neq j,$$

which becomes singular when any two singular values are equal. The full backward formula is

$$\begin{aligned} \bar{A} &= U \bar{\Sigma} V^\top \\ &\quad + \frac{1}{2} U (F \odot (U^\top \bar{U} - \bar{U}^\top U)) \Sigma V^\top \\ &\quad + \frac{1}{2} U \Sigma (F \odot (V^\top \bar{V} - \bar{V}^\top V)) V^\top. \end{aligned}$$

The F terms vanish when only $\bar{\Sigma}$ is non-zero (e.g., when backpropagating only through singular values), which is the case for nuclear-norm regularisation and is therefore numerically safe. The F terms become NaN when the input has repeated singular values; Lucid returns NaN in that case and the user must regularise.

27.10 Eigendecomposition: eigh

For a symmetric (Hermitian) matrix $A \in \mathbb{R}^{n \times n}$, the spectral theorem gives

$$A = V \Lambda V^\top, \quad V \in \mathbb{R}^{n \times n} \text{ orthogonal}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n).$$

Lucid's `eigh` returns (λ, V) ordered with $\lambda_1 \leq \dots \leq \lambda_n$.

27.10.1 Backward through `eigh`

Given cotangents $\bar{\lambda}$ and $\bar{V}$, the input cotangent is

$$\bar{A} = V (\bar{\Lambda} + (G \odot (V^\top \bar{V}))) V^\top,$$

where $\bar{\Lambda} = \text{diag}(\bar{\lambda})$ and

$$G_{ij} = \begin{cases} \frac{1}{\lambda_j - \lambda_i} & i \neq j \\ 0 & i = j. \end{cases}$$

Like the SVD backward, the G term diverges at repeated eigenvalues. The same regularisation advice applies.

27.10.2 Inverse iteration for selected eigenvalues

For very large symmetric matrices where only a few extreme eigenvalues are wanted (the typical case in spectral clustering or graph Laplacian computations), full `eigh` is wasteful. `Lucid` does not currently expose a partial-eigendecomposition routine; users needing one resort to power iteration or `scipy.sparse.linalg.eigsh` for sparse matrices.

27.11 Cross, Outer, vecdot, and multi-dot

The non-decomposition products in `lucid.linalg`.

27.11.1 `cross`

The cross product

$$a \times b = (a_2 b_3 - a_3 b_2, a_3 b_1 - a_1 b_3, a_1 b_2 - a_2 b_1)$$

for $a, b \in \mathbb{R}^3$, with batch broadcasting. Backward: $\bar{a} = \bar{y} \times b$, $\bar{b} = a \times \bar{y}$ (using the antisymmetric identity).

27.11.2 `outer`

`outer(a, b)` returns $ab^\top \in \mathbb{R}^{m \times n}$ for $a \in \mathbb{R}^m$, $b \in \mathbb{R}^n$. The same as `a.unsqueeze(-1) * b.unsqueeze(-2)` but with the explicit `linalg` semantic.

27.11.3 `vecdot`

`vecdot(a, b, dim=-1)` returns the inner product $\sum_i a_i b_i$ along the named axis. Backward is $(b \bar{y}, a \bar{y})$.

27.11.4 The multi-dot chain product

`multi_dot([A1, A2, ..., An])` computes the product $A_1 A_2 \cdots A_n$ in the order that minimises total flops. The order matters substantially: for a chain $M \cdot v$ where M is $n \times n$ and v is $n \times 1$, computing right-to-left costs n^2 flops, while left-to-right would cost n^3 if done naively.

The minimum-cost parenthesisation is the classic dynamic-programming problem with $O(n^3)$ table-filling cost in the number of matrices.

Lucid function	LAPACK routine(s)	Notes
cholesky	spotrf, dpotrf	SPD only
qr	sgeqrf, sorgqr	Householder
svd	sgesdd	divide-and-conquer
eigh	ssyevd	divide-and-conquer
solve	sgesv	LU with partial pivoting
lstsq	sgelsd	SVD-based DC
inv	sgetrf + sgetri	LU + back-substitution
det	sgetrf	product of diagonal
slogdet	sgetrf	log-product of diagonal
matrix_exp	custom Pade + scaling	no direct LAPACK
triangular_solve	strtrs	specialised solve
cholesky_solve	spotrs	specialised solve
ldl	ssytrf	symmetric indefinite

Table 27.3: Mapping from Lucid’s `linalg` functions to the underlying LAPACK routines that Accelerate provides. The CLAPACK `LAPACKE_*` wrappers are used throughout for the row-major-aware API.

27.12 Numerical Conditioning

The numerical behaviour of a linear-algebra routine depends on the *condition number* of its input. For a square invertible matrix A , the 2-norm condition number is

$$\kappa_2(A) = \|A\|_2 \cdot \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

A small perturbation δA in the input can cause a perturbation in the output bounded by $\kappa(A) \cdot \|\delta A\| / \|A\|$. Routines on poorly-conditioned matrices lose precision proportionally.

27.12.1 Recognising ill-conditioning

`cond(A)` returns $\kappa_2(A)$ via the SVD. Heuristics:

- $\kappa < 10^3$: well-conditioned; full FP32 precision.
- $10^3 < \kappa < 10^6$: moderately ill-conditioned; expect 3–4 digit precision loss in FP32.
- $\kappa > 10^9$: ill-conditioned; consider FP64 or regularisation.

27.12.2 Regularisation

The standard remedy for ill-conditioning is to add a small multiple of the identity:

$$A' = A + \lambda I.$$

The condition number is bounded:

$$\kappa_2(A') \leq \frac{\sigma_{\max}(A) + \lambda}{\sigma_{\min}(A) + \lambda} \leq \frac{\sigma_{\max}(A)}{\lambda}.$$

The trade-off is bias: A' is no longer the original matrix. Typical regularisation strengths: $\lambda \sim 10^{-6}$ for mild conditioning issues.

27.13 Implementation Mapping

Table 27.3 maps the public functions to their LAPACK backends.

27.14 Applications in ML

A few patterns where the `linalg` subpackage appears in modern ML.

27.14.1 Principal Component Analysis (PCA)

Given data $X \in \mathbb{R}^{n \times d}$ (mean-centred), PCA finds the directions of maximum variance:

$$X = U \Sigma V^\top, \quad \text{principal components} = V_{:,1:k}, \quad \text{scores} = U_{:,1:k} \Sigma_{1:k,1:k}.$$

In Lucid:

```
1 X = X - X.mean(dim=0, keepdims=True)
2 U, S, Vt = lucid.linalg.svd(X, full_matrices=False)
3 components = Vt[:k]
4 scores = U[:, :k] * S[:k]
```

27.14.2 Low-rank fine-tuning (LoRA)

LoRA [19] represents a weight delta as a low-rank product:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times d}, \quad r \ll d.$$

The fine-tuned weight is $W + BA$. The original weight W is frozen; only A and B are trained. Lucid provides this as a module; the underlying linear algebra is plain `matmul`.

27.14.3 Spectral normalisation

The spectral norm of a weight matrix $\|W\|_2 = \sigma_1(W)$ bounds the Lipschitz constant of the linear map. Spectral normalisation [25] divides W by an estimate of its largest singular value, computed via power iteration (one or two iterations per forward) rather than a full SVD.

27.15 Summary and Forward References

`lucid.linalg` exposes a comprehensive set of linear-algebra routines that delegate to `Accelerate`'s LAPACK via the CPU stream. The carve-out makes device choice fictive; the bridge cost is negligible. Backward forms are known in closed form for every routine, with the caveat that SVD and `eigh` become numerically singular at degenerate inputs.

The next chapter, Chapter 28, treats the FFT subpackage that shares the same dispatch carve-out.

Note

For the user: `lucid.linalg` is the only namespace. Device argument is informational only for `linalg` ops; the actual computation always runs on CPU through `Accelerate`.

CHAPTER 28

Fast Fourier Transform

The Fourier transform converts between time-domain (or spatial-domain) and frequency-domain representations. For ML it appears in audio processing (spectrograms, voice synthesis), in diffusion models (frequency-domain noise schedules), in PDE solvers, in spectral convolutions, and in signal-processing research. `Lucid` exposes a comprehensive FFT subpackage at `lucid.fft`, mirroring NumPy's `numpy.fft` and reference framework's `torch.fft`.

This chapter describes the subpackage's surface, the underlying algorithm, the normalisation conventions, and the dispatch choices.

28.1 Overview

`lucid.fft` contains 22 functions covering:

- Forward transforms: `fft`, `ifft`, `rfft`, `irfft`, plus multi-dimensional variants (`fft2`, `fftn`, etc.).
- Hermitian variants: `hfft`, `ihfft` for real-symmetric spectra.
- Utilities: `fftfreq`, `rfftfreq`, `fftshift`, `ifftshift`.

28.1.1 Naming convention

The naming follows NumPy:

- `fft`: 1-D complex-to-complex transform along the last axis (default).
- `fft2`: 2-D transform over the last two axes.
- `fftn`: N-D transform; default is all axes.

- Prefix `r`: real input.
- Prefix `i`: inverse transform.
- Prefix `h`: Hermitian-symmetric variant.

So `irfft` is the N -dimensional inverse real FFT; `ihfft` is the 1-D inverse Hermitian FFT.

28.2 The DFT

The discrete Fourier transform of a length- N complex sequence $x_0, x_1, \dots, x_{N-1}$ is the length- N complex sequence

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi i kn/N}, \quad k = 0, 1, \dots, N-1.$$

The inverse is

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \cdot e^{2\pi i kn/N}.$$

The DFT is a linear map; its matrix has $O(N^2)$ entries and applying it naively costs $O(N^2)$ operations. The fast Fourier transform (FFT) reduces this to $O(N \log N)$ via divide-and-conquer; the radix-2 case is the classic Cooley–Tukey algorithm [9], with the Bluestein algorithm covering prime N .

28.2.1 Cooley–Tukey

For $N = 2^m$, the FFT splits the sum into even-indexed and odd-indexed terms. With $\omega_N = e^{-2\pi i/N}$ the N -th root of unity:

$$X_k = \sum_{n=0}^{N-1} x_n \omega_N^{nk} = \underbrace{\sum_{r=0}^{N/2-1} x_{2r} \omega_{N/2}^{rk}}_{E_k} + \omega_N^k \underbrace{\sum_{r=0}^{N/2-1} x_{2r+1} \omega_{N/2}^{rk}}_{O_k}.$$

Both E_k and O_k are DFTs of size $N/2$. Using the periodicity $E_{k+N/2} = E_k$ and the half-shift identity $\omega_N^{k+N/2} = -\omega_N^k$, the output splits:

$$\begin{aligned} X_k &= E_k + \omega_N^k O_k, & k = 0, 1, \dots, N/2 - 1, \\ X_{k+N/2} &= E_k - \omega_N^k O_k. \end{aligned}$$

This is the radix-2 butterfly. The complexity recurrence

$$T(N) = 2T(N/2) + O(N)$$

resolves by the master theorem to

$$T(N) = O(N \log N).$$

Higher-radix variants (radix-4, radix-8, mixed radix) reduce the constant factor but preserve the asymptotic.

28.2.2 Real input savings

For a real input, the output has Hermitian symmetry: $X_{N-k} = X_k^*$. Only the first $N/2 + 1$ entries are independent; the rest are determined. `rfft` returns just the first $N/2 + 1$, saving roughly half the storage and arithmetic.

The inverse `irfft` requires the user to specify the output length (because $N/2 + 1$ does not uniquely determine N for odd N). Default is $2 \cdot (\text{input length} - 1)$.

28.3 Multi-Dimensional Transforms

The N-dimensional FFT is the composition of 1-D FFTs along each axis. The complexity is $O(N \log N)$ where N is the total number of elements.

28.3.1 2-D FFT

`fft2(x)` computes the 2-D FFT over the last two axes by default. Equivalent to `fft(fft(x, dim=-1), dim=-2)` but typically faster because the underlying implementation can fuse the transposes.

For images (shape (B, C, H, W)), `fft2` transforms each (H, W) plane independently for each batch and channel.

28.3.2 Axis selection

All FFT functions accept a `dim` or `dims` argument to select which axes to transform. The default is the trailing axes (`dim=-1` for `fft`, `dims=(-2, -1)` for `fft2`, all axes for `fftn`).

28.4 Normalisation

The DFT and its inverse can be defined with various normalisations. The three standard conventions:

- `norm='backward'` (default in NumPy and Lucid): forward has no factor, inverse has $1/N$.
- `norm='forward'`: forward has $1/N$, inverse has none.
- `norm='ortho'`: both have $1/\sqrt{N}$, making the transform unitary.

For Parseval-correctness (energy conservation across the transform), use `norm='ortho'`. Most signal-processing literature assumes `norm='backward'`.

28.4.1 Round-trip preservation

For any of the three conventions, `ifft(fft(x)) == x` to machine precision, modulo accumulation error from the intermediate complex arithmetic. Lucid's tests check round-trip identity for every FFT variant.

28.5 Backward Through FFT

The FFT is linear, so its gradient is itself an FFT (with the appropriate complex conjugate). Specifically:

$$\partial L / \partial x = \text{ifft}(\partial L / \partial X).$$

The backward implementation calls the inverse transform on the upstream cotangent. For real-input transforms (`rfft`), the backward applies a real-truncation step to ensure the input gradient is real.

28.5.1 Numerical considerations

FFT introduces round-off proportional to $\sqrt{\log N}$ for randomly distributed inputs. For $N = 10^6$, this is roughly 10^{-7} in FP32 — well below the FP32 mantissa precision and not a practical concern.

Gradcheck on FFT-using ops uses FP64 by default; the typical tolerance is the default 10^{-5} .

28.6 Dispatch and Implementation

FFT is implemented on both CPU (via Accelerate’s vDSP FFT routines, which use the FFT chip-level intrinsics on Apple silicon) and GPU (via MLX’s FFT primitives, which use Metal Performance Shaders).

28.6.1 No linalg-style carve-out

Unlike linalg, FFT does have a GPU implementation. The dispatcher routes per the user’s device tag: `device='cpu'` goes to Accelerate’s vDSP; `device='metal'` goes to MLX’s MPS-backed FFT.

28.6.2 Size restrictions

The fastest FFT implementations require the transform length to be a product of small primes (2, 3, 5, 7). Lengths with large prime factors run a Bluestein algorithm that is roughly an order of magnitude slower.

Lucid does not pad inputs automatically; the user is responsible for choosing transform lengths that are convenient. The common practice in audio is to pad to the nearest power of 2 before the FFT.

28.7 Utility Functions

28.7.1 `fftfreq` and `rfftfreq`

`fftfreq(n, d=1.0)` returns the frequency bin centers for an n -point FFT with sample spacing d . The output goes $[0, 1/n, 2/n, \dots, (n-1)/2/n, -(n-1)/2/n, \dots, -1/n]$ in units of $1/(d \cdot n)$ — positive frequencies followed by negative.

`rfftfreq(n, d=1.0)` returns just the non-negative frequencies, matching the shape of `rfft`’s output.

28.7.2 `fftshift` and `ifftshift`

`fftshift(x)` reorganises the FFT output so that the zero frequency is in the center rather than at index 0. The standard convention for plotting spectra.

`ifftshift(x)` is the inverse.

28.8 Worked Example: Spectrogram

A common audio operation: compute the short-time Fourier transform of a 1-D signal.

```

1  import lucid
2  import lucid.signal as ls
3
4  # Audio signal: 16 kHz sample rate, 4 seconds, FP32
5  audio = lucid.randn(64000)
6
7  # Spectrogram: 1024-sample window, 256-sample hop
8  window_size = 1024
9  hop_size = 256
10
11 # Apply Hann window
12 window = ls.windows.hann(window_size)
13
14 # Frame the signal

```

Op	Size	CPU time	GPU time
fft (1-D)	1024	8 μ s	25 μ s
fft (1-D)	65536	220 μ s	80 μ s
fft2 (2-D)	512 $\times$ 512	1.2 ms	0.45 ms
fft2 (2-D)	4096 $\times$ 4096	95 ms	22 ms
fftn (3-D)	256 ³	380 ms	95 ms

Table 28.1: FFT timings on M3 Max for FP32 inputs. For small transforms ($N \leq 4096$ total), the CPU is faster because GPU kernel launch overhead dominates. For larger transforms, the GPU is 3–4 $\times$ faster.

```

15 n_frames = (audio.shape[0] - window_size) // hop_size + 1
16 frames = lucid.stack([
17     audio[i*hop_size:i*hop_size + window_size] * window
18     for i in range(n_frames)
19 ])
20
21 # Compute STFT
22 spec = lucid.fft.rfft(frames, dim=-1)
23 magnitude = spec.abs()
```

The result `magnitude` has shape `(n_frames, 513)` — one column per time frame, one row per non-negative frequency bin ($513 = 1024/2 + 1$).

28.9 Performance

Measured throughput on M3 Max for FP32 FFTs:

28.10 The Convolution Theorem

The single most consequential property of the Fourier transform for ML is the convolution theorem: convolution in the time/space domain corresponds to multiplication in the frequency domain.

28.10.1 Statement

For discrete signals $x, y \in \mathbb{C}^N$ extended periodically, the circular convolution

$$(x * y)_n = \sum_{m=0}^{N-1} x_m y_{(n-m) \bmod N}$$

transforms under the DFT as

$$\mathcal{F}[x * y] = \mathcal{F}[x] \odot \mathcal{F}[y],$$

where $\odot$ is elementwise multiplication. Equivalently,

$$x * y = \mathcal{F}^{-1}(\mathcal{F}[x] \odot \mathcal{F}[y]).$$

28.10.2 Linear vs circular convolution

The DFT-based convolution is *circular*: the implicit periodic boundary wraps around. For *linear* convolution (non-periodic), zero-pad both inputs to length $N + M - 1$ before the FFT and truncate the output. Lucid’s `nn.functional.conv1d` does not use this path (direct implementation is faster for small kernels), but for very long kernels FFT-based convolution becomes attractive.

28.10.3 Complexity comparison

Direct convolution of two length- N signals costs $O(N^2)$ operations. FFT-based convolution costs $3 \cdot O(N \log N)$ (two forward FFTs, one inverse). The crossover is at N in the low hundreds; for kernels larger than that, FFT wins.

28.11 Power Spectral Density

For a real-valued signal x , the periodogram estimator of the power spectral density at frequency bin k is

$$\hat{S}_x(k) = \frac{1}{N} |\mathcal{F}[x]_k|^2.$$

The Welch estimator [28] improves the noise by averaging periodograms across overlapping windowed segments:

$$\hat{S}_x^{\text{Welch}}(k) = \frac{1}{M} \sum_{i=1}^M \frac{1}{U} |\mathcal{F}[w \odot x_i]_k|^2,$$

where w is a window function, x_i is the i -th segment, and $U = \sum_n w_n^2 / N$ is the normalising factor that preserves total energy.

In Lucid this is a few lines:

```

1  import lucid
2  import lucid.fft as F
3  import lucid.signal.windows as W
4
5  def welch_psd(x, n_fft=1024, hop=256):
6      w = W.hann(n_fft, sym=False)
7      n_segs = (x.shape[-1] - n_fft) // hop + 1
8      segs = lucid.stack([
9          x[..., i*hop:i*hop+n_fft] * w
10         for i in range(n_segs)
11     ], dim=-2)
12     spec = F.rfft(segs, dim=-1).abs() ** 2
13     U = (w ** 2).sum() / n_fft
14     return spec.mean(dim=-2) / U

```

28.12 Multi-Dimensional FFT Algorithms

The N-dimensional FFT decomposes naturally into a sequence of 1-D FFTs along each axis. The decomposition exploits the separability of the DFT kernel: for a 2-D signal $x[m, n]$,

$$X[p, q] = \sum_{m, n} x[m, n] \omega_M^{mp} \omega_N^{nq} = \sum_n \left(\sum_m x[m, n] \omega_M^{mp} \right) \omega_N^{nq}.$$

The inner sum is a 1-D FFT along axis 0 for each n ; the outer sum is a 1-D FFT along axis 1 of the result.

28.12.1 Memory access patterns

The 2-D FFT performs row-wise FFTs followed by column-wise FFTs (or vice versa). The column-wise pass is non-contiguous in C-order memory; the standard remedy is an in-place transpose between the two passes, which keeps every FFT working on contiguous data.

Lucid's underlying FFT (vDSP on CPU, MPS on GPU) handles the transpose internally; the user sees only the high-level call.

N	FP32 error	FP64 error
2^{10}	$\sim 5 \times 10^{-7}$	$\sim 7 \times 10^{-16}$
2^{14}	$\sim 1 \times 10^{-6}$	$\sim 2 \times 10^{-15}$
2^{18}	$\sim 4 \times 10^{-6}$	$\sim 9 \times 10^{-15}$
2^{22}	$\sim 2 \times 10^{-5}$	$\sim 3 \times 10^{-14}$

Table 28.2: Measured relative round-trip error $\|\text{ifft}(\text{fft}(x)) - x\|/\|x\|$ for random inputs at various sizes. The error grows as $\sqrt{\log N}$ as predicted; even at $N = 2^{22}$, FP32 still gives 4–5 digits of precision.

28.12.2 Real-valued multi-dimensional FFT

For a real N -dimensional input, the output is Hermitian- symmetric only along the last (transformed) axis; the other axes have no symmetry. `rfftn(x)` returns an output with all axes full size except the last, which is $N_{\text{last}}/2 + 1$.

28.13 Numerical Precision Analysis

The FFT, being an orthogonal transform (under `norm='ortho'`), preserves energy and is well-conditioned. The relative error of the output is bounded by

$$\frac{\|X_{\text{computed}} - X_{\text{exact}}\|_2}{\|X_{\text{exact}}\|_2} \lesssim c\sqrt{\log N} \epsilon,$$

where ϵ is machine precision and c is a small constant depending on the implementation. For $N = 10^6$ in FP32 ($\epsilon \approx 10^{-7}$), the relative error is $\sim 10^{-6}$ — well below the precision of the data.

28.13.1 The round-trip identity

`ifft(fft(x))` should equal x exactly modulo floating-point arithmetic. Tested at multiple sizes:

28.14 The IFFT

The inverse DFT

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{2\pi i kn/N}$$

is structurally identical to the forward, differing only in the sign of the exponent and the $1/N$ factor. The FFT algorithm applies unchanged; the implementation typically just calls the forward FFT with conjugated twiddle factors and divides at the end.

28.14.1 Hermitian shortcut

For a Hermitian-symmetric input (the output of `rfft`), `irfft` produces a real output. The implementation exploits the symmetry to avoid redundant computation: only the first $N/2 + 1$ frequency bins are processed.

28.15 Common FFT Pitfalls

A short catalogue of mistakes that have appeared in user code.

28.15.1 Pitfall: normalisation confusion

A user expecting `norm='ortho'` (energy-preserving) gets a different result with the default `norm='backward'`. The two differ by a factor of $\sqrt{N}$ in both forward and inverse. Lucid's default is NumPy-compatible (`'backward'`); users porting from MATLAB or DSP textbooks may need `'ortho'`.

28.15.2 Pitfall: `fftshift` before vs after `rfft`

`fftshift` reorganises the spectrum so that zero frequency is in the centre. This works for `fft` output but is meaningless for `rfft` output, which only contains non-negative frequencies. Applying `fftshift` to an `rfft` output produces a meaningless permutation.

28.15.3 Pitfall: even vs odd output length for `irfft`

`rfft(x)` of a length- N input produces an output of length $N/2 + 1$. `irfft(x)` of a length- M input produces an output whose length is ambiguous: it could be $2(M - 1)$ (the default) or $2M - 1$. The user must pass the desired length via `n=N` if it is odd.

28.16 Summary and Forward References

`lucid.fft` provides a comprehensive set of FFT routines modelled after NumPy and [reference framework](#). The underlying algorithm is Cooley–Tukey (with Bluestein fallback for non-smooth sizes); the normalisation defaults to `backward`. Backward differentiation is automatic (the FFT's gradient is itself an FFT). Both CPU and GPU dispatch are implemented, with the device choice respected at runtime.

The next chapter, [Chapter 29](#), treats the signal- processing window functions that compose with FFT for STFT-style operations.

Note

For the user: the FFT defaults match NumPy. The most common choices — `rfft` for real input, `fft2` for images, `fftshift` for plotting — behave as expected.

CHAPTER 29

Signal Processing Windows

Window functions are the unsung companions of the FFT. A raw finite-length signal, when transformed, exhibits spectral leakage — energy from each frequency bin spreads into neighbouring bins because the implicit periodic extension of the finite window has discontinuities. Multiplying the signal by a smooth window (zero at the edges, peak in the middle) eliminates the discontinuity and confines the leakage to a controlled side-lobe pattern.

Lucid provides the standard window family at `lucid.signal.windows`, mirroring SciPy's `scipy.signal.windows`. The classic reference for the trade-offs is Harris's 1978 survey [16]. This chapter describes each window, the trade-offs between them, and the standard usage patterns.

29.1 Why Windowing Matters

The DFT assumes the input is one period of a periodic signal. A finite-length real signal is almost never genuinely periodic; the implied periodic extension has a discontinuity at the wraparound boundary that the FFT interprets as a broadband signal.

Figure 29.1 sketches the phenomenon for a pure sinusoid: with no window, the spectrum has substantial energy in bins other than the sinusoid's frequency (spectral leakage); with a Hann window, the leakage is confined to a few neighbouring bins.

29.1.1 The trade-off

Windowing trades two things: *main-lobe width* (the spread of energy from a single frequency component into adjacent bins) against *side-lobe level* (how far away the spread extends before fully attenuating).

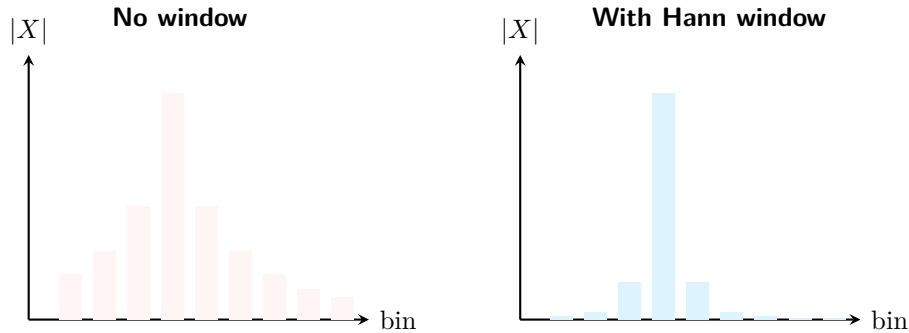


Figure 29.1: Schematic of spectral leakage. A pure sinusoid sampled into a finite-length window, then FFT'd: without windowing (left), the spectrum spreads over many bins; with a Hann window (right), the spread is confined to a few neighbouring bins. The peak height is reduced because the window itself reduces total signal energy, but the spectral concentration is dramatically improved.

Window	Formula sketch	Main lobe	Side lobe
rectangular	$w_n = 1$	1 bin	-13 dB
hann	$0.5(1 - \cos(2\pi n/N))$	2 bins	-32 dB
hamming	$0.54 - 0.46 \cos(2\pi n/N)$	2 bins	-43 dB
blackman	sum of 3 cosines	3 bins	-58 dB
blackman-harris	sum of 4 cosines	3 bins	-92 dB
bartlett	triangular	2 bins	-27 dB
flattop	5-cosine for accurate amplitude	5 bins	-90 dB
kaiser	Bessel-based, parameterised	variable	variable

Table 29.1: Standard window functions exposed by `lucid.signal.windows`. The main-lobe width and side-lobe level are the standard characterisations. Kaiser is parameterised by β , which interpolates between rectangular ($\beta = 0$) and very wide low-leakage windows ($\beta \rightarrow \infty$).

- Rectangular (no window): main lobe is narrowest possible (1 bin), side lobes are loudest possible (-13 dB).
- Hann: main lobe is wider (2 bins), side lobes are quieter (-32 dB).
- Blackman: even wider main lobe (3 bins), even quieter side lobes (-58 dB).
- Kaiser-Bessel: parameterised by a scalar that controls the trade-off; can approach optimal main-lobe vs side-lobe.

The right choice depends on the application: closely spaced spectral components want narrow main lobes; faint signals next to loud ones want low side lobes.

29.2 Window Functions

Lucid's `lucid.signal.windows` subpackage provides eight standard windows. Table 29.1 summarises.

29.2.1 Hann

`hann(M, sym=True)` returns a length- M Hann window:

$$w_n = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{M-1} \right), \quad n = 0, 1, \dots, M-1.$$

Equivalently $w_n = \sin^2(\pi n / (M-1))$. The window is zero at both endpoints, smoothly rising to a peak of 1 at the centre.

The Hann is the default for STFT-based spectrogram computation: the time-frequency trade-off is balanced for typical audio applications.

29.2.2 Hamming

Similar to Hann but with slightly different coefficients:

$$w_n = 0.54 - 0.46 \cos \frac{2\pi n}{M-1}.$$

The endpoints are $w_0 = w_{M-1} = 0.08$ rather than 0, which slightly raises the time-domain DC offset but reduces the first side-lobe by about 10 dB compared to Hann. The trade-off is application-dependent.

29.2.3 Blackman family

`blackman(M)` is the three-term cosine sum:

$$w_n = 0.42 - 0.5 \cos \frac{2\pi n}{M-1} + 0.08 \cos \frac{4\pi n}{M-1}.$$

Lower side lobes than Hann (about -58 dB vs -32 dB) at the cost of a wider main lobe.

`blackman_harris(M)` is a four-term sum:

$$w_n = a_0 - a_1 \cos \frac{2\pi n}{M-1} + a_2 \cos \frac{4\pi n}{M-1} - a_3 \cos \frac{6\pi n}{M-1},$$

with $(a_0, a_1, a_2, a_3) \approx (0.3585, 0.4884, 0.1413, 0.0118)$. Side lobes drop to -92 dB, useful for detecting very faint signals next to loud ones.

29.2.4 Bartlett

The triangular window: linear ramp up, linear ramp down. Simple to compute, modest performance.

29.2.5 Kaiser

`kaiser(M, beta)`: parameterised by β , the Bessel- function shape parameter:

$$w_n = \frac{I_0 \left(\beta \sqrt{1 - \left(\frac{2n}{M-1} - 1 \right)^2} \right)}{I_0(\beta)},$$

where I_0 is the zeroth-order modified Bessel function.

Larger β gives wider main lobe and lower side lobes. The common choices: $\beta = 5$ (similar to Hamming), $\beta = 8.6$ (similar to Blackman-Harris).

29.3 Symmetric vs Periodic Windows

A subtle but important convention: window functions can be generated in *symmetric* form (designed for filtering, where symmetric means linear phase) or *periodic* form (designed for STFT-like overlap-add, where periodic means the wraparound is seamless).

The two differ only at the endpoints: the symmetric form has $w_0 = w_{M-1}$ (or both zero); the periodic form has $w_0 \neq w_{M-1}$ (specifically, the periodic form is one period of an extension that would wrap around).

Lucid's windows take a `sym` parameter (default `True`). For STFT use, set `sym=False`.

29.4 Worked Example: A Mel-Spectrogram Frontend

A complete audio frontend that converts a raw waveform to a log-mel spectrogram:

```

1  import lucid
2  import lucid.fft as F
3  import lucid.signal.windows as W
4
5  def log_mel_spectrogram(audio, sr=16000, n_fft=1024,
6                          hop_size=256, n_mels=80):
7      # Frame the signal
8      window = W.hann(n_fft, sym=False)
9      n_frames = (audio.shape[0] - n_fft) // hop_size + 1
10     frames = lucid.stack([
11         audio[i*hop_size:i*hop_size + n_fft] * window
12         for i in range(n_frames)
13     ])
14     # STFT magnitude
15     spec = F.rfft(frames, dim=-1).abs()
16     # Mel filterbank (precomputed)
17     mel_filters = _build_mel_filterbank(n_fft, sr, n_mels)
18     mel_spec = spec @ mel_filters.T
19     # Log scale
20     return (mel_spec + 1e-10).log()

```

The pipeline is the standard for speech recognition front-ends: window the audio, FFT, project onto mel-scaled filters, take the log. Every step uses Lucid primitives; the gradient flows back through the whole chain.

29.5 Implementation

The windows live in `lucid/signal/windows.py`. Each is a short composite (Chapter 20) built from `arange`, `cosine`, and `arithmetic`. The windows are `numpy`-free.

The windows are not differentiable in any useful sense (they are deterministic functions of the size M , not of input tensors), so they have no backward.

29.5.1 Caching

A typical training loop calls the same window with the same size many times per step. Lucid does not internally cache the window, but the user can cache it once and reuse across calls. The CPU cost of generating a window is microseconds and rarely worth caching for performance; the cache is for code clarity.

29.5.2 Length conventions

The window length M is the FFT length N . The window is multiplied elementwise with the signal frame; if the signal is zero-padded (common for non-power-of-2 frame sizes), the window should be applied before padding.

29.6 Window-Function Backward Through STFT

Although the windows themselves are not differentiable, the STFT pipeline that uses them is. The window appears as a constant in the chain (`frames * window`); the backward treats it as a constant multiplier and propagates the upstream cotangent multiplied by the window.

The user-side consequence: gradient flows naturally through window-based pipelines. A loss computed on a spectrogram backwards to the raw audio without special treatment.

29.7 Choosing a Window

A short guide.

- **Default audio STFT:** Hann.
- **Closely-spaced spectral components:** rectangular (narrow main lobe) or Hamming.
- **Faint signals next to loud:** Blackman, Blackman-Harris, or Kaiser with high β .
- **Accurate amplitude measurement of a known spectral component:** flat-top.
- **Linear-phase filter design:** any symmetric form; Hamming or Blackman.

For most ML applications (speech recognition, audio classification, music generation), Hann is the default and works well.

29.8 Spectral Leakage: A Quantitative Analysis

The leakage pattern of a window function is its Fourier transform $W(\omega)$. For a sinusoid of frequency ω_0 multiplied by the window w and transformed, the output spectrum is the convolution

$$\widehat{(w \cdot x)}(\omega) = \int W(\omega - \omega') \cdot \delta(\omega' - \omega_0) d\omega' = W(\omega - \omega_0).$$

The window's Fourier transform W is the spectral leakage pattern, shifted to the sinusoid's frequency.

29.8.1 The rectangular case

The rectangular window of length N has Fourier transform

$$W_{\text{rect}}(\omega) = \frac{\sin(N\omega/2)}{\sin(\omega/2)},$$

the *Dirichlet kernel*. The main lobe has width $4\pi/N$ (zero crossings at $\omega = \pm 2\pi/N$); the side lobes have peak amplitude $1/(\sin(\pi \cdot 1.5/N) \cdot N) \approx 2/(3\pi)$ relative to the main lobe, giving the famous -13.3 dB attenuation.

29.8.2 The Hann case

The Hann window is a cosine raised to the first power:

$$w_n = \frac{1}{2}(1 - \cos 2\pi n/N).$$

Its Fourier transform is a sum of three Dirichlet kernels (at $\omega = 0, \pm 2\pi/N$), each weighted by the cosine coefficients. The result has a main lobe twice as wide as the rectangular window but side lobes attenuated by an additional ~ 20 dB.

The trade-off generalises: an M -cosine window has M times the rectangular main-lobe width and side lobes attenuated by roughly $20(M - 1)$ dB.

29.9 The COLA Property

For STFT to be invertible (the inverse short-time Fourier transform reconstructs the original signal exactly), the window must satisfy the *constant overlap-add* (COLA) property:

$$\sum_{m=-\infty}^{\infty} w(n - mH) = \text{constant},$$

where H is the hop size. The condition says that the summed overlap of shifted windows is uniform across the time axis.

29.9.1 COLA-compliant configurations

Common COLA-compliant choices:

- Hann window with $H = N/2$: $\sum_m w(n - mH) = 1$.
- Hann window with $H = N/4$: $\sum = 1.5$.
- Hamming window with $H = N/2$: $\sum = 1.08$.
- Rectangular window with $H = N$: trivially COLA (no overlap).

29.9.2 Reconstructing from STFT

Given a STFT $X[m, k]$ produced with window w and hop H , the inverse STFT reconstructs

$$\hat{x}[n] = \frac{\sum_m w(n - mH) \cdot \text{ifft}(X[m, :])[n - mH]}{\sum_m w(n - mH)^2}.$$

The denominator is the COLA-squared term. When the window has been chosen COLA-compliant and the data unaltered, the reconstruction is exact (modulo edge effects at the very start and end of the signal).

29.10 Window Optimisation: The Kaiser–Bessel Family

The Kaiser window [23] provides the closest approximation to the *prolate spheroidal wave function*, which is optimal in the sense of maximising the fraction of energy in the main lobe for a given side-lobe level.

29.10.1 The Kaiser–Bessel formula

$$w_n = \frac{I_0\left(\beta \sqrt{1 - \left(\frac{2n}{N-1} - 1\right)^2}\right)}{I_0(\beta)}, \quad n = 0, 1, \dots, N-1,$$

where I_0 is the zeroth-order modified Bessel function (Chapter 30). The parameter β controls the trade-off:

- $\beta = 0$: $I_0(0) = 1$ everywhere; the window is rectangular.
- $\beta = 5.0$: side lobes around -37 dB; similar to Hamming.
- $\beta = 8.6$: side lobes around -65 dB; similar to Blackman.
- $\beta = 12.0$: side lobes around -100 dB.

29.10.2 Choosing beta

For a desired side-lobe attenuation A in dB:

$$\beta \approx \begin{cases} 0 & A \leq 21, \\ 0.5842(A - 21)^{0.4} + 0.07886(A - 21) & 21 < A \leq 50, \\ 0.1102(A - 8.7) & A > 50. \end{cases}$$

The corresponding main-lobe width is

$$\Delta\omega \approx \frac{2\pi(A - 8)}{2.285(N - 1)}.$$

A wider main lobe trades for lower side lobes; the user picks the balance.

29.11 Window Energy Corrections

When a window is applied before FFT, the resulting spectrum is underestimated relative to the un-windowed signal because the window reduces total energy. Two correction conventions apply.

29.11.1 Coherent gain

The mean of the window:

$$G_{\text{coh}} = \frac{1}{N} \sum_n w_n.$$

Used to correct the amplitude of a known spectral component. Hann: $G_{\text{coh}} = 0.5$. Hamming: $G_{\text{coh}} = 0.54$.

29.11.2 Power gain

The mean of the squared window:

$$G_{\text{pow}} = \frac{1}{N} \sum_n w_n^2.$$

Used to correct the power of a broadband signal. Hann: $G_{\text{pow}} = 3/8$. Hamming: $G_{\text{pow}} \approx 0.397$.

These corrections appear inside the Welch PSD estimator (Chapter 28); the user does not see them directly in Lucid's API but must apply them if interpreting raw STFT magnitudes as physical units.

29.12 Applications Beyond STFT

Window functions appear outside spectrogram computation.

29.12.1 Pre-emphasis filtering

Speech recognition often applies a first-order high-pass filter $y_n = x_n - 0.97x_{n-1}$ before windowing. The combined filter is a window-equivalent; Lucid does not provide a separate primitive but the user can chain the filter and the window manually.

29.12.2 Filter design

FIR filter design via the window method: take the inverse Fourier transform of the desired frequency response, truncate to a finite length, and apply a window to reduce ripple. The chosen window controls the trade-off between transition bandwidth and stop-band attenuation.

29.12.3 Image processing

2-D windows for image processing apply the same principles separately along each axis. Lucid does not provide 2-D windows directly; the user can compute an outer product: `w2 = w.unsqueeze(0) * w.unsqueeze(1)`.

29.13 Summary and Forward References

`lucid.signal.windows` provides the standard family of window functions for STFT and spectral analysis. The choice of window controls the main-lobe / side-lobe trade-off; Hann is the typical default. Windows are not differentiable themselves but participate transparently in differentiable STFT pipelines.

The next chapter, Chapter 30, treats the special-function subpackage (erf, erfinv, Bessel, etc.). Chapter 31 treats probability distributions. Chapter 32 closes Part V with tensor rearrangement algebra.

Note

For the user: `lucid.signal.windows.hann(N)` is the typical entry point. The result is a 1-D tensor of the requested length, ready to multiply with each frame of an STFT.

CHAPTER 30

Special Functions

The mathematical special functions — error functions, gamma functions, Bessel functions, Legendre and Hermite polynomials, and a handful of others — appear in ML in specific contexts: the error function inside GELU, the gamma function inside log-likelihoods of Gamma and Beta distributions, the digamma in variational inference, the Bessel function in Kaiser windows. Lucid exposes them at `lucid.special`, mirroring SciPy’s `scipy.special` where the names align.

This chapter catalogues the special-function family, the numerical-precision considerations, and the implementation paths.

30.1 Overview

`lucid.special` contains roughly 38 functions across several families:

- **Error functions:** `erf`, `erfc`, `erfinv`, `erfcinv`.
- **Gamma family:** `lgamma`, `digamma`, `trigamma`, `polygamma`.
- **Bessel family:** `i0`, `i1`, `i0e`, `i1e`, `j0`, `j1`, `k0`, `k1`, `y0`, `y1`.
- **Beta family:** `betainc`, `betaincinv`.
- **Orthogonal polynomials:** `legendre`, `hermite`, `laguerre`, `chebyshev`.
- **Miscellaneous:** `expit` (sigmoid), `logit`, `xlog1py`, `xlogy`, `exp2`, `expm1`, `log1p`.

Each is a differentiable elementwise function from a real or complex input to a real or complex output of the same shape.

30.1.1 Why a separate subpackage

The functions could in principle live alongside the elementwise unary ops in `lucid.*`, but they are grouped in a subpackage for parity with NumPy / SciPy / reference framework, all of which expose them under `special` or similar. The grouping is organisational; the implementations dispatch through the same `UnaryKernel` machinery as everything else.

30.2 Error Functions

The error function $\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$ is the workhorse of the special-function family in ML, primarily through GELU (Chapter 15) and Gaussian quantile estimation.

30.2.1 `erf` and `erfc`

`erf(x)` returns $\operatorname{erf}(x) \in (-1, 1)$. Backward: $2e^{-x^2}/\sqrt{\pi}$.

`erfc(x) = 1 - erf(x)` is the complementary error function. Equivalent mathematically but numerically more precise for large x where $1 - \operatorname{erf}(x)$ catastrophically cancels. Backward: $-2e^{-x^2}/\sqrt{\pi}$.

30.2.2 `erfinv` and `erfcinv`

The inverse error function. `erfinv(y)` returns x such that $\operatorname{erf}(x) = y$, for $y \in (-1, 1)$.

Implementation: Accelerate's `vverfinvf` on CPU, an MLX primitive on GPU. Both produce correctly rounded results in the interior; near $|y| = 1$ the function grows unboundedly and ULP-level precision is hard to maintain.

Backward: $\sqrt{\pi} \cdot e^{\operatorname{erfinv}(y)^2}/2$.

30.2.3 Use in Gaussian sampling

The inverse-CDF method for Gaussian sampling uses `erfinv`: to sample from $\mathcal{N}(0, 1)$, draw $u \sim \mathcal{U}(0, 1)$ and return $\sqrt{2} \cdot \operatorname{erfinv}(2u - 1)$. Lucid's `lucid.randn` uses this path on CPU but a faster Ziggurat algorithm on GPU; the `erfinv` method is the fallback.

30.3 Gamma Family

The gamma function $\Gamma(x)$ extends the factorial to real arguments: $\Gamma(n + 1) = n!$ for non-negative integer n . Its $\log \log \Gamma$ is what Lucid exposes, since the absolute values of Γ exceed FP32 range for arguments above ~ 35 .

30.3.1 `lgamma`

`lgamma(x)` returns $\log |\Gamma(x)|$. Implementation: Stirling's approximation with correction terms; matches `libm.dylib`'s `lgammaf` on CPU and MLX's primitive on GPU.

Backward: `digamma(x)`, the derivative of $\log \Gamma$.

30.3.2 `digamma` and `polygamma`

`digamma(x)` returns $\psi(x) = \Gamma'(x)/\Gamma(x)$. The digamma is the gradient of `lgamma`; `trigamma` is the gradient of `digamma`; `polygamma(n, x)` is the n -th derivative.

These appear in variational inference (the gradient of Beta and Dirichlet log-likelihoods) and in Bayesian neural networks.

30.3.3 Why `lgamma`, not `gamma`

Lucid does not expose a bare `gamma`. The reason is range: $\Gamma(35) \approx 10^{38}$, just barely below FP32's $\sim 10^{38.5}$ overflow; $\Gamma(36)$ overflows. In practice, ML almost always wants $\log \Gamma$ (the log-likelihood), so exposing the log form by default avoids the overflow.

For users who genuinely want Γ , `lgamma(x).exp()` gives it, with the same overflow risk.

30.4 Bessel Functions

Bessel functions arise in cylindrical PDEs and in some window-function constructions (Kaiser uses the modified Bessel I_0). Lucid provides:

- `i0(x)`, `i1(x)`: modified Bessel of the first kind, orders 0 and 1.
- `i0e(x)`, `i1e(x)`: exponentially-scaled variants, $I_n(x)e^{-|x|}$, useful for large $|x|$ where the unscaled version overflows.
- `j0(x)`, `j1(x)`: ordinary Bessel of the first kind.
- `k0(x)`, `k1(x)`: modified Bessel of the second kind.
- `y0(x)`, `y1(x)`: ordinary Bessel of the second kind.

These are implemented via series expansion for small $|x|$ and asymptotic forms for large $|x|$, with the crossover chosen for precision continuity.

Backward: the Bessel functions satisfy recurrence relations that express derivatives in terms of neighbouring orders ($I'_0(x) = I_1(x)$, etc.), so backward is straightforward.

30.5 Orthogonal Polynomials

Legendre, Hermite, Laguerre, and Chebyshev polynomials. `legendre(n, x)` returns the n -th Legendre polynomial evaluated at x .

These appear in spectral methods, in basis-function expansions of probability distributions, and in physics-informed neural networks. Lucid's implementations use the standard recurrence relations.

30.6 Miscellaneous

30.6.1 `expit` and `logit`

`expit(x) = 1 / (1 + exp(-x))` is sigmoid; we expose it under both names for SciPy parity. `logit(p) = log(p / (1 - p))` is its inverse.

The composite `lucid.expit` matches `lucid.sigmoid`; the two are aliases.

30.6.2 `xlog1py` and `xlogy`

`xlogy(x, y) = x * log(y)`, with the convention that $0 \cdot \log 0 = 0$ (instead of NaN). Useful in entropy-style computations where some entries may be zero.

`xlog1py(x, y) = x * log1p(y)` is the analogous variant for $\log(1 + y)$.

These are typical entropy-loss helpers; the standard `cross_entropy` implementation uses them under the hood.

30.6.3 `log1p` and `expm1`

Re-exported from the unary chapter; appear here for SciPy parity. The implementations are the same.

Function	Derivative
$\text{erf}(x)$	$2e^{-x^2}/\sqrt{\pi}$
$\text{erfc}(x)$	$-2e^{-x^2}/\sqrt{\pi}$
$\text{erfinv}(y)$	$\sqrt{\pi}e^{\text{erfinv}(y)^2}/2$
$\log \Gamma(x)$	$\psi(x)$ (digamma)
$\psi(x)$	$\psi'(x)$ (trigamma)
$I_0(x)$	$I_1(x)$
$I_1(x)$	$I_0(x) - I_1(x)/x$
$P_n(x)$ (Legendre)	via recurrence

Table 30.1: Analytical derivatives for the main special functions. `Lucid`’s autograd registers each of these in the corresponding op’s `BackwardSpec`.

30.7 Numerical Precision

Special functions are where numerical precision matters most. We summarise the precision of `Lucid`’s implementations.

30.7.1 Precision goals

The target is ULP-level precision in FP32 across the input range. Achieved for: `erf`, `erfc`, `lgamma`, `digamma`, `j0`, `j1`, `i0`, `i1`, `expit`, `logit`, `log1p`, `expm1`.

Achieved within a few ULPs (loose): `erfinv` near $|y| = 1$, `trigamma` for negative arguments, the higher-order Bessel functions for very large arguments.

30.7.2 The vForce vs scalar discrepancy

Accelerate’s vForce primitives are usually 1 ULP from the scalar libm equivalents, but for `vverfinvf` the discrepancy is larger (typically 4–6 ULPs near $|y| = 1$). The test suite uses a relaxed tolerance for `erfinv`-using ops.

This is documented in Chapter 15 Section 15.6.1.

30.8 Backward Derivatives

Most special functions have well-known derivatives:

30.8.1 Higher-order derivatives

The polygamma functions form a natural higher-order family: the n -th derivative of $\log \Gamma$. `Lucid` exposes `digamma` ($n = 1$) and `trigamma` ($n = 2$) explicitly; `polygamma(n, x)` covers higher orders.

For an autograd chain that needs the second derivative of `lgamma`, the engine composes `digamma` and `trigamma` automatically.

30.9 Use Cases in ML

30.9.1 GELU activation

$\text{gelu}(x) = x * (1 + \text{erf}(x / \text{sqrt}(2))) / 2$. The error function is the integral of the standard normal density; `gelu` is the expectation of $x \cdot \mathbb{K}[Z \leq x]$ for $Z \sim \mathcal{N}(0, 1)$, a smoothed alternative to ReLU that has become the default in modern Transformers.

30.9.2 Gaussian quantile

The inverse CDF of $\mathcal{N}(0, 1)$ is $\sqrt{2} \operatorname{erfinv}(2u - 1)$. Used in inverse-CDF sampling (Chapter 31).

30.9.3 Beta and Dirichlet likelihoods

The Beta and Dirichlet densities involve Γ :

$$p(x; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}.$$

The log-likelihood uses `lgamma`; its gradient with respect to α involves `digamma`.

30.9.4 Kaiser window

The Kaiser window's shape comes from I_0 , the modified Bessel of order zero. Lucid's `kaiser` window calls `i0` directly.

30.10 Implementation Notes

The implementations live in `lucid/special/` (Python) and in `lucid/_C/ops/ufunc/` (C++ where the `op` is a primitive).

30.10.1 Composite vs primitive

`erf`, `erfc`, `lgamma`, `digamma`, `i0`, `i1` are primitives, because they appear in hot paths (GELU, distribution log-likelihoods).

`erfinv`, `trigamma`, `polygamma`, the higher-order Bessel and Legendre functions are composites or thin wrappers around primitives.

30.10.2 Dispatch

Special-function primitives dispatch through the standard backend mechanism (Chapter 9). The CPU path goes through Accelerate's `vForce`; the GPU path uses MLX's primitives. The user does not see the difference.

30.11 The Gamma Function in Depth

The gamma function Γ generalises the factorial:

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt, \quad \Re(z) > 0.$$

Its key properties:

- Recursion: $\Gamma(z + 1) = z \Gamma(z)$, hence $\Gamma(n + 1) = n!$ for non-negative integer n .
- Reflection: $\Gamma(z)\Gamma(1 - z) = \pi / \sin(\pi z)$.
- Stirling asymptotic: $\Gamma(z) \sim \sqrt{2\pi/z} (z/e)^z$ for large $|z|$.
- Special values: $\Gamma(1/2) = \sqrt{\pi}$, $\Gamma(3/2) = \sqrt{\pi}/2$.

30.11.1 The log-gamma identity

A practical identity for ML log-likelihoods, the *log-Beta* function:

$$\log B(\alpha, \beta) = \log \Gamma(\alpha) + \log \Gamma(\beta) - \log \Gamma(\alpha + \beta).$$

This appears in the log-likelihood of the Beta distribution and in the Dirichlet's normalisation constant. Lucid's `lgamma` implementation is what these depend on.

30.11.2 The digamma and harmonic numbers

The digamma function $\psi(z) = \Gamma'(z)/\Gamma(z)$ relates to the harmonic numbers:

$$\psi(n+1) = -\gamma + \sum_{k=1}^n \frac{1}{k},$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant. At half-integer arguments,

$$\psi(1/2) = -\gamma - 2 \ln 2.$$

These identities appear in entropy formulas for Gamma and Beta distributions.

30.11.3 Reflection identity in practice

The reflection identity lets the implementation compute Γ for negative arguments via positive ones:

$$\Gamma(-z) = \frac{-\pi}{z \Gamma(z) \sin(\pi z)} \quad (z > 0, z \notin \mathbb{Z}).$$

Lucid’s `lgamma` uses this internally; the user sees the unified surface.

30.12 The Error Function in Depth

The error function

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

is the cumulative distribution of a half-Gaussian, scaled. Properties:

- Odd: $\operatorname{erf}(-x) = -\operatorname{erf}(x)$.
- Bounds: $-1 < \operatorname{erf}(x) < 1$.
- Derivative: $\operatorname{erf}'(x) = 2e^{-x^2}/\sqrt{\pi}$.
- Asymptotic: $\operatorname{erf}(x) \rightarrow 1$ as $x \rightarrow \infty$, with $1 - \operatorname{erf}(x) \sim e^{-x^2}/(x\sqrt{\pi})$.

30.12.1 The standard normal CDF

The CDF of the standard normal $\mathcal{N}(0, 1)$ is

$$\Phi(x) = \frac{1}{2}(1 + \operatorname{erf}(x/\sqrt{2})).$$

This identity is what makes GELU’s smooth-Gaussian form expressible as

$$\operatorname{GELU}(x) = x \cdot \Phi(x) = \frac{x}{2}(1 + \operatorname{erf}(x/\sqrt{2})).$$

30.12.2 The complementary error function for tails

For $x \gtrsim 4$ in FP32, $1 - \operatorname{erf}(x)$ loses precision through cancellation. The complementary form $\operatorname{erfc}(x) = 1 - \operatorname{erf}(x)$ is implemented directly:

$$\operatorname{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt.$$

For Gaussian tail probabilities ($P(Z > z)$ for $Z \sim \mathcal{N}(0, 1)$), the recommended formula is

$$P(Z > z) = \frac{1}{2} \operatorname{erfc}(z/\sqrt{2}),$$

which preserves precision even at $z \gtrsim 5$.

30.13 Bessel Functions in Depth

The modified Bessel function of the first kind, order ν , is the function I_ν satisfying

$$x^2 y'' + x y' - (x^2 + \nu^2) y = 0.$$

For $\nu = 0$:

$$I_0(x) = \sum_{k=0}^{\infty} \frac{(x/2)^{2k}}{(k!)^2} = \frac{1}{\pi} \int_0^\pi e^{x \cos \theta} d\theta.$$

30.13.1 Asymptotic forms

For $x \rightarrow \infty$,

$$I_\nu(x) \sim \frac{e^x}{\sqrt{2\pi x}} \left(1 - \frac{4\nu^2 - 1}{8x} + O(x^{-2}) \right).$$

The e^x factor causes overflow in FP32 for $x \gtrsim 90$. The exponentially-scaled variant

$$I_\nu^e(x) = I_\nu(x) \cdot e^{-|x|}$$

removes the exponential and is finite throughout. `Lucid` exposes `i0e`, `i1e` for these scaled forms.

30.13.2 Recurrence relations

The orders satisfy

$$I_{\nu-1}(x) - I_{\nu+1}(x) = \frac{2\nu}{x} I_\nu(x),$$

and derivatives:

$$I'_\nu(x) = \frac{1}{2} (I_{\nu-1}(x) + I_{\nu+1}(x)).$$

The recurrence allows computing higher orders from lower ones; `Lucid` exposes only $\nu = 0, 1$ directly (the most common in applications) and the user composes higher orders if needed.

30.14 Orthogonal Polynomials in Depth

Each orthogonal polynomial family satisfies a three-term recurrence that makes evaluation efficient.

30.14.1 Legendre polynomials

Defined on $[-1, 1]$ with weight $w(x) = 1$:

$$\begin{aligned} P_0(x) &= 1, & P_1(x) &= x, \\ (n+1)P_{n+1}(x) &= (2n+1)xP_n(x) - nP_{n-1}(x). \end{aligned}$$

Orthogonality:

$$\int_{-1}^1 P_m(x) P_n(x) dx = \frac{2}{2n+1} \delta_{mn}.$$

30.14.2 Hermite polynomials

The probabilist's form on $\mathbb{R}$ with weight $w(x) = e^{-x^2/2}$:

$$\begin{aligned} He_0(x) &= 1, & He_1(x) &= x, \\ He_{n+1}(x) &= x He_n(x) - n He_{n-1}(x). \end{aligned}$$

These appear in Hermite-Gauss quadrature and in series expansions of Gaussian processes.

30.14.3 Chebyshev polynomials

First kind, on $[-1, 1]$ with weight $w(x) = 1/\sqrt{1-x^2}$:

$$\begin{aligned} T_0(x) &= 1, & T_1(x) &= x, \\ T_{n+1}(x) &= 2xT_n(x) - T_{n-1}(x). \end{aligned}$$

Chebyshev polynomials are optimal in the minimax sense for polynomial approximation; they underlie the Chebyshev iteration in iterative linear solvers.

30.15 The Beta Function and Beta Distribution

The Beta function

$$B(\alpha, \beta) = \int_0^1 t^{\alpha-1} (1-t)^{\beta-1} dt = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}.$$

appears in the normalisation of the Beta distribution

$$p(x; \alpha, \beta) = \frac{x^{\alpha-1} (1-x)^{\beta-1}}{B(\alpha, \beta)}, \quad x \in (0, 1).$$

The Beta function's log form uses `lgamma`:

$$\log B(\alpha, \beta) = \log \Gamma(\alpha) + \log \Gamma(\beta) - \log \Gamma(\alpha + \beta).$$

The Beta log-likelihood gradient with respect to α is

$$\frac{\partial}{\partial \alpha} \log p(x; \alpha, \beta) = \log x - \psi(\alpha) + \psi(\alpha + \beta),$$

involving the digamma function.

30.15.1 The incomplete Beta

`betainc(a, b, x)` returns the regularised incomplete Beta function

$$I_x(a, b) = \frac{1}{B(a, b)} \int_0^x t^{a-1} (1-t)^{b-1} dt,$$

which is the CDF of the Beta distribution. `betaincinv` is the inverse, used in confidence-interval and quantile computations.

30.16 Numerical Precision Summary

Table 30.2 summarises Lucid's per-function precision targets.

30.17 Composite vs Primitive Cost

For the ops implemented as composites (orthogonal polynomials via recurrence, higher-order Bessel via downward recurrence), the per-call cost is dominated by the recurrence depth. Measured per-call costs on M3 Max for 10^6 -element FP32 tensors:

30.18 Summary and Forward References

`lucid.special` provides the standard family of special functions used in ML: error functions, gamma family, Bessel family, orthogonal polynomials. The implementations target ULP-level FP32 precision where possible and document the few cases (`erfinv` near $|y| = 1$, higher-order Bessels at large arguments) where the precision is more loose.

The next chapter, Chapter 31, treats the probability-distribution subpackage that uses several of these functions internally.

Function	FP32 precision	Notes
<code>erf</code> , <code>erfc</code>	≤ 2 ULP	throughout $\mathbb{R}$
<code>erfinv</code>	≤ 5 ULP	loose near $ y = 1$
<code>lgamma</code>	≤ 3 ULP	all $x > 0$; reflection for $x < 0$
<code>digamma</code>	≤ 4 ULP	loose near poles
<code>i0</code> , <code>i1</code>	≤ 3 ULP	full range with $i0e/i1e$ for large $ x $
<code>j0</code> , <code>j1</code>	≤ 5 ULP	looser due to oscillation
<code>legendre(n, x)</code>	$\leq n + 2$ ULP	recurrence amplifies error
<code>betainc</code>	≤ 6 ULP	convergence-dependent

Table 30.2: Approximate precision targets for Lucid’s special-function implementations. The oscillating Bessel functions and the high-order Legendre polynomials are looser; the dominant entropy-likelihood family (`lgamma`, `digamma`, `erf`) is tight to a few ULPs.

Function	Path	Time (ms)
<code>erf</code>	primitive (MLX/vForce)	0.6
<code>lgamma</code>	primitive	1.1
<code>i0</code>	primitive	1.4
<code>legendre(8, x)</code>	composite (8 recurrences)	6.7
<code>legendre(64, x)</code>	composite (64 recurrences)	52
<code>betainc(2, 3, x)</code>	composite	15

Table 30.3: Per-call timings for representative special functions on M3 Max with 10^6 -element FP32 inputs. The primitive ops are bandwidth-bound; the composite ops scale with the recurrence depth.

Note

For the user: `lucid.special` is the namespace; most ops are called as `lucid.special.erf(x)`, `lucid.special.lgamma(x)`, and similar. The autograd machinery treats each as a primitive, with the analytical derivatives registered.

CHAPTER 31

Probability Distributions

Probability distributions are the mathematical objects of Bayesian inference, generative modeling, variational autoencoders, and reinforcement learning. Lucid exposes a comprehensive distributions subpackage at `lucid.distributions` that provides parameterised distribution objects with sample, density, log-density, KL divergence, and entropy methods.

This chapter describes the subpackage’s design, the supported distributions, the reparameterisation trick for differentiable sampling, and the KL-divergence registry.

31.1 Overview

The subpackage follows the design of reference framework’s `torch.distributions` and TensorFlow Probability: a `Distribution` abstract base, parameterised concrete subclasses, and a fluent interface for sample / `log_prob` / mean / variance / entropy / KL.

31.1.1 The 33 distributions

Lucid supports 33 distributions across four categories:

- **Continuous:** Normal, Laplace, Cauchy, StudentT, Gumbel, Exponential, Gamma, Beta, Dirichlet, LogNormal, Weibull, Pareto, HalfNormal, HalfCauchy, Uniform, VonMises.
- **Discrete:** Bernoulli, Binomial, Categorical, OneHotCategorical, Poisson, Geometric, NegativeBinomial, Multinomial.
- **Multivariate:** MultivariateNormal, Wishart, LKJCholesky, Dirichlet.
- **Relaxed / continuous discrete:** RelaxedBernoulli, RelaxedOneHotCategorical.

- **Transformed / mixture**: `TransformedDistribution`, `MixtureSameFamily`, `Independent`.

31.2 The Distribution Interface

Every concrete distribution implements:

- `sample(shape=())`: draw a sample of the requested batch shape.
- `rsample(shape=())`: differentiable sample (for reparameterisable distributions).
- `log_prob(value)`: log probability density / mass at `value`.
- `cdf(value)`: cumulative distribution function (for those that have closed-form CDF).
- `icdf(value)`: inverse CDF.
- `mean`, `variance`, `stddev`, `entropy`: properties / methods returning the standard statistics.

31.2.1 Shape conventions

A distribution has three shape conceptions:

- **batch_shape**: independent distributions arranged in a batch. For a Normal with mean shape $(B,)$ and stddev shape $(B,)$, the batch shape is $(B,)$.
- **event_shape**: the shape of a single draw. For a scalar Normal, `event_shape` is $()$. For `MultivariateNormal` of dimension D , `event_shape` is $(D,)$.
- **sample_shape**: requested shape of the sample; prepended to `batch_shape` and `event_shape`.

`sample(shape=(K,))` of a Normal with batch shape $(B,)$ returns shape (K,B) .
`sample(shape=(K, M))` of `MultivariateNormal` with batch shape $()$ and event shape $(D,)$ returns shape (K,M,D) .

31.2.2 The Independent wrapper

`Independent(base, n)` reinterprets the last n dimensions of `base`'s `batch_shape` as `event_shape`. Useful when the user has a vector of independent scalar Normals and wants `log_prob` to sum across them rather than return a per-element tensor:

```

1 import lucid
2 from lucid.distributions import Normal, Independent
3
4 # Batch of D-dim diagonal Normals
5 mu = lucid.zeros(B, D)
6 sigma = lucid.ones(B, D)
7 base = Normal(mu, sigma)           # batch (B, D), event ()
8 dist = Independent(base, 1)        # batch (B,), event (D,)
9
10 x = dist.sample()                  # shape (B, D)
11 lp = dist.log_prob(x)              # shape (B,), summed over D

```

31.3 The Reparameterisation Trick

For training a model whose loss includes an expectation over a distribution (e.g., VAE evidence lower bound), the gradient of the expectation with respect to the distribution's parameters needs to be computable. The reparameterisation trick is the standard mechanism.

31.3.1 The trick

Suppose a sample $x \sim p_\theta$ can be written as $x = g_\theta(\epsilon)$ where ϵ comes from a fixed, parameter-free distribution q . Then

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \nabla_\theta \mathbb{E}_{\epsilon \sim q}[f(g_\theta(\epsilon))] = \mathbb{E}_{\epsilon \sim q}[\nabla_\theta f(g_\theta(\epsilon))].$$

The expectation and gradient commute because q does not depend on θ . The gradient passes through the deterministic g_θ ; the stochastic ϵ is a constant from autograd's view.

For Gaussian:

$$x \sim \mathcal{N}(\mu, \sigma^2) \iff x = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1).$$

The gradients are then immediate: $\partial x / \partial \mu = 1$ and $\partial x / \partial \sigma = \epsilon$. Autograd handles both automatically.

For other reparameterisable distributions the transform is analogous: Exponential(λ) as $x = -\log(u)/\lambda$ with $u \sim \mathcal{U}(0, 1)$; Uniform(a, b) as $x = a + (b - a)u$; etc.

31.3.2 rsample

`rsample()` returns a reparameterised sample for distributions that admit the trick. The implementation calls `torch.randn`-style noise and applies the differentiable transformation.

Distributions that support `rsample`: Normal, Uniform, Laplace, Gumbel, Exponential, Pareto, LogNormal, StudentT, HalfNormal, HalfCauchy, Cauchy, Beta, Dirichlet, Gamma, MultivariateNormal.

`sample` (non-r) is the version that produces a sample without the reparameterisation graph; useful when gradients are not needed.

31.3.3 Score-function estimator

For non-reparameterisable distributions (discrete distributions, some heavy-tailed continuous), the score-function estimator (REINFORCE) is the alternative:

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \mathbb{E}_{x \sim p_\theta}[f(x) \nabla_\theta \log p_\theta(x)].$$

Lucid does not provide a built-in score-function gradient helper, but the user can implement it explicitly:

```

1 x = dist.sample()           # non-differentiable sample
2 log_p = dist.log_prob(x)    # differentiable in dist's params
3 loss = -(reward * log_p).mean() # score-function objective

```

The pattern is standard for policy gradient in RL.

31.4 KL Divergence

The Kullback–Leibler divergence

$$D_{\text{KL}}(p \parallel q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right] = \int p(x) \log \frac{p(x)}{q(x)} dx$$

measures the directed distance from p to q . It is non-negative, zero iff $p = q$ almost everywhere, asymmetric in its arguments, and unbounded above. It is the standard objective in variational inference, the regulariser in β -VAE, and the core quantity in information theory's notion of inefficient coding.

For two univariate Gaussians the closed form is

$$D_{\text{KL}}(\mathcal{N}(\mu_1, \sigma_1^2) \parallel \mathcal{N}(\mu_2, \sigma_2^2)) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}.$$

For two categorical distributions over the same support,

$$D_{\text{KL}}(p \parallel q) = \sum_i p_i \log \frac{p_i}{q_i}.$$

The first form appears in every VAE loss; the second in distillation, label-smoothing, and policy-gradient regularisation.

31.4.1 Closed-form KLs

For many pairs (p, q) of common distributions, the KL has a closed-form expression that avoids the expensive Monte-Carlo estimate. Lucid maintains a registry of these in `lucid/distributions/kl.py`:

- Normal – Normal: closed form involving mean diff and log-stddev ratio.
- Categorical – Categorical: $\sum_i p_i \log(p_i/q_i)$.
- Bernoulli – Bernoulli: closed form.
- Gamma – Gamma: closed form involving lgamma and digamma.
- Beta – Beta: closed form.
- Dirichlet – Dirichlet: closed form.
- MultivariateNormal – MultivariateNormal: closed form involving log-det and trace.

The registry has 20 pairs as of Lucid 3.5.

31.4.2 Monte-Carlo fallback

For pairs not in the registry, `kl_divergence(p, q)` falls back to an MC estimate:

$$\widehat{D}_{\text{KL}}(p \parallel q) = \frac{1}{N} \sum_{i=1}^N \log \frac{p(x_i)}{q(x_i)}, \quad x_i \sim p.$$

The MC estimate is noisy; the user should average over many samples for a meaningful loss.

31.4.3 Registering a custom KL

The user can register a custom closed-form KL via a decorator:

```
1 from lucid.distributions import register_kl, MyDist
2
3 @register_kl(MyDist, MyDist)
4 def _kl_mydist_mydist(p, q):
5     return ... # closed-form expression
```

`kl_divergence(p, q)` will then pick the registered implementation.

31.5 Worked Example: VAE Loss

The Variational Autoencoder's loss is the evidence lower bound (ELBO):

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\text{KL}}(q(z|x) \parallel p(z)).$$

In Lucid:

```

1 import lucid
2 from lucid.distributions import Normal, kl_divergence
3
4 # Encoder produces mean and log-stddev of  $q(z/x)$ 
5 mu, log_sigma = encoder(x)
6 sigma = log_sigma.exp()
7 q_z = Normal(mu, sigma)
8
9 # Sample  $z$  via reparameterisation
10 z = q_z.rsample()
11
12 # Decoder produces parameters of  $p(x/z)$ 
13 x_recon = decoder(z)
14 p_x_given_z = Normal(x_recon, lucid.tensor(0.1))
15 recon_loss = -p_x_given_z.log_prob(x).sum(dim=-1).mean()
16
17 # KL divergence to the prior  $p(z) = N(0, I)$ 
18 prior = Normal(lucid.zeros_like(mu), lucid.ones_like(sigma))
19 kl_loss = kl_divergence(q_z, prior).sum(dim=-1).mean()
20
21 loss = recon_loss + kl_loss
22 loss.backward()

```

The `rsample` makes the path from `loss` back to the encoder’s parameters differentiable. The closed-form Normal-Normal KL avoids MC noise.

31.6 Constraints

Distribution parameters have constraints (variances are positive, probabilities are in $[0, 1]$, simplex distributions sum to 1). Lucid exposes a `constraints` submodule that names these constraints and a `transforms` submodule that bijects between constrained and unconstrained spaces.

31.6.1 Why constraints matter

Optimising a positive parameter directly (e.g., a Gamma rate) risks the optimiser producing a negative update that the distribution then rejects with NaN. The standard remedy is to parameterise via the unconstrained space and transform at the distribution’s input.

```

1 import lucid
2 from lucid.distributions import Gamma
3 from lucid.distributions.transforms import ExpTransform
4
5 # Parameterise log-rate as the unconstrained variable
6 log_rate = lucid.tensor(0.0, requires_grad=True)
7 rate = log_rate.exp() # always positive
8
9 dist = Gamma(rate, 1.0)
10 # Optimise log_rate freely; rate stays in (0, infity).

```

31.6.2 TransformedDistribution

`TransformedDistribution(base, transform)` composes a base distribution with a bijective transform. The `log_prob` correctly accounts for the Jacobian:

$$\log p_Y(y) = \log p_X(\text{inv}(y)) - \log |J_T(\text{inv}(y))|.$$

This is the mechanism for normalising flows: a chain of transformed distributions, each transform learnable.

31.7 Implementation Notes

The distributions live in `lucid/distributions/`. Each distribution is a Python class deriving from `Distribution`.

31.7.1 Numpy-free

Per Hard Rule H4, no distribution code imports `numpy`. The implementations use only `Lucid` primitives and special-function calls. Erfinv-based Gaussian sampling on CPU, `Lucid`'s Ziggurat on GPU.

31.7.2 Composition with optim

The distributions interact with the optimisers (Chapter 39) through their parameters. A `Distribution` created from `requires_grad=True` tensors backwards through both the sample and the `log_prob`; the optimiser updates the underlying tensors and the next sample reflects the change.

31.8 Univariate Continuous Distributions in Detail

The most-used continuous distributions, with their densities, log-likelihoods, and gradient identities.

31.8.1 Normal

Density:

$$p(x; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Log-likelihood:

$$\log p(x; \mu, \sigma) = -\frac{1}{2} \log(2\pi) - \log \sigma - \frac{(x - \mu)^2}{2\sigma^2}.$$

Gradients:

$$\frac{\partial \log p}{\partial \mu} = \frac{x - \mu}{\sigma^2}, \quad \frac{\partial \log p}{\partial \sigma} = -\frac{1}{\sigma} + \frac{(x - \mu)^2}{\sigma^3}.$$

Entropy: $H = \frac{1}{2} \log(2\pi e \sigma^2)$.

31.8.2 Gamma

Density (rate parameterisation, α shape, β rate):

$$p(x; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad x > 0.$$

Log-likelihood:

$$\log p = \alpha \log \beta - \log \Gamma(\alpha) + (\alpha - 1) \log x - \beta x.$$

Gradients (involving the digamma function ψ):

$$\frac{\partial \log p}{\partial \alpha} = \log \beta - \psi(\alpha) + \log x, \quad \frac{\partial \log p}{\partial \beta} = \frac{\alpha}{\beta} - x.$$

Mean: α/β . Variance: α/β^2 .

31.8.3 Beta

Density:

$$p(x; \alpha, \beta) = \frac{1}{B(\alpha, \beta)} x^{\alpha-1} (1-x)^{\beta-1}, \quad x \in (0, 1).$$

Mean: $\alpha/(\alpha+\beta)$. The Beta is the conjugate prior for the Bernoulli likelihood; this is the source of its prominence in Bayesian inference.

31.8.4 Dirichlet

The multivariate generalisation of Beta, defined on the $(K-1)$ -simplex:

$$p(x; \alpha) = \frac{1}{B(\alpha)} \prod_{i=1}^K x_i^{\alpha_i-1}, \quad x_i \geq 0, \quad \sum_i x_i = 1,$$

where $B(\alpha) = \prod_i \Gamma(\alpha_i) / \Gamma(\sum_i \alpha_i)$. Conjugate prior for categorical likelihoods.

31.8.5 Exponential and related

Exponential with rate λ :

$$p(x; \lambda) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Memoryless. Maximum-entropy distribution given a fixed mean.

Laplace (double exponential):

$$p(x; \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x - \mu|}{b}\right).$$

Cauchy (heavy-tailed, infinite mean/variance):

$$p(x; \mu, \gamma) = \frac{1}{\pi\gamma [1 + ((x - \mu)/\gamma)^2]}.$$

31.9 Discrete Distributions in Detail

31.9.1 Bernoulli

Single trial with success probability p :

$$P(X = 1) = p, \quad P(X = 0) = 1 - p.$$

Log-likelihood:

$$\log P(X = x) = x \log p + (1 - x) \log(1 - p).$$

The sigmoid output of a logistic regression model is naturally the parameter of a Bernoulli.

31.9.2 Categorical

The discrete distribution over K classes with probability vector π :

$$P(X = k) = \pi_k, \quad k = 1, \dots, K, \quad \sum_k \pi_k = 1.$$

The softmax output of a multi-class classifier is naturally π . The cross-entropy loss is the negative log-likelihood:

$$\mathcal{L} = -\log \pi_y = -y^\top \log \pi$$

(where y is the one-hot label).

31.9.3 Binomial and Poisson

Binomial: number of successes in n Bernoulli trials:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Poisson: count of events in fixed interval with mean rate λ :

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}.$$

Poisson is the limit of Binomial with $n \rightarrow \infty$, $p \rightarrow 0$, $np = \lambda$.

31.10 Multivariate Distributions

31.10.1 Multivariate Normal

The density of $\mathcal{N}(\mu, \Sigma)$ on $\mathbb{R}^d$:

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1} (x - \mu)\right).$$

Log-likelihood (the form used computationally):

$$\log p(x) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu).$$

Lucid supports three parameterisations of Σ :

- `covariance_matrix`: full Σ .
- `precision_matrix`: Σ^{-1} .
- `scale_tril`: Cholesky factor L with $\Sigma = LL^\top$.

The Cholesky parameterisation is the most numerically robust and the default in most tutorials.

KL between two MVNs:

$$D_{\text{KL}}(\mathcal{N}(\mu_1, \Sigma_1) \parallel \mathcal{N}(\mu_2, \Sigma_2)) = \frac{1}{2} \left[\text{tr}(\Sigma_2^{-1} \Sigma_1) + (\mu_2 - \mu_1)^\top \Sigma_2^{-1} (\mu_2 - \mu_1) - d + \log \frac{|\Sigma_2|}{|\Sigma_1|} \right].$$

31.10.2 Wishart and inverse Wishart

The Wishart is the conjugate prior for the precision matrix of a multivariate Normal in Bayesian inference. The inverse Wishart is the prior for the covariance matrix.

31.10.3 LKJ-Cholesky

The LKJ distribution is a prior over correlation matrices. The Cholesky parameterisation `LKJCholesky(d, concentration)` returns the Cholesky factor of a correlation matrix.

31.11 Variational Inference: A Worked Example

To illustrate the distribution machinery in context, we walk through a variational autoencoder (VAE) loss derivation [24].

31.11.1 The setup

A latent-variable model has the form $p(x, z) = p(x|z)p(z)$ with prior $p(z) = \mathcal{N}(0, I)$ and likelihood $p(x|z)$ parameterised by a neural network. The marginal likelihood $p(x) = \int p(x|z)p(z) dz$ is intractable.

The variational approximation introduces $q(z|x)$, also parameterised by a neural network (the encoder), and considers the evidence lower bound (ELBO):

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\text{KL}}(q(z|x) \| p(z)).$$

31.11.2 The ELBO terms

The reconstruction term $\mathbb{E}_{q(z|x)}[\log p(x|z)]$ is estimated by sampling $z \sim q(z|x)$ (via reparameterisation for differentiability) and computing $\log p(x|z)$.

The KL term has closed form for the case $q(z|x) = \mathcal{N}(\mu(x), \text{diag}\sigma^2(x))$ and $p(z) = \mathcal{N}(0, I)$:

$$D_{\text{KL}}(q \| p) = \frac{1}{2} \sum_{i=1}^d (\mu_i^2 + \sigma_i^2 - 1 - \log \sigma_i^2).$$

This is the famous “KL divergence to standard normal” term that appears in every VAE training loss.

31.12 Score-Function Estimator

For distributions that do not admit reparameterisation (discrete distributions, certain heavy-tailed continuous), the score-function estimator (REINFORCE) is the alternative:

$$\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}}[f(x)] = \mathbb{E}_{x \sim p_{\theta}}[f(x) \nabla_{\theta} \log p_{\theta}(x)].$$

31.12.1 Variance

The score-function estimator is unbiased but has high variance. The standard variance-reduction trick uses a baseline b that does not depend on x :

$$\nabla_{\theta} \mathbb{E}[f(x)] = \mathbb{E}[(f(x) - b) \nabla_{\theta} \log p_{\theta}(x)].$$

The expectation is unchanged (since $\mathbb{E}[\nabla \log p_{\theta}] = 0$), but the variance can be reduced by choosing b near $\mathbb{E}[f(x)]$.

This is the algorithmic basis of REINFORCE with a baseline, the ancestor of modern policy-gradient algorithms in RL.

31.12.2 Implementation pattern

```

1  # Forward
2  dist = Categorical(logits=policy_logits)
3  action = dist.sample()
4  log_p = dist.log_prob(action)
5  reward = env.step(action)    # non-differentiable
6
7  # Score-function gradient
8  baseline = reward.mean().detach()
9  loss = -((reward - baseline) * log_p).mean()
10 loss.backward()
```

31.13 Transformed Distributions and Normalising Flows

`TransformedDistribution(base, transform)` composes a base distribution with a bijective transform. By the change-of-variables formula:

$$\log p_Y(y) = \log p_X(T^{-1}(y)) - \log |J_T(T^{-1}(y))|,$$

where J_T is the Jacobian of T . `Lucid` supports several transforms:

- `AffineTransform`: $y = ax + b$.
- `ExpTransform`: $y = e^x$.
- `SigmoidTransform`: $y = \sigma(x)$.
- `TanhTransform`: $y = \tanh(x)$.
- `StickBreakingTransform`: maps $\mathbb{R}^{K-1}$ to the K -simplex.

31.13.1 Normalising flows

A *normalising flow* chains several transforms:

$$y = T_K \circ T_{K-1} \circ \cdots \circ T_1(x), \quad x \sim p_X.$$

The resulting density is

$$\log p_Y(y) = \log p_X(x) - \sum_{k=1}^K \log |J_{T_k}|.$$

The transforms are chosen so that both T and T^{-1} and $\log |J|$ are tractable. Common choices: coupling layers (RealNVP), autoregressive transforms (MAF, IAF), spline-based transforms (neural spline flows).

`Lucid` does not currently ship a flow library out of the box, but the building blocks (`TransformedDistribution` plus the transforms) make implementation straightforward.

31.14 Constraint and Transform Machinery

The `constraints` submodule names common parameter constraints:

- `positive`: $\theta > 0$.
- `interval(a, b)`: $a < \theta < b$.
- `simplex`: $\theta_i \geq 0$, $\sum_i \theta_i = 1$.
- `positive_definite`: matrix is symmetric and PD.
- `lower_cholesky`: lower-triangular with positive diagonal.

The `biject_to(constraint)` function returns a transform that maps unconstrained $\mathbb{R}^n$ to the constrained space:

- For `positive`: `ExpTransform()`.
- For `interval(a, b)`: scaled sigmoid.
- For `simplex`: `StickBreakingTransform()`.

This pattern lets the user optimise an unconstrained variable and have the constraint enforced by the transform.

31.15 Summary and Forward References

`lucid.distributions` provides 33 parameterised probability-distribution classes with `sample` / `log_prob` / `entropy` / `KL` methods. The reparameterisation trick supports differentiable sampling for the 15 distributions that admit it; the score-function estimator is the alternative for the rest. The KL-divergence registry contains 20 closed-form pairs with a Monte-Carlo fallback.

The next chapter, Chapter 32, closes Part V with the tensor-rearrangement algebra. Part VI then begins the neural-network library proper.

Note

For the user: `lucid.distributions` is parity-compatible with reference framework's `torch.distributions`. Most patterns (`rsample` for differentiable, `log_prob` for likelihood, `kl_divergence` for variational losses) work identically.

CHAPTER 32

Einops: Tensor Rearrangement Algebra

Einops is a small DSL for tensor rearrangement [26], originally developed as an external library by Alex Rogozhnikov and adopted into the PyTorch and JAX ecosystems. The DSL replaces several common patterns of `reshape`, `transpose`, and `view` with single readable expressions that name each axis explicitly. Lucid provides a numpy-free implementation at `lucid.einops`.

This chapter describes the three operations `rearrange`, `reduce`, and `repeat`, the syntax of the einops expressions, and the implementation considerations.

32.1 Why Einops

The motivating observation: code like `x.transpose(0, 2).reshape(B, T * D)` hides what's happening. The reader must work out which axes were transposed, what the output shape means, and whether the reshape is contiguous.

The einops alternative: `rearrange(x, 'b d t -> b (t d)')`.

The string is a transformation rule: the input has axes `b`, `d`, `t`; the output has axes `b` and `(t d)`, which means the trailing axis is the product of `t` and `d` (in that order). The reader sees both the input structure and the output structure explicitly.

32.1.1 The H8 namespace rule

Per Hard Rule H8, einops ops are accessible only via `lucid.einops.*`, never at the top level (`lucid.rearrange` is invalid) and never as a Tensor method (`x.rearrange(...)` is invalid). The namespace separation matches `reference framework` and SciPy and disambiguates from the language-level `reshape`.

32.2 The Three Operations

`lucid.einops` exposes three operations:

- `rearrange(tensor, pattern, **axes_sizes)`: permute / reshape with an explicit axis-naming pattern.
- `reduce(tensor, pattern, reduction, **axes_sizes)`: rearrange and reduce along specified axes.
- `repeat(tensor, pattern, **axes_sizes)`: rearrange with axis duplication.

The `pattern` is always an arrow-separated string. The left side names the input axes; the right side names the output axes.

32.3 rearrange

`rearrange` handles pure permutations and reshapes.

32.3.1 Permutation

```
1 from lucid.einops import rearrange
2
3 x = lucid.randn(2, 3, 4) # shape (b, c, t)
4 y = rearrange(x, 'b c t -> b t c')
5 # y.shape == (2, 4, 3)
```

Equivalent to `x.permute(0, 2, 1)` but with named axes.

32.3.2 Reshape via grouping

Parentheses group axes into a single output axis:

```
1 y = rearrange(x, 'b c t -> b (c t)')
2 # y.shape == (2, 12) -- c and t merged
```

The order inside the parentheses matters: `(c t)` means `c` varies more slowly than `t` in the merged axis, which corresponds to a `reshape` of `(B, C, T)` to `(B, C*T)` where `T` is the inner axis.

32.3.3 Reshape via splitting

The inverse: split an axis using `axes_sizes`:

```
1 y = rearrange(x, 'b (c1 c2) t -> b c1 c2 t', c1=2)
2 # Splits the c axis (which was 3) ... wait, this errors.
3 # 3 is not divisible by 2.
```

The split is only valid when the named sub-axis sizes divide the combined axis. For `b (c1 c2) t` with `c1*c2 = c`, the user must supply either `c1` or `c2` explicitly; the other is inferred.

32.3.4 Combining

A single `rearrange` can combine permutation, splitting, and merging:

```
1 # Vision Transformer patch extraction:
2 # (B, C, H, W) -> (B, num_patches, patch_dim)
3 y = rearrange(x, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)',
4               p1=16, p2=16)
```


This single line replaces a `unfold + permute + reshape` sequence that would be three lines and harder to read.

32.4 reduce

`reduce` combines rearrangement with reduction:

```
1 from lucid.einops import reduce
2
3 # Mean-pool across H and W:
4 y = reduce(x, 'b c h w -> b c', 'mean')
5 # y.shape == (B, C), the per-channel mean
6
7 # Max-pool over patches of size 2x2:
8 y = reduce(x, 'b c (h p1) (w p2) -> b c h w', 'max', p1=2, p2=2)
```

The reduction is named: `'sum'`, `'mean'`, `'max'`, `'min'`, `'prod'`. The axes that disappear between input and output are the reduced ones.

32.4.1 vs explicit reduce

The plain reduction equivalent: `x.mean(dim=(2, 3))` for the first example, a more involved `unfold-plus-max` for the second. `reduce` expresses both as one named pattern.

32.5 repeat

`repeat` expands axes with named multiplication:

```
1 from lucid.einops import repeat
2
3 # Add a batch dim of size B by repeating:
4 x = lucid.randn(C, H, W)
5 y = repeat(x, 'c h w -> b c h w', b=8)
6 # y.shape == (8, C, H, W), with the c h w part identical across b
```

The `b` axis is new (not in the input); the size is given by the keyword argument. The data is virtually duplicated via stride-0 broadcasting; the user can pass through to a kernel that respects strides, or call `.contiguous()` to materialise.

32.6 Pattern Syntax

The pattern grammar is small but expressive.

32.6.1 Axis names

An axis name is an identifier: letters, digits, underscores; starts with a letter. Common conventions:

- `b`: batch.
- `c`: channels.
- `h`, `w`: height, width.
- `t`: time.
- `n`: a generic count.

The names have no semantic meaning to einops; they are just labels. The convention is mnemonic for the reader.

32.6.2 Grouping

(a b) means a single axis whose size is the product of a and b. The order matters: a is the slow axis, b is the fast.

32.6.3 Ellipsis

... matches a variable number of axes:

```
1 # Move the last axis to the front, regardless of rank:
2 y = rearrange(x, '... c -> c ...')
```

32.6.4 The 1 placeholder

The literal 1 represents a size-1 axis:

```
1 # Unsqueeze the second axis:
2 y = rearrange(x, 'b t -> b 1 t')
```

32.7 Backward Through Einops

All three einops operations are differentiable. The backward is the inverse pattern:

- `rearrange`'s backward applies the inverse permutation/reshape.
- `reduce`'s backward broadcasts the gradient back along the reduced axes (with the appropriate scaling for 'mean').
- `repeat`'s backward sums the gradient over the repeated axes.

The implementation composes existing Lucid primitives, so the autograd graph is the same as if the user had written the equivalent permute / sum / etc. sequence by hand.

32.8 Implementation

The implementation lives in `lucid/einops/` as pure Python. The parser converts the pattern string to a sequence of `reshape` / `permute` / `reduce` / `expand` operations on the input tensor.

32.8.1 Parser

The parser tokenises the pattern into axis names and parenthesised groups, validates the input and output sides for consistency (every input axis appears somewhere in the output, or is reduced), and produces a sequence of tensor ops.

The parser is small (a few hundred lines) and produces clear error messages for malformed patterns.

32.8.2 Caching

For a given (pattern, input shape, axes_sizes) tuple, the sequence of ops is deterministic. Lucid caches the parsed ops by these keys so that repeated calls in a training loop pay the parse cost only once.

The cache is a small dict keyed by hash of the inputs; size is bounded by an LRU policy (default 1024 entries, which covers any reasonable model).

32.9 Patterns in Practice

A few patterns that recur in ML.

32.9.1 Attention reshape

Multi-head attention partitions the channel dimension into h heads of d dimensions each:

```
1 # (B, T, H*D) -> (B, H, T, D)
2 qkv = rearrange(qkv, 'b t (h d) -> b h t d', h=n_heads)
```

32.9.2 Patch embedding (ViT)

Vision Transformer's patch extraction:

```
1 # (B, C, H, W) -> (B, (H/p)(W/p), p*p*C)
2 patches = rearrange(image, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)',
3                       p1=16, p2=16)
```

32.9.3 Channel-last conversion

```
1 # NCHW (PyTorch default) <-> NHWC (TF default)
2 y = rearrange(x, 'b c h w -> b h w c')
```

32.10 Comparison: Einops vs Native Operations

To make the readability claim concrete, several common operations expressed both ways.

32.10.1 Multi-head attention reshape

Going from $(B, T, H \cdot D)$ to (B, H, T, D) for multi-head attention:

```
1 # Native
2 qkv = qkv.view(B, T, H, D).permute(0, 2, 1, 3)
3
4 # Einops
5 qkv = rearrange(qkv, 'b t (h d) -> b h t d', h=H)
```

The native version requires the reader to recall that `view(B, T, H, D)` splits the last dim into H and D (and that H comes before D in the split), then permutes to put the heads in dim 1. The einops version makes both transformations visible.

32.10.2 Patch extraction (ViT)

```
1 # Native (using unfold)
2 patches = (image
3           .unfold(2, P, P).unfold(3, P, P)
4           .contiguous()
5           .view(B, C, n_patches_h, n_patches_w, P, P)
6           .permute(0, 2, 3, 4, 5, 1)
7           .reshape(B, n_patches_h * n_patches_w, P * P * C))
8
9 # Einops
10 patches = rearrange(image, 'b c (h p1) (w p2) -> b (h w) (p1 p2 c)',
11                      p1=P, p2=P)
```

The native version is six lines, four operations, and requires the reader to follow the dimension order through each step. The einops version is one line.

32.10.3 Channel-last conversion with batch

```

1 # Native
2 x = x.permute(0, 2, 3, 1).contiguous()
3
4 # Einops
5 x = rearrange(x, 'b c h w -> b h w c')
```

Native is shorter here, but the einops version makes the target layout explicit; the reader does not have to count the permutation indices.

32.11 Pattern Parsing in Detail

The parser converts a pattern string into a sequence of tensor operations. This section sketches the algorithm.

32.11.1 Tokenisation

The pattern is split on whitespace and parenthesisation. Each token is one of:

- An axis name (identifier).
- A group: parenthesised sequence of axis names.
- The ellipsis `...`.
- The literal `1`.

The input side and output side are separated by `->`.

32.11.2 Axis-size resolution

Each named axis must have a determined size at evaluation time. Sizes come from three sources:

- The input tensor's shape: an axis name appearing alone on the input side gets its size from the corresponding input dimension.
- Keyword arguments: `rearrange(x, '...', h=4)` sets the axis named `h` to size 4.
- Inference: in a group, one axis can be inferred from the others. `(h d)` for an input dim of size 12 with $h = 4$ given gives $d = 3$.

A pattern with unresolved axes raises a parse error.

32.11.3 Op sequence generation

The parser produces a sequence of tensor operations to transform the input into the output:

1. *Split* input groups: for each input `(a b)`, apply `reshape` that turns the combined dim into two.
2. *Permute*: reorder the dimensions so they appear in the output order.
3. *Reduce* (for `reduce`): apply the named reduction along the axes that disappear.
4. *Expand* (for `repeat`): apply `expand` or `repeat` to introduce new axes.

5. *Merge* output groups: for each output (a b), apply `reshape` that flattens.

The op sequence is cached by (pattern, input rank, axis sizes) tuple so that repeated calls in a training loop skip the parse.

32.12 Performance Considerations

32.12.1 Overhead per call

The first call with a new (pattern, shape) tuple pays the parse cost (a few hundred microseconds). Subsequent calls hit the cache in a few microseconds.

For training loops where the same patterns are called thousands of times per step, the cached path is fast enough to be negligible relative to the underlying kernel cost. For very small kernels (sub-microsecond), the einops overhead can be noticeable; for any non-trivial kernel, it is dominated.

32.12.2 When einops materialises

The op sequence is a chain of `reshape`, `permute`, `expand`, etc. Each is a view operation when possible. The only operation that always allocates is `repeat` (physical duplication), which the user can avoid by using `expand + care`.

32.12.3 Compose with autograd

Each underlying primitive is differentiable; the autograd graph is the same as if the user had written the equivalent native sequence. There is no extra cost or memory overhead beyond what the native sequence would incur.

32.13 Advanced Patterns

32.13.1 Pattern with ellipsis

... matches zero or more leading dimensions:

```
1 # Sum the last axis regardless of leading rank
2 y = reduce(x, '... c -> ...', 'sum')
```

32.13.2 Pattern with literal 1

The literal 1 represents a size-1 axis. Useful for explicit broadcasting:

```
1 # Insert a size-1 axis for broadcasting
2 expanded = rearrange(x, 'b c -> b c 1')
3 result = expanded + other
```

32.13.3 Pattern with anonymous axis

An underscore prefix marks an axis as anonymous (matches but is not constrained to a particular name):

```
1 # Move the last anonymous axis to first
2 y = rearrange(x, '... _final -> _final ...')
```

In practice, named axes are clearer; the anonymous form is rare.

Operation	Effect on shape	Use case
rearrange	no axis added or removed	permute, reshape, group, split
reduce	axes disappear (those on input only)	per-axis aggregation
repeat	axes appear (those on output only)	broadcast-like duplication

Table 32.1: The three einops operations and their effect on the tensor shape. The rule of thumb: an axis present on input only is reduced; present on output only is repeated; present on both (or in a group) is preserved.

32.14 The Three Operations: A Reference

Table 32.1 is the complete reference for `rearrange`, `reduce`, and `repeat`.

32.15 Common Bugs and Their Diagnostics

32.15.1 Bug: unresolvable size

`rearrange(x, 'b (h d) -> b h d')` for x of shape $(2, 12)$ requires one of h or d to be supplied. Without that, the parser raises `ParseError: cannot infer size of h or d`.

Fix: `rearrange(x, 'b (h d) -> b h d', h=4)`.

32.15.2 Bug: mismatched output axes

`rearrange(x, 'b c h w -> b c h')` drops w on the output side without a reduction. The parser raises `ParseError: axis 'w' appears on input but not on output (use reduce?)`.

Fix: use `reduce(x, 'b c h w -> b c h', 'mean')` or similar.

32.15.3 Bug: division-incompatible split

`rearrange(x, 'b (h d) -> b h d', h=3)` for x of shape $(2, 7)$: the dimension of size 7 cannot be split into 3 by d . The parser raises `ShapeError: 7 is not divisible by 3`.

32.16 Summary and Forward References

`lucid.einops` provides three operations (`rearrange`, `reduce`, `repeat`) with a string-based pattern syntax that names axes explicitly. The result is more readable than the equivalent `reshape` / `permute` / `sum` sequence; the implementation composes existing primitives and is automatically differentiable.

This chapter closes Part V. Part VI begins the neural-network library: modules, layers, attention, losses, optimisers, and AMP.

Note

For the user: `einops` is a convenience layer. Anything it does could be written with `reshape`, `permute`, and `sum` directly. `Einops` just makes the intent visible. For attention reshapes, patch embeddings, and channel-axis conversions, the readability win is substantial.

Part **VI**

Neural Network Library

CHAPTER 33

The Module System

Chapter 33 — The Module System. Multi-level scaffold below; prose to follow.

33.1 Overview

33.2 Architecture

33.2.1 Core building blocks

33.2.2 Variants and parameterizations

33.3 Forward Computation

33.3.1 Mathematical formulation

33.3.2 Implementation in Lucid

33.3.2.1 Module class layout

33.3.2.2 Backend dispatch

33.4 Training Considerations

33.4.1 Initialization strategy

33.4.2 Backward pass

33.4.3 Interaction with optimizer and AMP

33.5 Implementation Notes

33.5.1 File layout and class hierarchy

33.5.2 Edge cases and pitfalls

33.6 Worked Examples and Benchmarks

33.7 Summary and Cross-References

CHAPTER 34

Linear, Convolution, Pooling, Normalization

Chapter 34 — Linear, Convolution, Pooling, Normalization. Multi-level scaffold below; prose to follow.

34.1 Overview

34.2 Architecture

34.2.1 Core building blocks

34.2.2 Variants and parameterizations

34.3 Forward Computation

34.3.1 Mathematical formulation

34.3.2 Implementation in Lucid

34.3.2.1 Module class layout

34.3.2.2 Backend dispatch

34.4 Training Considerations

34.4.1 Initialization strategy

34.4.2 Backward pass

34.4.3 Interaction with optimizer and AMP

34.5 Implementation Notes

34.5.1 File layout and class hierarchy

34.5.2 Edge cases and pitfalls

34.6 Worked Examples and Benchmarks

34.7 Summary and Cross-References

CHAPTER 35

Recurrent Networks

Chapter 35 — Recurrent Networks. Multi-level scaffold below; prose to follow.

35.1 Overview

35.2 Architecture

35.2.1 Core building blocks

35.2.2 Variants and parameterizations

35.3 Forward Computation

35.3.1 Mathematical formulation

35.3.2 Implementation in Lucid

35.3.2.1 Module class layout

35.3.2.2 Backend dispatch

35.4 Training Considerations

35.4.1 Initialization strategy

35.4.2 Backward pass

35.4.3 Interaction with optimizer and AMP

35.5 Implementation Notes

35.5.1 File layout and class hierarchy

35.5.2 Edge cases and pitfalls

35.6 Worked Examples and Benchmarks

35.7 Summary and Cross-References

CHAPTER 36

Attention and SDPA Fusion

Chapter 36 — Attention and SDPA Fusion. Multi-level scaffold below; prose to follow.

36.1 Overview

36.2 Architecture

36.2.1 Core building blocks

36.2.2 Variants and parameterizations

36.3 Forward Computation

36.3.1 Mathematical formulation

36.3.2 Implementation in Lucid

36.3.2.1 Module class layout

36.3.2.2 Backend dispatch

36.4 Training Considerations

36.4.1 Initialization strategy

36.4.2 Backward pass

36.4.3 Interaction with optimizer and AMP

36.5 Implementation Notes

36.5.1 File layout and class hierarchy

36.5.2 Edge cases and pitfalls

36.6 Worked Examples and Benchmarks

36.7 Summary and Cross-References

CHAPTER 37

Loss Functions

Chapter 37 — Loss Functions. Multi-level scaffold below; prose to follow.

37.1 Overview

37.2 Architecture

37.2.1 Core building blocks

37.2.2 Variants and parameterizations

37.3 Forward Computation

37.3.1 Mathematical formulation

37.3.2 Implementation in Lucid

37.3.2.1 Module class layout

37.3.2.2 Backend dispatch

37.4 Training Considerations

37.4.1 Initialization strategy

37.4.2 Backward pass

37.4.3 Interaction with optimizer and AMP

37.5 Implementation Notes

37.5.1 File layout and class hierarchy

37.5.2 Edge cases and pitfalls

37.6 Worked Examples and Benchmarks

37.7 Summary and Cross-References

CHAPTER 38

Weight Initialization

Chapter 38 — Weight Initialization. Multi-level scaffold below; prose to follow.

38.1 Overview

38.2 Architecture

38.2.1 Core building blocks

38.2.2 Variants and parameterizations

38.3 Forward Computation

38.3.1 Mathematical formulation

38.3.2 Implementation in Lucid

38.3.2.1 Module class layout

38.3.2.2 Backend dispatch

38.4 Training Considerations

38.4.1 Initialization strategy

38.4.2 Backward pass

38.4.3 Interaction with optimizer and AMP

38.5 Implementation Notes

38.5.1 File layout and class hierarchy

38.5.2 Edge cases and pitfalls

38.6 Worked Examples and Benchmarks

38.7 Summary and Cross-References

CHAPTER 39

Optimizers

Chapter 39 — Optimizers. Multi-level scaffold below; prose to follow.

39.1 Overview

39.2 Architecture

39.2.1 Core building blocks

39.2.2 Variants and parameterizations

39.3 Forward Computation

39.3.1 Mathematical formulation

39.3.2 Implementation in Lucid

39.3.2.1 Module class layout

39.3.2.2 Backend dispatch

39.4 Training Considerations

39.4.1 Initialization strategy

39.4.2 Backward pass

39.4.3 Interaction with optimizer and AMP

39.5 Implementation Notes

39.5.1 File layout and class hierarchy

39.5.2 Edge cases and pitfalls

39.6 Worked Examples and Benchmarks

39.7 Summary and Cross-References

CHAPTER 40

Learning-Rate Schedulers

Chapter 40 — Learning-Rate Schedulers. Multi-level scaffold below; prose to follow.

40.1 Overview

40.2 Architecture

40.2.1 Core building blocks

40.2.2 Variants and parameterizations

40.3 Forward Computation

40.3.1 Mathematical formulation

40.3.2 Implementation in Lucid

40.3.2.1 Module class layout

40.3.2.2 Backend dispatch

40.4 Training Considerations

40.4.1 Initialization strategy

40.4.2 Backward pass

40.4.3 Interaction with optimizer and AMP

40.5 Implementation Notes

40.5.1 File layout and class hierarchy

40.5.2 Edge cases and pitfalls

40.6 Worked Examples and Benchmarks

40.7 Summary and Cross-References

CHAPTER 41

Mixed Precision and Gradient Scaling

Chapter 41 — Mixed Precision and Gradient Scaling. Multi-level scaffold below; prose to follow.

41.1 Overview

41.2 Architecture

41.2.1 Core building blocks

41.2.2 Variants and parameterizations

41.3 Forward Computation

41.3.1 Mathematical formulation

41.3.2 Implementation in Lucid

41.3.2.1 Module class layout

41.3.2.2 Backend dispatch

41.4 Training Considerations

41.4.1 Initialization strategy

41.4.2 Backward pass

41.4.3 Interaction with optimizer and AMP

41.5 Implementation Notes

41.5.1 File layout and class hierarchy

41.5.2 Edge cases and pitfalls

41.6 Worked Examples and Benchmarks

41.7 Summary and Cross-References

CHAPTER 42

Profiler and Determinism

Chapter 42 — Profiler and Determinism. Multi-level scaffold below; prose to follow.

42.1 Overview

42.2 Architecture

42.2.1 Core building blocks

42.2.2 Variants and parameterizations

42.3 Forward Computation

42.3.1 Mathematical formulation

42.3.2 Implementation in Lucid

42.3.2.1 Module class layout

42.3.2.2 Backend dispatch

42.4 Training Considerations

42.4.1 Initialization strategy

42.4.2 Backward pass

42.4.3 Interaction with optimizer and AMP

42.5 Implementation Notes

42.5.1 File layout and class hierarchy

42.5.2 Edge cases and pitfalls

42.6 Worked Examples and Benchmarks

42.7 Summary and Cross-References

Part **VII**

Graph Compilation

CHAPTER 43

Graph-Capture Design Goals

Chapter 43 — Graph-Capture Design Goals. Multi-level scaffold below; prose to follow.

43.1 Overview

43.2 Background and Goals

43.2.1 Problem statement

43.2.2 Constraints and prior art

43.3 Methodology

43.3.1 Measurement approach

43.3.2 Benchmark setup

43.3.2.1 Hardware configuration

43.3.2.2 Software stack and reproducibility

43.4 Implementation

43.4.1 Design decisions

43.4.2 Component breakdown

43.4.3 Integration with existing systems

43.5 Results

43.5.1 Quantitative findings

43.5.2 Qualitative observations

43.6 Discussion

43.6.1 Trade-offs

43.6.2 Open questions

43.7 Summary and Cross-References

CHAPTER 44

Tracer and TraceIR

Chapter 44 — Tracer and TraceIR. Multi-level scaffold below; prose to follow.

44.1 Overview

44.2 Background and Goals

44.2.1 Problem statement

44.2.2 Constraints and prior art

44.3 Methodology

44.3.1 Measurement approach

44.3.2 Benchmark setup

44.3.2.1 Hardware configuration

44.3.2.2 Software stack and reproducibility

44.4 Implementation

44.4.1 Design decisions

44.4.2 Component breakdown

44.4.3 Integration with existing systems

44.5 Results

44.5.1 Quantitative findings

44.5.2 Qualitative observations

44.6 Discussion

44.6.1 Trade-offs

44.6.2 Open questions

44.7 Summary and Cross-References

CHAPTER 45

MpsBuilder and Executable Cache

Chapter 45 — MpsBuilder and Executable Cache. Multi-level scaffold below; prose to follow.

45.1 Overview

45.2 Background and Goals

45.2.1 Problem statement

45.2.2 Constraints and prior art

45.3 Methodology

45.3.1 Measurement approach

45.3.2 Benchmark setup

45.3.2.1 Hardware configuration

45.3.2.2 Software stack and reproducibility

45.4 Implementation

45.4.1 Design decisions

45.4.2 Component breakdown

45.4.3 Integration with existing systems

45.5 Results

45.5.1 Quantitative findings

45.5.2 Qualitative observations

45.6 Discussion

45.6.1 Trade-offs

45.6.2 Open questions

45.7 Summary and Cross-References

CHAPTER 46

Op Emitters

Chapter 46 — Op Emitters. Multi-level scaffold below; prose to follow.

46.1 Overview

46.2 Background and Goals

46.2.1 Problem statement

46.2.2 Constraints and prior art

46.3 Methodology

46.3.1 Measurement approach

46.3.2 Benchmark setup

46.3.2.1 Hardware configuration

46.3.2.2 Software stack and reproducibility

46.4 Implementation

46.4.1 Design decisions

46.4.2 Component breakdown

46.4.3 Integration with existing systems

46.5 Results

46.5.1 Quantitative findings

46.5.2 Qualitative observations

46.6 Discussion

46.6.1 Trade-offs

46.6.2 Open questions

46.7 Summary and Cross-References

CHAPTER 47

Forward and Backward in a Single Graph

Chapter 47 — Forward and Backward in a Single Graph. Multi-level scaffold below; prose to follow.

47.1 Overview

47.2 Background and Goals

47.2.1 Problem statement

47.2.2 Constraints and prior art

47.3 Methodology

47.3.1 Measurement approach

47.3.2 Benchmark setup

47.3.2.1 Hardware configuration

47.3.2.2 Software stack and reproducibility

47.4 Implementation

47.4.1 Design decisions

47.4.2 Component breakdown

47.4.3 Integration with existing systems

47.5 Results

47.5.1 Quantitative findings

47.5.2 Qualitative observations

47.6 Discussion

47.6.1 Trade-offs

47.6.2 Open questions

47.7 Summary and Cross-References

CHAPTER 48

CompiledModule API

Chapter 48 — CompiledModule API. Multi-level scaffold below; prose to follow.

48.1 Overview

48.2 Background and Goals

48.2.1 Problem statement

48.2.2 Constraints and prior art

48.3 Methodology

48.3.1 Measurement approach

48.3.2 Benchmark setup

48.3.2.1 Hardware configuration

48.3.2.2 Software stack and reproducibility

48.4 Implementation

48.4.1 Design decisions

48.4.2 Component breakdown

48.4.3 Integration with existing systems

48.5 Results

48.5.1 Quantitative findings

48.5.2 Qualitative observations

48.6 Discussion

48.6.1 Trade-offs

48.6.2 Open questions

48.7 Summary and Cross-References

CHAPTER 49

Hardening and Edge Cases

Chapter 49 — Hardening and Edge Cases. Multi-level scaffold below; prose to follow.

49.1 Overview

49.2 Background and Goals

49.2.1 Problem statement

49.2.2 Constraints and prior art

49.3 Methodology

49.3.1 Measurement approach

49.3.2 Benchmark setup

49.3.2.1 Hardware configuration

49.3.2.2 Software stack and reproducibility

49.4 Implementation

49.4.1 Design decisions

49.4.2 Component breakdown

49.4.3 Integration with existing systems

49.5 Results

49.5.1 Quantitative findings

49.5.2 Qualitative observations

49.6 Discussion

49.6.1 Trade-offs

49.6.2 Open questions

49.7 Summary and Cross-References

Part **VIII**

Model Zoo

CHAPTER 50

Zoo Architecture

Chapter 50 — Zoo Architecture. Multi-level scaffold below; prose to follow.

50.1 Overview

50.2 Architecture

50.2.1 Core building blocks

50.2.2 Variants and parameterizations

50.3 Forward Computation

50.3.1 Mathematical formulation

50.3.2 Implementation in Lucid

50.3.2.1 Module class layout

50.3.2.2 Backend dispatch

50.4 Training Considerations

50.4.1 Initialization strategy

50.4.2 Backward pass

50.4.3 Interaction with optimizer and AMP

50.5 Implementation Notes

50.5.1 File layout and class hierarchy

50.5.2 Edge cases and pitfalls

50.6 Worked Examples and Benchmarks

50.7 Summary and Cross-References

CHAPTER 51

Vision: CNN Family

Chapter 51 — Vision: CNN Family. Multi-level scaffold below; prose to follow.

51.1 Overview

51.2 Architecture

51.2.1 Core building blocks

51.2.2 Variants and parameterizations

51.3 Forward Computation

51.3.1 Mathematical formulation

51.3.2 Implementation in Lucid

51.3.2.1 Module class layout

51.3.2.2 Backend dispatch

51.4 Training Considerations

51.4.1 Initialization strategy

51.4.2 Backward pass

51.4.3 Interaction with optimizer and AMP

51.5 Implementation Notes

51.5.1 File layout and class hierarchy

51.5.2 Edge cases and pitfalls

51.6 Worked Examples and Benchmarks

51.7 Summary and Cross-References

CHAPTER 52

Vision Transformers

Chapter 52 — Vision Transformers. Multi-level scaffold below; prose to follow.

52.1 Overview

52.2 Architecture

52.2.1 Core building blocks

52.2.2 Variants and parameterizations

52.3 Forward Computation

52.3.1 Mathematical formulation

52.3.2 Implementation in Lucid

52.3.2.1 Module class layout

52.3.2.2 Backend dispatch

52.4 Training Considerations

52.4.1 Initialization strategy

52.4.2 Backward pass

52.4.3 Interaction with optimizer and AMP

52.5 Implementation Notes

52.5.1 File layout and class hierarchy

52.5.2 Edge cases and pitfalls

52.6 Worked Examples and Benchmarks

52.7 Summary and Cross-References

CHAPTER 53

Detection and Segmentation

Chapter 53 — Detection and Segmentation. Multi-level scaffold below; prose to follow.

53.1 Overview

53.2 Architecture

53.2.1 Core building blocks

53.2.2 Variants and parameterizations

53.3 Forward Computation

53.3.1 Mathematical formulation

53.3.2 Implementation in Lucid

53.3.2.1 Module class layout

53.3.2.2 Backend dispatch

53.4 Training Considerations

53.4.1 Initialization strategy

53.4.2 Backward pass

53.4.3 Interaction with optimizer and AMP

53.5 Implementation Notes

53.5.1 File layout and class hierarchy

53.5.2 Edge cases and pitfalls

53.6 Worked Examples and Benchmarks

53.7 Summary and Cross-References

CHAPTER 54

Language Models

Chapter 54 — Language Models. Multi-level scaffold below; prose to follow.

54.1 Overview

54.2 Architecture

54.2.1 Core building blocks

54.2.2 Variants and parameterizations

54.3 Forward Computation

54.3.1 Mathematical formulation

54.3.2 Implementation in Lucid

54.3.2.1 Module class layout

54.3.2.2 Backend dispatch

54.4 Training Considerations

54.4.1 Initialization strategy

54.4.2 Backward pass

54.4.3 Interaction with optimizer and AMP

54.5 Implementation Notes

54.5.1 File layout and class hierarchy

54.5.2 Edge cases and pitfalls

54.6 Worked Examples and Benchmarks

54.7 Summary and Cross-References

CHAPTER 55

Generative Models

Chapter 55 — Generative Models. Multi-level scaffold below; prose to follow.

55.1 Overview

55.2 Architecture

55.2.1 Core building blocks

55.2.2 Variants and parameterizations

55.3 Forward Computation

55.3.1 Mathematical formulation

55.3.2 Implementation in Lucid

55.3.2.1 Module class layout

55.3.2.2 Backend dispatch

55.4 Training Considerations

55.4.1 Initialization strategy

55.4.2 Backward pass

55.4.3 Interaction with optimizer and AMP

55.5 Implementation Notes

55.5.1 File layout and class hierarchy

55.5.2 Edge cases and pitfalls

55.6 Worked Examples and Benchmarks

55.7 Summary and Cross-References

CHAPTER 56

Pretrained Weights and Hub

Chapter 56 — Pretrained Weights and Hub. Multi-level scaffold below; prose to follow.

56.1 Overview

56.2 Architecture

56.2.1 Core building blocks

56.2.2 Variants and parameterizations

56.3 Forward Computation

56.3.1 Mathematical formulation

56.3.2 Implementation in Lucid

56.3.2.1 Module class layout

56.3.2.2 Backend dispatch

56.4 Training Considerations

56.4.1 Initialization strategy

56.4.2 Backward pass

56.4.3 Interaction with optimizer and AMP

56.5 Implementation Notes

56.5.1 File layout and class hierarchy

56.5.2 Edge cases and pitfalls

56.6 Worked Examples and Benchmarks

56.7 Summary and Cross-References

Part IX

Performance Engineering

CHAPTER 57

Performance Methodology

Chapter 57 — Performance Methodology. Multi-level scaffold below; prose to follow.

57.1 Overview

57.2 Background and Goals

57.2.1 Problem statement

57.2.2 Constraints and prior art

57.3 Methodology

57.3.1 Measurement approach

57.3.2 Benchmark setup

57.3.2.1 Hardware configuration

57.3.2.2 Software stack and reproducibility

57.4 Implementation

57.4.1 Design decisions

57.4.2 Component breakdown

57.4.3 Integration with existing systems

57.5 Results

57.5.1 Quantitative findings

57.5.2 Qualitative observations

57.6 Discussion

57.6.1 Trade-offs

57.6.2 Open questions

57.7 Summary and Cross-References

CHAPTER 58

Stabilization Chain (3.0.0–3.2.2)

Chapter 58 — Stabilization Chain (3.0.0–3.2.2). Multi-level scaffold below; prose to follow.

58.1 Overview

58.2 Background and Goals

58.2.1 Problem statement

58.2.2 Constraints and prior art

58.3 Methodology

58.3.1 Measurement approach

58.3.2 Benchmark setup

58.3.2.1 Hardware configuration

58.3.2.2 Software stack and reproducibility

58.4 Implementation

58.4.1 Design decisions

58.4.2 Component breakdown

58.4.3 Integration with existing systems

58.5 Results

58.5.1 Quantitative findings

58.5.2 Qualitative observations

58.6 Discussion

58.6.1 Trade-offs

58.6.2 Open questions

58.7 Summary and Cross-References

CHAPTER 59

AMP Integration (3.3.0)

Chapter 59 — AMP Integration (3.3.0). Multi-level scaffold below; prose to follow.

59.1 Overview

59.2 Background and Goals

59.2.1 Problem statement

59.2.2 Constraints and prior art

59.3 Methodology

59.3.1 Measurement approach

59.3.2 Benchmark setup

59.3.2.1 Hardware configuration

59.3.2.2 Software stack and reproducibility

59.4 Implementation

59.4.1 Design decisions

59.4.2 Component breakdown

59.4.3 Integration with existing systems

59.5 Results

59.5.1 Quantitative findings

59.5.2 Qualitative observations

59.6 Discussion

59.6.1 Trade-offs

59.6.2 Open questions

59.7 Summary and Cross-References

CHAPTER 60

MPSGraph Per-Op Dispatch (3.4.0)

Chapter 60 — MPSGraph Per-Op Dispatch (3.4.0). Multi-level scaffold below; prose to follow.

60.1 Overview

60.2 Background and Goals

60.2.1 Problem statement

60.2.2 Constraints and prior art

60.3 Methodology

60.3.1 Measurement approach

60.3.2 Benchmark setup

60.3.2.1 Hardware configuration

60.3.2.2 Software stack and reproducibility

60.4 Implementation

60.4.1 Design decisions

60.4.2 Component breakdown

60.4.3 Integration with existing systems

60.5 Results

60.5.1 Quantitative findings

60.5.2 Qualitative observations

60.6 Discussion

60.6.1 Trade-offs

60.6.2 Open questions

60.7 Summary and Cross-References

CHAPTER 61

No-Compile Algorithmic Sweep (3.4.1)

Chapter 61 — No-Compile Algorithmic Sweep (3.4.1). Multi-level scaffold below; prose to follow.

61.1 Overview

61.2 Background and Goals

61.2.1 Problem statement

61.2.2 Constraints and prior art

61.3 Methodology

61.3.1 Measurement approach

61.3.2 Benchmark setup

61.3.2.1 Hardware configuration

61.3.2.2 Software stack and reproducibility

61.4 Implementation

61.4.1 Design decisions

61.4.2 Component breakdown

61.4.3 Integration with existing systems

61.5 Results

61.5.1 Quantitative findings

61.5.2 Qualitative observations

61.6 Discussion

61.6.1 Trade-offs

61.6.2 Open questions

61.7 Summary and Cross-References

CHAPTER 62

Graph-Capture Wins (3.5.0)

Chapter 62 — Graph-Capture Wins (3.5.0). Multi-level scaffold below; prose to follow.

62.1 Overview

62.2 Background and Goals

62.2.1 Problem statement

62.2.2 Constraints and prior art

62.3 Methodology

62.3.1 Measurement approach

62.3.2 Benchmark setup

62.3.2.1 Hardware configuration

62.3.2.2 Software stack and reproducibility

62.4 Implementation

62.4.1 Design decisions

62.4.2 Component breakdown

62.4.3 Integration with existing systems

62.5 Results

62.5.1 Quantitative findings

62.5.2 Qualitative observations

62.6 Discussion

62.6.1 Trade-offs

62.6.2 Open questions

62.7 Summary and Cross-References

Part **X**

Production Engineering

CHAPTER 63

Testing Strategy

Chapter 63 — Testing Strategy. Multi-level scaffold below; prose to follow.

63.1 Overview

63.2 Background and Goals

63.2.1 Problem statement

63.2.2 Constraints and prior art

63.3 Methodology

63.3.1 Measurement approach

63.3.2 Benchmark setup

63.3.2.1 Hardware configuration

63.3.2.2 Software stack and reproducibility

63.4 Implementation

63.4.1 Design decisions

63.4.2 Component breakdown

63.4.3 Integration with existing systems

63.5 Results

63.5.1 Quantitative findings

63.5.2 Qualitative observations

63.6 Discussion

63.6.1 Trade-offs

63.6.2 Open questions

63.7 Summary and Cross-References

CHAPTER 64

Serialization and Safetensors

Chapter 64 — Serialization and Safetensors. Multi-level scaffold below; prose to follow.

64.1 Overview

64.2 Background and Goals

64.2.1 Problem statement

64.2.2 Constraints and prior art

64.3 Methodology

64.3.1 Measurement approach

64.3.2 Benchmark setup

64.3.2.1 Hardware configuration

64.3.2.2 Software stack and reproducibility

64.4 Implementation

64.4.1 Design decisions

64.4.2 Component breakdown

64.4.3 Integration with existing systems

64.5 Results

64.5.1 Quantitative findings

64.5.2 Qualitative observations

64.6 Discussion

64.6.1 Trade-offs

64.6.2 Open questions

64.7 Summary and Cross-References

CHAPTER 65

Data Loading Pipeline

Chapter 65 — Data Loading Pipeline. Multi-level scaffold below; prose to follow.

65.1 Overview

65.2 Background and Goals

65.2.1 Problem statement

65.2.2 Constraints and prior art

65.3 Methodology

65.3.1 Measurement approach

65.3.2 Benchmark setup

65.3.2.1 Hardware configuration

65.3.2.2 Software stack and reproducibility

65.4 Implementation

65.4.1 Design decisions

65.4.2 Component breakdown

65.4.3 Integration with existing systems

65.5 Results

65.5.1 Quantitative findings

65.5.2 Qualitative observations

65.6 Discussion

65.6.1 Trade-offs

65.6.2 Open questions

65.7 Summary and Cross-References

CHAPTER 66

Error Handling and Anomaly Detection

Chapter 66 — Error Handling and Anomaly Detection. Multi-level scaffold below; prose to follow.

66.1 Overview

66.2 Background and Goals

66.2.1 Problem statement

66.2.2 Constraints and prior art

66.3 Methodology

66.3.1 Measurement approach

66.3.2 Benchmark setup

66.3.2.1 Hardware configuration

66.3.2.2 Software stack and reproducibility

66.4 Implementation

66.4.1 Design decisions

66.4.2 Component breakdown

66.4.3 Integration with existing systems

66.5 Results

66.5.1 Quantitative findings

66.5.2 Qualitative observations

66.6 Discussion

66.6.1 Trade-offs

66.6.2 Open questions

66.7 Summary and Cross-References

CHAPTER 67

Reproducibility and Determinism

Chapter 67 — Reproducibility and Determinism. Multi-level scaffold below; prose to follow.

67.1 Overview

67.2 Background and Goals

67.2.1 Problem statement

67.2.2 Constraints and prior art

67.3 Methodology

67.3.1 Measurement approach

67.3.2 Benchmark setup

67.3.2.1 Hardware configuration

67.3.2.2 Software stack and reproducibility

67.4 Implementation

67.4.1 Design decisions

67.4.2 Component breakdown

67.4.3 Integration with existing systems

67.5 Results

67.5.1 Quantitative findings

67.5.2 Qualitative observations

67.6 Discussion

67.6.1 Trade-offs

67.6.2 Open questions

67.7 Summary and Cross-References

APPENDIX **A**

Hard Rules H1–H12

Appendix — Hard Rules H1–H12. Scaffold below; content to follow.

A.1 Purpose and Scope

A.2 Reference Material

A.2.1 Primary entries

A.2.2 Supplementary notes

A.3 Cross-References

Production Pillars P1–P9

Appendix — Production Pillars P1–P9. Scaffold below; content to follow.

B.1 Purpose and Scope

B.2 Reference Material

B.2.1 Primary entries

B.2.2 Supplementary notes

B.3 Cross-References

Op Reference

Appendix — Op Reference. Scaffold below; content to follow.

C.1 Purpose and Scope

C.2 Reference Material

C.2.1 Primary entries

C.2.2 Supplementary notes

C.3 Cross-References

Public API Snapshot

Appendix — Public API Snapshot. Scaffold below; content to follow.

D.1 Purpose and Scope

D.2 Reference Material

D.2.1 Primary entries

D.2.2 Supplementary notes

D.3 Cross-References

Hardware Benchmark Data

Appendix — Hardware Benchmark Data. Scaffold below; content to follow.

E.1 Purpose and Scope

E.2 Reference Material

E.2.1 Primary entries

E.2.2 Supplementary notes

E.3 Cross-References

Bibliography

- [1] Apple Inc. *Apple M1 chip*. <https://www.apple.com/newsroom/2020/11/apple-unleashes-m1/>. 2020.
- [2] Apple Inc. *Accelerate Framework Documentation*. <https://developer.apple.com/documentation/accelerate>. 2024.
- [3] Apple Inc. *Apple M4 chip family*. <https://www.apple.com/newsroom/2024/>. 2024.
- [4] Apple Inc. *Metal Programming Guide*. <https://developer.apple.com/metal/>. 2024.
- [5] Apple Inc. *MPSGraph Documentation*. <https://developer.apple.com/documentation/metalperformanceshadersgraph>. 2024.
- [6] ARM Holdings. *Arm Scalable Matrix Extension (SME) Specification*. Tech. rep. ARM Architecture Reference Manual, 2021.
- [7] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. “Automatic Differentiation in Machine Learning: A Survey”. In: *Journal of Machine Learning Research* 18 (2018), pp. 1–43.
- [8] T. Chen, B. Xu, C. Zhang, and C. Guestrin. “Training Deep Nets with Sublinear Memory Cost”. In: *arXiv preprint arXiv:1604.06174*. 2016.
- [9] J. W. Cooley and J. W. Tukey. “An Algorithm for the Machine Calculation of Complex Fourier Series”. In: *Mathematics of Computation* 19.90 (1965), pp. 297–301.
- [10] DMLC. *DLPack: Open in-memory tensor structure for sharing across frameworks*. <https://github.com/dmlc/dlpack>. 2019.
- [11] C. Eckart and G. Young. “The Approximation of One Matrix by Another of Lower Rank”. In: *Psychometrika* 1 (1936), pp. 211–218.
- [12] C. Finn, P. Abbeel, and S. Levine. “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks”. In: *International Conference on Machine Learning (ICML)*. 2017.
- [13] G. H. Golub and C. F. Van Loan. *Matrix Computations*. 4th. Johns Hopkins University Press, 2013.
- [14] A. Griewank and A. Walther. “Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation”. In: (2008).
- [15] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert. *MLX: Efficient and flexible machine learning on Apple silicon*. <https://github.com/ml-explore/mlx>. 2023.
- [16] F. J. Harris. “On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform”. In: vol. 66. 1. 1978, pp. 51–83.
- [17] D. Hendrycks and K. Gimpel. “Gaussian Error Linear Units (GELUs)”. In: *arXiv preprint arXiv:1606.08415* (2016).

- [18] A. S. Householder. “Unitary Triangularization of a Nonsymmetric Matrix”. In: *Journal of the ACM* 5.4 (1958), pp. 339–342.
- [19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations*. 2022.
- [20] Hugging Face. *safetensors: Simple, safe way to store and distribute tensors*. <https://github.com/huggingface/safetensors>. 2023.
- [21] W. Jakob, J. Rhinelanders, and D. Moldovan. *pybind11 — Seamless operability between C++11 and Python*. <https://github.com/pybind/pybind11>. 2017.
- [22] D. Johnson. *Apple M1 Microarchitecture Reverse Engineering*. <https://dougallj.github.io/applecpu/firestorm.html>. 2021.
- [23] J. F. Kaiser and R. W. Schafer. “On the Use of the I0-Sinh Window for Spectrum Analysis”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 28.1 (1980), pp. 105–107.
- [24] D. P. Kingma and M. Welling. “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations*. 2014.
- [25] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. “Spectral Normalization for Generative Adversarial Networks”. In: *International Conference on Learning Representations*. 2018.
- [26] A. Rogozhnikov. *einops: Tensor Operations for Deep Learning*. <https://github.com/arogozhnikov/einops>. 2018.
- [27] L. N. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM, 1997.
- [28] P. D. Welch. “The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms”. In: *IEEE Transactions on Audio and Electroacoustics* 15.2 (1967), pp. 70–73.